

Content-dependent chunking for differential compression, the local maximum approach

Nikolaj Bjørner^{a,*}, Andreas Blass^b, Yuri Gurevich^a

^a Microsoft Research, One Microsoft Way, Redmond, WA 98052, USA
^b Mathematics Department, University of Michigan, Ann Arbor, MI 48109-1043, USA

ARTICLE INFO

Article history:
Received 10 July 2008
Received in revised form 29 May 2009
Available online 13 June 2009

Keywords:
Networking
Distributed file systems
Compression
Ergodic theory

ABSTRACT

When a file is to be transmitted from a sender to a recipient and when the latter already has a file somewhat similar to it, remote differential compression seeks to determine the similarities interactively so as to transmit only the part of the new file not already in the recipient's old file. Content-dependent chunking means that the sender and recipient chop their files into chunks, with the cutpoints determined by some internal features of the files, so that when segments of the two files agree (possibly in different locations within the files) the cutpoints in such segments tend to be in corresponding locations, and so the chunks agree. By exchanging hash values of the chunks, the sender and recipient can determine which chunks of the new file are absent from the old one and thus need to be transmitted.

We propose two new algorithms for content-dependent chunking, and we compare their behavior, on random files, with each other and with previously used algorithms. One of our algorithms, the local maximum chunking method, has been implemented and found to work better in practice than previously used algorithms.

Theoretical comparisons between the various algorithms can be based on several criteria, most of which seek to formalize the idea that chunks should be neither too small (so that hashing and sending hash values become inefficient) nor too large (so that agreements of entire chunks become unlikely). We propose a new criterion, called the slack of a chunking method, which seeks to measure how much of an interval of agreement between two files is wasted because it lies in chunks that don't agree.

Finally, we show how to efficiently find the cutpoints for local maximum chunking.

© 2009 Elsevier Inc. All rights reserved.

Contents

1.	Introduction	155
1.1.	Applicability	158
2.	Preliminaries	159
2.1.	Files	159
2.2.	Ergodic theory	160
2.3.	Useful formulas	162
2.4.	Greedy increasing sequences	163
3.	Chunking methods and slack	165
3.1.	Chunking methods and locality	165

* Corresponding author.
E-mail addresses: nbjorner@microsoft.com (N. Bjørner), ablass@umich.edu (A. Blass), gurevich@microsoft.com (Y. Gurevich).



内容相关分块用于差分压缩：局部最大值方法

Nikolaj Bjørner^{a,*}, Andreas Blass^b, Yuri Gurevich^a

^a 微软研究院, One Microsoft Way, 雷德蒙德, 华盛顿州 98052, 美国
^b 数学系, 密歇根大学, 安娜堡, 密歇根州 48109-1043, 美国

ARTICLE INFO

文章历史：收到于 2008 年 7 月 10 日收到修改稿于 2009 年 5 月 29 日网络上可用于 2009 年 6 月 13 日

关键词：网络 分布式文件系统 压缩

遍历理论

abstract

当文件需要从发送方传输到接收方，且接收方已拥有一个与该文件部分相似的文件时，远程差分压缩旨在交互式地确定相似性，以便仅传输新文件中未包含在接收方旧文件中的部分。内容相关分块意味着发送方和接收方将文件分割成块，切点由文件的一些内部特征决定，以便当两个文件的片段一致（可能位于文件的不同位置）时，这些片段中的切点倾向于位于相应位置，从而使块一致。通过交换块的哈希值，发送方和接收方可以确定新文件中哪些块缺失于旧文件，因此需要传输。我们提出了两种新的内容相关分块算法，并将它们在随机文件上的行为与其他先前使用的算法进行了比较。我们其中一种算法，即局部最大值分块方法，已被实现，并在实践中发现比先前使用的算法效果更好。各种算法之间的理论比较可以基于多个标准进行，其中大多数标准旨在将“块不应太小（以便哈希和发送哈希值变得低效）也不应太大（以便整个块的一致性变得不太可能）”这一概念形式化。我们提出了一个新的标准，称为分块方法的松弛度，旨在衡量两个文件之间一致区间中有多少因位于不一致的块中而被浪费。

nall呀，我们how how to efficiently find the cutpoints for local maximum chunking.

© 2009 Elsevier Inc. 版权所有。

目录

1.	简介.155
1.1.	适用性.158
2.	预备知识.159
2.1.	文件.159
2.2.	遍历理论.160
2.3.	有用公式.162
2.4.	贪婪递增序列.163
3.	分块方法与松弛.165
3.1.	分块方法与局部性.165

* 通讯作者。邮箱地址：nbjorner@microsoft.com (N. Bjørner), ablass@umich.edu (A. Blass), gurevich@microsoft.com (Y. Gurevich)。0022-0000/\$ – 请参见前页© 2009 Elsevier Inc. 保留所有权利。
doi:10.1016/j.jcss.2009.06.004

3.2.	Length of chunks	165
3.3.	Costs	168
3.4.	Slack from the left	169
3.5.	Slack from the right	170
3.6.	Quality of chunking methods	171
4.	Point filter methods	171
4.1.	Pure point filters	171
4.2.	Point filters without short chunks	172
4.3.	The slack of LBFS chunking	174
4.3.1.	Piece 1	176
4.3.2.	Piece 2	176
4.3.3.	Piece 3	177
4.3.4.	Piece 4	177
4.3.5.	Assembling the pieces	178
4.4.	The reverse slack of LBFS chunking	180
5.	Interval filter methods	180
5.1.	Definition of interval filter chunking	180
5.2.	Statistics of interval filter chunking	181
5.3.	Slack of the interval filter method	183
5.4.	Probability of long chunks	184
5.5.	Reverse slack of the interval filter method	187
6.	Local maximum chunking	188
6.1.	Statistics of local maximum chunking	188
7.	Probability of long chunks	189
7.1.	Pure point filter method	190
7.2.	LBFS method	190
7.3.	Interval filter method	191
7.4.	Local maximum method	191
7.4.1.	Splitting intervals without local maxima	192
7.4.2.	Greedy increasing sequence	192
7.4.3.	Good cuts	192
7.4.4.	Probabilities of $\rightarrow$ maxima	193
7.4.5.	Comparison with other methods	194
8.	Computing local maxima	195
9.	Evaluation and experiments	198
9.1.	Computation overhead	199
9.2.	Intra-file compression	199
9.3.	Distribution of long chunks	199
9.4.	Run-length compression	200
10.	Related work, variants, and open problems	202
10.1.	Previous work	202
10.2.	Hash-less local maxima	202
10.3.	Open problems	202
	Acknowledgments	203
	References	203

1. Introduction

The proliferation of networks such as intranets, extranets, and the internet has led to a large growth in the number of users that share information across wide networks. However, the amount of data that is transferred over the networks is still limited by cost and bandwidth constraints. As a result of limited network bandwidth, users can experience long delays or high costs in retrieving and transferring large amounts of data across a network.

Fortunately, there are stratagems for reducing the amount of data that must be transmitted. Data compression algorithms take advantage of the redundancy that is present in many files. They allow one to transmit not the file itself but information enabling the recipient to reconstruct the file; because of redundancy, this information may be much shorter than the original file.

In this paper, we are concerned with taking advantage of another frequently occurring situation, namely that the recipient already has a file similar to the one being transmitted. The idea here is, of course, to transmit only the new content, not the content that the recipient already has. Because what is transmitted is just the part of one file that differs from the other, one calls this compression method “differential compression.” What makes differential compression non-trivial is that, to implement this idea, one must first decide what part of the file doesn’t need to be sent, and one must decide this without sending massive amounts of information.

3.2. 块的长度.

1. 简介

随着内网、外网和互联网等网络的普及，共享信息跨广域网络用户的数量已大幅增长。然而，网络传输的数据量仍然受成本和带宽限制。由于网络带宽有限，用户在跨网络检索和传输大量数据时可能会遇到长时间延迟或高昂成本。

幸运的是，有一些策略可以减少必须传输的数据量。数据压缩算法利用了许多文件中存在的冗余性。它们允许传输的不是文件本身，而是使接收方能够重建文件的信息；由于冗余性，这些信息可能比原始文件短得多。

在本文中，我们关注的是利用另一种频繁出现的情况，即接收方已经拥有与正在传输的文件相似的文件。这里的想法当然是要仅传输新内容，而不是接收方已经拥有的内容。因为传输的只是与其他文件不同的文件部分，因此将这种压缩方法称为“差异压缩”。差异压缩之所以不简单，是因为要实现这一想法，必须首先决定文件中哪些部分不需要发送，并且必须在不发送大量信息的情况下做出这个决定。

The most favorable situation of this sort arises, for example, in the distribution of software updates. Here, a client computer can tell a distribution server “Please update my program X to version 3.14159¹; I currently have version 2.71828.” The distribution server has a copy of the old version 2.71828, compares it with the new version 3.14159, and sends the client essentially just the difference between the two [27,2,30,15,16]. What makes this situation so pleasant is that the sender knows exactly what file the recipient already has (and that the sender got this information via a very short message from the recipient). “Local differential compression” refers to this situation where, after the initial message from the recipient, the sender can locally decide what information needs to be sent. Copious use of local differential compression is found in source and revision control systems [25,29], in file systems [19], and in domain specific versions, such as binary code differencing used for distributing software patches [9,21].

We shall treat the more difficult situation where the recipient has a file F_1 that is believed to be similar to the file F_2 to be transmitted, but the exact content of F_1 is not known to the sender. We shall discuss some existing protocols and propose new ones for handling this situation, i.e., for taking advantage of similarity between F_1 and F_2 to reduce the amount of data being sent. “Remote differential compression” (RDC) refers to such protocols, where the difference between the two files cannot be produced by the sender alone but must be determined interactively by the sender and the recipient. We also suggest a new measure, which we call slack, for comparing such protocols. And we compare the various protocols using several criteria, including slack.

A comprehensive overview of how chunking is used for RDC, how RDC can be used recursively, how similar files are identified for RDC, and how RDC can be used within a file replication system is presented in [28].

All the RDC protocols that we consider involve dividing at least one of the files into segments, which we call *chunks*, and computing hash values of these chunks to determine which chunks are common between the sender’s and the recipient’s files and thus do not need to be sent. The protocols differ in how the chunks are chosen.

Remark 1. RDC chunking protocols are designed for situations where reasonably large segments of F_1 and F_2 agree (but are perhaps in different locations in the two files). Other sorts of similarity would require other sorts of RDC protocols.

One sort of similarity that may fail to produce agreement of significant chunks occurs in compilation of programs. Compilers produce binaries with jump statements, where the jump locations are offsets into the file. These offsets are represented as absolute numbers. As a result, two almost equal source programs may compile to binaries with differences rather densely distributed throughout the files.

Something similar happens when files are compressed. Local differences between two files may result in densely distributed differences between their compressed versions.

Another sort of example arises from different methods of encoding. If a single file is encoded according to two different schemes, then the two encoded files are certainly similar in an intuitive sense, but that similarity may not result in any actual agreement between the encoded files.

All these situations are outside the scope of this paper. We are concerned here only with the use of chunking to detect and exploit agreement between reasonably long segments of two files.

The RSYNC protocol [31] uses the simplest and uniform choice of chunks: the recipient chops his file into chunks that are all of the same length l (except for the last chunk if l doesn’t divide the file length). He then sends a weak and a strong, collision resistant checksum (or hash value — we use the terms interchangeably) of each segment to the sender. The sender traverses his version of the file, computing weak checksums over a sliding window. The weak checksums are used in a crude, in-cache filter to find candidates to match the chunks hashed by the recipient. By using the strong checksums to validate the candidate local file chunks, the sender can then deduce which chunks the recipient already has and which parts of the file need to be transferred directly.

Note that, in this approach, it is necessary for the sender to compute weak checksums for all segments of length l in his file. It would not do for the sender to chop his file into chunks of length l as the recipient did and to compute checksums only for those chunks. Two files that differ merely by adding a single character at the beginning would almost surely have no chunks in common, so the nearly total similarity of the files would be entirely wasted.

The sender thus has to do considerable work, computing the hashes of all length l segments of the file he wants to send, and comparing the results with the list of hash values obtained from the recipient.

If a protocol of this sort is to be used repeatedly, to transfer F_2 to many recipients, who have different approximations F_1 , then the sender must either repeat all this work for each recipient or else store all the hash-values (considerably more data than the original file F_2) and then still do the comparisons separately for each recipient.

Remark 2. One could try to alleviate these problems by (partially) reversing roles. Let the sender chop F_2 into chunks of fixed size and send weak and strong hashes of these to the recipient. The recipient computes weak hashes in a sliding window to find chunks that might already be in F_1 . After using the strong checksums to confirm the candidates, he asks the sender for those chunks that he doesn’t already have. Of course, the total amount of work and communication here is essentially the same as in RSYNC, but if there are many recipients then much of the work is distributed among them, rather

¹ This choice of version number is used by TeX.

这种情况下最有利的情况，例如出现在软件更新的分发中。在这里，客户端计算机可以告诉分发服务器“请将我的程序X更新到版本3.14159¹；我目前拥有版本2.71828。”分发服务器拥有旧版本2.71828的副本，将其与新版本3.14159进行比较，并仅向客户端发送两者之间的差异 [27,2,30,15,16]。这种情况之所以令人愉快，是因为发送方确切地知道接收方已经拥有什么文件（并且发送方通过接收方发送的非常短的消息获得了这些信息）。“局部差异压缩”指的是这种情况，即接收方发送初始消息后，发送方可以本地决定需要发送哪些信息。在源代码和版本控制系统 [25,29]、文件系统 [19]，以及特定领域的版本中，例如用于分发软件补丁的二进制代码差异 [9,21]，都大量使用了局部差异压缩。

我们将处理更困难的情况，即接收方拥有一个文件 F_1 ，该文件被认为与要传输的文件 F_2 类似，但 F_1 的确切内容对发送方未知。我们将讨论一些现有协议并提出新的协议来处理这种情况，即利用 F_1 和 F_2 之间的相似性来减少发送的数据量。“远程差分压缩”（RDC）是指此类协议，其中两个文件之间的差异不能仅由发送方生成，而必须由发送方和接收方交互确定。我们还提出了一种新的度量标准，称为松弛，用于比较此类协议。我们使用多个标准（包括松弛）对各种协议进行了比较。

在 [28]中，详细介绍了如何使用分块技术进行 RDC，如何递归使用 RDC，如何为 RDC 识别相似文件，以及如何在文件复制系统中使用 RDC。

我们考虑的所有RDC协议都涉及将至少一个文件分割成我们称为块的片段，并计算这些块的哈希值，以确定发送方和接收方文件之间的哪些块是相同的，因此无需发送。这些协议在块的选择方式上有所不同。

备注1。RDC分块协议是为 F_1 和 F_2 的合理大片段（但可能位于两个文件的不同位置）相同时设计的。其他类型的相似性需要其他类型的RDC协议。

一种可能无法产生重要块一致性的相似性发生在程序编译中。编译器生成带有跳转语句的二进制文件，其中跳转位置是文件中的偏移量。这些偏移量表示为绝对数字。因此，两个几乎相等的源程序可能编译成在文件中分布相当密集的差异的二进制文件。

当文件被压缩时，也会发生类似的情况。两个文件之间的局部差异可能导致它们压缩版本之间出现密集分布的差异。

另一种例子则来自不同的编码方法。如果一个文件根据两种不同的方案进行编码，那么这两个编码后的文件在直观意义上肯定是相似的，但这种相似性可能不会导致编码文件之间有任何实际的一致性。

所有这些情况都不在本论文的范围內。我们这里只关注使用分块技术来检测和利用两个文件中合理长度的片段之间的一致性。

RSYNC协议 [31] 使用最简单且统一的块选择：接收方将他的文件分割成所有长度相同的块（如果长度 l 不能整除文件长度，则最后一个块除外）。然后他将每个片段的弱校验和以及强、抗冲突校验和（或哈希值——我们交替使用这些术语）发送给发送方。发送方遍历他的文件版本，在一个滑动窗口上计算弱校验和。弱校验和用于一个粗糙的缓存过滤器，以找到与接收方哈希的块匹配的候选。通过使用强校验和来验证候选本地文件块，发送方可以推断出接收方已经拥有哪些块，以及哪些文件部分需要直接传输。

请注意，在这种方法中，发送方必须对其文件中所有长度为 l 的片段计算弱校验和。发送方不能像接收方那样将其文件切成长度为 l 的块，并且仅对那些块计算校验和。两个文件如果仅在开头添加一个字符就不同，几乎肯定不会有任何共同的块，因此文件之间几乎完全的相似性将完全浪费。

发送方因此必须做大量工作，计算他想发送的文件所有长度为 l 的段的哈希值，并将结果与从接收方获得的哈希值列表进行比较。

如果一个协议要被反复使用，以将 F_2 转发给许多具有不同近似值 F_1 的接收方，那么发送方必须为每个接收方重复所有这些工作，或者存储所有哈希值（比原始文件 F_2 大得多的数据），然后仍然为每个接收方单独进行比较。

备注2。可以通过（部分）反转角色来缓解这些问题。让发送方将 F_2 切分成固定大小的块，并将这些块的弱哈希值和强哈希值发送给接收方。接收方在滑动窗口中计算弱哈希值，以找到可能已经存在于 F_1 中的块。在使用强校验和确认候选后，他向发送方请求那些他还没有的块。当然，这里的总工作和通信量与 RSYNC 基本相同，但如果接收方很多，那么大部分工作将分布在他们之间，而不是完全由发送方承担。另一方面，在许多应用中，例如更新日历日程或邮箱，从接收方的角度来看，文件传输只是附加工作，不是他们立即的工作的一部分。因此，将大部分工作分配给接收方可能不合适。

¹ 这个版本号的选择由TEX使用。

than being completely loaded on the sender. On the other hand, in many applications, such as updating calendar schedules or mailboxes, the file transfer is, from the recipients’ point of view, mere overhead, not part of their immediate work. So it may be inappropriate to assign most of the work to the recipients.

Remark 3. It is possible to reduce the communication overhead of the RSYNC protocol by using multiple rounds [18]. In the first round, use a relatively large chunk size. If there are large segments that match, they will be handled during this round. Subsequent rounds use progressively smaller chunk sizes.

Remark 4. We are concerned in this paper with reducing the communication needed for file transfer, but in particular applications other considerations may become especially important. For example, when files are sent to space-constrained devices, standard RSYNC has the drawback of requiring the receiver to create a fresh copy of the received file. For this situation, an in-place version of RSYNC is proposed in [24] to reduce the recipient’s storage needs.

The protocols that we treat in this paper, known ones as well as new, proceed differently from RSYNC in that both the sender and the recipient divide their files into chunks and compute (strong) checksums for the chunks. To avoid the pitfall described above, where a single character added to a file can make the chunks entirely different, the chunks in these protocols are not of a fixed length; rather, the places where the file is to be cut, the *chunk boundaries*, are determined by internal features of the files. This is the meaning of “content-dependent chunking.”

The protocols under consideration all proceed according to the following rough outline; details will be added later. As before, we use F_2 to denote the file to be transmitted and F_1 to denote a file that is already owned by the intended recipient of F_2 and that is believed to have substantial overlap with F_2 .

- (1) The sender chops F_2 into chunks and computes a hash value for each chunk.
- (2) The recipient does the same for F_1 .
- (3) The sender sends the recipient the hash values for F_2 (along with the lengths of the chunks).
- (4) The recipient compares those hash values with the ones he computed for F_1 . When two agree, he assumes that the corresponding chunks of F_1 and F_2 are the same, so there is no need for the sender to transmit those chunks of F_2 .
- (5) The recipient tells the sender which of the chunks of F_2 need to be sent.
- (6) The sender sends those chunks.

Remark 5. We describe in this remark a situation where content-dependent chunking has an important advantage over protocols like RSYNC that require a sliding window rather than independent chunking by the two parties. The situation is that the recipient is believed to have parts of F_2 in some file (or files) somewhere in his system, but it is not known where. In other words, the recipient doesn’t know which file should be F_1 (or perhaps he should use several files, each containing some part(s) of F_2). With a content-dependent protocol, the recipient can prepare (ahead of time) a list of hashes of all the chunks of all his relevant files. When he gets from the sender the hashes of the chunks of F_2 , he compares these with the contents of his list. If one tried to apply this idea to RSYNC, the recipient would send that whole list to the sender, who would have to compare everything listed with all the hashes produced in his sliding window. For the role-reversed variant of RSYNC, the situation is even worse; if the recipient wanted to prepare a list in advance, it would have to contain all the (weak) checksums of all the contents of the sliding window in all the relevant files.

Remark 6. The communication cost of content-dependent chunking can be reduced by using a chunking method recursively as follows [28]. Fix the parameter(s) of the chunking method to yield a relatively small chunk size c , just large enough to make it worthwhile to compute and send hashes of such chunks rather than the chunks themselves. Let Chk denote the length of a hash value. Apply the chunking method to produce a sequence of checksums, whose concatenation we regard as a new file $F^{(1)}$. On average, c symbols in F are represented by Chk symbols in $F^{(1)}$. Now apply the chunking protocol to $F^{(1)}$, obtaining a new file $F^{(2)}$. On average, Chk symbols here represent c symbols in $F^{(1)}$, i.e., c/Chk hash values in $F^{(1)}$, and thus c^2/Chk symbols in the original F . Repeating the process n times, we get a file $F^{(n)}$, each Chk symbols of which represent, on average c^n/Chk^{n-1} symbols of F . By choosing the number n of iterations suitably, we can arrange that each checksum in $F^{(n)}$ represents a rather large chunk of F . Now, to transmit F , the sender should first send $F^{(n)}$. When a hash value here matches one in the file already owned by the recipient, a large chunk of F has been transferred. For those hash values in $F^{(n)}$ that don’t match any of the recipient’s, the sender should next transfer the chunks of $F^{(n-1)}$ that were hashed to produce those values. Continue similarly for n rounds, sending the necessary chunks from $F^{(k)}$ for smaller and smaller k , where “necessary chunks” are those whose hashes didn’t match any of the recipient’s at the previous round. At the very end, when k has decreased to zero, send the remaining chunks of the original file F .

This sort of repeated chunking and hashing, converting F into $F^{(1)}$, then into $F^{(2)}$, and so on, would not work with a fixed chunk length protocol such as RSYNC. As we saw earlier, addition of a single character at the start of a file would completely change the checksums in $F^{(1)}$. As a result, all the later files $F^{(k)}$ would also be completely different, and the entire similarity between the files would be wasted. When one uses RSYNC repeatedly, sending large chunks first and then smaller ones as in Remark 3, it is necessary to process the whole original file F for each of the desired chunk sizes. With content-dependent methods, the files to be processed decrease in length at each step, by a factor c/Chk .

备注2。可以通过（部分）反转角色来缓解这些问题。让发送方将 $F[v1]$ 切分成固定大小的块，并将这些块的弱哈希值和强哈希值发送给接收方。接收方在滑动窗口中计算弱哈希值，以找到可能已经存在于 $F[v2]$ 中的块。在使用强校验和确认候选后，他向发送方请求那些他还没有的块。当然，这里的总工作和通信量与 RSYNC 基本相同，但如果接收方很多，那么大部分工作将分布在他们之间，而不是完全由发送方承担。另一方面，在许多应用中，例如更新日历日程或邮箱，从接收方的角度来看，文件传输只是附加工作，不是他们立即的工作的一部分。因此，将大部分工作分配给接收方可能不合适。

备注3. 通过使用多轮次 [18]可以减少 RSYNC 协议的通信开销。在第一轮次中，使用相对较大的块大小。如果有匹配的大型片段，它们将在这一轮次中处理。后续的轮次使用逐渐减小的块大小。

备注4. 本文关注的是减少文件传输所需的通信量，但在某些特定应用中，其他考虑因素可能变得尤为重要。例如，当文件发送到存储空间受限的设备时，标准 RSYNC 的缺点在于要求接收方创建接收文件的新副本。针对这种情况，在 [24] 中提出了 RSYNC 的就地版本，以减少接收方的存储需求。

本文讨论的协议，包括已知和新协议，与 RSYNC 不同之处在于，发送方和接收方都将文件分成块，并为这些块计算（强）校验和。为了避免上述提到的陷阱——即文件中添加单个字符就会使块完全不同——这些协议中的块不是固定长度的；相反，文件要被切割的位置，即块边界，是由文件内部特征决定的。这就是“内容相关分块”的含义。

所考虑的协议均按照以下大致框架进行；细节将在后续补充。如前所述，我们使用 F_2 表示待传输的文件，使用 F_1 表示 F_2 的预期接收方已拥有的文件，且该文件被认为与 F_2 有大量重叠。

- (1) 发送方将 F_2 切分成块，并为每个块计算哈希值。
- (2) 接收方对 F_1 执行相同的操作。
- (3) 发送方向接收方发送 F_2 的哈希值（以及块的长度）。
- (4) 接收方将收到的哈希值与为 F_1 计算的哈希值进行比较。当两者一致时，他假定 F_1 和 F_2 的相应块相同，因此发送方无需传输 F_2 的这些块。
- (5) 接收方告知发送方哪些 F_2 的块需要发送。
- (6) 发送方发送这些块。

备注5. 我们在本备注中描述了一种情况，即内容相关分块比需要滑动窗口而非双方独立分块的RSYNC等协议具有重要优势。这种情况是，接收方被认为在其系统中某些文件（或多个文件）中有 F_2 的部分，但不知道具体位置。换句话说，接收方不知道哪个文件应该是 F_1 （或者也许他应该使用多个文件，每个文件包含 F_2 的某些部分）。使用内容相关协议时，接收方可以提前准备一份其所有相关文件所有块哈希值的列表。当他从发送方收到 F_2 的块哈希值时，他会将这些与他的列表内容进行比较。如果尝试将此想法应用于RSYNC，接收方会将整个列表发送给发送方，发送方则必须将其与他在滑动窗口中生成的所有哈希值进行比较。对于RSYNC的反向角色变体，情况甚至更糟；如果接收方想提前准备一份列表，它必须包含所有相关文件中滑动窗口所有内容（弱）校验和。

备注6. 通过递归使用分块方法，可以降低内容相关分块通信成本，具体方法如下 [28]。将分块方法的参数设置为产生相对较小的块大小 c ，这个大小刚好足够使得计算和发送这些块的哈希值比发送块本身更值得。令 Chk 表示哈希值的长度。应用分块方法生成一系列校验和，我们将这些校验和的连接视为一个新文件 $F^{(1)}$ 。平均而言， F 中的 c 个符号由 $F^{(1)}$ 中的 Chk 个符号表示。现在应用分块协议到 $F^{(1)}$ ，得到一个新文件 $F^{(2)}$ 。平均而言，这里的 Chk 个符号表示 F 中的 c 个符号⁽¹⁾，即 F 中的 c/Chk 个哈希值⁽¹⁾，因此 c^2/Chk 个原始 F 中的符号。重复这个过程 n 次，我们得到文件 $F^{(n)}$ ，其中每个 Chk 个符号平均表示 F 中的 c^n/Chk^{n-1} 个符号。通过选择合适的迭代次数 n ，我们可以安排 $F^{(n)}$ 中的每个校验和表示 F 的一个相当大的块。现在，为了传输 F ，发送方应首先发送 $F^{(n)}$ 。当这里的哈希值与接收方已拥有的文件中的哈希值匹配时， F 的一个大块就已经传输完成。对于那些在 $F^{(n)}$ 中与接收方任何哈希值都不匹配的哈希值，发送方应接下来传输生成这些值的 $F^{(n-1)}$ 块的哈希值。类似地，对于 n 轮，发送必要的块 $F^{(k)}$ ，这些块的大小逐渐减小，其中“必要的块”是指在前一轮中其哈希值与接收方任何哈希值都不匹配的块。最后，当 k 减少到零时，发送原始文件 F 的剩余块。

这种重复分块和哈希的方法，将 F 转换为 $F^{(1)}$ ，再转换为 $F^{(2)}$ ，等等，无法在固定块长度的协议（如 RSYNC）中工作。如前所述，在文件开头添加单个字符会完全改变 $F^{(1)}$ 中的校验和。因此，所有后续文件 $F^{(k)}$ 也将完全不同，文件之间的所有相似性都将浪费。当反复使用 RSYNC 时，如注释 3 中所述，先发送大块再发送小块，必须为每个所需的块大小处理整个原始文件 F 。对于内容相关的方法，待处理的文件长度在每一步都会减少，减少因子为 c/Chk 。

The main difference between the various protocols we consider will be the *chunking methods*, i.e., how the files are to be divided into chunks at steps (1) and (2) in the outline above. In step (3), the sender should provide the lengths of the chunks of F_2 because these will not be fixed by the chunking method. He should also provide the locations in F_2 of those chunks (their *offsets*) if these cannot easily be computed from the lengths (e.g., if the information about different chunks might be received out of order). Then in step (5), the recipient can efficiently request the necessary chunks by sending their offsets to the sender.

This somewhat rough description of content-dependent chunking protocols makes some desiderata evident. First, the chunks should not be too short. The main reason is that the efficiency of the protocol depends on sending hash values that are significantly shorter than the chunks they represent. The hash values cannot be too short, lest accidental coincidences of hash values lead the recipient to think he already has a chunk when he doesn't. And the chunks themselves should be a good deal longer than the hash values; otherwise one might as well send the chunks themselves (i.e., send all of F_2) rather than computing and sending hash values.

There are other disadvantages associated with short chunks. One is that strong checksums have to be re-initialized for each small chunk, so setting up the computation for each strong checksum has an overhead. More importantly, each checksum is stored in a table and the table is searched for matches with checksums from the other files. There is a time overhead in storing and searching checksums.

On the other hand, the chunks should not be too long. With excessively long chunks, there is a risk that large segments of F_1 and F_2 might coincide yet no whole chunk coincides. Then the protocol would not detect any of the agreement between the two files, and the recipient would have to request all the chunks (i.e., all of F_2) from the sender.

A third desideratum is that similar files should be chopped into chunks at corresponding locations. Similarity of the files does us no good if the protocol fails to detect the similarity because the files were chopped into entirely different chunks. It is this requirement that prevents us from using chunks all of the same length in both files.

We shall describe and analyze a standard content-dependent chunking method, the one used in the Low Bandwidth File System (LBFS) proposed in [20], and we shall propose and analyze two new content-dependent chunking methods, called interval filter chunking and local maximum chunking. The analyses of these methods involve several measures, related to the desiderata described above. For example, since excessively long chunks and excessively short chunks both cause problems, it is desirable to keep the variance of the chunk length (on random files) low. For similar reasons, one may want to reduce the probability of getting chunks a great deal longer than the average chunk. We also introduce a more precise measure, though unfortunately rather difficult to compute, the slack of a chunking method, which takes into account not only the lengths of chunks but also the method's ability to take advantage of identical segments in files by putting chunk boundaries in matching places.

Let us say a few words on the history of the RDC project. It was conceived by and executed during 2003–2005 in the Core File Systems group in the Windows division of Microsoft. The project was successful, and the technology is widely used in Microsoft products. Results were reported in technical report [28]. The local-max method was the result of a collaboration of the Core File Systems group and Microsoft Research. The group performed experiments in order to evaluate different chunking methods. The local-max method proved to be superior to the competitors. The question arose whether there were a priori, mathematical reasons behind the better performance of the local-max method. We conceived this paper as a mathematical account with a relatively narrow purpose to clarify various issues related to content-dependent chunking. The narrow purpose is reflected in the title of the paper; the paper is not about the RDC project in general.

In more detail, the content of this paper is as follows. In Section 2, we present some preliminary information, including some mathematical tools needed later and some conventions concerning the files we consider. In Section 3, we introduce a simple probabilistic model of files with partial agreement, and we use it to define a measure, which we call slack, of the responsiveness of a chunking method to agreements between the files. That is, once two files start to agree, how much further in the files must one go until whole chunks agree? Section 4 is devoted to a description and analysis of point-filter methods, particularly the method used in LBFS [20]. In Section 5, we introduce and study one of our proposed new chunking methods, the interval filter method. Section 6 does the same for our second (and better) new method, the local maximum method. Section 7 is about the probabilities, under various chunking methods, of finding long intervals without any chunk boundaries. In Section 8, we give an efficient algorithm for finding the chunk boundaries in the local maximum method. (For the other methods, efficient algorithms are easy to see, but for the local maximum method this matter is not trivial.) Section 9 describes an experimental evaluation of the chunking methods in the context of Microsoft's RDC protocol. We also report a few experiments for evaluating chunking methods in isolation. Finally, in Section 10, we indicate connections with other work.

1.1. Applicability

Local maximum chunking is used as part of the RDC algorithms included in the Distributed File System Replication engine that was released as part of Windows Server 2003 R2 [4]. It is also being used as part of the file replication engine underlying Sharing Folders in the Windows Live Messenger 8.0 [5], and as part of Windows Meeting Space in Windows Vista [7]. The RDC algorithms are furthermore packaged as a stand-alone library that is made publicly available for application developers [6].

我们考虑的各种协议之间的主要区别在于分块方法，即文件在上述大纲步骤 (1) 和 (2) 中应如何分成块。在步骤 (3) 中，发送方应提供 F_2 的块长度，因为这些长度不会由分块方法固定。他还应提供 F_2 中这些块的位置（它们的偏移量），如果这些位置不能轻易地从长度计算得出（例如，如果关于不同块的信息可能会乱序接收）。然后在步骤 (5) 中，接收方可以通过将它们偏移量发送给发送方来高效地请求必要的块。

这种对内容相关分块协议的粗略描述使一些期望变得明显。首先，块不应太短。主要原因在于，协议的效率取决于发送的哈希值显著短于它们所代表的块。哈希值不能太短，否则偶然的哈希值巧合会导致接收方误以为他已经拥有一个块而实际上没有。而且块本身应比哈希值长得多；否则，与其计算并发送哈希值，不如直接发送块本身（即发送所有 F_2 ）。

短块还伴随着其他缺点。一个是，每个小块都需要重新初始化强校验和，因此为每个强校验和设置计算开销很大。更重要的是，每个校验和都存储在表中，表会搜索与其他文件校验和的匹配项。存储和搜索校验和存在时间开销。

另一方面，块不应该太长。块过长时，存在 F_1 和 F_2 的较大片段可能重合，但没有任何一个完整块重合的风险。那么协议将无法检测到两个文件之间的任何一致性，接收方将不得不从发送方请求所有块（即 F_2 的全部内容）。

第三条期望是，相似文件应该在相应位置被分割成块。如果协议因为文件被分割成完全不同的块而无法检测到相似性，那么文件的相似性就毫无用处。正是这一要求阻止我们在两个文件中都使用相同长度的块。

我们将描述和分析一种标准的基于内容的分块方法，即 [20]，中提出的低带宽文件系统（LBFS）所使用的方法，还将提出并分析两种新的基于内容的分块方法，称为区间过滤分块和局部最大值分块。这些方法的分析涉及多个与上述期望相关的指标。例如，由于过长的块和过短的块都会导致问题，因此希望（在随机文件上）保持块长度方差较低。出于类似的原因，人们可能希望降低获得远超平均块长度的块的概率。我们还引入了一个更精确的指标，尽管不幸的是计算起来相当困难，即分块方法的松弛度，它不仅考虑块长度，还考虑该方法通过在匹配位置放置块边界来利用文件中相同片段的能力。

让我们简单谈谈RDC项目的由来。该项目由微软Windows部门的核心文件系统小组在2003年至2005年间构思并执行。项目取得了成功，这项技术被广泛应用于微软的产品中。相关成果在技术报告 [28]中进行了报道。局部最大值方法是由核心文件系统小组和微软研究院合作研发的。该小组进行了实验，以评估不同的分块方法。局部最大值方法被证明优于其他方法。人们开始思考：局部最大值方法表现更好的背后是否存在先验的、数学上的原因？我们撰写这篇论文，旨在从数学角度解释与内容相关分块相关的一些问题。论文的狭窄目标体现在其标题中；本文并非关于RDC项目的一般性内容。

更详细地说，本文的内容如下。在第二节，我们介绍一些预备知识，包括后续所需的数学工具以及我们考虑的文件的相关约定。在第三节，我们介绍一种具有部分协议的文件简单概率模型，并利用该模型定义了一个度量，我们称之为松弛，用于衡量分块方法对文件间协议的响应程度。也就是说，当两个文件开始协议时，需要再深入文件多少才能使整个块达成协议？第四节专门描述和分析点过滤器方法，特别是LBFS [20]中使用的该方法。在第五节，我们介绍并研究了我们提出的一种新的分块方法——区间过滤器方法。第六节则针对我们的第二种（也是更好）的新方法——局部最大值方法进行介绍和研究。第七节讨论在各种分块方法下，找到没有块边界的长区间的概率。在第八节，我们给出一种在局部最大值方法中高效查找块边界的算法。（对于其他方法，容易看出高效的算法，但对于局部最大值方法，这一点并不简单。）第九节描述了在微软RDC协议背景下对分块方法的实验评估。我们还报告了一些用于独立评估分块方法的实验。最后，在第十节，我们指出了与其他工作的联系。

1.1. 适用范围

局部最大值分块作为 Windows Server 2003 R2 [4] 中发布的分布式文件系统复制引擎所包含的 RDC 算法的一部分被使用。它还作为 Windows Live Messenger 8.0 [5]，中共享文件夹底层文件复制引擎的一部分被使用，以及作为 Windows Vista [7] 中的 Windows Meeting Space 的一部分。此外，RDC 算法被打包为一个独立的库，供应用程序开发者公开使用 [6]。

2. Preliminaries

We collect in this section our conventions about files in general and random files in particular. We also recall some facts from probability theory, including ergodic theory, some formulas that will be used in our calculations, and some combinatorial information about greedy sequences. The reader may refer to [11,22], and [12] for further information about these topics.

2.1. Files

In the description and analysis of content-dependent chunking protocols, we shall use the following model of files.

We model a file as a sequence of elements from a finite set PFE of *potential file entries*. In reality, the sequence is always finite, its positions being indexed by a segment $[0, l - 1]$ of the natural numbers. (It is convenient to start the indexing at 0 rather than 1; we stop at $l - 1$ so that l denotes the length.) We shall, however, sometimes use infinite sequences, indexed by the set $\mathbb{N}$ of all natural numbers, or even doubly infinite sequences, indexed by the set $\mathbb{Z}$ of all integers. Infinite and doubly infinite sequences serve as a convenient mathematical approximation to long finite sequences.

When we use the words “left” and “right,” in connection with the positions in a file, we always assume the traditional picture of $\mathbb{Z}$; the integers lie on a horizontal line, with the smaller ones to the left of the larger ones. For example, we would call 0 the left end and $l - 1$ the right end of the interval $[0, l - 1]$.

For our analyses of various chunking methods, we shall assume that the entries of a file are probabilistically independent and uniformly distributed. That is, if I denotes the index set (a segment $[0, l - 1]$ or $\mathbb{N}$ or $\mathbb{Z}$), then we give the space PFE^I of files the product measure determined by the uniform measure on PFE. This means in particular that, if $i_1, \dots, i_k$ are distinct elements of I (i.e., distinct positions in a file), if $X_1, \dots, X_k$ are subsets of PFE, and if A is the set of those files F for which $F(i_j) \in X_j$ for each j (i.e., the entries at the positions i_j come from the corresponding sets X_j , all other entries being unconstrained), then A has probability (or measure)

$$\text{Prob}(A) = \frac{\prod_j |X_j|}{|\text{PFE}|^{|I|}},$$

where $|X|$ means the number of elements in the set X .

For finite I , it follows that any subset A of PFE^I has probability $|A|/(|\text{PFE}|^{|I|})$; that is, we have the uniform distribution on files. For infinite I , the laws of probability theory provide a unique measure, not for all subsets of PFE^I but for all reasonably well-behaved ones (known as measurable sets or as events). The measurable sets include all the sets that will arise in our discussion. This measure is also called the uniform measure, just as for finite I , even though it cannot be defined by simply saying that all individual elements of PFE^I have the same probability. (They do have the same probability, but it is zero, and so it tells us nothing about probabilities of more interesting events.)

We use standard terminology and notation from probability theory. For example, when A is a measurable set, we say that a random file has probability $\text{Prob}(A)$ of being in the set A . When this probability is 1, then we say that files are *almost surely* in A and that *almost all* files are in A . We use $\mathbf{E}(f)$ for the expectation and $\mathbf{Var}(f)$ for the variance of a random variable (i.e., a measurable, real-valued function on the probability space). We also use the standard notations $\mathbf{Prob}(A|B)$, $\mathbf{E}(f|B)$, and $\mathbf{Var}(f|B)$ for the conditional probability, expectation, and variance, conditional on the event B , assumed to have positive measure.

Whether the product measure accurately reflects the actual probabilities of files in the real world depends on the sort of files under consideration. Highly compressed files are close to random in our sense, but English text files are not, for two reasons. First, the probabilities of individual characters are not equal; the letter q occurs far less often than the uniform measure predicts, while the space occurs far more often. Second, the probabilities at different locations in the file are not independent; for example, the probability of the letter u is far higher immediately after q than elsewhere. Similarly, spreadsheets tend not to be random, as they often have considerable periodic content.

Fortunately, experimentation has shown that our protocols, particularly the local maximum chunking, work well even on common sorts of files, like English text, where our analysis becomes doubtful because our randomness assumptions fail.

Remark 7. There are rather easy ways of increasing the apparent randomness of a file. Given a file that is a sequence of symbols from an alphabet Σ (not our intended alphabet PFE), one can compute a hash value for each contiguous subsequence of some fixed window size w . The resulting sequence of hash values constitutes a new file, whose set PFE is the set of all possible hash values. Because of the hashing, this new file usually looks random even if the original file did not.

The time needed to compute hash values for all the windows of length w can be reduced by using a *rolling hash function*. This means that the hash value for each window except the first is computed from the hash value h of the immediately preceding window, the first symbol a in that preceding window (the symbol that is no longer present in the new window), and the last symbol b in the new window (the symbol that was not in the previous window).

If we assume that the symbols in Σ can be represented by bit-vectors of length w , then we can obtain a very simple rolling hash, using bit vectors of length w as hash values, and using the operations of bitwise exclusive or and rotation on these vectors, as follows. Given the hash value h for a particular window, given the first symbol a in that window (which is

2. 预备知识

在本节中，我们收集了关于文件的一般约定以及特定于随机文件的一些约定。我们还回顾了概率论的一些事实，包括遍历理论、将在我们的计算中使用的某些公式以及关于贪婪序列的某些组合信息。读者可以参考 [11,22]，和 [12] 获取这些主题的更多信息。

2.1. 文件

在内容相关分块协议的描述和分析中，我们将使用以下文件模型。

我们将文件建模为一个有限集合 PFE 中的潜在文件条目的元素序列。在实际中，该序列总是有限的，其位置由自然数的一个区间 $[0, l - 1]$ 索引。（从 0 而不是 1 开始索引很方便；我们停止在 $l - 1$ ，以便 l 表示长度。）然而，我们有时会使用由所有自然数的集合 $\mathbb{N}$ 索引的无穷序列，甚至是由所有整数的集合 $\mathbb{Z}$ 索引的双无穷序列。无穷和双无穷序列作为对长有限序列的方便数学近似。

当我们使用“左”和“右”这些词与文件中的位置相关联时，我们始终假设 $\mathbb{Z}$ 的传统图像；整数位于一条水平线上，较小的整数位于较大的整数左侧。例如，我们会称 0 为区间 $[0, l - 1]$ 的左端， $l - 1$ 为其右端。

在我们的各种分块方法分析中，我们将假设文件条目是概率独立的，并且均匀分布。也就是说，如果 I 表示索引集（一个包含 $[0, l - 1]$ 、 $\mathbb{N}$ 或 $\mathbb{Z}$ 的段），那么我们给文件空间 PFE^I 赋予由 PFE 上的均匀测度确定的乘积测度。这意味着，特别是，如果 $i_1 \dots i_k, \dots$ 是 I 的不同元素（即文件中的不同位置），如果 $X_1, \dots, X_k$ 是 PFE 的子集，并且如果 A 是那些文件 F 的集合，对于每个 j ， $F(i_j) \in X_j$ （即位置 i_j 处的条目来自相应的集合 X_j ，其他条目不受约束），那么 A 的概率（或测度）

$$\text{Prob}(A) = \frac{\prod_j |X_j|}{|\text{PFE}|^{|I|}},$$

其中 $|X|$ 表示集合 X 中的元素数量。

对于有限的 I ，由此可知 PFE^I 的任何子集 A 都有概率 $|A|/(|\text{PFE}|^{|I|})$ ；也就是说，我们得到了文件上的均匀分布。对于无限的 I ，概率论定律提供了一个唯一的测度，不是 PFE^I 的所有子集，而是所有合理良好行为的子集（称为可测集或事件）。可测集包括我们讨论中所有会出现的集合。这个测度也称为均匀测度，就像对于有限的 I 一样，尽管它不能简单地通过说 PFE^I 的所有单个元素具有相同的概率来定义。（它们确实具有相同的概率，但它是零，所以它告诉我们关于更有趣事件的概率的任何信息。）

我们使用概率论的标准术语和符号。例如，当 A 是一个可测集时，我们称随机文件在集合 A 中的概率为 $\text{Prob}(A)$ 。当这个概率为 1 时，我们就说文件几乎肯定在 A 中，并且几乎所有文件都在 A 中。我们使用 $\mathbf{E}(f)$ 表示随机变量（即概率空间上的可测、实值函数）的期望，使用 $\mathbf{Var}(f)$ 表示其方差。我们还使用标准符号 $\text{Prob}(A|B)$ 、 $\mathbf{E}(f|B)$ 和 $\mathbf{Var}(f|B)$ 表示在事件 B （假设具有正测度）下条件概率、期望和方差。

产品测度是否准确反映现实世界中文件的实际概率，取决于所考虑的文件类型。在我们的意义上，高度压缩的文件接近随机，但英文文本文件不是，原因有二。首先，单个字符的概率并不相等；字母 q 的出现频率远低于均匀测度所预测的，而空格的出现频率则远高于此。其次，文件不同位置的概率并不独立；例如，字母 u 在 q 之后立即出现的概率远高于其他位置。类似地，电子表格往往不是随机的，因为它们通常包含相当多的周期性内容。

幸运的是，实验表明我们的协议，尤其是局部最大值分块，即使对常见的文件类型，如英文文本，也能很好地工作，而在这些文件类型中，我们的分析变得可疑，因为我们的随机性假设失效了。

备注7。有相当简单的方法可以提高文件的可观随机性。给定一个由字母表 Σ （不是我们预期的字母表 PFE）中的符号序列组成的文件，可以为每个长度为固定窗口大小 w 的连续子序列计算一个哈希值。由此产生的哈希值序列构成一个新文件，其集合 PFE 是有可能哈希值的集合。由于哈希处理，这个新文件通常看起来是随机的，即使原始文件不是。

通过使用滚动哈希函数，可以减少为所有长度为 w 的窗口计算哈希值所需的时间。这意味着除了第一个窗口外，每个窗口的哈希值都是根据前一个窗口的哈希值 h 、该前一个窗口中的第一个符号 a （该符号不再存在于新窗口中）以及新窗口中的最后一个符号 b （该符号在前一个窗口中不存在）计算得出的。

如果我们假设 Σ 中的符号可以用长度为 w 的位向量表示，那么我们可以使用长度为 w 的位向量作为哈希值，并使用这些向量上的按位异或和旋转操作，得到一个非常简单的滚动哈希，如下所示。给定特定窗口的哈希值 h ，给定该窗口中的第一个符号 a （它是

about to leave the window), and given the next symbol b after that window (which is about to enter the window), regard a and b as bit vectors of length w . Compute the hash value of the next window by first taking the bit-wise exclusive-or $h \oplus a \oplus b$ and then rotating the resulting bit vector by one position (the last bit is removed and put in the front). Because the window size w equals the length of the bit vectors, when the element b that has just entered the window leaves the window w steps later, the hash vector will have been rotated by one full rotation. So the exclusive-or addition of b when it entered the window will be exactly canceled by the addition of b when it leaves the window.

A prime example of a rolling hash, for which the collision probabilities have been thoroughly analyzed, is the Rabin hash [23,17]. It is based on arithmetic modulo an irreducible polynomial with coefficients in $\mathbb{Z}/2$. The number $|\text{PFE}|$ of possible hash values can be adjusted by using polynomials of degree $\log(|\text{PFE}|)$.

The local maximum chunking method was originally proposed and implemented with a preliminary rolling hash, intended to introduce the randomness that our analysis presupposes. (Strictly speaking, a deterministic, length-preserving transformation cannot introduce or increase randomness. It can, however, mask any regularities so that they are unlikely to influence the analysis of chunking protocols.) Later, it was found experimentally that the local maximum method works well even without this preliminary hashing.

Rolling hashes essentially summarize the contents of a neighborhood in each position of the file, thus making the new file more resistant to local entropy variations.

Remark 8. There are additional actions that one can undertake in order to increase the entropy. For example, Mark Manasse noticed that if a short pattern repeats many times in succession, as in a long stretch of zeros, then that stretch can be compressed to a much shorter string before rolling hashes are applied. The idea is to replace many successive occurrences of the same string with one occurrence and the number of times to repeat it. (Care is needed to avoid possible ambiguity of such repetition instructions, but we need not concern ourselves with the details here.) Such *run-length encoding* is essential for the content-dependent chunking methods discussed in this paper when the file exhibits periodicity with period significantly shorter than the horizon of the chunking method. If no coding is undertaken, then such periodicity would produce undesirably long chunks under the interval filter and local maximum methods, because there would be no cutpoint in the periodic stretch of the file. Under the LBFS method, there would be cutpoints, but identical periodic segments in two files might well have their cutpoints in entirely different places.

We make an additional assumption about our files, namely that the set PFE of potential file entries is equipped with a linear ordering. In many situations, this assumption is clearly satisfied. If the potential file entries are hash values, or integers obtained in some other way, then we can use the usual ordering of integers. If they are characters, then we can order them by their ASCII codes or Unicodes.

One might even argue that, in real computers, potential file entries are always linearly ordered because they are ultimately represented by bit strings, and we can use the lexicographic ordering of these strings. This observation works as long as the sender and recipient use the same bit string representations. We need our linear orderings to be the same for the sender and the recipient, and whether the computers' internal bit strings can serve this purpose depends on the particular application.

Having assumed a linear ordering of PFE, we obtain a canonical bijection between PFE and $\{0, 1, \dots, |\text{PFE}| - 1\}$, namely the unique order-preserving bijection. We shall therefore, whenever it is convenient, assume without loss of generality that $\text{PFE} = \{0, 1, \dots, |\text{PFE}| - 1\}$.

2.2. Ergodic theory

We shall use a little ergodic theory in part of our analysis, so we summarize here what is needed. We state the results in their natural generality, namely a probability space Ω with a measure-preserving, one-to-one transformation T of Ω onto itself. In our applications of these results, however, Ω will always be the space $\text{PFE}^{\mathbb{Z}}$ of doubly infinite files, and T will always be the (leftward) *Bernoulli shift*, BS, which sends any file $F \in \text{PFE}^{\mathbb{Z}}$ to the file $G = \text{BS}(F)$ defined by $G(i) = F(i + 1)$. (The reader should see that, despite the impression one might get from the plus sign in $i + 1$, this really does shift a file to the left.) Thus, the reader can safely pretend that whenever we write Ω and T , we mean $\text{PFE}^{\mathbb{Z}}$ and BS. Clearly, the Bernoulli shift is a one-to-one function from $\text{PFE}^{\mathbb{Z}}$ onto itself. (Indeed, this is a major reason for using doubly infinite files.) It is also clear, from the definition of the probability measure on $\text{PFE}^{\mathbb{Z}}$, that this measure is invariant under BS. (So BS is an automorphism of the probability space.)

A measure-preserving bijection $T : \Omega \rightarrow \Omega$ is called *ergodic* if, whenever an event $A \subseteq \Omega$ is invariant (meaning $T(A) = A$), then its probability is 0 or 1. It is known that BS is ergodic (see [22, Section 2.4, Example (1)]), so all the following results about ergodic transformations apply to the particular case that we need later. Notice that the definition of ergodicity would be unchanged if we required probability 0 or 1 for all events A for which $T(A) \subseteq A$. This is because T is measure-preserving, so such an A would differ from $T(A)$ by a set of measure 0, and the intersection $\bigcap_{n \in \mathbb{N}} T^n(A)$ would be an invariant set differing from A by a set of measure 0.

We shall need three classical theorems of ergodic theory. Poincaré's recurrence theorem [22, Theorem II.3.2] implies that, if $T : \Omega \rightarrow \Omega$ is ergodic and if $A \subseteq \Omega$ is an event of positive probability, then almost all points $x \in \Omega$ have the property

给定特定窗口的哈希值 h , 给定该窗口中的第一个符号 a (即将离开窗口), 以及给定紧随其后的下一个符号 b (即将进入窗口), 将 a 和 b 视为长度为 w 的位向量。通过首先对 $h \oplus a \oplus b$ 进行按位异或, 然后旋转所得位向量一位 (最后一个位被移除并放到最前面), 来计算下一个窗口的哈希值。因为窗口大小 w 等于位向量的长度, 当刚刚进入窗口的元素 b 在 w 步之后离开窗口时, 哈希向量将旋转完整的一圈。因此, 当 b 进入窗口时的异或加法将正好被 b 离开窗口时的加法所抵消。

一个滚动哈希的典型例子是 Rabin 哈希[23,17], 其碰撞概率已被彻底分析。它基于系数在 $\mathbb{Z}/2$ 中的不可约多项式的算术模运算。可能的哈希值数量 $|\text{PFE}|$ 可以通过使用次数为 $\log(|\text{PFE}|)$ 的多项式来调整。

局部最大值分块方法最初是使用一个初步的滚动哈希提出的, 目的是引入我们分析所预设的随机性。(严格来说, 一个确定性、长度保持的转换不能引入或增加随机性。但它可以掩盖任何规律性, 从而使得它们不太可能影响分块协议的分析。)后来, 通过实验发现, 局部最大值方法即使没有这个初步哈希也能很好地工作。

滚动哈希本质上总结了文件每个位置邻域的内容, 从而使新文件更能抵抗局部熵变化。

备注8。可以通过采取额外操作来增加熵。例如, Mark Manasse注意到, 如果一个短模式连续重复多次, 比如一个很长的全零串, 那么这个串可以在应用滚动哈希之前被压缩成一个更短的字符串。这个想法是用一个出现次数和重复次数来替换多个连续出现的相同字符串。(需要注意避免这种重复指令可能产生的歧义, 但在这里我们不必关心细节。)这种行程长度编码对于本文讨论的内容相关分块方法至关重要, 当文件表现出周期性且周期显著短于分块方法的视野时。如果未进行编码, 那么在区间过滤器和局部最大值方法下, 这种周期性会产生不希望的长块, 因为在文件的周期性片段中不会有切割点。在LBFS方法下, 会有切割点, 但两个文件中相同的周期性片段的切割点可能完全不同。

我们对文件做出一个额外的假设, 即潜在的文件条目集合PFE配备了线性排序。在许多情况下, 这个假设显然成立。如果潜在的文件条目是哈希值, 或以其他方式获得的整数, 那么我们可以使用整数的常规排序。如果它们是字符, 那么我们可以按它们的ASCII码或Unicode码排序。

有人甚至可能认为, 在真实计算机中, 潜在的文件条目总是线性排序的, 因为它们最终由比特串表示, 我们可以使用这些串的词典排序。只要发送方和接收方使用相同的比特串表示, 这个观察就成立。我们需要发送方和接收方的线性排序相同, 而计算机的内部比特串能否实现这个目的则取决于具体的应用。

在假设了PFE的线性排序后, 我们得到了PFE和 $\{0, 1, \dots, |\text{PFE}| - 1\}$ 之间的规范双射, 即唯一的保序双射。因此, 只要方便, 我们无需损失普遍性就假设 $\text{PFE} = \{0, 1, \dots, |\text{PFE}| - 1\}$ 。

2.2. 遍历理论

在我们的分析中, 我们将使用一些遍历理论, 因此在此总结所需内容。我们以自然的一般性陈述结果, 即一个概率空间 Ω 及其上的测度保持、一一变换 T , 将 Ω 映射到自身。然而, 在应用这些结果时, Ω 始终是双无限文件空间 $\text{PFE}^{\mathbb{Z}}$, 而 T 始终是 (向左的) 伯努利移位 BS, 它将任何文件 $F \in \text{PFE}^{\mathbb{Z}}$ 映射到由 $G(i) = F(i + 1)$ 定义的文件 $G = \text{BS}(F)$ 。(读者应注意, 尽管 $i + 1$ 中的加号可能给人这样的印象, 但这确实将文件向左移位。)因此, 读者可以安全地假设, 每当我们在 Ω 和 T 时, 我们指的是 $\text{PFE}^{\mathbb{Z}}$ 和 BS。显然, 伯努利移位是 $\text{PFE}^{\mathbb{Z}}$ 到自身的一个一对一函数。(实际上, 这正是使用双无限文件的主要原因。)从 $\text{PFE}^{\mathbb{Z}}$ 上的概率测度定义来看, 该测度在 BS 下保持不变。(因此, BS 是概率空间的自同构。)

一个测度保持双射 $T : \Omega \rightarrow \Omega$ 被称为遍历的, 如果对于任何事件 $A \subseteq \Omega$, 只要它是不变的 (即 $T(A) = A$), 那么它的概率为 0 或 1。已知 BS 是遍历的 (参见 [22, 第 2.4 节, 示例 (1)]), 因此关于遍历变换的所有以下结果都适用于我们后面需要的特定情况。请注意, 如果我们要求所有满足 $T(A) \subseteq A$ 的事件 A 都具有概率 0 或 1, 那么遍历性的定义将保持不变。这是因为 T 是测度保持的, 因此这样的 A 与 $T(A)$ 的差异是一个测度为 0 的集合, 而交集 $\bigcap_{n \in \mathbb{N}} T^n(A)$ 将是一个与 A 差异为测度为 0 的不变集合。

我们需要三个遍历理论中的经典定理。Poincaré的回归定理 [22, 定理II.3.2] 意味着, 如果 $T : \Omega \rightarrow \Omega$ 是遍历的, 并且如果 $A \subseteq \Omega$ 是一个正概率事件, 那么几乎所有的点 $x \in \Omega$ 都具有这个性质

that $T^k(x) \in A$ for some positive integer k (in fact for infinitely many k). Birkhoff's ergodic theorem [22, Theorems II.2.3 and II.4.4] gives more detail about how often the sequence $T^k(x)$ visits A .

Proposition 9. Let $T : \Omega \rightarrow \Omega$ be ergodic and let $A \subseteq \Omega$ be any event. Then for almost all $x \in \Omega$,

$$\lim_{N \rightarrow \infty} \frac{\text{Number of } k \in [0, N-1] \text{ with } T^k(x) \in A}{N} = \mathbf{Prob}(A).$$

Another way to measure frequency of visits to A is the time until the first visit to A . For ergodic T , let $\rho(x)$ denote the least $k \geq 1$ with $T^k(x) \in A$. (Either define $\rho(x)$ arbitrarily on the measure-zero set of points x for which no such k exists, or simply ignore sets of measure zero.) Kac's theorem [22, Theorem II.4.6] gives the following result. Note that it is about random elements of A , not of the whole space Ω ; that is, the expectation in the conclusion of the theorem is conditional on $x \in A$.

Proposition 10. Let A be an event of positive probability p in Ω , and let x be a random member of A . The expectation of the return time $\rho(x)$, $\mathbf{E}(\rho|A)$, equals $1/p$.

It will be useful to have a companion result to Kac's theorem, giving the expectation of ρ on the whole space Ω rather than on A . Easy examples show that this $\mathbf{E}(\rho)$ is not determined by $\mathbf{Prob}(A)$ alone, but it turns out to be related to the variance of ρ on A .

Proposition 11. With notation as above,

$$\mathbf{E}(\rho) = \frac{1}{2} \left[\mathbf{Prob}(A) \mathbf{Var}(\rho|A) + \frac{1}{\mathbf{Prob}(A)} + 1 \right].$$

Proof. For the sake of brevity, we systematically ignore sets of measure zero; they do not affect any of the following computations. Partition A into the pieces $A_n = \{x \in A : \rho(x) = n\}$ ($n \in \mathbb{N} - \{0\}$).

It is not difficult to see that the sets $T^k(A_n)$ for $0 \leq k < n$ are all pairwise disjoint. Indeed, suppose, toward a contradiction, that we had $x \in T^k(A_n) \cap T^{k'}(A_{n'})$, and suppose this counterexample is chosen with k as small as possible. If neither k nor k' were 0, then $T^{-1}(x) \in T^{k-1}(A_n) \cap T^{k'-1}(A_{n'})$ would contradict the minimality of k . So we may assume $k = 0$ and so $x \in A_n \cap T^{k'}(A_{n'})$. In particular, $x \in A$ and $x = T^{k'}(y)$ for some $y \in A_{n'}$. But then from $k' < n' = \rho(y)$ we get $T^{k'}(y) \notin A$, a contradiction.

Consider how T acts on the sets $T^k(A_n)$ for $0 \leq k < n$. It sends each one to the one with k increased by 1, except that when $k = n-1$ it sends $T^{n-1}(A_n)$ into A , according to the definitions of A_n and ρ . Since A is the union of the various A_n 's, it follows that $\bigcup_{0 \leq k < n} T^k(A_n)$ (where both n and k vary) is mapped into itself by T . By ergodicity, its measure is 0 or 1. As it includes A , its measure cannot be 0, so it is almost all of the space Ω . Since we are ignoring sets of measure 0, we can say that Ω is partitioned into the sets $T^k(A_n)$, where, as before, $0 \leq k < n$.

From the definitions of A_n and ρ , it follows that ρ is constant on $T^k(A_n)$ with value $n-k$. Therefore,

$$\mathbf{E}(\rho) = \sum_{0 \leq k < n} (n-k) \mathbf{Prob}(T^k(A_n)) = \sum_{0 \leq k < n} (n-k) \mathbf{Prob}(A_n),$$

where the second equality uses the fact that T preserves the measure. Carrying out the summation over k for each fixed n , we get

$$\mathbf{E}(\rho) = \sum_{n=1}^{\infty} \left(\mathbf{Prob}(A_n) \frac{n(n+1)}{2} \right) = \frac{1}{2} \left[\sum_n n \mathbf{Prob}(A_n) + \sum_n n^2 \mathbf{Prob}(A_n) \right].$$

The first of the two sums in the brackets here can be rewritten as the sum of $\mathbf{Prob}(T^k(A_n))$ over all n and all $k < n$. So, as these sets $T^k(A_n)$ partition Ω , this sum is simply 1. (This observation is essentially a proof of Kac's theorem.) The second sum can be rewritten as follows, using the fact that, since $A_n \subseteq A$, $\mathbf{Prob}(A_n) = \mathbf{Prob}(A) \mathbf{Prob}(A_n|A)$,

$$\mathbf{Prob}(A) \sum_n n^2 \mathbf{Prob}(A_n|A) = \mathbf{Prob}(A) \mathbf{E}(\rho^2|A).$$

Furthermore, since all random variables satisfy $\mathbf{Var}(f) = \mathbf{E}(f^2) - \mathbf{E}(f)^2$, we can rewrite this in terms of the variance as

$$\mathbf{Prob}(A) [\mathbf{Var}(\rho|A) + \mathbf{E}(\rho|A)^2].$$

Remembering that $\mathbf{E}(\rho|A) = 1/\mathbf{Prob}(A)$ by Kac's theorem, and substituting the results of our computation back into the formula for $\mathbf{E}(\rho)$, we immediately get the proposition. $\square$

即 $T^k(x) \in A$ 对于某个正整数 k (实际上对于无限多个 k)。Birkhoff 的遍历定理 [22, 定理 II.2.3 和 II.4.4] 提供了更多关于序列 $T^k(x)$ 访问 A 的频率的细节

命题9. 设 $T : \Omega \rightarrow \Omega$ 是遍历的, 且 $A \subseteq \Omega$ 是任意事件. 那么对于几乎所有的 $x \in \Omega$,

$$\lim_{N \rightarrow \infty} \frac{\text{Number of } k \in [0, N-1] \text{ with } T^k(x) \in A}{N} = \mathbf{Prob}(A).$$

另一种衡量访问 A 频率的方法是第一次访问 A 之前的时间. 对于遍历的 T , 令 $\rho(x)$ 表示最小的 $k \geq 1$, 使得 $T^k(x) \in A$. (要么在不存在这样 k 的点集的测度为零的集合上任意定义 $\rho(x)$, 要么简单地忽略测度为零的集合.) 卡茨定理 [22, 定理 II.4.6] 给出了以下结果. 请注意, 它是关于 A 的随机元素, 而不是整个空间 Ω ; 也就是说, 定理结论中的期望是在 $x \in A$ 的条件下计算的.

命题10. 设 A 是 Ω 中概率为正数 p 的事件, 且 x 是 A 的一个随机成员. 返回时间 $\rho(x)$ 的期望等于 $1/p$.

要有一个与卡茨定理相配套的结果会很有用, 它给出的是 ρ 在整个空间 Ω 上的期望, 而不是在 A 上. 简单的例子表明, 这个 $\mathbf{E}(\rho)$ 并不是由 $\mathbf{Prob}(A)$ 单独决定的, 但它与 ρ 在 A 上的方差有关.

命题 11. 如上所述,

$$\mathbf{E}(\rho) = \frac{1}{2} \left[\mathbf{Prob}(A) \mathbf{Var}(\rho|A) + \frac{1}{\mathbf{Prob}(A)} + 1 \right].$$

证明. 为了简洁起见, 我们系统性地忽略测度为零的集合; 它们不会影响以下任何计算. 将 A 分成 $A_n = \{x \in A : \rho(x) = n\}$ ($n \in \mathbb{N} - \{0\}$) 的几块.

不难看出, 对于 $0 \leq k < n$ 的 $T^k(A_n)$ 都是两两不交的. 确实, 假设, 为了矛盾, 我们有了 $x \in T^k(A_n) \cap T^{k'}(A_{n'})$, 并且假设这个反例选择 k 尽可能小. 如果 k 和 k' 都不是 0, 那么 $T^{-1}(x) \in T^{k-1}(A_n) \cap T^{k'-1}(A_{n'})$ 会与 k 的最小性矛盾. 所以我们可以假设 $k = 0$, 因此 $x \in A_n \cap T^{k'}(A_{n'})$. 特别是, $x \in A$ 且 $x = T^{k'}(y)$ 对于某个 $y \in A_{n'}$. 但那么从 $k' < n' = \rho(y)$ 我们得到 $T^{k'}(y) \notin A$, 这是矛盾的.

考虑 T 如何作用于集合 $T^k(A_n)$ 上的 $0 \leq k < n$. 它将每个元素映射到 k 增加后的另一个元素, 但根据 A_n 和 ρ 的定义, 当 $k = n-1$ 时, 它将 $T^{n-1}(A_n)$ 映射到 A 中. 由于 A 是各种 A_n 's 的并集, 因此 $\bigcup_{0 \leq k < n} T^k(A_n)$ (其中 n 和 k 都变化) 被 T 映射到自身. 根据遍历性, 其测度为 0 或 1. 由于它包含 A , 其测度不能为 0, 因此它几乎占据了整个空间 Ω . 由于我们忽略了测度为 0 的集合, 我们可以说 Ω 被划分为集合 $T^k(A_n)$, 其中, 如前所述, $0 \leq k < n$.

根据 A_n 和 ρ 的定义, ρ 在 $T^k(A_n)$ 上是常数, 其值为 $n-k$. 因此,

$$\mathbf{E}(\rho) = \sum_{0 \leq k < n} (n-k) \mathbf{Prob}(T^k(A_n)) = \sum_{0 \leq k < n} (n-k) \mathbf{Prob}(A_n),$$

其中第二个等式使用了 T 保持测度的性质. 对每个固定的 n , 对 k 进行求和, 我们得到

$$\mathbf{E}(\rho) = \sum_{n=1}^{\infty} \left(\mathbf{Prob}(A_n) \frac{n(n+1)}{2} \right) = \frac{1}{2} \left[\sum_n n \mathbf{Prob}(A_n) + \sum_n n^2 \mathbf{Prob}(A_n) \right].$$

括号中两个和中的第一个可以重写为对所有 n 和所有 k 的 $\mathbf{Prob}(T^k(A_n))$ 的和. 因此, 由于这些集合 $T^k(A_n)$ 构成了 Ω 的划分, 这个和简单地等于 1. (这个观察本质上就是卡茨定理的证明.) 第二个和可以如下重写, 利用 $A_n \subseteq A$ 的性质, $\mathbf{Prob}(A_n) = \mathbf{Prob}(A) \mathbf{Prob}(A_n|A)$,

$$\mathbf{Prob}(A) \sum_n n^2 \mathbf{Prob}(A_n|A) = \mathbf{Prob}(A) \mathbf{E}(\rho^2|A).$$

此外, 由于所有随机变量都满足 $\mathbf{Var}(f) = \mathbf{E}(f^2) - \mathbf{E}(f)^2$, 我们可以用方差来重写这个式子

$$\mathbf{Prob}(A) [\mathbf{Var}(\rho|A) + \mathbf{E}(\rho|A)^2].$$

记住 $\mathbf{E}(\rho|A) = 1/\mathbf{Prob}(A)$ 根据卡茨定理, 并将我们计算的结果代入 $\mathbf{E}(\rho)$ 的公式中, 我们立即得到该命题. $\square$

2.3. Useful formulas

We collect here some formulas for use in the calculations in later sections. First, there is the well-known formula for the sum of a geometric series:

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1-x} \quad \text{for } |x| < 1.$$

Differentiating term by term (which is correct in the interior of the interval of convergence of any power series) and omitting the vanishing $i = 0$ term from the result, we get

$$\sum_{i=1}^{\infty} i x^{i-1} = \frac{1}{(1-x)^2} \quad \text{for } |x| < 1.$$

Multiplying this equation by x and then differentiating again, we get

$$\sum_{i=1}^{\infty} i^2 x^{i-1} = \frac{1+x}{(1-x)^3} \quad \text{for } |x| < 1.$$

We also need a special case of the familiar formula for the sum of an arithmetic progression

$$\sum_{m=1}^h m = \frac{h(h+1)}{2}.$$

A similar formula for adding values of a quadratic polynomial will be useful in the form

$$\sum_{m=2}^h m(m-1) = \frac{h^3 - h}{3}.$$

Although this formula, once it is proposed, is easily proved by induction, it can also be seen directly, in the equivalent form

$$\sum_{m=2}^h \binom{m}{2} = \binom{h+1}{3}.$$

The right side here counts the 3-element subsets of $\{0, 1, \dots, h\}$. The term with index m on the left side counts those 3-element subsets whose last element is m , since such a subset is determined by its other elements, a 2-element subset of $\{0, 1, \dots, m-1\}$. (The same proof gives the corresponding result for binomial coefficient sums with 2 and 3 changed to any k and $k+1$.) Note that the lower limit $m=2$ in the sums here can be changed to 1 or to 0, since the corresponding terms $m(m-1)$ vanish.

We shall also have use for estimates of sums of powers in the form

$$\sum_{k=0}^{m-1} \frac{1}{m} \left(\frac{k}{m}\right)^r.$$

This is a lower Riemann sum approximating the integral

$$\int_0^1 x^r dx = \frac{1}{r+1}.$$

An upper Riemann sum approximating the same integral is obtained by letting k range from 1 to m rather than from 0 to $m-1$. That amounts to adding $1/m$ to the sum, so we have

$$\frac{1}{r+1} - \frac{1}{m} < \sum_{k=0}^{m-1} \frac{1}{m} \left(\frac{k}{m}\right)^r < \frac{1}{r+1}.$$

So for large m we have

$$\sum_{k=0}^{m-1} \frac{1}{m} \left(\frac{k}{m}\right)^r \approx \frac{1}{r+1}.$$

Although these approximations suffice for our needs, we note that there is an exact expression in terms of the Bernoulli numbers B_k :

2.3. 有用公式

我们收集了一些公式，供后面计算使用。首先，有一个关于几何级数和的著名公式：

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1-x} \quad \text{for } |x| < 1.$$

逐项微分（这在任何幂级数的收敛区间内部都是正确的）并省略结果中消失的 $i = 0$ 项，我们得到

$$\sum_{i=1}^{\infty} i x^{i-1} = \frac{1}{(1-x)^2} \quad \text{for } |x| < 1.$$

将此方程乘以 x 然后再次微分，我们得到

$$\sum_{i=1}^{\infty} i^2 x^{i-1} = \frac{1+x}{(1-x)^3} \quad \text{for } |x| < 1.$$

我们还需要等差数列求和的熟悉公式的特殊情况

$$\sum_{m=1}^h m = \frac{h(h+1)}{2}.$$

一个用于求二次多项式值的类似公式将以以下形式有用

$$\sum_{m=2}^h m(m-1) = \frac{h^3 - h}{3}.$$

尽管这个公式一旦被提出，可以通过归纳法轻易证明，但它也可以直接以等价形式看出

$$\sum_{m=2}^h \binom{m}{2} = \binom{h+1}{3}.$$

这里的右侧计算了 $\{0, 1, \dots, h\}$ 的 3 元素子集。左侧的 m 索引项计算了那些最后一个元素是 m 的 3 元素子集，因为这样的子集由其他元素决定，即 $\{0, 1, \dots, m-1\}$ 的一个 2 元素子集。（同样的证明给出了对应的结果，将 2 和 3 更改为任何 k 和 $k+1$ 的二项式系数和。）注意，这里求和的下限 $m=2$ 可以更改为 1 或 0，因为相应的项 $m(m-1)$ 会消失。

我们还将用到形式为幂和的估计

$$\sum_{k=0}^{m-1} \frac{1}{m} \left(\frac{k}{m}\right)^r.$$

这是一个逼近积分的下黎曼和

$$\int_0^1 x^r dx = \frac{1}{r+1}.$$

通过让 k 从 1 到 m 范围而不是从 0 到 $m-1$ ，可以得到逼近同一积分的上黎曼和。这相当于将 $1/m$ 加到和中，所以我们有

$$\frac{1}{r+1} - \frac{1}{m} < \sum_{k=0}^{m-1} \frac{1}{m} \left(\frac{k}{m}\right)^r < \frac{1}{r+1}.$$

所以对于大的 m ，我们有

$$\sum_{k=0}^{m-1} \frac{1}{m} \left(\frac{k}{m}\right)^r \approx \frac{1}{r+1}.$$

尽管这些近似值能满足我们的需求，但请注意，存在一个以伯努利数 B_k 表示的精确表达式：

$$\begin{aligned}\sum_{k=0}^{m-1} \frac{1}{m} \binom{k}{m}^r &= \frac{1}{m^{r+1}} \frac{1}{r+1} \sum_{k=0}^r \binom{r+1}{k} B_k m^{r+1-k} \\ &= \frac{1}{r+1} - \frac{1}{2m} + \frac{r}{12m^2} + \cdots.\end{aligned}$$

The first term here is the integral approximation obtained above, and the second term says that the integral is approximately halfway between the upper and lower Riemann sums.

2.4. Greedy increasing sequences

Consider a finite file F or, more generally, a function F from any interval of integers $I = [a, b]$ to PFE. Recall that we identified the set PFE of potential file entries with a set of integers $\{0, 1, \dots, |\text{PFE}| - 1\}$. So it makes sense to talk about increasing (or decreasing) subsequences of F . It will be convenient here to discuss subsequences, not in terms of the values of F that constitute them, but in terms of the positions where those values occur.

For a fixed F , we define the *left-to-right greedy increasing sequence*, abbreviated $\rightarrow$ greedy sequence, in the interval $I = [a, b]$ as follows. Its first element g_0 is the first element a of I . Thereafter, g_{k+1} is defined as the smallest $n \in I$ such that $n > g_k$ and $F(n) > F(g_k)$. That is, we build an increasing sequence of elements of I such that the corresponding sequence of F -values is also increasing, and we do so greedily, always putting into our sequence the first available number. The sequence ends at g_k when there is no n satisfying the requirements for g_{k+1} . Notice that then $F(g_k)$ is the largest value that F attains on I .

There is an analogous definition of the *right-to-left greedy sequence* or $\leftarrow$ greedy sequence, which starts with the rightmost point b of I and thereafter takes g_{k+1} to be the rightmost point n to the left of g_k with $F(n) > F(g_n)$. Notice that the terms in this sequence are chosen in decreasing order, but their F -values are in increasing order, so that the corresponding restriction of the original sequence is decreasing. Because of this somewhat confusing situation, we do not use the words “increasing” or “decreasing” in connection with $\leftarrow$ greedy sequences.²

When we simply say “greedy sequence” without further modifiers, we mean the $\rightarrow$ greedy increasing sequence.

The elements g_k of the greedy increasing sequence admit a simple alternative characterization that does not involve recursion on k .

Proposition 12. *The $\rightarrow$ greedy increasing sequence in I consists of those $n \in I$ such that $F(n) > F(m)$ for all $m \in I$ such that $m < n$.*

In other words, they are the places where, as we read the sequence F from left to right, we see a new maximum value. We shall refer to such places as *left-to-right maxima* or $\rightarrow$ maxima of F . Analogously, of course, the $\leftarrow$ greedy sequence consists of the $\leftarrow$ maxima, those $n \in I$ such that $F(n) > F(m)$ for all $m \in I$ such that $m > n$.

Proof of Proposition 12. We first prove, by induction on k , that each g_k satisfies the condition in the lemma. For g_0 , this is vacuously true, since there is no smaller $m \in I$. As for g_{k+1} , we have $F(g_{k+1}) > F(g_k)$ by definition and $F(g_k) > F(m)$ for all $m < g_k$ in I by induction hypothesis, so it remains only to consider $m \in (g_k, g_{k+1})$. But the greediness in the definition of g_{k+1} implies that all such m have $F(m) \leq F(g_k) < F(g_{k+1})$.

Conversely, suppose $n \in I$ satisfies the condition in the proposition, and let k be the largest index for which $g_k \leq n$. (This exists because g_0 , being the first element of I , is $\leq n$.) If we had the strict inequality $g_k < n$, then, because g_{k+1} is chosen greedily and because $F(n) > F(g_k)$ by the assumption about n , we would have $g_{k+1} \leq n$, contrary to our choice of k . This contradiction shows that we must have $g_k = n$, and so the lemma is proved. $\square$

Corollary 13. *If $n \in I$ is not a member of the $\rightarrow$ greedy sequence, then there is some $m < n$ such that $F(m) \geq F(n)$ and m is a member of the $\rightarrow$ greedy sequence.*

Proof. This was, in effect, proved in the second half of the proof of Proposition 12, but it can also be obtained by applying the proposition itself, as follows. If n is not in the greedy subsequence, then there is, by the proposition, some $m < n$ with $F(m) \geq F(n)$. The smallest such m is, by the proposition again, in the greedy sequence. $\square$

We turn next to some probabilistic information about the $\rightarrow$ greedy sequence for a random file $F: [a, b] \rightarrow \text{PFE}$. We shall use this information in a context where $|\text{PFE}| \gg b - a$ and therefore it is very unlikely that two positions in $[a, b]$ will have the same F -value. We take advantage of this circumstance by doing our calculations under the assumption that F is one-to-one, i.e., that there are no “ties” between F -values. Formally, this means that we work not with the usual probability

² There are a half dozen more variants of greediness: We could take, in either the left-to-right or the right-to-left versions, successive points with smaller rather than larger F -values. And in all these situations, we could use weak rather than strict inequalities of F -values. We shall get by with the $\rightarrow$ greedy and $\leftarrow$ greedy sequences as defined above and avoid needing any of these other variants.

$$\begin{aligned}\sum_{k=0}^{m-1} \frac{1}{m} \binom{k}{m}^r &= \frac{1}{m^{r+1}} \frac{1}{r+1} \sum_{k=0}^r \binom{r+1}{k} B_k m^{r+1-k} \\ &= \frac{1}{r+1} - \frac{1}{2m} + \frac{r}{12m^2} + \cdots.\end{aligned}$$

其中第一项是上面得到的积分近似值，第二项则表明该积分值大约位于上黎曼和与下黎曼和的中点之间。

2.4. 贪婪递增序列

考虑一个有限文件 F 或更一般地，一个从任意整数区间 $I = [a, b]$ 到 PFE 的函数 F 。回想我们用一组整数 $\{0, 1, \dots, |\text{PFE}| - 1\}$ 识别了潜在文件条目集合 PFE。因此，谈论 F 的递增（或递减）子序列是有意义的。在这里，讨论子序列时，用这些值出现的位置来描述，而不是用构成它们的 F 值来描述，将更方便。

对于固定的 F ，我们在区间 $I = [a, b]$ 中定义从左到右的贪婪递增序列，缩写为 $\rightarrow$ greedy sequence，如下所示。它的第一个元素 g_0 是区间 I 的第一个元素 a 。之后， g_{k+1} 定义为满足 $n > g_k$ 和 $F(n) > F(g_k)$ 的最小 $n \in I$ 。也就是说，我们构建一个 I 的元素递增序列，使得对应的 F 值序列也是递增的，并且我们这样做是贪婪的，总是将第一个可用的数字放入我们的序列中。当没有满足 g_{k+1} 要求的 n 时，序列在 g_k 结束。请注意，在这种情况下， $F(g_k)$ 是 F 在 I 上达到的最大值。

存在一个与从右到左贪婪序列类似或 $\leftarrow$ 贪婪序列的等价定义，该序列以区间 I 中最右侧的点 b 开始，此后将 g_{k+1} 定义为 g_k 左侧最右侧的点 n ，并满足 $F(n) > F(g_n)$ 。请注意，该序列中的项按递减顺序选择，但它们的 F -值按递增顺序排列，因此原始序列的相应限制是递减的。由于这种有些令人困惑的情况，我们不在与 $\leftarrow$ 贪婪序列相关的语境中使用“递增”或“递减”等词语。²

当我们简单地说“贪婪序列”而不附加其他修饰时，我们指的是 $\rightarrow$ 贪婪递增序列。

贪婪递增序列中的元素 g_k 具有一个不涉及对 k 递归的简单替代描述。

命题12. 区间 I 中的 $\rightarrow$ 贪婪递增序列由所有满足 $F(n) > F(m)$ 的 $n \in I$ 组成，其中对于所有满足 $m \in I$ 且 $m < n$ 的 m 。

换句话说，它们就是当我们从左到右读取序列 F 时，看到新最大值的位置。我们将这样的位置称为从左到右最大值或 F 的 $\rightarrow$ 最大值。当然，类似地， $\leftarrow$ 贪婪序列由 $\leftarrow$ 最大值组成，即那些满足 $\{n\} \in I$ 使得 $F(n) > F(m)$ 对于所有 $\in I$ 中的 $m > n$ 。

命题12的证明。我们首先通过归纳法证明，每个 g_k 都满足引理中的条件。对于 g_0 ，这是空话，因为没有更小的 $m \in I$ 。至于 g_{k+1} ，根据定义和归纳假设对于所有 $m < g_k$ 在 I 中，因此只需考虑 $m \in (g_k, g_{k+1})$ 。但 g_{k+1} 定义中的贪婪性意味着所有这样的 m 都有 $F(m) \leq F(g_k) < F(g_{k+1})$ 。

反过来，假设 $n \in I$ 满足命题中的条件，并且设 k 是满足 $g_k \leq n$ 的最大索引。（这存在，因为 g_0 是 I 中 $is \leq n$ 。）的第一个元素，如果我们有严格不等式 $g_k < n$ ，那么，因为 g_{k+1} 是贪婪选择的，并且因为 $F(n) > F(g_k)$ 根据关于 n 的假设，我们会得到 $g_{k+1} \leq n$ ，这与我们选择 k 的方式矛盾。这个矛盾表明我们必须有 $g_k = n$ ，因此引理得证。 $\square$

推论13. 如果 $n \in I$ 不是 $\rightarrow$ 贪婪序列的成员，那么存在某个 $m < n$ 使得 $F(m) \geq F(n)$ 并且 m 是 $\rightarrow$ 贪婪序列的成员。

证明。这实际上在命题12证明的后半部分已经证明，但也可以通过应用命题本身得到，如下。如果 n 不在贪婪子序列中，那么根据命题，存在某个 $m < n$ 使得 $F(m) \geq F(n)$ 。根据命题，这样的最小 m 在贪婪序列中。 $\square$

接下来，我们讨论一些关于随机文件 $\rightarrow$ 贪婪序列 $F: [a, b] \rightarrow \text{PFE}$ 的概率信息。我们将在一个 $|\text{PFE}| \gg b - a$ 的上下文中使用这些信息，因此 $[a, b]$ 中的两个位置具有相同的 F 值的可能性非常小。我们利用这一情况，在假设 F 是一一对应的条件下进行计算，即不存在 F 值之间的“平局”。形式上，这意味着我们不是在通常的概率空间 $\text{PFE} \rightarrow a, b$ $F: [a, b] \rightarrow \text{PFE}$ 上工作，而是在由一一对应函数组成的子空间上工作（并且将概率测度限制在此子空间中，并重新归一化使其总概率为 1）。

² 贪婪还有另外六种变体：我们可以在从左到右或从右到左的版本中，选择 F 值较小而不是较大的连续点。并且在所有这些情况下，我们可以使用 F 值的弱不等式而不是严格不等式。我们将仅凭上述定义的 $\rightarrow$ 贪婪和 $\leftarrow$ 贪婪序列就能应对，并避免需要这些其他变体。

space $\text{PFE}^{[a,b]}$ but with the subspace consisting of the one-to-one functions (and with the probability measure restricted to this subspace and renormalized to have total probability 1).

Proposition 14. For any $n \in [a, b]$, the probability that n is in the $\rightarrow$ greedy sequence is $1/(n - a + 1)$.

Proof. For any $n \in [a, b]$, the largest F -value at the $n - a + 1$ positions in $[a, n]$ has an equal chance of occurring at any of these positions. So the probability that the largest F -value on $[a, n]$ occurs at n is $1/(n - a + 1)$. By Proposition 12, this is also the probability that n is in the greedy sequence. $\square$

Corollary 15. The expected length of the greedy sequence in $[a, b]$ is

$$\sum_{n=a}^b \frac{1}{n - a + 1} = \sum_{m=1}^{b-a+1} \frac{1}{m} \approx \ln(b - a + 1) = \ln|[a, b]|.$$

Proof. The expected size of any set is the sum, over all potential elements n , of the probability that n is in the set. So the first sum in the corollary follows immediately from the proposition. The logarithm is a well-known asymptotic (for large $b - a$) approximation to these harmonic sums. $\square$

Remark 16. It is known that the expected length of the *longest* (in contrast to the greedy) increasing sequence is $O(\sqrt{|[a, b]|})$. So the greedy method of selecting an increasing subsequence usually falls far short of the maximum achievable length. In fact, a theorem of Erdős and Szekeres [10,1] asserts that a sequence of integers of length $pq + 1$, without repetitions, must have an increasing subsequence of length $p + 1$ or a decreasing subsequence of length $q + 1$. So a sequence of length l will have a monotone sequence of length at least $\sqrt{l}$. By symmetry, there will be an increasing subsequence of this length at least half the time, so the expectation of the maximum length of an increasing subsequence is at least $\sqrt{l}/2$.

Although the greedy increasing sequences do not usually come near the length given by the Erdős–Szekeres theorem, they do provide an elegant proof of that theorem, as follows. Given a sequence F of distinct integers, let G_1 be the greedy increasing subsequence. Delete G_1 from F , and let G_2 be the greedy increasing subsequence of what remains. Continue in the same manner, forming and removing greedy increasing subsequences $G_3, \dots$, until nothing remains. If one of these sequences G_i has at least $p + 1$ terms, then we have the desired conclusion, so assume that each G_i has at most p terms. So the first q of our greedy sequences have altogether at most pq elements, not enough to exhaust the given sequence of length $pq + 1$. Pick any position that is not in any of $G_1, \dots, G_q$. Because it was not in G_q , Corollary 13 provides an earlier position that has a larger F value and is in G_q . This, in turn, was not in G_{q-1} , so Corollary 13 provides an even earlier position with an even larger F value in G_{q-1} . Continuing in this way, we get a decreasing subsequence of length $q + 1$ in F .

We shall need one additional, perhaps surprising piece of information about the $\rightarrow$ maxima of a random file, namely that different positions behave independently.

Proposition 17. For each $n \in I$, let E_n be the event that n is a $\rightarrow$ maximum of a random file F . Then these events are probabilistically independent.

Proof. Let $n_1 < n_2 < \dots < n_k$ be elements of I . We must show that

$$\text{Prob}(\text{All } n_i \text{ are } \rightarrow \text{ maxima}) = \prod_{i=1}^k \text{Prob}(n_i \text{ is a } \rightarrow \text{ maximum}),$$

and we shall do this by induction on k , the cases $k = 0$ and $k = 1$ being trivial. Suppose, therefore, that the result holds for $k - 1$. Consider the conditional probability

$$\text{Prob}(n_1, \dots, n_{k-1} \text{ are } \rightarrow \text{ maxima} \mid n_k \text{ is a } \rightarrow \text{ maximum}).$$

The event that all of $n_1, \dots, n_{k-1}$ are $\rightarrow$ maxima depends only on the relative order of the values of F at points $n < n_k$. The conditioning event, that n_k is a $\rightarrow$ maximum, means that all those values $F(n)$ are smaller than $F(n_k)$, but it says nothing about the order of those $F(n)$'s relative to each other. Thus, the conditional probability equals the absolute probability that $n_1, \dots, n_{k-1}$ are $\rightarrow$ maxima, which is, by the induction hypothesis,

$$\prod_{i=1}^{k-1} \text{Prob}(n_i \text{ is a } \rightarrow \text{ maximum}).$$

Finally, the probability that all of $n_1, \dots, n_k$ are $\rightarrow$ maxima is obtained by multiplying this conditional probability by the probability that n_k is a $\rightarrow$ maximum, so we get the required equality. $\square$

空间 $\text{PFE}^{[a,b]}$ 但是在由一一对应函数组成的子空间上工作（并且将概率测度限制在此子空间中，并重新归一化使其总概率为 1）。

命题14. 对于任意 $n \in [a, b]$, n 在贪婪序列中的概率是 $1/(n - a + 1)$ 。

证明. 对于任意 $n \in [a, b]$, $n - a + 1$ 位置中 $[a, n]$ 的最大 F 值在所有这些位置出现的概率相等。因此，最大 F 值在 $[a, n]$ 上出现的概率是 $1/(n - a + 1)$ 。根据命题12，这也是 n 在贪婪序列中的概率。 $\square$

推论15. 在 $[a, b]$ 中，贪婪序列的期望长度是

$$\sum_{n=a}^b \frac{1}{n - a + 1} = \sum_{m=1}^{b-a+1} \frac{1}{m} \approx \ln(b - a + 1) = \ln|[a, b]|.$$

证明. 任何集合的期望大小是对于所有潜在元素 n , n 在集合中的概率的总和。因此，推论中的第一个和式立即可以从命题中得出。对数是这些调和和的渐近（对于大的 $b - a$ ）近似。 $\square$

注释16。已知最长（与贪婪）递增序列的期望长度是 $O(\sqrt{|[a, b]|})$ 。因此，选择递增子序列的贪婪方法通常远低于可达到的最大长度。事实上，Erdős和Szekeres [10,1] 的一个定理断言，长度为 $pq + 1$ 且无重复的整数序列必须有一个长度为 $p + 1$ 的递增子序列或一个长度为 $q + 1$ 的递减子序列。因此，长度为 l 的序列将有一个长度至少为 $\sqrt{l}$ 的单调序列。由于对称性，至少有一半的时间会出现这个长度的递增子序列，所以递增子序列的最大长度的期望至少是 $\sqrt{l}/2$ 。

尽管贪婪递增序列通常远未达到Erdős–Szekeres定理给出的长度，但它们确实提供了该定理的一个优雅证明，如下所示。给定一个由不同整数组成的序列 F ，设 G_1 是贪婪递增子序列。从 F 中删除 G_1 ，并设 G_2 是剩余部分的贪婪递增子序列。以相同的方式继续，形成并删除贪婪递增子序列 $G_3, \dots$ ，直到什么都没有剩下。如果其中一个序列 G_i 至少有 $p + 1$ 个项，那么我们就得到了期望的结论，因此假设每个 G_i 最多有 p 个项。因此，我们的前 q 个贪婪序列总共最多有 pq 个元素，不足以耗尽长度为 $pq + 1$ 的给定序列。选择一个不在任何 $G_1, \dots, G_q$ 中的位置。因为它不在 G_q 中，推论13提供了一个更早的位置，该位置具有更大的 F 值且在 G_q 中。反过来，它不在 G_{q-1} 中，所以推论13提供了一个更早的位置，该位置具有更大的 F 值在 G_{q-1} 中。以此类推，我们在 F 中得到了一个长度为 $q + 1$ 的递减子序列。

我们需要关于随机文件 $\rightarrow$ 最大值的另一条信息，或许令人惊讶，即不同位置独立地表现。

命题17. 对于每个 $n \in I$ ，令 E_n 是事件 n 是随机文件 F 的 $\rightarrow$ 最大值。那么这些事件在概率上是独立的。

证明. 令 $n_1 < n_2 < \dots < n_k$ 是 I 的元素。我们必须证明

$$\text{Prob}(\text{All } n_i \text{ are } \rightarrow \text{ maxima}) = \prod_{i=1}^k \text{Prob}(n_i \text{ is a } \rightarrow \text{ maximum}),$$

我们将通过归纳法对 k 进行证明，其中 $k = 0$ 和 $k = 1$ 的情况是平凡的。因此，假设结果对 $k - 1$ 成立。考虑条件概率

$$\text{Prob}(n_1, \dots, n_{k-1} \text{ are } \rightarrow \text{ maxima} \mid n_k \text{ is a } \rightarrow \text{ maximum}).$$

事件 $n_1, \dots, n_{k-1}$ 是 $\rightarrow$ 最大值仅取决于 F 在点 $n < n_k$ 处的值的相对顺序。条件事件，即 n_k 是 $\rightarrow$ 最大值，意味着所有这些值 $F(n)$ 都小于 $F(n_k)$ ，但它没有说明这些 $F(n)$'s 相对于彼此的顺序。因此，条件概率等于 $n_1, \dots, n_{k-1}$ 是 $\rightarrow$ 最大值的绝对概率，这是根据归纳假设

$$\prod_{i=1}^{k-1} \text{Prob}(n_i \text{ is a } \rightarrow \text{ maximum}).$$

最后，所有 $n_1, \dots, n_k$ 都是 $\rightarrow$ 极大值的概率，通过将此条件概率乘以 n_k 是一个 $\rightarrow$ 极大值的概率来获得，因此我们得到所需的等式。 $\square$

3. Chunking methods and slack

3.1. Chunking methods and locality

We begin our discussion of content-dependent chunking methods for remote differential compression by defining what we mean by a chunking method. Our definition is general enough to also cover content-independent methods, such as chopping a file into chunks of a fixed length, but we shall use it only in the content-dependent case.

Definition 18. A *chunking method* is an operation assigning to every file a set of locations in that file, called the *cutpoints* of the file. That is, for files in PFE^I where I is $[0, l - 1]$ or $\mathbb{N}$ or $\mathbb{Z}$, the cutpoints form a subset of I . The *chunks* of a file F are the segments beginning at one cutpoint and ending immediately before the next cutpoint, as well as the segment from the beginning of the file (when there is a beginning, i.e., when $I \neq \mathbb{Z}$) up to and not including the first cutpoint and the segment from the last cutpoint (if there is one) to the end of the file. In the degenerate case where the set of cutpoints is empty, the whole file counts as a single chunk.

We have adopted here the arbitrary convention that a cutpoint belongs to the chunk on its right rather than the one on its left. So the actual cutting occurs just to the left of the cutpoints.

When the files are infinite ($I = \mathbb{N}$ or $\mathbb{Z}$), infinite chunks can occur. In all the chunking methods that we consider, however, the probability of such an occurrence is zero. That is, almost all files will be chopped into finite chunks. We shall often ignore the measure-zero set of exceptions. Indeed, we have already ignored it in our rough description of protocols, where the first two steps involve applying a hash function to each chunk.

Definition 19. Let h be a non-negative integer, and let $F \in \text{PFE}^I$ be a file. A position $i \in I$ is *h-internal* to the file F if the interval $[i - h, i + h]$ is included in I . In this case, the restriction of F to this interval, consisting of the $2h + 1$ entries $F(i - h), \dots, F(i + h)$ of F , is called the *h-neighborhood* of i in F . We may omit the prefix h when it is clear from the context.

One of the advantages of dealing with doubly infinite files is that all positions are internal. In a singly infinite file, all except the first h positions are h -internal; in a finite file, the exceptions are the first and last h positions.

We shall need to compare vicinities at different positions, and for this purpose it is useful to have a brief expression for “being the same except that the positions have been shifted.”

Definition 20. Consider two finite sequences of potential file entries, of the same length, but indexed by (possibly) different segments of $\mathbb{Z}$, say $\sigma \in \text{PFE}^{[a, a+l]}$ and $\tau \in \text{PFE}^{[b, b+l]}$. We say that σ and τ *agree* if they differ only by shifting the indices from a to b , i.e., if $\sigma(a + i) = \tau(b + i)$ for $i = 0, \dots, l$.

Definition 21. A chunking method is *local* if there exist a non-negative integer h and a non-empty set C of sequences of length $2h + 1$, $C \subseteq \text{PFE}^{[-h, h]}$ such that the cutpoints of any file F are exactly those h -internal positions in F whose h -neighborhood agrees with some $\sigma \in C$. We call h the *horizon* of the method and C its *criterion* (for cutpoints).

The requirement that C be non-empty avoids trivialities; if it were violated, no file would have cutpoints. With this requirement, not only do some files have cutpoints, but almost all infinite files in $\text{PFE}^{\mathbb{N}}$ have infinitely many cutpoints and almost all doubly infinite files in $\text{PFE}^{\mathbb{Z}}$ have infinitely many positive and infinitely many negative cutpoints. Thus, almost all files are chopped into finite chunks.

Definition 22. A chunking method is *shift-invariant* if, whenever i is a cutpoint of a doubly infinite file F and s is an arbitrary integer, then $i - s$ is a cutpoint of $\text{BS}^s(F)$.

It follows immediately from the definition of locality that any local chunking method is shift-invariant, simply because the vicinity of $i - s$ in $\text{BS}^s(F)$ agrees with that of i in F .

3.2. Length of chunks

Fix a shift-invariant chunking method for doubly infinite files. Because of shift-invariance, each position $i \in \mathbb{Z}$ has the same probability p of being a cutpoint. We call this p the *cutpoint probability* of the method. In the case of local chunking methods, we have, with notation as in the definition of locality,

$$p = \frac{|C|}{|\text{PFE}|^{2h+1}}.$$

In the case of singly infinite files, locality requires all cutpoints to be h -internal, i.e., to be positions $\geq h$. All these positions have the same probability of being cutpoints, and that probability is given by the same formula as for the doubly infinite

3. 分块方法与松弛

3.1. 分块方法与局部性

我们通过定义什么是分块方法来开始对远程差分压缩的内容相关分块方法的讨论。我们的定义足够通用，也可以涵盖内容无关方法，例如将文件分割成固定长度的块，但我们仅将在内容相关的情况下使用它。

定义 18. 分块方法是一种操作，它为每个文件分配该文件中的一组位置，称为文件的切点。也就是说，对于 PFE^I 中 I 是 $[0, l - 1]$ 或 $\mathbb{N}$ 或 $\mathbb{Z}$ 的文件，切点形成 I 的一个子集。文件 F 的块是从一个切点开始到下一个切点立即结束的段，以及从文件开头（当存在开头时，即当 $I \neq \mathbb{Z}$ ）到不包括第一个切点的段，以及从最后一个切点（如果存在）到文件末尾的段。在切点集为空的退化情况下，整个文件计为一个块。

我们在这里采用了任意约定，即切割点属于其右侧的块，而不是左侧的块。因此，实际切割发生在切割点左侧。

当文件是无限时 ($I = \mathbb{N}$ 或 $\mathbb{Z}$)，可能会出现无限块。然而，在我们考虑的所有分块方法中，这种出现的概率为零。也就是说，几乎所有文件都会被切割成有限块。我们将经常忽略测度为零的例外情况。事实上，在我们的协议粗略描述中已经忽略了这一点，其中前两个步骤涉及对每个块应用哈希函数。

定义 19. 设 h 为一个非负整数，且 $F \in \text{PFE}^I$ 为一个文件。如果区间 $[i - h, i + h]$ 包含在 I 中，则位置 $i \in I$ 称为文件 F 中的 h -内部位置。在这种情况下，将 F 限制在这个区间内，即由 F 的 $2h + 1$ 个条目 $F(i - h), \dots, F(i + h)$ ，组成的限制，称为 i 在 F 中的 h -邻近。当上下文清晰时，我们可以省略前缀 h 。

处理双无限文件的一个优势在于所有位置都是内部位置。在单无限文件中，除了前 h 个位置外，所有位置都是 h -内部位置；在有限文件中，例外情况是前 h 个位置和最后 h 个位置。

我们将需要比较不同位置处的邻近区域，为此目的，有一个关于“除了位置被平移外相同”的简短表达方式会很有用。

定义 20. 考虑两个长度相同但索引由 $\mathbb{Z}$ 的不同片段（可能）索引的潜在文件条目有限序列，比如 $\sigma \in \text{PFE}^{[a, a+l]}$ 和 $\tau \in \text{PFE}^{[b, b+l]}$ 。如果 σ 和 τ 只通过将索引从 a 平移到 b 而不同，即如果对于 $i = 0, \dots, l$ 有 $\sigma(a + i) = \tau(b + i)$ ，则称 σ 和 τ 一致。

定义 21. 如果存在一个非负整数 h 和一个非空长度为 $2h$ 的序列集合 C ，其中 $C \subseteq \text{PFE}^{[-h, h]}$ ，使得任何文件 F 的切点正好是 F 中所有满足其 h -邻近与 C 中某个序列一致的 h -内部位置，则称分块方法为局部。我们将 h 称为该方法的地平线，将 C 称为其切点标准。

要求 C 为非空可避免平凡情况；如果违反该要求，则没有任何文件具有切割点。在此要求下，不仅某些文件具有切割点，而且 $\text{PFE}^{\mathbb{N}}$ 中几乎所有无限文件都有无限多个切割点， $\text{PFE}^{\mathbb{Z}}$ 中几乎所有双无限文件都有无限多个正切割点和无限多个负切割点。因此，几乎所有文件都被切分成有限块。

定义 22. 若对于双无限文件 F 的任意切割点 i 和任意整数 s ， $i - s$ 都是 $\text{BS}^s(F)$ 的切割点，则称分块方法为平移不变。

根据局部性的定义，任何局部分块方法都是平移不变的，因为 $\text{BS}^s(F)$ 中 $i - s$ 的邻域与 F 中 i 的邻域相同。

3.2. 块的长度

固定一个用于双无限文件的分块方法。由于平移不变性，每个位置 $i \in \mathbb{Z}$ 成为切割点的概率相同，记为 p 。我们将此 p 称为该方法的切割点概率。对于局部分块方法，我们有，如局部性定义中的符号所示，

$$p = \frac{|C|}{|\text{PFE}|^{2h+1}}.$$

对于单无限文件，局部性要求所有切点必须是 h -内部位置，即必须是位置 $\geq h$ 。所有这些位置成为切点的概率相同，该概率由与双无限情况相同的公式给出

case. Similarly, for finite files, the same formula gives the probability that any particular h -internal position will be a cutpoint.

Definition 23. For a file F with a cutpoint at 0, we define the *chunk length* $L(F)$ to be the first positive cutpoint.

Thus, the chunk length of F is the length of the chunk whose first element is 0. We shall comment later, in Remark 28, on why we consider only files with a cutpoint at 0, rather than extending the definition to arbitrary files.

It is possible for a file to have a cutpoint at 0 but no cutpoints farther to the right, so that the chunk length is not defined. But, with all local chunking methods and indeed with all shift-invariant methods that have non-zero cutpoint probability, the Poincaré recurrence theorem ensures that the files with no chunk length form a set of measure zero, so they will not affect any of the considerations below.

For finite files, on the other hand, there is a non-zero probability that the chunk length doesn't exist. But this probability approaches zero exponentially fast as the length of the file increases. So for very long files, there is only a negligible danger that the chunk length is undefined. It would also be reasonable to modify the definition of chunk length, to cover the case where a finite file $F \in \text{PFE}^{[0,l-1]}$ has no positive cutpoint, by letting the chunk length in this case be the whole length l of the file.

Notation 24. Given a shift-invariant chunking method, we write p for the cutpoint probability. If a file F with a cutpoint at 0 is also given, then we write $L(F)$ for its chunk length. Thus, L is a random variable (defined almost everywhere) on the subspace Cut0 of $\text{PFE}^{\mathbb{Z}}$ consisting of files with a cutpoint at 0.

Because the random variable L is defined only on the subspace Cut0 , any statement about its statistical properties must be interpreted as conditional on the event Cut0 . Nevertheless, to reduce the chance of confusion, we shall often (as in the next proposition) indicate the conditioning explicitly.

Proposition 25. For doubly infinite files, the expectation of the chunk length for any shift-invariant chunking method is the reciprocal of the cutpoint probability, $\mathbf{E}(L|\text{Cut0}) = 1/p$.

Proof. Notice that the first positive cutpoint of F is, by shift-invariance, the smallest positive number k such that 0 is a cutpoint of the shifted file $\text{BS}^k(F)$, where BS denotes the leftward Bernoulli shift as above. Thus the expectation of the chunk length is the conditional expectation, conditioned on $F \in \text{Cut0}$, of the smallest positive k with $\text{BS}^k(F) \in \text{Cut0}$. This is precisely the situation covered by Kac's theorem, Proposition 10. According to that theorem, the expectation under consideration is the reciprocal of the probability of Cut0 , so it is $1/p$. $\square$

Can one associate chunk lengths to (almost) all $F \in \text{PFE}^{\mathbb{Z}}$, rather than only to those with a cutpoint at 0? The answer is yes, to some extent, but the right approach is more complicated than one might guess. It involves averaging over all the chunks within the file, as follows.

Definition 26. The *average chunk length* of a file $F \in \text{PFE}^{\mathbb{Z}}$ is the limit

$$\lim_{N \rightarrow \infty} \frac{2N + 1}{\text{Number of cutpoints in } [-N, N]}.$$

This limit and thus the average chunk length may be undefined for some files F , but these form a set of measure zero. In fact, we have the following consequence of Birkhoff's ergodic theorem.

Proposition 27. For almost all files in $\text{PFE}^{\mathbb{Z}}$, the average chunk length equals $1/p$.

Proof. Before starting the main part of the proof, we note that the definition of average chunk length would be unaffected if we replaced $2N + 1$ by $2N$ in the numerator (without changing the denominator), because $\frac{2N}{2N+1} \rightarrow 1$ as $N \rightarrow \infty$. We refer to such a change, also in other similar situations, as a “trivial modification” of the fraction.

Applying Birkhoff's result, Proposition 9, to the space $\text{PFE}^{\mathbb{Z}}$, the leftward Bernoulli shift, and the event Cut0 , and recalling that the cutpoints of a file F are exactly those k for which $\text{BS}^k(F) \in \text{Cut0}$, we find that

$$\lim_{N \rightarrow \infty} \frac{\text{Number of cutpoints in } [0, N]}{N} = p,$$

where we have made a trivial modification to get N rather than $N + 1$ in the denominator. Symmetrically, using BS^{-1} and shifting the file by one unit (and not needing a trivial modification this time), we get

$$\lim_{N \rightarrow \infty} \frac{\text{Number of cutpoints in } [-N, 1]}{N} = p.$$

给出。类似地, 对于有限文件, 相同的公式给出了任何特定h-内部位置成为切点的概率。

定义 23. 对于在 0 处有切割点的文件 F, 我们定义块长度 L(F) 为第一个正切割点。

因此, F 的块长度是第一个元素为 0 的块的长度。我们稍后将在注释 28 中说明, 为什么我们只考虑在 0 处有切割点的文件, 而不是将定义扩展到任意文件。

一个文件可能在 0 处有切割点但没有更右边的切割点, 因此块长度未定义。但是, 对于所有局部分块方法以及所有具有非零切割点概率的平移不变方法, 庞加莱回归定理确保没有块长度的文件形成一个测度为零的集合, 因此它们不会影响以下任何考虑。

另一方面, 对于有限文件, 块长度不存在有一个非零概率。但是, 随着文件长度的增加, 这个概率呈指数级快速趋近于零。因此, 对于非常长的文件, 块长度未定义的风险可以忽略不计。此外, 也可以修改块长度的定义, 以涵盖有限文件 $F \in \text{PFE}^{[0,l-1]}$ 没有正切割点的情况, 在这种情况下让块长度为文件的总长度 l。

符号 24. 给定一个平移不变块划分方法, 我们用 p 表示断点概率。如果给定一个在 0 处有断点的文件 F, 那么我们用 L(F) 表示其块长度。因此, L 是一个在 $\text{PFE}^{\mathbb{Z}}$ 的子空间 Cut0 上 (几乎处处定义的) 随机变量, 该子空间由在 0 处有断点的文件组成。

因为随机变量 L 仅定义在子空间 Cut0 上, 关于其统计特性的任何陈述都必须解释为在事件 Cut0 的条件期望下。尽管如此, 为了减少混淆的可能性, 我们将经常 (如在下一个命题中) 明确指出条件。

命题 25. 对于双无限文件, 任何平移不变块划分方法的块长度期望是断点概率的倒数, $\mathbf{E}(L|\text{Cut0}) = 1/p$ 。

证明。注意到 F 的第一个正断点, 根据移位不变性, 是满足 0 是左移 Bernoulli 变换 $\text{BS}^k(F)$ (其中 BS 如上所述表示左移 Bernoulli 变换) 的断点的最小正数 k。因此块长度的期望是条件期望, 条件是 $F \in \text{Cut0}$, 关于具有 $\text{BS}^k(F) \in \text{Cut0}$ 的最小正 k。这恰好是卡茨定理, 命题 10 所涵盖的情况。根据该定理, 所考虑的期望是 Cut0 概率的倒数, 因此是 $1/p$ 。 $\square$

能否将块长度与 (几乎所有) $F \in \text{PFE}^{\mathbb{Z}}$ 相关联, 而不仅仅是与在 0 处有切割点的那些相关联? 答案是肯定的, 在某种程度上, 但正确的方法比人们想象的要复杂得多。它涉及对所有文件内的块进行平均, 如下所示。

定义 26. 文件 $F \in \text{PFE}^{\mathbb{Z}}$ 的平均块长度是其极限。

$$\lim_{N \rightarrow \infty} \frac{2N + 1}{\text{Number of cutpoints in } [-N, N]}.$$

这个极限以及因此的平均块长度对于某些文件 F 可能未定义, 但这些形成一个测度为零的集合。事实上, 我们有 Birkhoff 的遍历定理的如下推论。

命题 27. 对于 $\text{PFE}^{\mathbb{Z}}$ 中的几乎所有文件, 平均块长度等于 $1/p$ 。

证明。在开始证明的主要部分之前, 我们注意到, 如果我们用 $2N$ 替换分子中的 $2N + 1$ (而不改变分母), 平均块长度的定义将不受影响, 因为 $\frac{2N}{2N+1} \rightarrow 1$ 作为 $N \rightarrow \infty$ 。我们将这种改变, 以及其他类似情况中的类似改变, 称为分数的 “平凡修改”。

将 Birkhoff 的结果、命题 9 应用于空间 $\text{PFE}^{\mathbb{Z}}$ 、左移 Bernoulli 变换以及事件 Cut0 , 并回忆文件 F 的切点正是那些满足 $\{k\}$ 的 k, 我们发现

$$\lim_{N \rightarrow \infty} \frac{\text{Number of cutpoints in } [0, N]}{N} = p,$$

我们在分母中得到了 N 而不是 $N + 1$, 这是通过一个微小的修改实现的。对称地, 使用 BS^{-1} 并将文件移动一个单位 (这次不需要微小的修改), 我们得到

$$\lim_{N \rightarrow \infty} \frac{\text{Number of cutpoints in } [-N, 1]}{N} = p.$$

Average these two equations to get

$$\lim_{N \rightarrow \infty} \frac{\text{Number of cutpoints in } [-N, N]}{2N} = p.$$

Finally, take reciprocals of both sides and make a trivial modification to get the desired result. $\square$

Thus, Proposition 25 would remain true if we replaced chunk length by average chunk length and took the expectation over all of $\text{PFE}^{\mathbb{Z}}$ rather than over Cut0. Notice, however, that average chunk length cannot replace chunk length in other contexts. For example, in a non-trivial chunking method, the chunk length will have non-zero variance, essentially because not all chunks have the same length. But the average chunk length is, according to Proposition 27, constant almost everywhere, so its variance is 0.

Remark 28. It is tempting to associate, to (almost) every file $F \in \text{PFE}^{\mathbb{Z}}$, the length of a particular, chosen chunk to serve as the chunk length of F . Such a definition would avoid both the restriction to Cut0 in our definition of chunk length and the limiting process in our definition of average chunk length. In fact, an earlier draft of this paper defined the chunk length of F to be the distance between the first two non-negative cutpoints. Unfortunately, the analog of Proposition 25, averaging this chunk length over all files, is in general false, even for local chunking methods. Here is a simple counterexample. Suppose $\text{PFE} = \{0, 1\}$, and let the chunking method put cutpoints at both of the 1's wherever the pattern 011 occurs in a file. (Formally, this is a local chunking method with horizon $h = 2$ and criterion C consisting of the 8 sequences $*011*$ and $011**$, where the stars represent either 0 or 1 independently.) The cutpoint probability is $1/4$, and the expectation of the chunk length, as we have defined it, is 4. But the older definition, using the first two non-negative cutpoints, results in the expectation of the chunk length being only $7/4$.

One can see, intuitively, what goes wrong in this example. The chunking method guarantees that cutpoints occur in adjacent pairs. So half the chunks have length 1 (extending from the first to the second 1 in a 011 pattern) while the other half are longer (of length at least 2 and on average 7). Position 0 is considerably more likely to lie in one of the long chunks, so the next chunk, the one between the first two non-negative cutpoints, is more likely to be a short one, of length 1. And of course this drags down the expectation of this version of chunk length.

Another approach to assigning a chunk length to (almost) every file is to take the length of the chunk that contains a specific position, say 0. This also fails to work properly, for similar reasons. If there are chunks of different lengths (as there will be under non-trivial chunking methods) then 0 is more likely to lie in one of the longer chunks. A specific counterexample is even easier to produce than for the “first two non-negative cutpoints” version. Let $\text{PFE} = \{0, 1\}$ again and let the cutpoints be the positions where the file entry is 1. (This is a special case of the pure point filter method discussed in more detail below.) Then the cutpoint probability is $1/2$ and the average chunk length is 2. But the expectation of the length of the chunk containing 0 is 3.

Typically a chunking method has parameters which can be manipulated so that the expected chunk length $\mathbf{E}(L)$ is as desired. But the chunks in a particular file may be shorter or longer than this average. As indicated earlier, both too short and too long chunks are undesirable. On the one hand, the overhead of assembling and communicating the checksum of too short a chunk may outweigh the cost of sending the chunk itself. On the other hand, excessively long chunks are unlikely to match between the sender's and recipient's files. Therefore, one would prefer a chunking method with lower deviation from the average chunk length.

In a sense, too long chunks create a smaller problem than too short chunks. If necessary, a too long chunk can be subdivided by adding additional “artificial” cutpoints; all “indigenous” cutpoints remain intact. For example, if the desired chunk length is L , then a chunk of size $cL + d$ may be subdivided into $c - 1$ chunks of size L , and one chunk of size $L + d$. In this way, a sender never transmits a chunk of size larger than $2L$. Of course, this method can run into the same problem that motivated content-dependent chunking in the first place: Since the artificial cutpoints are at fixed positions, inserting a single character into a file, near the beginning of a long chunk, may disrupt agreement between the artificial subdivisions. There are more intrinsic ways of subdividing long chunks. For example, in the case of local-maximum chunking method, discussed in Section 6, a long chunk can be subdivided by means of local maxima with smaller horizon.

On the other hand, imposing a minimal length may require removing indigenous cutpoints. The following example is admittedly extreme and improbable but it gives a good idea of a discoordination that may result from removing indigenous cutpoints.

Example 29. The indigenous cutpoints partition the recipient's file F_1 into $2n$ distinct chunks $C_1, \dots, C_{2n}$ that are too small. By removing half of the cutpoints, we have n bigger chunks $C_1C_2, \dots, C_{2n-1}C_{2n}$. The sender's file F_2 was obtained from F_1 by moving C_1 to the end. The indigenous cutpoints partition F_2 into $2n$ chunks $C_2, \dots, C_{2n}, C_1$. By removing half of the cutpoints, we get bigger chunks $C_2C_3, \dots, C_{2n}C_1$. But none of these bigger chunks occurs in F_1 .

Lemma 30. *No local chunking method can have an absolute guarantee that the chunks are not too short and not too long.*

对这两个方程取平均值，得到

$$\lim_{N \rightarrow \infty} \frac{\text{Number of cutpoints in } [-N, N]}{2N} = p.$$

最后，对等式两边取倒数并对结果进行平凡修改，即可得到所需结果。 $\square$

因此，如果我们将块长度替换为平均块长度，并对所有 $\text{PFE}^{\mathbb{Z}}$ 取期望而不是对 Cut0 取期望，命题 25 仍然成立。然而，需要注意的是，平均块长度在其他上下文中不能替代块长度。例如，在非平凡的块分方法中，块长度将有非零方差，本质上是因为并非所有块的长度都相同。但根据命题 27，平均块长度几乎处处是常数，因此其方差为 0。

注释 28。人们很自然地倾向于将（几乎）每个文件 $F \in \text{PFE}^{\mathbb{Z}}$ 的特定选定块的长度与 F 的块长度相关联。这样的定义将避免我们在块长度定义中对 Cut0 的限制，以及我们在平均块长度定义中的极限过程。事实上，本文早期草稿将 F 的块长度定义为第一个两个非负切点之间的距离。不幸的是，命题 25 的类似命题，即对所有文件平均这个块长度，在一般情况下是错误的，即使对于局部块分方法也是如此。这里有一个简单的反例。假设 $\text{PFE} = \{0, 1\}$ ，并且块分方法在文件中每个出现的模式 011 处都在两个 1 的位置放置切点。（形式上，这是一种具有视野 $h = 2$ 和由 8 个序列 $*011*$ 和 $011*$ 组成的标准 C 的局部块分方法，其中星号代表独立地为 0 或 1。）切点概率是 $1/4$ ，而我们定义的块长度的期望是 4。但旧的定义，使用第一个两个非负切点，导致块长度的期望仅为 $7/4$ 。

人们可以直观地看出这个例子中出了什么问题。分块方法保证了切点以相邻对出现。因此，一半的块长度为 1（从 011 模式中的第一个 1 延伸到第二个 1），而另一半则更长（长度至少为 2，平均为 7）。位置 0 更有可能位于较长的块中，因此下一个块——即第一个和第二个非负切点之间的块——更有可能是长度为 1 的短块。当然，这也拉低了这种版本块长度的期望值。

另一种为（几乎）每个文件分配块长度的方法是，取包含特定位置（比如 0）的块的长度。这也无法正常工作，原因类似。如果存在不同长度的块（在非平凡分块方法下会存在这种情况），那么 0 更有可能位于较长的块中。一个具体的反例甚至比“前两个非负切点”版本更容易构造。再次设 $\text{PFE} = \{0, 1\}$ ，并设切点为文件条目为 1 的位置。（这是下面将更详细讨论的纯点过滤器方法的一个特例。）那么切点概率是 $1/2$ ，平均块长度是 2。但包含 0 的块的长度期望值是 3。

通常，分块方法具有可调整的参数，以便期望的块长度 $\mathbf{E}(L)$ 符合要求。但特定文件中的块可能比这个平均值短或长。如前所述，块过长或过短都不理想。一方面，组装和通经过短块的校验和的开销可能超过发送块本身的成本。另一方面，过长的块在发送方和接收方的文件中不太可能匹配。因此，人们更倾向于块长度与平均值偏差较小的分块方法。

从某种意义上说，块过长比块过短造成的问题更小。如有必要，过长的块可以通过添加额外的“人工”切点进行细分；所有“原生”切点保持完整。例如，如果期望的块长度是 L ，那么大小为 $cL + d$ 的块可以细分为 $c - 1$ 个大小为 L 的块，以及一个大小为 $L + d$ 的块。这样，发送方永远不会发送大小超过 $2L$ 的块。当然，这种方法可能会遇到最初促使采用内容相关分块的问题：由于人工切点位于固定位置，在长块的开头附近插入单个字符可能会破坏人工细分之间的协议。有更内在的方法来细分长块。例如，在第六节讨论的局部最大值分块方法中，长块可以通过具有较小视界的局部最大值进行细分。

另一方面，强制设置最小长度可能需要移除本土切割点。以下示例虽属极端且不太可能，但它能很好地说明移除本土切割点可能导致的失配情况。

示例 29。本土切割点将接收方的文件 F_1 分割成 $2n$ 个不同的块 $C_1 \dots C_{2n}$ ，这些块太小。通过移除一半的切割点，我们得到 n 个更大的块 $C_1C_2, \dots, C_{2n-1}C_{2n}$ 。发送方的文件 F_2 是从 F_1 通过将 C_1 移到末尾得到的。本土切割点将 F_2 分割成 $2n$ 个块 $C_2 \dots C_{2n} C_1$ ，，通过移除一半的切割点，我们得到更大的块 $C_2C_3 \dots C_{2n}C_1$ ，但这些更大的块在 F_1 中都没有出现。

第 30 条。没有一种局部分块方法能够绝对保证块不会太短也不会太长。

Proof. Consider a doubly infinite file F with the same entry in every position. Let h be the horizon of the chunking method. Every position of F has the same h -vicinity. It follows that either every position of F is a cutpoint, which violates the minimality requirement, or else no position of F is a cutpoint, which violates the maximality requirement. $\square$

Although files of the sort used in this proof form a set of measure zero, there is non-zero (albeit small) probability for a file to have a very long finite stretch of identical entries. In such files one will have either very short chunks (of length one) or a very long chunk (at least as long as the stretch of identical entries minus twice the horizon).

The methods that we propose in Sections 5 and 6 below provide absolute lower bounds on the chunk lengths. Upper bounds and stricter lower bounds hold with high probability. In practice, absolute guarantees are not crucial; high probabilistic guarantees are almost as good.

3.3. Costs

How can one compare the efficiency of different chunking methods? The cost of executing a remote differential compression protocol has several components, including

- (1) The number of bytes sent over the wire (in each direction).
- (2) The number of communication rounds.
- (3) The cumulative time of hard-disk accesses.
- (4) The computation complexity of finding the cutpoints.

In this section we concentrate on the first component. It is about minimizing the bandwidth used by a file transfer. Components 2 and 3 are highly relevant as well, but they do not depend on particular chunking methods. In particular, for any protocol that fits the rough outline in Section 1, the number of communication rounds is three, namely steps 3, 5, and 6 of that outline. (The number of communication rounds would increase to $2n + 1$ if a protocol is used recursively to depth n , as described in Remark 6. So one should, when using recursion in this way, keep in mind the trade-off between the benefit in Component 1 and the cost in Component 2.)

Component 3 can be reduced by using multiple disks and carefully laying out data on disks. A more detailed discussion of Components 2 and 3 is however outside the scope of this paper. Component 4 is addressed in subsequent sections in conjunction with the particular chunking methods. In particular, we show in Section 8 that local maxima can be found efficiently.

We now turn to our primary topic, Component 1. What is sent over the wire in order to transfer a file F_2 from a sender to a recipient?

- (S1) The chunk checksums sent to the recipient. Suppose that the chunks are $B_1, \dots, B_n$ and let $\mathbf{E}(L)$ be the expected chunk length. So n is usually close to $|F_2|/\mathbf{E}(L)$. All checksums have the same length Chk . So the number of bytes sent is

$$n \cdot \text{Chk} \approx (|F_2|/\mathbf{E}(L)) \cdot \text{Chk}.$$

- (S2) The indication, from the recipient, which of the chunks he wants to receive; the requested chunks are the chunks of F_2 that are not chunks of F_1 .
- (S3) The requested chunks B_j sent to the recipient. If k is the number of F_2 chunks that are also F_1 chunks then the number of bytes sent is

$$\sum_{B_j \text{ is wanted}} |B_j| \approx (n - k) \cdot \mathbf{E}(L).$$

The estimate in (S3) is necessarily a rough approximation because it assumes that the average length of requested chunks is the same as the average length $\mathbf{E}(L)$ of chunks in general. In fact, shorter chunks usually have a better chance of matching than long ones do. For an extreme example, suppose F_2 is obtained from F_1 by modifying every $\mathbf{E}(L)$ th symbol. Then no chunks of length more than $\mathbf{E}(L)$ will match, and only chunks of length smaller than $\mathbf{E}(L)$ will ever have a chance to match. In such a case the number of requested bytes will be larger than $(n - k) \cdot \mathbf{E}(L)$. Fortunately, for the purpose of the following discussion, the important part of the formula in (S3) is not the questionable factor $\mathbf{E}(L)$ but the factor $n - k$ which indicates that we should aim for large k .

(S1) and (S2) do not depend much on the chunking method. (S1) just depends, as indicated above, on the average chunk length and the size of the checksums, both of which can be chosen independently of the choice of chunking method.

(S2) is negligible compared to (S1) + (S3).

As indicated earlier, (S1) + (S3) can be reduced by applying chunking methods recursively; see Remark 6. We shall, however, analyze and compare content-dependent chunking protocols in a simple, non-recursive context. The comparisons carry over to the corresponding recursive versions, as the benefits of recursion are essentially independent of the benefits of choosing a good chunking method.

证明。考虑一个双无限文件 F ，它在每个位置都有相同的条目。令 h 为分块方法的视界。 F 的每个位置都有相同的 h -邻近。因此，要么 F 的每个位置都是一个切割点，这违反了最小性要求，要么 F 的每个位置都不是切割点，这违反了最大性要求。 $\square$

尽管在本证明中使用的文件形成一个测度为零的集合，但文件具有非常长的有限相同条目片段的非零（尽管很小）概率。在这样的文件中，要么会存在非常短的块（长度为1），要么会存在一个非常长的块（至少与相同条目片段的长度一样长减去两倍的视野）。

我们在第5节和第6节下面提出的那些方法为块长度提供了绝对下界。高概率下存在上限和更严格的下界。在实践中，绝对保证并不关键；高概率保证几乎一样好。

3.3. 成本

如何比较不同分块方法的效率？执行远程差异压缩协议的成本包含多个组成部分，包括

- (1) 通过线缆发送的字节数（每个方向）。
- (2) 通信轮次的数量。
- (3) 硬盘访问的累积时间。
- (4) 寻找切点的计算复杂度。

在本节中，我们专注于第一个组件。它涉及最小化文件传输使用的带宽。组件2和3也非常相关，但它们不依赖于特定的分块方法。特别是，对于符合第1节粗略概述的任何协议，通信轮次为三个，即该概述的第3、5和6步。（如果像注释6中描述的那样使用递归协议，通信轮次将增加到 $2n + 1$ ，因此在使用递归这种方式时，应记住组件1的收益与组件2的成本之间的权衡。）

组件3可以通过使用多个磁盘并仔细在磁盘上布置数据来减少。然而，关于组件2和3的更详细讨论超出了本文的范围。组件4将在后续章节中与特定的分块方法一起讨论。特别是，我们在第8节中展示了局部最大值可以高效地找到。

我们现在转向我们的主要话题，组件1。为了将文件 F_2 从发送方传输到接收方，在电线上发送的是什么？

- (S1) 发送给接收方的块校验和。假设块是 $B_1, \dots, B_n$ ，并且让 $\mathbf{E}(L)$ 是预期的块长度。所以 n 通常接近 $|F_2|/\mathbf{E}(L)$ 。所有校验和具有相同的长度 Chk 。所以发送的字节数是

$$n \cdot \text{Chk} \approx (|F_2|/\mathbf{E}(L)) \cdot \text{Chk}.$$

- (S2) 接收方指示他想接收哪些块；请求的块是 F_2 的块，但不是 F_1 的块。

- (S3) 请求的块 B_j 已发送给接收方。如果 k 是既是 F_2 块又是 F_1 块的数量，那么发送的字节数是

$$\sum_{B_j \text{ is wanted}} |B_j| \approx (n - k) \cdot \mathbf{E}(L).$$

(S3) 中的估计必然是一个粗略的近似，因为它假设请求的块的平均长度与一般块的平均长度 $\mathbf{E}(L)$ 相同。事实上，较短的块通常比长的块有更好的匹配机会。为了举一个极端的例子，假设 F_2 是通过修改 F_1 中的每个 $\mathbf{E}(L)$ 个符号得到的。那么，长度超过 $\mathbf{E}(L)$ 的块将不会匹配，只有长度小于 $\mathbf{E}(L)$ 的块才有可能匹配。在这种情况下，请求的字节数将大于 $(n - k) \cdot \mathbf{E}(L)$ 。幸运的是，对于以下讨论的目的，(S3) 中公式的重要部分并不是可疑的因子 $\mathbf{E}(L)$ ，而是因子 $n - k$ ，该因子表明我们应该追求较大的 k 。

(S1) 和 (S2) 对分块方法依赖不大。(S1) 仅依赖于，如上所述，平均块长度，以及校验和的大小，这两者都可以独立于分块方法的选择来选择。长度和校验和的大小，这两者都可以独立于分块方法的选择来选择。

(S2) 与 (S1) + (S3) 相比可以忽略不计。

如前所述，(S1) + (S3) 可以通过递归应用分块方法来减少；参见注释 6。然而，我们将在一个简单、非递归的上下文中分析和比较内容相关分块协议。这些比较可以推广到相应的递归版本，因为递归的好处基本上与选择一个好的分块方法的好处无关。

To minimize (S3) we would like to maximize the number k of common chunks. Consider a maximal interval I common to the two files F_1 and F_2 , and assume that I is long enough to contain at least one common chunk. I has the form $S \cdot C_1 \cdots C_n \cdot S'$ where $C_1, \dots, C_n$ are chunks, S is a proper final segment of the preceding chunk, and S' is a proper initial segment of the subsequent chunk. The parts S and S' of the agreement interval I are wasted in the sense that the agreement of these segments of the two files doesn't reduce the transmissions needed in (S3). If I had not included any common chunks then the whole I would have been wasted.

Thus, the efficiency of a chunking method depends, in large part, on its ability to keep the wasted agreements, the segments S and S' , small. In the next subsection, we introduce a mathematically convenient way to assess this ability. In that discussion, we also take into account that, although the interval I is common to the two files, its subdivision into chunks may be different near the ends. This is because whether a position is a cutpoint depends on its h -vicinity, and that may extend beyond I .

3.4. Slack from the left

We introduce an idealized model of what happens near the beginning of an interval of agreement between two files. The model uses two doubly infinite files $F_1, F_2 \in \text{PFE}^{\mathbb{Z}}$ that coincide at all non-negative positions but are independent elsewhere. We write F^+ for their common non-negative part (an element of $\text{PFE}^{\mathbb{N}}$) and F_1^- and F_2^- for their respective negative parts. Thus, we envision a pair of doubly infinite files that start out independent but at some point (position 0) merge and are identical thereafter.

Formally, we work with the probability space $\text{PFE}^{\mathbb{M}}$ where $\mathbb{M}$ (the symbol stands for “merge”) consists of all the non-negative integers and all pairs $(i, 1)$ and $(i, 2)$ for negative integers i . So $\mathbb{M} = ((\mathbb{Z} - \mathbb{N}) \times \{1, 2\}) \cup \mathbb{N}$. As with previous probability spaces, we use the product measure derived from the uniform measure on PFE. Thus, the entries in a random file $F \in \text{PFE}^{\mathbb{M}}$ are chosen independently and uniformly from PFE. If $F \in \text{PFE}^{\mathbb{M}}$ then F determines a merging pair of files (F_1, F_2) by

$$F_k(i) = \begin{cases} F(i, k) & \text{if } i < 0, \\ F(i) & \text{if } i \geq 0. \end{cases}$$

The extraction of the two files F_k from F amounts to a pair of projection functions

$$\pi_k : \text{PFE}^{\mathbb{M}} \rightarrow \text{PFE}^{\mathbb{Z}} : F \mapsto F_k.$$

Both of these projection functions are clearly measure-preserving; that is, $\mathbf{Prob}(\pi_k^{-1}(A)) = \mathbf{Prob}(A)$ for all events $A \subseteq \text{PFE}^{\mathbb{Z}}$.

Remark 31. We think of position 0 as where the agreement between F_1 and F_2 begins. Strictly speaking, 0 is where agreement begins to be enforced by the definition of the model. It is possible for the files to already have the same entry “accidentally” at -1 ; this happens with probability $1/|\text{PFE}|$. And the actual interval of agreement may begin even earlier, though with even smaller probability. Modifying the model to prohibit such coincidences would introduce additional cases into our computations without significantly changing the results. So we abstain from such a modification and use the model as presented above.

Although F_1 and F_2 are infinite, they are intended to serve as mathematically convenient models for the behavior of a pair of real, finite files with an interval I of agreement as in the discussion at the end of Section 3.3. More precisely, they model the behavior near the beginning of I , where the files agree to the right and are (as a mathematical idealization) independent to the left. As indicated earlier, we would like our chunking method to minimize the wasted part S of the interval of agreement. We also want to minimize the wasted part S' at the other end of I , but that can be treated almost symmetrically (see Section 3.5 below), so we concentrate, for now, on S .

In our model, the wasted part extends at least from the merge point 0 to the first non-negative position that is a common cutpoint of both files. (It may extend further, if, after the first common cutpoint, the files have different cutpoints. This will not happen with any of the chunking methods we consider.) That motivates the following definition.

Definition 32. Let a chunking method for doubly infinite files be given. The *slack* of any $F \in \text{PFE}^{\mathbb{M}}$, written $\text{Slack}(F)$, is the smallest non-negative integer i that is a cutpoint of both of the derived files F_1 and F_2 .

Since the slack gives information about the behavior of a chunking method when one enters an interval of agreement from the left end, we may refer to it as the $\rightarrow$ slack, especially if we need to contrast it with the analogous $\leftarrow$ slack defined below.

As with some previous definitions, we confess that the slack may not be defined for some F , if the files F_1 and F_2 have no common cutpoint. For local chunking methods, the set of such bad F has probability zero and can therefore safely be ignored. Indeed, if h is the horizon of the chunking method, then any cutpoint $\geq h$ in either F_1 or F_2 is also a cutpoint of the other, since the h -vicinities agree. So the only way for F to have undefined slack is for each F_k to have no cutpoints

为了最小化 (S3)，我们希望最大化公共块的数量 k 。考虑两个文件 F_1 和 F_2 的一个最大区间 I ，并假设 I 足够长，可以包含至少一个公共块。 I 的形式为 $S \cdot C_1 \cdots C_n \cdot S'$ ，其中 $C_1 \dots C_n$ 是块， S 是前一个块的一个适当最终片段，而 S' 是后续块的一个适当初始片段。协议区间 I 的 S 和 S' 部分是浪费的，因为这两个文件这些段的协议不会减少 (S3) 中需要的传输量。如果 I 没有包含任何公共块，那么整个 I 都会被浪费。

因此，分块方法的效率在很大程度上取决于其保持浪费的协议、段 S 和 S' 小的能力。在下一小节中，我们介绍一种方便的数学方法来评估这种能力。在讨论中，我们还考虑了这样一个事实：尽管区间 I 对两个文件来说是公共的，但其在两端附近的分块可能不同。这是因为一个位置是否是切割点取决于其 h -邻近，而 h -邻近可能超出 I 。

3.4. 左侧的松弛

我们介绍一个理想化的模型，描述两个文件在协议区间开始附近发生的情况。该模型使用两个双无限文件 $F_1, F_2 \in \text{PFE}^{\mathbb{Z}}$ ，它们在所有非负位置上重合，但在其他位置上相互独立。我们用 F^+ 表示它们的公共非负部分 ($\text{PFE}^{\mathbb{N}}$ 的一个元素)，用 F_1^- 和 F_2^- 表示它们各自的负部分。因此，我们设想了一对双无限文件，它们起初是独立的，但在某个点（位置 0）合并，此后完全相同。

形式上，我们处理的是概率空间 $\text{PFE}^{\mathbb{M}}$ ，其中 $\mathbb{M}$ （该符号表示“合并”）包含所有非负整数以及所有负整数 i 对应的对 $(i, 1)$ 和 $(i, 2)$ 。因此 $\mathbb{M} = ((\mathbb{Z} - \mathbb{N}) \times \{1, 2\}) \cup \mathbb{N}$ 。与之前的概率空间一样，我们使用从 PFE 上均匀测度导出的乘积测度。因此，随机文件 $F \in \text{PFE}^{\mathbb{M}}$ 中的条目是从 PFE 中独立且均匀地选择的。如果 $F \in \text{PFE}^{\mathbb{M}}$ ，则 F 通过以下方式确定一对合并的文件 (F_1, F_2) ，

$$F_k(i) = \begin{cases} F(i, k) & \text{if } i < 0, \\ F(i) & \text{if } i \geq 0. \end{cases}$$

从 F 中提取两个文件 F_k 相当于一对投影函数

$$\pi_k : \text{PFE}^{\mathbb{M}} \rightarrow \text{PFE}^{\mathbb{Z}} : F \mapsto F_k.$$

这两个投影函数显然都是测度保持的；也就是说， $\mathbf{Prob}(\pi_k^{-1}(A)) = \mathbf{Prob}(A)$ 对于所有事件 $A \subseteq \text{PFE}^{\mathbb{Z}}$ 。

注释31。我们认为位置0是 F_1 和 F_2 之间开始一致的地方。严格来说，0是根据模型定义开始强制一致的地方。文件在 -1 处已经偶然具有相同的条目是可能的；这种情况发生的概率是 $1/|\text{PFE}|$ 。并且实际的一致区间可能更早开始，尽管概率更小。修改模型以禁止这种巧合会在我们的计算中引入更多情况，但不会显著改变结果。因此，我们避免这种修改，并使用上述模型。

尽管 F_1 和 F_2 是无限的，但它们旨在作为数学上方便的模型，用于描述一对具有区间 I 的真实、有限文件，正如第 3.3 节末尾的讨论中所描述的那样。更精确地说，它们模拟了 I 开头附近的行为，在这些行为中，文件在右侧保持一致，并且在左侧（作为数学理想化）是独立的。如前所述，我们希望我们的分块方法能够最小化区间 I 的一致部分 S 的浪费部分。我们还希望最小化 I 另一端的浪费部分 S' ，但这几乎可以对称地处理（见下文第 3.5 节），因此我们现在集中精力关注 S 。

在我们的模型中，浪费的部分至少从合并点 0 延伸到第一个非负位置，该位置是两个文件的一个公共切点。（如果第一个公共切点之后，文件具有不同的切点，它可能会进一步延伸。在我们考虑的所有分块方法中，这种情况都不会发生。）这引出了以下定义。

定义32。设给定一种用于双无限文件的分块方法。任意 $F \in \text{PFE}^{\mathbb{M}}$ 的松弛，记作 $\text{Slack}(F)$ ，是同时为派生文件 F_1 和 F_2 的切割点的最小非负整数 i 。

由于松弛提供了关于分块方法在从左端进入一致区间时的行为信息，因此我们可以将其称为 $\rightarrow$ 松弛，尤其是在我们需要将其与下面定义的类似 $\leftarrow$ 松弛进行对比时。

与一些之前的定义类似，我们承认，如果文件 F_1 和 F_2 没有公共切割点，那么松弛可能对某些 F 未定义。对于局部分块方法，这样的坏 F 集合的概率为零，因此可以安全地忽略。实际上，如果 h 是分块方法的视野，那么在 F_1 或 F_2 中的任何切割点 $\geq h$ 也是另一个的切割点，因为 h -邻域是一致的。因此， F 具有未定义松弛的唯一方式是每个 F_k 都没有切割点。

$\geq h$. But we already saw that almost all files in $\text{PFE}^{\mathbb{Z}}$ have infinitely many positive cutpoints, so, invoking the fact that π_k preserves measure, we conclude that almost all $F \in \text{PFE}^{\mathbb{M}}$ have well-defined slack.

For non-local chunking methods, it is not so clear that the slack is almost everywhere defined, but this will be the case for the one non-local method that we shall analyze and compare with our local methods.

It is intuitively plausible that chunking methods with large chunks will have larger slack, because chunks that start in the independent negative parts F_k^- of the two files will extend farther into the common positive part F^+ . Accordingly, it makes sense to measure slack relative to expected chunk length.

Definition 33. The *normalized slack* of $F \in \text{PFE}^{\mathbb{M}}$ is defined as $S(F) = \text{Slack}(F)/\mathbf{E}(L)$.

As indicated by our discussion of (S3) above, remote differential compression benefits from a chunking method with small slack. Accordingly, we shall use the expectation of the (normalized) slack as one measure of the quality of chunking methods.

3.5. Slack from the right

Recall that it is advantageous for a chunking method to waste as little as possible from either end of an interval of agreement. That is, if two files coincide on a long interval I , then the chunking method should produce a common cutpoint near the left end and another common cutpoint near the right end of I . The slack measures how well a method does at the left end. The situation at the right end is almost but not quite symmetrical. For mathematical simplicity, we make the definitions exactly symmetrical. Afterward, we discuss how reality deviates slightly from this perfect symmetry.

Symmetrical to $\mathbb{M}$ is the index set

$$\mathbb{D} = \{i \in \mathbb{Z}: i \leq 0\} \cup \{(i, k) \in \mathbb{Z} \times \{1, 2\}: i > 0\}.$$

(The symbol stands for “diverge.”) An element F of $\text{PFE}^{\mathbb{D}}$ amounts to two files $F_k \in \text{PFE}^{\mathbb{Z}}$

$$F_k(i) = \begin{cases} F(i) & \text{if } i \leq 0, \\ F(i, k) & \text{if } i > 0 \end{cases}$$

that coincide at non-positive locations but are independent at positive locations. The two projections

$$\pi_k: \text{PFE}^{\mathbb{D}} \rightarrow \text{PFE}^{\mathbb{Z}}: F \mapsto F_k$$

preserve measure just as before. Symmetrically to the earlier notation, we write F^- for the common part of F_1 and F_2 , i.e., the restriction of F to non-positive integers, and we write F_k^+ for the independent positive parts of the two files F_k .

Definition 34. Let a chunking method for doubly infinite files be given. The *reverse slack*, or $\leftarrow$ slack, of any $F \in \text{PFE}^{\mathbb{D}}$, which is written $\leftarrow \text{Slack}(F)$, is the smallest non-negative integer i such that $-i$ is a cutpoint of both of the derived files F_1 and F_2 .

The two files F_k extracted from any $F \in \text{PFE}^{\mathbb{D}}$ agree at all positions ≤ 0 , and the $\leftarrow$ slack of F measures how much of this agreement is wasted. This is intended to model what happens at the right end of a long interval of agreement between two real files.

Remark 35. As a measure of wasted agreement, the $\leftarrow$ slack suffers from a few small inaccuracies. One was already pointed out in Remark 31 in connection with the $\rightarrow$ slack, namely that there is a slight chance that the interval of agreement between F_1 and F_2 is actually longer than what the model enforces. Even though $F_1(1)$ and $F_2(1)$ are independent, they might happen to coincide. As with $\rightarrow$ slack, we choose to ignore this problem because it is unlikely to occur at all and even more unlikely to have a significant influence (more than one or two positions) on the amount of wasted agreement.

Two other inaccuracies arise from our convention that a cutpoint is included in the chunk to its right, not the one to its left. Suppose the $\leftarrow$ slack of $F \in \text{PFE}^{\mathbb{D}}$ is s . So the two files F_1 and F_2 have a common cutpoint at $-s$ and no later common cutpoints ≤ 0 . Ordinarily (but see the exception in the next paragraph), this means that the chunks of F_1 and F_2 that begin at $-s$ will differ, either because they have different lengths or, if they have the same length, because they extend to positive positions where the files differ. (Remember that, as discussed above, we are ignoring possible accidental agreement at position 1.) So the agreement of F_1 and F_2 at the $s+1$ positions $-s, \dots, 0$ is wasted. The $\leftarrow$ slack s underestimates the waste by 1. Thus, were it not for the next paragraph, this inaccuracy could be corrected by simply adding 1 to the $\leftarrow$ slack. In most situations, this correction will be negligible; the slack is comparable to the average chunk length, which is much larger than 1.

There is, however, an exceptional situation where s overestimates the waste. This occurs when both F_1 and F_2 have a cutpoint at 1. Being > 0 , this cutpoint has no influence on the $\leftarrow$ slack s , but the chunk that begins at $-s$ is $[-s, 0]$, which lies entirely in the interval of agreement of the two files. So there is no waste at all in this case. The correction needed

$\geq h$. 但我们已经看到 $\text{PFE}^{\mathbb{Z}}$ 中的几乎所有文件都有无限多个正切割点，因此，利用 π_k 保持测量的事实，我们得出结论，几乎所有 $F \in \text{PFE}^{\mathbb{M}}$ 都有定义良好的松弛。

对于非局部分块方法，尚不清楚松弛是否几乎处处有定义，但我们将分析并同我们的局部方法进行比较的那一种非局部方法将满足这一条件。

直观上似乎合理的观点是，块长度较大的分块方法将具有较大的松弛，因为当两个文件中的独立负部分 F_k^- 开始时，块将扩展到公共正部分 F^+ 更远。因此，相对于预期块长度来衡量松弛是有意义的。

定义 33. $F \in \text{PFE}^{\mathbb{M}}$ 的标准化松弛定义为 $S(F) = \text{Slack}(F)/\mathbf{E}(L)$ 。

正如我们上面关于 (S3) 的讨论所表明的，远程差分压缩受益于具有较小松弛的分块方法。因此，我们将使用（标准化）松弛的期望值作为衡量分块方法质量的一个指标。

3.5. 从右侧的松弛

回想一下，对于分块方法来说，浪费掉区间一致性的两端尽可能少是有利的。也就是说，如果两个文件在长区间 I 上重合，那么分块方法应该在 I 的左端附近产生一个公共切割点，并在 I 的右端附近产生另一个公共切割点。松弛度衡量了方法在左端的表现。右端的情况几乎对称，但不完全对称。为了数学上的简化，我们使定义完全对称。之后，我们讨论现实如何轻微偏离这种完美的对称性。

$\mathbb{M}$ 的对称物是索引集

$$\mathbb{D} = \{i \in \mathbb{Z}: i \leq 0\} \cup \{(i, k) \in \mathbb{Z} \times \{1, 2\}: i > 0\}.$$

(该符号表示“发散”。) $\text{PFE}^{\mathbb{D}}$ 中的元素 F 等于两个文件 $F_k \in \text{PFE}^{\mathbb{Z}}$

$$F_k(i) = \begin{cases} F(i) & \text{if } i \leq 0, \\ F(i, k) & \text{if } i > 0 \end{cases}$$

在非正位置处重合但在正位置处独立的。这两个投影

$$\pi_k: \text{PFE}^{\mathbb{D}} \rightarrow \text{PFE}^{\mathbb{Z}}: F \mapsto F_k$$

正如之前一样，我们用 F^- 表示 F_1 和 F_2 的公共部分，即 F 在非正整数上的限制，并用 F_k^+ 表示两个文件 F_k 的独立正部分。

定义34. 设给定一个用于双无限文件的分块方法。任何 $F \in \text{PFE}^{\mathbb{D}}$ 的逆向松弛，或 $\leftarrow$ 松弛，是满足 $-i$ 为派生文件 F_1 和 F_2 的共同切割点的最小非负整数 i 。

从任何 $F \in \text{PFE}^{\mathbb{D}}$ 提取的两个文件在所有位置 ≤ 0 上完全一致，而 F 的 $\leftarrow$ 松弛衡量了这种一致性中有多少被浪费了。这旨在模拟两个真实文件之间在一个长区间内一致性的右端发生的情况。

注释35. 作为衡量被浪费一致性的一种指标， $\leftarrow$ 松弛存在一些小的误差。其中一个已经在注释31中与 $\rightarrow$ 松弛相关时指出，即 F_1 和 F_2 之间的一致性区间实际上可能比模型强制执行的要长。尽管 $F_1(1)$ 和 $F_2(1)$ 是独立的，但它们可能会恰好重合。与 $\rightarrow$ 松弛一样，我们选择忽略这个问题，因为它不太可能发生，而且即使发生，对被浪费一致性的数量（超过一两个位置）的影响也极小。

我们的约定中存在两个不准确之处：切割点包含在它右侧的块中，而不是左侧的块中。假设 $\leftarrow$ 松弛值为 $F \in \text{PFE}^{\mathbb{D}}$ 的 s 。因此，文件 F_1 和 F_2 在 $-s$ 处有一个公共切割点，且没有更晚的公共切割点 ≤ 0 。通常情况下（但见下一段的例外情况），这意味着以 $-s$ 开头的 F_1 和 F_2 的块将不同，要么因为它们的长度不同，要么如果它们的长度相同，则因为它们扩展到文件不同的正位置。（记住，如上所述，我们忽略了位置 1 的可能偶然一致。）因此， F_1 和 F_2 在 $s+1$ 位置 $-s, \dots, 0$ 处的一致性是不浪费的。 $\leftarrow$ 松弛值 s 低估了浪费量 1。因此，如果不是因为下一段，这个不准确之处可以通过简单地将 $\leftarrow$ 松弛值加 1 来纠正。在大多数情况下，这种纠正将是微不足道的；松弛值与平均块长度相当，而平均块长度远大于 1。

然而，存在一种特殊情况，其中 s 会高估浪费。这种情况发生在 F_1 和 F_2 都在 1 处有一个切割点时。由于 > 0 ，这个切割点对 $\leftarrow$ 松弛 s 没有影响，但以 $-s$ 开始的块是 $[-s, 0]$ ，它完全位于两个文件的协议区间内。因此，在这种情况下完全没有浪费。所需的修正

in this case is (to not add 1 as in the preceding paragraph and) to subtract s , i.e., to replace the $\leftarrow$ slack with 0. This correction, though it may be large for an individual file, is usually negligible on average, especially in comparison with the whole $\leftarrow$ slack s , because the probability q that 1 is a cutpoint in both F_1 and F_2 is so small (clearly $q \leq p$ and usually $q \ll p$).

Because all the inaccuracies in the reverse slack are relatively small, we shall neglect them and use the average reverse slack as a measure of a chunking method's waste of agreement at the right end of an interval on which two files coincide.

3.6. Quality of chunking methods

Recall the three desiderata for a chunking method: The chunks should not be too short. The chunks should not be too long. And agreements between parts of files should be promptly reflected in agreements between chunks. Of course, these desiderata are interrelated. For example, the trouble with excessively long chunks is that a long interval of agreement between files might not contain any whole chunks. Nevertheless, it is convenient to consider the three desiderata separately, because the first two are somewhat easier to deal with. We must not, however, be so focused on the first two that we ignore the third, because the first two can be satisfied by using chunks of a single fixed length, and we have seen that this chunking method can take two nearly identical files (differing by the addition of a single character) and produce no agreements at all between chunks.

The chunking methods that we consider will have one or two adjustable parameters, so that we can control, for example, the average chunk size, or the minimum chunk size, or sometimes both. So it is not too difficult to achieve the first or second desideratum; it is the interplay between the two that imposes a non-trivial requirement on a chunking method. This interplay can be summarized by saying that we do not want too much variation in the chunk lengths.

An obvious measure, therefore, is the variance (or its square root, the standard deviation) of the chunk length. Another possible measure for the same purpose is the ratio of the average to the minimum chunk length. A third is the probability of finding no cutpoints in a long interval, say an interval whose length is 5 or more times the average chunk length. We shall calculate or at least estimate these three measures for all of the chunking methods that we treat in the following sections, except that the variance of the local maximum method remains an open problem and the probability of long chunks and the $\leftarrow$ slack of the interval filter are expressed not explicitly but in terms of the smallest root of a certain polynomial equation.

The quality of a chunking method from the viewpoint of the third desideratum is, we propose, reasonably measured by the slack and reverse slack. We shall therefore also compute or at least estimate the $\rightarrow$ slack and $\leftarrow$ slack of the methods in the following sections, except that we have not been able to accurately estimate the slack of the local maximum method (in either direction — they are the same by symmetry).

4. Point filter methods

4.1. Pure point filters

The pure point-filter chunking is the most local chunking method: the vicinity of a position i that determines whether i is a cutpoint of F consists of $F(i)$ alone. This chunking method has an integer $c \geq 2$ as a parameter. In the following discussion, we consider c as fixed and we identify PFE with the set of integers from 0 to $|PFE| - 1$.

Definition 36. *Pure point filter chunking* is the chunking method where the cutpoints of a file F are those positions i where $F(i)$ is divisible by c .

In practice, the number $|PFE|$ of potential file entries (often hashes resulting from a rolling hash) is usually a power of 2 and c is a smaller power of 2. This simplifies the task of finding cutpoints, because, instead of dividing $F(i)$ by c , one can just test whether the bit-pattern of $F(i)$ ends with enough 0's.

When $|PFE| \geq c$ are powers of 2, or more generally when c divides $|PFE|$, then the cutpoint probability is clearly $1/c$. In general, it is

$$p = \frac{1}{|PFE|} \left\lceil \frac{|PFE|}{c} \right\rceil,$$

i.e., $1/c$ rounded up to the next larger multiple of $1/|PFE|$. (Had we chosen PFE to start with 1 rather than 0, then we would round down rather than up.) In practice, the rounding is negligible because $|PFE|$ is much larger than c .

Remark 37. It is only a matter of practical convenience that the cutpoints are defined in terms of divisibility. In practice, one uses divisibility by powers of two, because that can be checked by inspecting a bit pattern, which takes just one CPU cycle. But our analysis applies equally well to less efficient criteria. One could use an arbitrary subset C of PFE, defining the cutpoints of a file F to be those positions i where the value $F(i)$ belongs to C . Then the cutpoint probability is $p = |C|/|PFE|$. All the following results, except for approximations involving c , hold in this more general situation.

在这种情况下是（不在前一段中加 1）减去 s ，即用 0 替换 $\leftarrow$ 松弛。虽然这个修正对单个文件可能很大，但在平均情况下通常是微不足道的，尤其是在与整个 $\leftarrow$ 松弛 s 相比时，因为 1 在 F_1 和 F_2 中都是切割点的概率 q 如此之小（显然 $q \leq p$ ，并且通常 $q \ll p$ ）。

由于反向松弛中的所有误差都相对较小，我们将忽略它们，并使用平均反向松弛作为衡量分块方法在两个文件重合的区间右端浪费协议的一种指标。

3.6. 分块方法的质量

回想一下分块方法的三项期望：块不应太短。块不应太长。文件各部分之间的协议应迅速反映在块之间的协议中。当然，这三项期望是相互关联的。例如，过长的块的问题在于，文件之间长度的协议区间可能不包含任何完整的块。尽管如此，将这三项期望分开考虑是方便的，因为前两项相对更容易处理。然而，我们不应如此专注于前两项而忽略第三项，因为前两项可以通过使用固定长度的块来满足，而我们已看到这种分块方法可以将两个几乎相同的文件（通过添加一个字符而不同）产生块之间没有任何协议。

我们考虑的分块方法将有一个或两个可调节参数，以便我们可以控制，例如，平均块长度，或最小块长度，有时两者都控制。因此，实现第一个或第二个目标要求并不太难；是这两个参数之间的相互作用对分块方法提出了一个非平凡的要求。这种相互作用可以总结为：我们不希望块长度变化太大。

因此，一个明显的度量是块长度的方差（或其平方根，标准差）。另一个可能的度量是平均块长度与最小块长度的比率。第三种是找到长区间（例如，长度是平均块长度5倍或更多的区间）中无切割点的概率。我们将为以下各节中处理的全部分块方法计算或至少估计这三个度量，但局部最大值方法的方差仍然是一个悬而未决的问题，长块的概率和区间过滤器的 $\leftarrow$ 松弛则不是明确表达，而是用某个多项式方程的最小根来表示。

从第三个期望的角度来看，我们建议分块方法的品质可以通过松弛和反向松弛来合理衡量。因此，在接下来的几节中，我们也将计算或至少估计这些方法中的 $\rightarrow$ 松弛和 $\leftarrow$ 松弛，不过我们未能准确估计局部最大值方法的松弛（无论哪个方向——它们由于对称性是相同的）。

4. 点过滤器方法

4.1. 纯点过滤器

纯点过滤器分块是最局部的分块方法：确定位置*i*是否为F的切点F(i) 的邻域仅包含F(i)。这种分块方法有一个整数*c* ≥ 2 作为参数。在接下来的讨论中，我们将*c*视为固定，并将PFE与从0到 $|PFE| - 1$ 的整数集等同起来。

定义36. 纯点过滤器分块是指切点为文件F中所有满足F(i) 能被*c*整除的位置*i*的分块方法。

实际上，潜在文件条目（通常是滚动哈希产生的哈希值）的数量 $|PFE|$ 通常是 2 的幂，且 c 是一个较小的 2 的幂。这简化了寻找切割点的任务，因为，与其将 $F(i)$ 除以 c ，可以直接测试 $F(i)$ 的位模式是否以足够的 0 结尾。

当 $|PFE| \geq c$ 是 2 的幂时，或者更一般地，当 c 整除 $|PFE|$ 时，切割点概率显然是 $1/c$ 。通常情况下，它是

$$p = \frac{1}{|PFE|} \left\lceil \frac{|PFE|}{c} \right\rceil,$$

即， $1/c$ 向上舍入到下一个更大的 $1/|PFE|$ 的倍数。（如果我们一开始选择 PFE 为 1 而不是 0，那么我们会向下舍入而不是向上舍入。）实际上，舍入可以忽略不计，因为 $|PFE|$ 远大于 c 。

备注37. 切点的定义基于可整除性，这纯粹是出于实用便利的考虑。在实际应用中，人们通常使用以2的幂次为基准的可整除性，因为可以通过检查位模式来验证，而位模式只需一个CPU周期即可处理。但我们的分析同样适用于效率较低的标准。人们可以使用 PFE 的任意子集*C*，将文件F的切点定义为那些值F(i) 属于*C*的位置*i*。那么，断点概率就是 $p = |C|/|PFE|$ 。以下所有结果，除了涉及*c*的近似计算外，都适用于这种更一般的情况。

The probability distribution of the chunk length L is geometric with parameter p . That is, L takes each positive integer i as a value with probability

$$\mathbf{Prob}(L = i) = (1 - p)^{i-1} p \quad \text{for } i \geq 1.$$

(Recall that we defined chunk length only for files with a cutpoint at 0, so the probabilistic notions here are all conditional on the event Cut0.) Using the formulas in Subsection 2.3 to sum the relevant series, we obtain that the expectation of L is

$$\mathbf{E}(L) = \frac{1}{p} \approx c$$

in agreement with Proposition 25, that

$$\mathbf{E}(L^2) = \frac{2 - p}{p^2},$$

and therefore that the variance of L is

$$\mathbf{Var}(L) = \mathbf{E}(L^2) - E(L)^2 = \frac{1 - p}{p^2} \approx c^2 - c.$$

The slack is also geometrically distributed, but taking values starting with 0 rather than 1. (The slack of $F \in \text{PFE}^{\mathbb{M}}$ can be zero; the chunk length of $F \in \text{PFE}^{\mathbb{Z}}$ cannot.) We have

$$\mathbf{Prob}(\text{Slack} = i) = (1 - p)^i p \quad \text{for } i \geq 0$$

and therefore

$$\mathbf{E}(\text{Slack}) = \frac{1}{p} - 1 \approx c - 1.$$

The expectation of the normalized slack is thus $1 - p \approx 1 - \frac{1}{c}$. Of course, the reverse slack has the same expectation, because of the left-right symmetry of the chunking method.

4.2. Point filters without short chunks

The pure point filter method allows chunks to be as small as a single element of a file. To avoid excessively small chunks, a modification of the method was proposed, in [20], forcing all chunks to be larger than a certain length h ; we refer to this modification as the LBFS chunking method. In [20], the parameters were chosen to be $h = 2^{11}$ and $c = 2^{13}$, but the chunking method can be applied with any desired h and c . It proceeds as follows, given a file in $\text{PFE}^{[0, l-1]}$ or in $\text{PFE}^{\mathbb{N}}$. Ignore the first $h + 1$ positions (0 to h) because cutting there would produce an impermissibly small chunk. Beginning at position $h + 1$, look for positions i where c divides $F(i)$, and declare the first such i to be a cutpoint. Then ignore the next h positions, $i + 1$ to $i + h$, again because cutting there would produce an impermissibly small chunk. Starting at position $i + h + 1$, look again for a position where the entry in F is divisible by c , declare it to be a cutpoint, and so forth.

Because this chunking method was introduced as a part of LBFS (low bandwidth file system) in [20], we shall refer to it as *LBFS chunking*. (LBFS includes other aspects in addition to chunking, such as maintaining a system-wide database of chunks indexed by their hashes (see Remark 5), but we are concerned in this paper only with the chunking method.)

Definition 38. In connection with LBFS chunking applied to a file F , we call a position i a *candidate* if c divides $F(i)$. The *cutpoints* are those candidates i such that $i > h$ and none of the preceding h positions $i - h, \dots, i - 1$ is a cutpoint. If a candidate i fails to be a cutpoint because there was a cutpoint j in the range $i - h \leq j \leq i - 1$, then we say that j *blocks* i (from being a cutpoint). We use the symbol k for the ratio h/c .

Notice that the candidates for the LBFS chunking method are exactly the cutpoints for the pure point filter method with the same c .

We shall occasionally give particular attention to the case $k = 1/4$ that corresponds to the choice of parameters proposed in [20].

Remark 39. The LBFS chunking method is not local (except, of course, when $h = 0$ and it reduces to the pure point filter method). To see this, consider how to tell whether a position i is a cutpoint. First, check whether c divides $F(i)$. If not, then you have the answer, “no.” But if c does divide $F(i)$, then you still have to check whether any of $i - 1, \dots, i - h$ was a cutpoint, which would block i . So check whether c divides any of $F(i - 1), \dots, F(i - h)$. If all the answers are “no,” then you have the answer; i is a cutpoint. But if you find a candidate among $i - 1, \dots, i - h$, then you still need to check whether it was a cutpoint, which involves checking the preceding h positions. And if you find a candidate there, then you have to check h positions farther back from that candidate, and so forth. Thus, there is no a priori bound on how far back you might have to look in order to decide whether i is a cutpoint. That is, the chunking method is not local.

块长度L的概率分布是参数为p的几何分布。也就是说，L以概率取每个正整数i作为值

$$\mathbf{Prob}(L = i) = (1 - p)^{i-1} p \quad \text{for } i \geq 1.$$

(回想我们仅对在0处有切割点的文件定义了块长度，因此这里的概率概念都是基于事件Cut0的条件。)使用2.3小节中的公式来求和相关级数，我们得到L的期望是

$$\mathbf{E}(L) = \frac{1}{p} \approx c$$

与命题25一致，即

$$\mathbf{E}(L^2) = \frac{2 - p}{p^2},$$

因此L的方差是

$$\mathbf{Var}(L) = \mathbf{E}(L^2) - E(L)^2 = \frac{1 - p}{p^2} \approx c^2 - c.$$

松弛也服从几何分布，但取值从0开始而不是1。($F \in \text{PFE}^{\mathbb{M}}$ 的松弛可以为零； $F \in \text{PFE}^{\mathbb{Z}}$ 的块长度不能为零。)我们有

$$\mathbf{Prob}(\text{Slack} = i) = (1 - p)^i p \quad \text{for } i \geq 0$$

因此

$$\mathbf{E}(\text{Slack}) = \frac{1}{p} - 1 \approx c - 1.$$

因此，标准化松弛的期望是 $1 - p \approx 1 - \frac{1}{c}$ 。当然，反向松弛具有相同的期望，因为分块方法具有左右对称性。

4.2. 没有短块的点过滤器

纯点过滤器方法允许块小到一个文件的单个元素。为了避免过小的块，提出了该方法的一种修改，在 [20]，强制所有块的大小必须大于某个长度 h ；我们将此修改称为 LBFS 分块方法。在 [20]，参数被选择为 $h = 2^{11}$ 和 $c = 2^{13}$ ，但分块方法可以用任何期望的 h 和 c 应用。它按以下步骤进行，给定一个在 $\text{PFE}^{[0, l-1]}$ 或 $\text{PFE}^{\mathbb{N}}$ 中的文件。忽略前 $h + 1$ 个位置 (0 到 h)，因为在那里切割会产生一个不允许的过小块。从位置 $h + 1$ 开始，寻找位置 i ，其中 c 整除 $F(i)$ ，并将第一个这样的 i 声明为切割点。然后忽略下一个 h 个位置， $i + 1$ 到 $i + h$ ，同样因为在那里切割会产生一个不允许的过小块。从位置 $i + h + 1$ 开始，再次寻找 F 中可被 c 整除的位置，将其声明为切割点，等等。

因为这种分块方法是在 [20]，作为 LBFS（低带宽文件系统）的一部分引入的，我们将它称为 LBFSchunking。（LBFS 除了分块之外还包括其他方面，例如维护一个按其哈希值索引的块的全局数据库（见备注 5），但在本文中我们只关心分块方法。）

定义38. 在将LBFS分块应用于文件F的情况下，如果c整除F(i)，则称位置i为候选。切点是指满足 $i > h$ 并且其之前的h个位置 $i - h \dots i - 1$ ，中没有一个位置是切点的候选i。如果一个候选i因为范围内存在切点j $i - h \leq j \leq i - 1$ 而未能成为切点，则我们称j阻塞i（成为切点）。我们使用符号k表示比率h/c。

请注意，LBFS分块方法的候选与纯pointfilter方法（使用相同c参数）的切点完全一致。

我们将偶尔特别关注对应于 [20]中提出的参数选择的 $k = 1/4$ 情况。

备注39. LBFS分块方法不是局部的（当然，当 $h = 0$ 时，它会简化为纯点过滤器方法）。要理解这一点，考虑如何判断位置i是否是切割点。首先，检查c是否整除F(i)。如果不是，那么答案就是“否”。但如果c确实整除F(i)，那么你仍然需要检查 $i - 1 \dots i - h$ 、…是否是切割点，这会阻塞i。所以检查c是否整除 $F(i - 1) \dots F(i - h)$ 、…。如果所有答案都是“否”，那么你就有答案了；i是切割点。但如果你在 $i - 1 \dots i - h$ 、…中找到一个候选，那么你仍然需要检查它是否是切割点，这涉及到检查前面的h个位置。如果在那里找到一个候选，那么你必须从该候选位置再往前检查h个位置，等等。因此，在决定i是否是切割点之前，你可能需要向后查找的距离没有先验界限。也就是说，分块方法不是局部的。

We observed earlier that, for any local chunking method, almost every infinite file $F \in \text{PFE}^{\mathbb{N}}$ has infinitely many cutpoints. Since the LBFS method is not local, this observation cannot be applied to it directly. Nevertheless, one can easily deduce the desired information, that the LBFS method produces infinitely many cutpoints in almost all $F \in \text{PFE}^{\mathbb{N}}$, from the corresponding result for the pure point filter method, which is local. To see this, consider an arbitrary position $i \in \mathbb{N}$. For almost all F , the pure point filter method will have a cutpoint $j > i + h$. This j is a candidate in the LBFS method, so either it is a cutpoint or it is blocked by some cutpoint in the interval from $j - h$ to $j - 1$. In either case, there is a cutpoint $> i$, either j or the one blocking it. Since this holds with probability 1 for each i (and since probability is countably additive), almost all F have infinitely many cutpoints under the LBFS chunking method.

Remark 40. We defined the LBFS method for finite and singly infinite files but not for doubly infinite files. If one applies the same idea to doubly infinite files, it may fail to give a well-defined set of cutpoints. That is, it may not really be a chunking method. The source of this problem is the same as the source of non-locality in the previous remark. To determine whether a position i is a cutpoint, we may have to look farther and farther back in the file. In the case of doubly infinite files, the process may never terminate, so there is no decision whether i is a cutpoint.

Consider, for example, a file $F \in \text{PFE}^{\mathbb{Z}}$ such that $F(i)$ is divisible by c if and only if i is divisible by h . So every h th position is a candidate, but each such position, if it is a cutpoint, will block the next candidate from being a cutpoint. It would be consistent with the LBFS method to say that the cutpoints are all of the positions divisible by $2h$, i.e., every second candidate. Each of these candidates $2nh$ blocks $(2n + 1)h$, but then $(2n + 2)h$ is unblocked and serves as the next cutpoint. But it would be equally consistent to say that the cutpoints are the positions of the form $(2n + 1)h$, the odd multiples of h . Each blocks the next even multiple of h , and then the next odd multiple of h is unblocked and serves as the next cutpoint. The LBFS method gives no way to choose between these two possible selections of cutpoints from among the candidates. One could amplify the method by specifying the choice arbitrarily in all such situations; the result would be a chunking method, but it would not be shift invariant.

Fortunately, the LBFS method works for almost all doubly infinite files. To see this, notice first that, if a file has no candidates in some interval of length h , then the method will determine the cutpoints to the right of that interval. Specifically, the first candidate to the right of the interval is not blocked, because there is no candidate in the preceding h positions. So this first candidate is a cutpoint. Knowing this, one can proceed to the right, inductively determining which candidates are cutpoints and which are blocked. Furthermore, an easy calculation shows that almost all files $F \in \text{PFE}^{\mathbb{Z}}$ have candidate-free intervals of length h arbitrarily far to the left. So, in almost all files, all the cutpoints are uniquely determined.

From now on, we shall work with the LBFS method as though it were a genuine chunking method even for doubly infinite files. That is, we shall ignore the measure-zero set of exceptional files for which the method fails to determine the cutpoints.

Having made the LBFS chunking method applicable to doubly infinite files, by ignoring a set of probability zero, we note that the method is clearly shift-invariant even though it is not local.

Remark 41. In the preceding remark, we used the fact that a candidate-free interval of length h in a doubly infinite file is sufficient to disambiguate the choice of cutpoints to the right. There are other intervals that would serve the same purpose. For example, if $h = 3$ and if a file contains a segment of the form NCCCCNNC, where C means “candidate” and N means “non-candidate,” then the last candidate in this segment will be a cutpoint, and the chunking method to the right of this segment is therefore well-defined. To see that the last C in NCCCCNNC is a cutpoint, suppose not. Then it is blocked by a cutpoint at one of the $h = 3$ preceding positions, which can only be the last of the four consecutive C’s. Then the preceding three C’s are not cutpoints, lest they block the fourth one. But why is the third of the four consecutive C’s not a cutpoint? It’s not blocked by the two preceding C’s (as they’re not cutpoints), nor by the initial N (as a non-candidate is certainly not a cutpoint). This contradiction shows that the last C in NCCCCNNC must be a cutpoint. It is clearly possible to devise analogous examples, and more complicated ones, also for other values of h .

The LBFS chunking method clearly ensures that all chunks have length at least $h + 1$. The following proposition gives basic probabilistic information about the behavior of this method. The slack is more complicated and is treated in the next subsection.

For notational simplicity, we assume henceforth that $|\text{PFE}|$ is divisible by c . In the general case, the following results are still approximately correct and would become exactly correct if c were replaced with $|\text{PFE}|/\lceil |\text{PFE}|/c \rceil$.

Proposition 42. *The LBFS chunking method applied to (almost all) doubly infinite files has the following properties.*

- (1) *Each position is a candidate with probability $1/c$ independently.*
- (2) *The expectation of the chunk length is $h + c = c(1 + k)$.*
- (3) *Each point has the same probability to be a cutpoint, namely $1/(c + h)$.*
- (4) *Any interval of $l \leq h + 1$ positions contains a cutpoint with probability $l/(c + h)$.*
- (5) *The variance of the chunk length is $\text{Var}(L) = c^2 - c$.*

我们之前观察到，对于任何局部分块方法，几乎每个无限文件 $F \in \text{PFE}^{\mathbb{N}}$ 都有无限多个切割点。由于LBFS方法不是局部的，这个观察结果不能直接应用于它。然而，人们可以很容易地从纯点过滤器方法的相应结果中推导出所需信息，即LBFS方法在几乎所有 $F \in \text{PFE}^{\mathbb{N}}$ 中产生无限多个切割点，纯点过滤器方法是局部的。要理解这一点，考虑任意位置 $i \in \mathbb{N}$ 。对于几乎所有的 F ，纯点过滤器方法将有一个切割点 $j > i + h$ 。这个 j 是LBFS方法中的一个候选，所以要么它是切割点，要么被 $j - h$ 到 $j - 1$ 区间中的某个切割点阻塞。在任一情况下，都有一个切割点 $> i$ ，要么是 j ，要么是阻塞它的那个。由于这对每个 i 都成立（并且概率是可数可加的），因此几乎所有 F 在LBFS分块方法下都有无限多个切割点。

备注40. 我们定义了适用于有限文件和单无限文件的LBFS方法，但未定义适用于双无限文件的方法。如果将相同的思想应用于双无限文件，可能无法给出一组定义良好的切点。也就是说，它可能并非真正的分块方法。这个问题与上一条备注中非局部性的来源相同。要确定位置 i 是否为切点，我们可能需要不断向后查看文件中的内容。对于双无限文件，这个过程可能永远不会终止，因此无法判断 i 是否为切点。

例如，考虑一个文件 $F \in \text{PFE}^{\mathbb{Z}}$ ，其中 $F(i)$ 能被 c 整除当且仅当 i 能被 h 整除。因此，每隔 h 个位置都是一个候选，但如果某个这样的位置是切点，它将阻止下一个候选成为切点。按照LBFS方法的说法，切点应该是所有能被 $2h$ 整除的位置，即每两个候选中的一个。这些候选中的每一个 $2nh$ 会阻塞 $\{n\}h$ ，但随后 $\{n\}h$ 将被解锁，并作为下一个切点。但同样，也可以认为切点是形式为 $\{n\}h$ 的位置，即 h 的奇数倍。每个会阻塞下一个 h 的偶数倍，然后下一个 h 的奇数倍将被解锁，并作为下一个切点。LBFS方法没有提供在候选中选择这两种可能的切点的方法。可以通过在所有此类情况下任意指定选择来扩展该方法；结果将是一种分块方法，但它将不具有平移不变性。

幸运的是，LBFS方法适用于几乎所有双无限文件。要看到这一点，首先要注意的是，如果一个文件在长度为 h 的某个区间内没有候选，那么该方法将确定该区间右侧的切点。具体来说，该区间右侧的第一个候选没有被阻塞，因为在该区间之前的 h 个位置上没有候选。因此，这个第一个候选是一个切点。知道了这一点，人们可以向右进行归纳，确定哪些候选是切点，哪些被阻塞。此外，一个简单的计算表明，几乎所有文件 $F \in \text{PFE}^{\mathbb{Z}}$ 都有任意远左侧的无候选区间，长度为 h 。因此，在几乎所有文件中，所有切点都是唯一确定的。

从现在起，我们将像它是一个真正的分块方法一样使用LBFS方法，即使对于双无限文件也是如此。也就是说，我们将忽略那些方法无法确定切点的零测度异常文件集。

通过忽略概率为0的一组，我们将LBFS分块方法应用于双无限文件，我们注意到该方法显然是平移不变的，尽管它不是局部的。

备注41. 在之前的备注中，我们使用了这样一个事实：长度为 h 的无候选区间是双无限文件，足以消除右侧切点的选择歧义。还有其他区间可以实现相同的目的。例如，如果 $h = 3$ ，并且文件包含形式为NCCCCNNC的片段，其中C表示“候选”，N表示“非候选”，那么该片段中的最后一个候选将是切点，因此该片段右侧的分块方法是明确定义的。要证明NCCCCNNC中的最后一个C是切点，假设不是这样。那么它被 $h = 3$ 之前某个位置上的切点阻挡，这只能是四个连续的C中的最后一个。那么前面的三个C不是切点，否则它们会阻挡第四个。但是为什么四个连续的C中的第三个不是切点？它没有被前面的两个C阻挡（因为它们不是切点），也没有被初始的N阻挡（因为非候选肯定不是切点）。这个矛盾表明NCCCCNNC中的最后一个C必须是切点。显然可以设计类似的其他例子，以及更复杂的例子，也适用于其他值的 h 。

LBFS分块方法明确确保所有块长度至少为 $h + 1$ 。以下命题给出了关于此方法行为的基本概率信息。松弛更为复杂，将在下一小节中讨论。

为简化记号，我们假设今后 $|\text{PFE}|$ 能被 c 整除。在一般情况下，以下结果仍然近似正确，并且如果用 $|\text{PFE}|/\lceil |\text{PFE}|/c \rceil$ 替换 c ，则会完全正确。

命题42. 将LBFS分块方法应用于（几乎所有）双无限文件，具有以下性质。

- (1) 每个位置以概率 $1/c$ 独立地成为候选。
- (2) 块长度的期望为 $h + c = c(1 + k)$ 。
- (3) 每个点成为切割点的概率相同，即 $1/(c + h)$ 。
- (4) 任何包含 $l \leq h + 1$ 个位置区间的切割点概率为 $l/(c + h)$ 。
- (5) 块长度的方差是 $\text{Var}(L) = c^2 - c$ 。

Proof. Item 1 was already established in our discussion of the pure point filter method, since the cutpoints of that method are exactly the candidates of LBFS.

In item 2, the summand h represents the blocked positions immediately after a cutpoint (at 0), and c is the expected number of subsequent positions needed to reach a candidate.

That each point has the same probability of being a cutpoint is obvious by shift-invariance. The value of the probability in item 3 follows from the expected chunk length via Kac's theorem, as applied in Proposition 25.

For item 4, recall that an interval of length $\leq h + 1$ can't contain more than one cutpoint. So, as i ranges over the l points in the interval, the events “ i is a cutpoint” are mutually exclusive and have probability $1/(c + h)$ each.

Finally, item 5 simply says that the variance of the chunk length is the same as for the pure point filter method. Informally, this is true because waiting h steps before looking for candidates increases chunk lengths by h but doesn't affect differences between lengths. Formally, simply observe that the probability that a chunk, starting at a particular cutpoint, has length i is 0 for $i \leq h$ and $(1 - \frac{1}{c})^{i-h-1} \frac{1}{c}$ for $i > h$. That is, the probability distribution of the chunk lengths is obtained from that of the pure point filter method by shifting h steps to the right. The shift increases the expectation by h but has no effect on the variance. $\square$

For future reference, it will be convenient to express the variance of L in terms of the cutpoint probability and the parameter h .

Corollary 43. For the LBFS method,

$$\text{Var}(L) = \frac{1}{p^2} - \frac{2h+1}{p} + h^2 + h.$$

Proof. Substitute $c = \frac{1}{p} - h$ into item 5 of the proposition. $\square$

Remark 44. The mutual exclusion used in the proof of item 4 can also be used to obtain the probability $1/(c + h)$ in item 3 without invoking Kac's theorem. Letting p be the probability that a particular position i is a cutpoint, we find that

$$p = \frac{1}{c}(1 - hp).$$

The first factor here, $1/c$, is the probability that i is a candidate. The second factor is the independent probability that it is unblocked. Indeed, for each of the h immediately preceding positions $j = i - h, \dots, i - 1$, there is probability p that j is a cutpoint, and these events are mutually exclusive. So hp is the probability that i is blocked. Independence follows from the observation that whether j is a cutpoint depends only on the file entries at j and to the left, not on the entry at i . Solving the equation above for p , we get $1/(c + h)$.

Of course it is also possible to get the expected chunk length by a direct computation (using formulas from Section 2.3) and the probability distribution described in the proof of item 5.

Remark 45. The number of candidates skipped by the LBFS method, after it finds a cutpoint and before it begins to look for the next candidate, is binomially distributed, for it is the number of “successes” (candidates) in h independent “trials” (positions), each trial having success probability $1/c$. Thus, the expected number of skipped candidates is $h/c = k$. The probability that at least one candidate will be skipped, i.e., that the chunk is larger than what the pure point filter method would produce, is

$$1 - \left(1 - \frac{1}{c}\right)^h \approx 1 - e^{-k}$$

for large c . If, for example, we want the minimum chunk size $h + 1$ to be about half of the average chunk size $c + h$ (as is the case for the local maximum method in Section 6), then we would have $k = 1$ and so the probability that LBFS skips a candidate is approximately $1 - (1/e) \approx 63\%$ – and the probability of skipping at least two candidates is $\approx 26\%$. Thus in this situation, the LBFS method and the pure point filter method differ on a large fraction of the chunks.

4.3. The slack of LBFS chunking

In this subsection, we shall estimate the expected slack of the LBFS method. Before proceeding, we must check that the notion of slack makes sense in this context. Immediately after defining slack (Definition 32), we observed that, although some $F \in \text{PFE}^{\text{M}}$ may fail to have a slack, because the component files F_1 and F_2 have no common non-negative cutpoint, the probability of this event is zero for any local chunking method. Since the LBFS method is not local, a separate argument is needed here, but it is an easy one. With probability 1, the common positive part F^+ of F_1 and F_2 will contain an interval of length h with no candidates and therefore no cutpoints, and it will have a candidate to the right of this interval. The

证明。第1点在我们的纯点过滤器方法讨论中已经确立，因为该方法的切点正是LBFS的候选。

在项目2中，求和项 h 表示在切割点（位置0）之后立即的阻塞位置， c 是到达候选位置所需的预期后续位置数量。

每个点成为切割点的概率相同，这是由平移不变性显而易见的。项目3中的概率值源自于块长度的期望，通过Kac定理应用在命题25中得出。

对于项目4，回想一下，长度为 $\leq h + 1$ 的区间不能包含超过一个切割点。因此，当 i 在区间内的 l 个点中变化时，“ i 是切割点”

这些事件是互斥的，每个事件的概率为 $1/(c + h)$ 。

最后，项目5只是说块长度的方差与纯点过滤器方法相同。非正式地说，这是因为等待 h 步后再寻找候选会增加块长度，但不会影响长度之间的差异。正式地说，只需观察到，一个块从特定的切割点开始，其长度为 i 的概率在 $i \leq h$ 时为0，在 $i > h$ 时为 $(1 - \frac{1}{c})^{i-h-1} \frac{1}{c}$ 。也就是说，块长度的概率分布是通过将纯点过滤器方法的分布向右移动 h 步得到的。这个移动使期望增加 h ，但对方差没有影响。 $\square$

为方便后续参考，用断点概率和参数 h 表示 L 的方差将更为便捷。

推论43. 对于 LBFS 方法,

$$\text{Var}(L) = \frac{1}{p^2} - \frac{2h+1}{p} + h^2 + h.$$

证明。将 $c = \frac{1}{p} - h$ 代入命题的第5项。 $\square$

注释44. 在证明第4项时使用的互斥性，也可以在不调用卡茨定理的情况下用于获得第3项中的概率 $1/(c + h)$ 。令 p 为特定位置 i 是切割点的概率，我们发现，

$$p = \frac{1}{c}(1 - hp).$$

这里的第一个因子 $1/c$ 是 i 是候选的概率。第二个因子是它未被阻塞的独立概率。确实，对于 h 个紧邻的前置位置

$j = i - h \dots i - 1$ ，， j 是切割点的概率为 p ，且这些事件是互斥的。因此 hp 是 i 被阻塞的概率。独立性源于 j 是否是切割点仅取决于 j 及其左侧的文件条目，而与 i 处的条目无关。解上述方程得到 p ，我们得到 $1/(c + h)$ 。

当然，也可以通过直接计算（使用第2.3节的公式）和第5项证明中描述的概率分布来获得预期的块长度。

注释45. LBFS方法在找到一个切割点并开始寻找下一个候选之前跳过的候选数量是二项分布的，因为它是在 h 个独立的“试验”（位置）中“成功”（候选）的数量，每个试验的成功概率为 $1/c$ 。因此，跳过的候选数量的期望值是 $h/c = k$ 。至少会跳过一个候选的概率，即块大于纯点过滤器方法会产生大小，是

$$1 - \left(1 - \frac{1}{c}\right)^h \approx 1 - e^{-k}$$

对于较大的 c 。例如，如果我们希望最小块大小 $h + 1$ 大约是平均块大小 $c + h$ 的一半（如第6节中的局部最大值方法的情况），那么我们会 $k = 1$ ，因此LBFS跳过一个候选的概率大约是 $1 - (1/e) \approx 63\%$ – and，跳过至少两个候选的概率是 $\approx 26\%$ 。因此在这种情况下，LBFS方法和纯点过滤器方法在很大一部分块上存在差异。

4.3. LBFS 分块的松弛

在本小节中，我们将估计LBFS方法的预期松弛。在进行之前，我们必须检查松弛的概念在此上下文中是否有意义。在定义松弛（定义32）之后，我们立即观察到，尽管某些 $F \in \text{PFE}^{\text{M}}$ 可能没有松弛，因为组件文件 F_1 和 F_2 没有共同的非负切割点，但这种事件的概率对于任何局部分块方法是零。由于LBFS方法不是局部的，因此在这里需要一个单独的论证，但它很容易。以概率1， F_1 和 F_2 的共同正部分 F^+ 将包含一个长度为 h 且没有候选且因此没有切割点的区间，并且它在这个区间右侧有一个候选。第一个这样的候选是 $F \in \text{PFE}^{\text{M}}$ 和 1 的共同切割点，因此定义了 $\text{Slack}_2 F^+$ 。由于我们对预期松弛感兴趣，我们可以忽略那些松弛未定义的 F 的度量0集。

first such candidate is a common cutpoint of F_1 and F_2 , and so $\text{Slack}(F)$ is defined. Since we are interested in the expected slack, we can ignore the measure 0 set of F 's whose slack is undefined.

The exact value of the expected slack seems to be difficult to compute, so we shall estimate it from below.

Consider a random $F \in \text{PFE}^{\mathbb{M}}$, giving rise to a merging pair of files $F_1, F_2 \in \text{PFE}^{\mathbb{Z}}$. The slack depends on the location of candidates in the common part F^+ and also on any cutpoints that may be present in F_1^- and F_2^- in the *critical range* of positions $-h, -h+1, \dots, -2, -1$, the last h positions before the merge. These positions are called critical because a cutpoint there, in either file, could block a candidate in F^+ from being a cutpoint of that file and could thus delay the appearance of a common cutpoint.

With bad luck, the delay could be quite large. Suppose, for example, that F^+ has candidates at exactly the positions $i_n = hn$ for all non-negative n , and suppose further that F_1 has a cutpoint in the critical range but F_2 does not. The critical cutpoint in F_1 will block the candidate at 0, so the next cutpoint of F_1 will be at h . That will, in turn, block $2h$, so the next cutpoint is at $3h$, and so forth. In F_2 , on the other hand, 0 is not blocked, so it is a cutpoint, and it blocks h . The next cutpoint in F_2 is $2h$, blocking $3h$, and so forth. Thus, the cutpoints of F_1 (resp. F_2) are hn for odd (resp. even) n , and there are no common cutpoints. (The example doesn't really depend on the assumed precise spacing of the candidates. It would suffice that the distance $i_{n+1} - i_n$ be $\leq h$ and that $i_{n+2} - i_n$ be $> h$ for all n .) As indicated above, the probability of this situation is zero, but there is a non-zero probability that this sort of alternation continues for a large (but finite) number of steps.

We must take such situations into account when estimating the slack, because the delay in getting the files to agree on a cutpoint can greatly increase the slack.

We shall take the delay into account, but only partially. In other words, we shall take too optimistic a view of the possible delay. This is why our estimate of the slack will be low. More specifically, we shall take into account that, if one or both files F_1 and F_2 have a cutpoint in the critical range, then the last of these cutpoints, say at position $-j$ (where $1 \leq j \leq h$), blocks, in at least one of the files, all candidates up to and including $h - j$. If one of the files has a cutpoint z in $[0, h - j]$ that is blocked in the other file, then there cannot be a common cutpoint until at least position $z + h + 1$. We shall pretend that the next candidate after $z + h$ in F^+ is a common cutpoint of the two files and is therefore the slack. In reality, it can happen as in the situation described above that, while one of the files has its candidates up to $z + h$ blocked, the other has a cutpoint in that part of F^+ , which blocks the candidate that we pretend is a common cutpoint. That is why our pretense is too optimistic and our estimate of the expected slack too low.

We define $\text{Slack}'(F)$, for almost all $F \in \text{PFE}^{\mathbb{M}}$, to be

- the first candidate $> z + h$ in F^+ if $z \geq 0$ is a cutpoint of one of the files F_1 and F_2 but is blocked in the other file by a cutpoint in the critical range, and
- $\text{Slack}(F)$ if there is no such z .

Thus, $\text{Slack}'(F)$ is where we optimistically pretend to have the first common cutpoint of F_1 and F_2 . It is our low estimate of $\text{Slack}(F)$. We write S' for $\text{Slack}'/(c + h)$; it is our low estimate of the normalized slack $S = \text{Slack}/(c + h)$.

We compute the expectation of Slack' by dividing the probability space $\text{PFE}^{\mathbb{M}}$ (or rather the measure one subset where the cutpoints of F_1 and F_2 are well defined) into several pieces P_r , computing for each piece its probability p_r and the conditional expectation e_r of Slack' , and finally adding all the products $p_r e_r$ to get the overall expectation $\mathbf{E}(\text{Slack}')$, our lower bound for $\mathbf{E}(\text{Slack})$. There are six pieces, defined as follows; in each case, we describe the conditions on the component files F_1 and F_2 that put $F \in \text{PFE}^{\mathbb{M}}$ into that piece. Remember that F_1^- and F_2^- are the independent negative parts of F_1 and F_2 while F^+ is their common positive part. Remember also that no two cutpoints of a file can be within a distance h of each other, so each of F_1 and F_2 has at most one cutpoint in the critical range.

- (1) Neither F_1 nor F_2 has a cutpoint in the critical range $[-h, -1]$.
- (2) Both F_1 and F_2 have a cutpoint at the same position $-j \in [-h, -1]$.
- (3) F_1 has a cutpoint $-j \in [-h, -1]$ and F^+ has a candidate z that is a cutpoint of F_2 but blocked by $-j$ in F_1 .
- (4) F_1 has a cutpoint $-j \in [-h, -1]$, F_2 has no cutpoint in $[-j, -1]$, and there is no z as in (3).
- (5)–(6) Like (3)–(4) but with the roles of F_1 and F_2 interchanged.

Before treating these pieces individually, we check that they constitute a partition of $\text{PFE}^{\mathbb{M}}$ (minus, as always, the set of measure zero where the cutpoints are not well defined). It is clear that (1) is disjoint from all the others. Observe that, in (3), no point in $[-j, -1]$ can be a cutpoint of F_2 , because such a cutpoint would block z from being a cutpoint of F_2 . This observation and the analogous one in (5) immediately show that all six pieces are disjoint. To see that they cover (almost) all of $\text{PFE}^{\mathbb{M}}$, consider any F for which the cutpoints of F_1 and F_2 are well defined. If it is not in piece (1) or (2), then at least one of F_1 and F_2 has a cutpoint in $[-h, -1]$ and, if both do, then they are not at the same point. We can thus classify F according to which of the files has its critical cutpoint farther to the right. If this is F_1 , then we clearly have (3) or (4), and if it is F_2 then, symmetrically, we have (5) or (6).

Now let us compute the probabilities p_r and the conditional expectations e_r of Slack' for the six pieces in turn. Of course there are only four computations, since (5) and (6) give the same results as (3) and (4) by symmetry.

第一个这样的候选是 F_1 和 F_2 的共同切割点，因此定义了 $\text{Slack}(F)$ 。由于我们对预期松弛感兴趣，我们可以忽略那些松弛未定义的 F 的度量 0 集。

期望松弛的精确值似乎难以计算，因此我们将从下方进行估计。

考虑一个随机 $F \in \text{PFE}^{\mathbb{M}}$ ，由此产生一对文件 $F_1, F_2 \in \text{PFE}^{\mathbb{Z}}$ ，. 松弛取决于候选在公共部分 F^+ 中的位置，也取决于 F_1^- 和 F_2^- 在关键位置范围 $-h, -h+1, \dots, -2, -1$ ，，，，最后 h 个位置在合并之前。这些位置被称为关键，因为在其中任何一个文件中的切割点可能会阻止 F^+ 中的候选成为该文件的切割点，从而可能延迟公共切割点的出现。

运气不好时，延迟可能相当大。例如，假设 F^+ 在所有非负 n 的位置 $i_n = hn$ 处都有候选，并且进一步假设 F_1 在关键范围内有一个切割点，但 F_2 没有切割点。 F_1 中的关键切割点会阻止 0 位置的候选，因此下一个 F_1 的切割点将在 h 处。反过来，它会阻止 $2h$ ，因此下一个切割点在 $3h$ 处，等等。另一方面，在 F_2 中，0 没有被阻止，因此它是一个切割点，并阻止 h 。 F_2 中的下一个切割点是 $2h$ ，阻止 $3h$ ，等等。因此， F_1 （分别 F_2 ）的切割点是奇数（分别偶数） n 的 hn ，并且没有公共切割点。（这个例子并不真正依赖于假设的候选精确间距。只要距离 $i_{n+1} - i_n$ 是 $\leq h$ ，并且 $i_{n+2} - i_n$ 是 $> h$ 对所有 n 就足够了。）如上所述，这种情况的概率为零，但存在非零的概率，这种交替会持续一段很大的（但有限的）步骤数。

在估算松弛时，我们必须考虑这种情况，因为文件就 cutpoint 达成一致时的延迟可能会大大增加松弛。

我们将考虑延迟，但仅部分考虑。换句话说，我们将对可能的延迟过于乐观。这就是为什么我们对松弛的估计会偏低。更具体地说，我们将考虑这一点：如果文件 F_1 和 F_2 中有一个或两个在关键范围内有 cutpoint，那么这些 cutpoints 中的最后一个，比如在位置 $-j$ （其中 $1 \leq j \leq h$ ），块，至少在其中一个文件中，所有候选者直到并包括 $h - j$ 。如果一个文件在 $[0, h - j]$ 中有 cutpoint z ，而在另一个文件中被阻塞，那么直到至少位置 $z + h + 1$ 之前不可能有一个公共 cutpoint。我们将假装文件 F^+ 中 z 之后的下一个候选者 $+h$ 是两个文件的公共 cutpoint，因此就是松弛。实际上，正如上面描述的情况那样，可能会发生这种情况：虽然一个文件中的候选者直到 $z + h$ 被阻塞，但另一个文件在该部分的 F^+ 中有 cutpoint，这阻塞了我们假装的公共 cutpoint。这就是为什么我们的假设过于乐观，我们对预期松弛的估计也过低。

我们定义松弛'(F)，对于几乎所有的 $F \in \text{PFE}^{\mathbb{M}}$ ，为

- 第一个候选 $> z + h$ ，在 F^+ if $z \geq 0$ is 是文件 F_1 和 F_2 的一个切割点，但在另一个文件中被阻塞，在关键范围内有一个切割点，并且，
- 松弛(F) 如果没有这样的 z 。

因此，松弛'(F) 是我们乐观地假设 F_1 和 F_2 的第一个共同切割点。它是我们的低估值松弛(F)。我们用 S' 表示松弛'/($c + h$)；它是我们归一化松弛 $S = \text{Slack}/(c + h)$ 的低估价值。

我们通过将概率空间 $\text{PFE}^{\mathbb{M}}$ （或者更准确地说，是其中一个子集，其中 F_1 和 F_2 的切割点定义良好）分成几个部分 P_r ，计算每个部分的概率 p_r 和松弛'的条件期望 e_r ，最后将所有 $p_r e_r$ 相加得到整体期望 $\mathbf{E}(\text{Slack}')$ ，即 $\mathbf{E}(\text{Slack})$ 的下界。有六个部分，定义如下；在每个情况下，我们描述将 $F \in \text{PFE}^{\mathbb{M}}$ 放入该部分的组件文件 F_1 和 F_2 的条件。记住 F_1^- 和 F_2^- 是 F_1 和 F_2 的独立负部分，而 F^+ 是它们的共同正部分。还要记住，一个文件的两个切割点不能在距离 h 内，因此 F_1 和 F_2 在关键范围内最多只有一个切割点。

- (1) F_1 和 F_2 在关键范围 $[-h, -1]$ 中都没有切割点。
- (2) F_1 和 F_2 在相同位置 $-j \in [-h, -1]$ 都有切割点。
- (3) F_1 有一个切割点 $-j \in [-h, -1]$ ， F^+ 有一个候选 z ，它是 F_2 的切割点，但被 F_1 中的 $-j$ 阻塞。
- (4) F_1 有一个切割点 $-j \in [-h, -1]$ ， F_2 在 $[-j, -1]$ 中没有切割点，并且没有像 (3) 中那样的 z 。
- (5)–(6) 类似 (3)–(4)，但 F_1 和 F_2 的角色互换。

在单独处理这些片段之前，我们检查它们是否构成 $\text{PFE}^{\mathbb{M}}$ 的划分（始终排除那些切割点未定义的测度为零的集合）。很明显，(1) 与其他所有片段都不相交。注意到，在 (3) 中， $[-j, -1]$ 中的任何点都不能是 F_2 的切割点，因为这样的切割点会阻塞 z 成为 F_2 的切割点。这个观察结果和 (5) 中的类似观察结果立即表明所有六个片段都不相交。要看到它们覆盖（几乎）所有的 $\text{PFE}^{\mathbb{M}}$ ，考虑任何 F ，其 F_1 和 F_2 的切割点都定义良好。如果它不在片段 (1) 或 (2) 中，那么 F_1 和 F_2 中至少有一个在 $[-h, -1]$ 中有切割点，如果两个都有，那么它们不在同一点。因此，我们可以根据哪个文件具有其关键切割点更靠右来对 F 进行分类。如果这是 F_1 ，那么我们显然有 (3) 或 (4)，如果它是 F_2 ，那么，对称地，我们有 (5) 或 (6)。

现在让我们逐一计算 Slack' 的概率 p_r 和条件期望 e_r 。当然，实际上只有四次计算，因为 (5) 和 (6) 与 (3) 和 (4) 具有相同的对称结果。

A preliminary observation will be useful in these computations. The first non-negative candidate in a random $F \in \text{PFE}^{\mathbb{M}}$ is exactly the slack of F under the pure point filter method. We have already computed its expectation as $c - 1$. More generally, the first candidate at a position $> i$ in a random file has expectation $c + i$. This follows from the special case already done (where i is -1) by shift invariance.

4.3.1. Piece 1

By Proposition 42, the probability that F_1 has a cutpoint in the critical range of length h is $h/(c + h)$. So the probability that it has no cutpoint there is $c/(c + h)$, and similarly for F_2 . Since the negative parts F_1^- and F_2^- are independent, we get

$$p_1 = \left(\frac{c}{c + h} \right)^2.$$

Because there are no cutpoints in $[-h, -1]$, the first candidate in the non-negative part F^+ will be a common cutpoint of F_1 and F_2 . So the conditional expectation of the slack (and Slack') is the (conditional) expectation of the first non-negative candidate. We put “conditional” in parentheses here, because the condition (1) makes no difference. The condition refers only to the negative parts of the files while the candidate we seek is determined by the non-negative parts. Thus, by the preliminary observation, we have

$$e_1 = c - 1$$

and the contribution of piece (1) to the overall expectation is

$$p_1 e_1 = \frac{c^2(c - 1)}{(c + h)^2}.$$

4.3.2. Piece 2

We split piece (2) into h sub-pieces, according to the value of $j \in [1, h]$, and we compute the probabilities $p_2(j)$, the expectations $e_2(j)$, and the contributions to the overall expectation separately for the sub-pieces.

Consider therefore, a fixed $j \in [1, h]$. The probability that F_1 has a cutpoint at $-j$ is, according to Proposition 42, $1/(c + h)$, and the same for F_2 . Since these events refer only to the negative parts F_1^- and F_2^- , they are independent, and so

$$p_2(j) = \frac{1}{(c + h)^2}.$$

When F_1 and F_2 have cutpoints at $-j$, these cutpoints block candidates up to and including $h - j$. So the slack (and Slack') is the first candidate position $> h - j$. By the preliminary observation, this has expectation

$$e_2(j) = c + h - j.$$

The contribution of sub-piece j to the overall $\mathbf{E}(\text{Slack}')$ is the product $p_2(j)e_2(j)$, and summing over all j we get the contribution of piece (2)

$$\begin{aligned} p_2 e_2 &= \sum_{j=1}^h \frac{1}{(c + h)^2} (c + h - j) \\ &= \frac{h}{c + h} - \frac{1}{(c + h)^2} \frac{h(h + 1)}{2} \\ &= \frac{ch + \frac{1}{2}h^2 - \frac{1}{2}h}{(c + h)^2} \\ &= \frac{h}{(c + h)^2} \left(c + \frac{h - 1}{2} \right). \end{aligned}$$

We note that p_2 has the simple formula $h/(c + h)^2$, and this combines with the preceding formula for $p_2 e_2$ to give $e_2 = c + (h - 1)/2$. The latter has the intuitive interpretation that j , being random in the interval $[1, h]$ is on average $(h + 1)/2$ and so the cutpoints at $-j$ in both files block candidates up to and including, on average, $(h - 1)/2$. So the first common cutpoint will be the first candidate $> (h - 1)/2$, and its expectation is, by the preliminary observation, $c + (h - 1)/2$. Using the average j instead of considering each j individually can be justified, but the justification ultimately amounts to the computation we did above, treating each j separately and adding the resulting contributions.

在进行这些计算之前，一个初步的观察将很有用。随机 $F \in \text{PFE}^{\mathbb{M}}$ 中的第一个非负候选正好是 F 在纯点过滤器方法下的松弛。我们已经计算了它的期望为 $c - 1$ 。更一般地，随机文件中位置 $> i$ 处的第一个候选的期望为 $c + i$ 。这源于已经完成的特殊情况（其中 i 是 -1 ），通过平移不变性得出。

4.3.1. 片段 1

根据命题 42， F_1 在长度为 h 的关键范围内具有切割点的概率是 $h/(c + h)$ 。因此，它在其中没有切割点的概率是 $c/(c + h)$ ，对于 F_2 也是如此。由于负部分 F_1^- 和 F_2^- 是独立的，我们得到

$$p_1 = \left(\frac{c}{c + h} \right)^2.$$

由于 $[-h, -1]$ 中没有切割点，非负部分 F^+ 中的第一个候选将是 F_1 和 F_2 的公共切割点。因此，松弛（和 Slack'）的条件期望是第一个非负候选的期望。我们在这里将“条件”放在括号中，因为条件 (1) 没有区别。条件仅指文件的负部分，而我们寻求的候选是由非负部分决定的。因此，根据初步观察，我们有

$$e_1 = c - 1$$

并且，片段 (1) 对整体期望的贡献是

$$p_1 e_1 = \frac{c^2(c - 1)}{(c + h)^2}.$$

4.3.2. 片段 2

我们将片段 (2) 根据 $j \in [1, h]$ 的值分成 h 个子片段，并分别计算子片段的概率 $p_2(j)$ 、期望 $e_2(j)$ 以及对整体期望的贡献。

因此，考虑一个固定的 $j \in [1, h]$ 。根据命题 42， F_1 在 $-j$ 处有切割点的概率是 $1/(c + h)$ ， F_2 也是如此。由于这些事件仅涉及负部分 F_1^- 和 F_2^- ，它们是独立的，因此

$$p_2(j) = \frac{1}{(c + h)^2}.$$

当 F_1 和 F_2 在 $-j$ 处有切割点时，这些切割点会阻塞从候选到 $h - j$ 的所有候选。因此，松弛（和 Slack'）是第一个候选位置 $> h - j$ 。根据初步观察，这的期望是

$$e_2(j) = c + h - j.$$

子片段 j 对整体 $\mathbf{E}(\text{松弛})$ 的贡献是 $p_2(j)e_2(j)$ 的乘积，对所有 j 求和后得到片段 (2) 的贡献

$$\begin{aligned} p_2 e_2 &= \sum_{j=1}^h \frac{1}{(c + h)^2} (c + h - j) \\ &= \frac{h}{c + h} - \frac{1}{(c + h)^2} \frac{h(h + 1)}{2} \\ &= \frac{ch + \frac{1}{2}h^2 - \frac{1}{2}h}{(c + h)^2} \\ &= \frac{h}{(c + h)^2} \left(c + \frac{h - 1}{2} \right). \end{aligned}$$

我们注意到 p_2 有简单的公式 $h/(c + h)^2$ ，并且这与前面的 $p_2 e_2$ 公式结合给出 $e_2 = c + (h - 1)/2$ 。后者具有直观的解释，即 j 在区间 $[1, h]$ 中是随机的，平均值为 $(h + 1)/2$ ，因此在两个文件中，切割点在 $-j$ 处的候选块直到平均 $(h - 1)/2$ 。因此，第一个公共切割点将是第一个候选 $> (h - 1)/2$ ，并且根据初步观察，其期望值为 $c + (h - 1)/2$ 。使用平均 j 而不是单独考虑每个 j 是可以证明的，但最终证明归结为上述分别处理每个 j 并将结果贡献相加的计算。

4.3.3. Piece 3

With j and z as in (3), notice that $0 \leq z \leq h - j$, because z is blocked in F_1 by $-j$. Since z is a cutpoint in F_2 , it blocks all candidates up to and including $h + z$. So to find a common non-negative cutpoint of F_1 and F_2 , we must look for candidates in F^+ strictly to the right of $h + z$. As indicated above, we will pretend that the first such candidate is a common cutpoint; formally, this means that we deal with $\text{Slack}'(F)$ instead of $\text{Slack}(F)$.

As in the previous subsection, we split the piece under consideration into sub-pieces, this time indexed by both j and z , where j ranges from 1 to h and z ranges from 0 to $h - j$.

We begin by computing the probability $p_3(j, z)$ of the sub-piece indexed by j and z . The probability that F_1 has a cutpoint at $-j$ is, by Proposition 42, $1/(c + h)$. The probability that F_2 has a cutpoint at z is the same. And these two events are independent because the former depends only on F_1^- while the second depends only on F_2 , i.e., on F_2^- and F^+ , and these parts of F are independent. We need not consider separately the requirement in (3) that z is blocked by $-j$ in F_1 ; this is already covered by the fact that j and z are in the appropriate ranges, specifically that $-j < z \leq h - j$. So we infer that

$$p_3(j, z) = \frac{1}{(c + h)^2}.$$

The conditional expectation of Slack' is the expectation of the first cutpoint of F^+ strictly after $h + z$, so, by our preliminary observation,

$$e_3(j, z) = c + h + z.$$

Thus, the contribution of this sub-piece to the overall $\mathbf{E}(\text{Slack}')$ is

$$p_3(j, z)e_3(j, z) = \frac{c + h + z}{(c + h)^2}.$$

To get the contribution of the whole piece (3) to $\mathbf{E}(\text{Slack}')$, we must sum this over z from 0 to $h - j$ and then over j from 1 to h . Since the expression being summed is independent of j , we prefer to carry out the sum over j first; converting the limits of summation to the new order, we get

$$\begin{aligned} p_3e_3 &= \sum_{z=0}^{h-1} \sum_{j=1}^{h-z} \frac{c + h + z}{(c + h)^2} \\ &= \sum_{z=0}^{h-1} \frac{(c + h + z)(h - z)}{(c + h)^2}. \end{aligned}$$

We postpone simplifying the sum over z in order to make it easier to combine with the forthcoming result from piece (4).

4.3.4. Piece 4

In the description of piece (4), the requirement that there is no z as in (3) is equivalent to saying that F_2 has no cutpoint in the interval $[0, h - j]$. Indeed, a cutpoint in that interval would be a candidate of F^+ and would be blocked in F_1 by $-j$, whereas a cutpoint farther to the left would not be in F^+ while one farther to the right would not be blocked by $-j$.

Combining this requirement with the other requirement in (4) that F_2 have no cutpoint in $[-j, -1]$, we find that we can restate (4) as follows: F_1 has a cutpoint $-j$ in the critical range, but F_2 has no cutpoint in the interval $[-j, h - j]$ of length $h + 1$.

As before, we divide our piece into sub-pieces, according to the value of j , and we do the calculation for each j separately.

To calculate the probability $p_4(j)$ of the sub-piece indexed by j , we again apply Proposition 42. The probability that F_1 has a cutpoint at $-j$ is $1/(c + h)$. The probability that F_2 has a cutpoint in $[-j, h - j]$ is $(h + 1)/(c + h)$, so the complementary probability, that it has no cutpoint in this interval, is $(c - 1)/(c + h)$. Since F_2 is independent of the negative part F_1^- of F_1 , it follows that we can simply multiply the two probabilities to get

$$p_4(j) = \frac{1}{c + h} \cdot \frac{c - 1}{c + h} = \frac{c - 1}{(c + h)^2}.$$

Notice that neither F_1 nor F_2 has a cutpoint in $[-j + 1, h - j]$. We already pointed this out for F_2 , with the slightly longer interval $[-j, h - j]$, but it also holds for F_1 because this file has a cutpoint at $-j$, which blocks the next h positions. Therefore, the next candidate in F^+ strictly to the right of $h - j$ will be a common cutpoint and will thus be the slack (and Slack') of F . The expectation is, by our preliminary observation,

$$e_4(j) = c + h - j.$$

Therefore, the contribution of this sub-piece to $\mathbf{E}(\text{Slack}')$ is

$$p_4(j)e_4(j) = \frac{(c - 1)(c + h - j)}{(c + h)^2}.$$

4.3.3. 片段 3

对于 (3) 中的 j 和 z , 请注意 $0 \leq z \leq h - j$, 因为 z 在 F_1 中被 $-j$ 阻塞。由于 z 是 F_2 中的一个切割点, 它阻塞所有候选直到包括 $h + z$ 。因此, 为了找到 F_1 和 F_2 的公共非负切割点, 我们必须在 F^+ 中严格位于 $h + z$ 右侧寻找候选。如上所述, 我们将假装第一个这样的候选是公共切割点; 形式上, 这意味着我们处理 $\text{Slack}'(F)$ 而不是 $\text{Slack}(F)$ 。

与上一小节相同, 我们将所考虑的片段分割成子片段, 这次由 j 和 z 索引, 其中 j 的范围是 1 到 h , z 的范围是 0 到 $h - j$ 。

我们首先计算由 j 和 z 索引的子片段的概率 $p_3(j, z)$ 。根据命题 42, F_1 在 $-j$ 处有切割点的概率是 $1/(c + h)$ 。 F_2 在 z 处有切割点的概率相同。这两个事件是独立的, 因为前者仅取决于 F_1^- , 而后者仅取决于 F_2 , 即仅取决于 F_2^- 和 F^+ , 而 F 的这些部分是独立的。我们无需单独考虑 (3) 中 z 被 $-j$ 在 F_1 中阻塞的要求; 这已经包含在 j 和 z 处于适当范围的事实中, 具体来说就是 $-j < z \leq h - j$ 。因此我们推断

$$p_3(j, z) = \frac{1}{(c + h)^2}.$$

Slack' 的条件期望是 F^+ 在 $h + z$ 之后严格的第一切割点的期望, 因此, 根据我们的初步观察,

$$e_3(j, z) = c + h + z.$$

因此, 这个子片段对整体 $\mathbf{E}(\text{Slack}')$ 的贡献是

$$p_3(j, z)e_3(j, z) = \frac{c + h + z}{(c + h)^2}.$$

要得到整个片段 (3) 对 $\mathbf{E}(\text{Slack}')$ 的贡献, 我们必须对 z 从 0 到 $h - j$ 求和, 然后对 j 从 1 到 h 求和。由于求和的表达式与 j 无关, 我们更倾向于先对 j 求和; 将求和的上下限转换为新的顺序, 我们得到

$$\begin{aligned} p_3e_3 &= \sum_{z=0}^{h-1} \sum_{j=1}^{h-z} \frac{c + h + z}{(c + h)^2} \\ &= \sum_{z=0}^{h-1} \frac{(c + h + z)(h - z)}{(c + h)^2}. \end{aligned}$$

我们推迟对 z 的求和简化, 以便更容易将其与即将到来的片段 (4) 的结果结合起来。

4.3.4. 片段 4

在片段 (4) 的描述中, 没有 z (如 (3)) 的要求相当于说 F_2 在区间 $[0, h - j]$ 中没有切割点。确实, 该区间内的切割点会是 F^+ 的候选, 并且会被 F_1 中的 $-j$ 阻挡, 而更靠左的切割点则不在 F^+ 中, 而更靠右的切割点则不会被 $-j$ 阻挡。

将此要求与 (4) 中的另一要求相结合, 即 F_2 在 $[-j, -1]$ 中没有切割点, 我们发现可以将 (4) 重新表述如下: F_1 在关键区间内有一个切割点 $\{j\}$, 但 F_2 在长度为 h 的区间 $[-j, h - j]$ 中 + 1 没有切割点。

如前所述, 我们根据 j 的值将片段分成子片段, 并对每个 j 分别进行计算。

要计算索引为 j 的子片段的概率 $p_4(j)$, 我们再次应用命题 42。 F_1 在 $-j$ 处有切割点的概率是 $1/(c + h)$ 。 F_2 在 $[-j, h - j]$ 是 $(h + 1)/(c + h)$ 中有切割点的概率, 因此其互补概率, 即在该区间内没有切割点的概率, 是 $(c - 1)/(c + h)$ 。由于 F_2 独立于 F_1 的负部分 F_1^- , 因此我们可以简单地将这两个概率相乘得到

$$p_4(j) = \frac{1}{c + h} \cdot \frac{c - 1}{c + h} = \frac{c - 1}{(c + h)^2}.$$

请注意, F_1 和 F_2 在 $[-j + 1, h - j]$ 中都没有切割点。我们之前已经针对 F_2 和稍长的区间 $[-j, h - j]$ 指出了这一点, 但它同样适用于 F_1 , 因为该文件在 $-j$ 处有一个切割点, 这会阻塞接下来的 h 个位置。因此, 在 F^+ 中位于 $h - j$ 严格右侧的下一个候选将是一个公共切割点, 从而成为 F 的松弛 (和 Slack')。根据我们的初步观察, 期望是,

$$e_4(j) = c + h - j.$$

因此, 该子片段对 $\mathbf{E}(\text{Slack}')$ 的贡献是

$$p_4(j)e_4(j) = \frac{(c - 1)(c + h - j)}{(c + h)^2}.$$

To get the contribution of piece (4) to $\mathbf{E}(\text{Slack}')$, we must sum over j ; it is convenient to postpone actually doing this summation until after we combine the results from pieces (3) and (4), but we prepare for the job of combining these pieces by changing the summation variable from j to $z = h - j$. (In contrast to previous computations, z does not represent a position here. It is merely a formal variable. We chose the letter z to facilitate combination with the sum from piece (3).)

$$p_4 e_4 = \sum_{j=1}^h \frac{(c-1)(c+h-j)}{(c+h)^2} = \sum_{z=0}^{h-1} \frac{(c-1)(c+z)}{(c+h)^2}.$$

4.3.5. Assembling the pieces

Since all the contributions to $\mathbf{E}(\text{Slack}')$ computed above had $(c+h)^2$ in their denominators, we put those denominators aside for the time being. That is, we assemble the contributions to $(c+h)^2 \mathbf{E}(\text{Slack}')$.

As indicated above, we first combine the contributions from pieces (3) and (4). We get

$$\begin{aligned} & \sum_{z=0}^{h-1} (c+h+z)(h-z) + \sum_{z=0}^{h-1} (c-1)(c+z) \\ &= \sum_{z=0}^{h-1} (ch + h^2 + c^2 - c - z(z+1)) \\ &= ch^2 + h^3 + c^2h - ch - \sum_{m=1}^h m(m-1) \\ &= ch^2 + h^3 + c^2h - ch - \frac{h^3 - h}{3}, \end{aligned}$$

where we introduced the new summation variable $m = z + 1$ and then applied one of the summation identities from Section 2.3. Simplifying slightly by combining the two h^3 terms, and doubling the result so as to account for pieces (5) and (6) as well as (3) and (4), we get as the contribution of these four pieces to $(c+h)^2 \mathbf{E}(\text{Slack}')$

$$2ch^2 + \frac{4}{3}h^3 + 2c^2h - 2ch + \frac{2}{3}h.$$

Finally, adding the contributions $c^3 - c$ from piece (1) and $ch + \frac{1}{2}h^2 - \frac{1}{2}h$ from piece (2), and restoring the $(c+h)^2$ denominator, we get

$$\mathbf{E}(\text{Slack}') = \left(2ch^2 + \frac{4}{3}h^3 + 2c^2h - ch + \frac{1}{6}h + c^3 - c^2 + \frac{1}{2}h^2 \right) / (c+h)^2.$$

Recall that $\text{Slack}'(F) \leq \text{Slack}(F)$ for all F , with the usual exception of a measure zero set where the chunking method doesn't work or too few cutpoints exist. Thus, our calculation proves the following lower bound for the expected slack.

Proposition 46. *The expectation of the slack $\mathbf{E}(\text{Slack})$ of the LBFS method with parameters c and h is greater than or equal to*

$$\left(2ch^2 + \frac{4}{3}h^3 + 2c^2h - ch + \frac{1}{6}h + c^3 - c^2 + \frac{1}{2}h^2 \right) / (c+h)^2.$$

The expectation $\mathbf{E}(S)$ of the normalized slack is the same except that the denominator $(c+h)^2$ is replaced with $(c+h)^3$, because, by Proposition 42, $c+h$ is the expectation of the chunk length.

When, as is usually the case in practice, c and h are fairly large numbers, then, since the coefficients in our formulas are not particularly large, we can approximate the formulas by keeping only the terms of highest degree.

Corollary 47. *For large c and h , the expectation of the normalized slack is asymptotically bounded below by*

$$\left(2ch^2 + \frac{4}{3}h^3 + 2c^2h + c^3 \right) / (c+h)^3.$$

In terms of the parameter $k = h/c$, this asymptotic lower bound is

$$\left(\frac{4}{3}k^3 + 2k^2 + 2k + 1 \right) / (1+k)^3.$$

要得到片段 (4) 对 $\mathbf{E}(\text{Slack}')$ 的贡献，我们必须对 j 求和；为了方便，我们推迟实际进行这个求和，直到我们结合片段 (3) 和 (4) 的结果之后，但我们通过将求和变量从 j 改为 $z = h - j$ 来为结合这些片段做准备。（与之前的计算不同，这里的 z 不代表一个位置。它只是一个形式变量。我们选择字母 z 是为了方便与片段 (3) 的求和结合。）

$$p_4 e_4 = \sum_{j=1}^h \frac{(c-1)(c+h-j)}{(c+h)^2} = \sum_{z=0}^{h-1} \frac{(c-1)(c+z)}{(c+h)^2}.$$

4.3.5. 组装各部分

由于 $\mathbf{E}(\text{Slack}')$ 计算出的所有贡献项的分母中都有 $(c+h)^2$ ，我们将这些分母暂时搁置。也就是说，我们组装对 $(c+h)^2 \mathbf{E}(\text{Slack}')$ 的贡献项。

如上所述，我们首先组合来自 (3) 和 (4) 的贡献项。我们得到

$$\begin{aligned} & \sum_{z=0}^{h-1} (c+h+z)(h-z) + \sum_{z=0}^{h-1} (c-1)(c+z) \\ &= \sum_{z=0}^{h-1} (ch + h^2 + c^2 - c - z(z+1)) \\ &= ch^2 + h^3 + c^2h - ch - \sum_{m=1}^h m(m-1) \\ &= ch^2 + h^3 + c^2h - ch - \frac{h^3 - h}{3}, \end{aligned}$$

其中我们引入了新的求和变量 $m = z + 1$ ，然后应用了第 2.3 节中的一个求和公式。通过合并两个 h^3 项并使结果翻倍（以便考虑 (5) 和 (6) 以及 (3) 和 (4)），我们得到这四个部分的贡献项为 $(c+h)^2 \mathbf{E}(\text{Slack}')$

$$2ch^2 + \frac{4}{3}h^3 + 2c^2h - 2ch + \frac{2}{3}h.$$

最后，将来自片段(1)的 $c^3 - c$ 贡献和来自片段(2)的 $ch + \frac{1}{2}h^2 - \frac{1}{2}h$ 贡献相加，并恢复 $(c+h)^2$ 分母，我们得到

$$\mathbf{E}(\text{Slack}') = \left(2ch^2 + \frac{4}{3}h^3 + 2c^2h - ch + \frac{1}{6}h + c^3 - c^2 + \frac{1}{2}h^2 \right) / (c+h)^2.$$

回想一下，对于所有 F ，通常有一个几乎为0的集合除外，在该集合中分块方法不适用或切点太少。因此，我们的计算证明了期望松弛的下界。

命题46. *LBFS方法（参数为 c 和 h ）的松弛期望 $E(\text{Slack})$ 大于或等于*

$$\left(2ch^2 + \frac{4}{3}h^3 + 2c^2h - ch + \frac{1}{6}h + c^3 - c^2 + \frac{1}{2}h^2 \right) / (c+h)^2.$$

标准化松弛的期望 $\mathbf{E}(S)$ 相同，只是分母 $(c+h)^2$ 被替换为 $(c+h)^3$ ，因为根据命题42， $c+h$ 是块长度的期望。

当，如实践中通常情况那样， c 和 h 是相当大的数时，由于我们公式中的系数并不特别大，我们可以通过仅保留最高次项来近似公式。

推论47. *对于较大的 c 和 h ，标准化松弛的期望渐近下界为*

$$\left(2ch^2 + \frac{4}{3}h^3 + 2c^2h + c^3 \right) / (c+h)^3.$$

在参数 $k = h/c$ 方面，这个渐近下界是

$$\left(\frac{4}{3}k^3 + 2k^2 + 2k + 1 \right) / (1+k)^3.$$

In particular, for the LBFS method with $k = 1/4$ (the choice proposed in [20]), we find that the expectation of the normalized slack is asymptotically at least 0.842. Here is a table of values of the asymptotic lower bound in the corollary, as k ranges from 0 to 1 in steps of $1/10$.

k	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
$\mathbf{E}(S)$	0.92	0.86	0.83	0.80	0.79	0.78	0.78	0.78	0.79	0.79

This function of k attains its minimum (over positive values of k) at $k = 1/\sqrt{2}$, the minimum value being

$$\frac{8}{3} \left(1 - \frac{1}{\sqrt{2}} \right) \approx 0.781049.$$

Corollary 48. For large c and h , the expectation of the normalized slack of the LBFS method is at least 0.781.

Remark 49. The preceding estimates do not imply that the normalized slack of the LBFS method is optimized (i.e., minimized) by choosing k to be $1/\sqrt{2}$. This choice optimizes (for large c and h) the expectation of S' , but the difference between Slack' and Slack , and thus also the difference between the normalizations S' and S , increase as k increases, and so we would expect the optimum of $\mathbf{E}(S)$ to occur at a value of k somewhat smaller than $1/\sqrt{2}$.

To see why the difference between Slack' and Slack should grow with k , recall that Slack' is what Slack becomes if we pretend that files do not have cutpoints in a certain interval whose length is, on average, about $h/2$. The probability that this pretense is an error, i.e., the probability that there is a cutpoint in such an interval, is near

$$\frac{h/2}{c+h} = \frac{k/2}{1+k},$$

which is an increasing function of k for positive k .

Remark 50. The preceding observations suggest the question of the behavior of $\mathbf{E}(S)$ as $k \rightarrow \infty$. The lower bound $\mathbf{E}(S')$ computed above approaches $4/3$, but we expect $\mathbf{E}(S)$ to be significantly larger. Here is a rough argument to show that $\mathbf{E}(S)$ is approximately proportional to k^2 for large k .

Since k is large, h is much larger than c , and so the cutpoints in a random file will be spaced approximately h positions apart. Indeed, the LBFS method always skips h positions after a cutpoint, but then it will very quickly, in about c more steps, find a candidate, which will serve as the next cutpoint. Notice that the h skipped positions will usually include a large number (approximately k) of blocked candidates.

Thus, an interval of length h is very likely to contain a cutpoint, but where in the interval that cutpoint lies, or equivalently the remainder of the cutpoint modulo h , is essentially random, being determined by some event in the distant past (where there were h consecutive non-candidates or some other special configuration of file entries that makes the LBFS cutpoints well-defined thereafter) and the small (relative to h) differences between h and the actual chunk lengths.

In particular, in the model PFE^{M} of merging files, we expect F_1 and F_2 to have cutpoints in the critical range $[-h, -1]$, and we expect the location of these cutpoints to be uniformly randomly distributed in this range. Now when two points are chosen at random from an interval, the expectation of the smaller (resp. larger) is at the left (resp. right) trisection point of the interval. So, on average, one of our files has a cutpoint at $-2h/3$ and the other at $-h/3$. (This use of averages is one of several reasons why this is a rough argument.) The next cutpoints after these will come about h positions later, i.e., near $h/3$ and $2h/3$, respectively.

Our optimistic estimate for the slack takes into account that one file's cutpoint at $h/3$ blocks the other's cutpoint at $2h/3$ from being a common cutpoint, but it then assumes that the next cutpoint of the former file, h units past the cutpoint at $h/3$, is a common cutpoint. So it pretends to see a common cutpoint near $4h/3$. This accounts, since the chunk length is essentially h , for the value $4/3$ that we found for the expectation of the normalized slack.

But the assumption that there is a common cutpoint near $4h/3$ is usually wrong; one file has a cutpoint near $2h/3$ blocking any candidate near $4h/3$. Were it not for random fluctuations, the two files would alternate cutpoints, one having cutpoints near $(n + \frac{1}{3})h$ and the other near $(n + \frac{2}{3})h$ for all n , and there would never be a common cutpoint. In reality, the positions of the cutpoints modulo h will slowly drift as we proceed farther to the right in the file. How long will it take them to drift to a collision, i.e., to a common cutpoint?

The distance between the cutpoints is initially (i.e., just before the merge at position 0) proportional to h ; on average it is $h/3$. From one cutpoint to the next, each increases, modulo h , by a random amount of size approximately c . So the distance between them increases or decreases randomly, by amounts roughly proportional to c . So the number of steps needed for the distance to become zero can be approximated by the number of steps needed for a random walk, in steps of length roughly $c = h/k$, to cover a distance proportional to h . But the distance covered by a random walk is known to be proportional to the size of the step times the square root of the number of steps. So the number of steps needed will be roughly proportional to k^2 . Since each step of the random walk corresponds to advancing a distance roughly equal to the chunk length in the file, the number of steps is approximately the normalized slack. Thus, we expect the normalized slack, for large k , to be roughly proportional to k^2 .

特别地，对于使用 $k = 1/4$ （在 [20]中提出的选择）的LBFS方法，我们发现标准化松弛的期望渐近至少为0.842。这里是一个引理中渐近下界的数值表，当 k 以 $1/10$ 为步长从0变化到1时。

k	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
$\mathbf{E}(S)$	0.92	0.86	0.83	0.80	0.79	0.78	0.78	0.78	0.79	0.79

该函数 k 在 $k = 1/\sqrt{2}$ 处取得其最小值（在正值的 k 上），最小值为

$$\frac{8}{3} \left(1 - \frac{1}{\sqrt{2}} \right) \approx 0.781049.$$

推论 48. 对于较大的 c 和 h ，LBFS 方法的标准化松弛的期望至少为 0.781。

注释 49. 上述估计并不意味着通过选择 k 为 $1/\sqrt{2}$ 可以优化（即最小化）LBFS 方法的标准化松弛。这种选择优化了（对于较大的 c 和 h ） S' 的期望，但 Slack' 与 Slack 以及 S' 与 S 之间的差异随着 k 的增加而增加，因此我们预计 $\mathbf{E}(S)$ 的最优值出现在比 $1/\sqrt{2}$ 稍小的 k 值处。

要理解为什么 Slack' 与 Slack 之间的差异会随着 k 的增长而增大，回想一下，如果我们将文件在某个长度平均约为 $h/2$ 的区间内假装没有切点， Slack' 就是 Slack 会变成的样子。这种假装是错误的概率，即该区间内存在切点的概率，接近

$$\frac{h/2}{c+h} = \frac{k/2}{1+k},$$

这是一个关于正 k 的递增函数。

注释50. 前面的观察表明了 $\mathbf{E}(S)$ 在 $k \rightarrow \infty$ 时的行为问题。上面计算的 $\mathbf{E}(S')$ 下界趋近于 $4/3$ ，但我们预期 $\mathbf{E}(S)$ 会显著更大。这里有一个粗略的论证，表明 $\mathbf{E}(S)$ 对于大的 k 大致与 k^2 成正比。

由于 k 很大， h 远大于 c ，因此随机文件中的切点将大致相隔 h 个位置。实际上，LBFS方法在切点后总是跳过 h 个位置，但随后它会非常快，大约再经过 c 步，找到一个候选点，该候选点将作为下一个切点。请注意，被跳过的 h 个位置通常会包含大量（大约 k 个）被阻塞的候选点。

因此，一个长度为 h 的区间很可能包含一个切割点，但在该区间中切割点位于何处，或者换句话说，切割点模 h 的余数，基本上是随机的，它由过去某个事件决定（当时有 h 个连续的非候选或文件条目的其他特殊配置，使得 LBFS 切割点在此之后得以很好地定义）以及 h 与实际块长度之间的微小差异（相对于 h ）。

在合并文件的模型 PFE^{M} 中，我们期望 F_1 和 F_2 在关键区间 $[-h, -1]$ 中有切点，并且期望这些切点的位置在这个区间内均匀随机分布。现在当从区间中随机选择两个点时，较小（resp. 较大）的期望位于区间的左（resp. 右）三等分点。因此，平均而言，我们的一个文件在 $-2h/3$ 处有一个切点，另一个文件在 $-h/3$ 处有一个切点。（这种使用平均值的方法是导致这个论证粗糙的几个原因之一。）这些切点之后的位置将在 h 位置之后出现，即分别接近 $h/3$ 和 $2h/3$ 。

我们对松弛的乐观估计考虑了这样一个事实：一个文件在 $h/3$ 块处会阻塞另一个文件在 $2h/3$ 处成为公共切割点的切割点，但它随后假设前者文件的下一个切割点，在 $h/3$ 处的切割点 h 个单位之后，是一个公共切割点。因此，它假装在 $4h/3$ 附近看到一个公共切割点。由于块长度基本上是 h ，这解释了我们为标准化松弛的期望找到的值 $4/3$ 。

但是，假设在 $4h/3$ 附近存在一个公共切割点通常是错误的；一个文件在 $2h/3$ 附近有一个切割点会阻塞任何在 $4h/3$ 附近的候选点。如果不是因为随机波动，这两个文件会交替切割点，一个在 $(n + \frac{1}{3})h$ 附近有切割点，另一个在 $(n + \frac{2}{3})h$ 附近有切割点，对于所有 n 都是这样，并且永远不会有一个公共切割点。实际上，随着我们向文件右侧移动，切割点的位置模 h 会缓慢漂移。它们需要多长时间才能漂移到碰撞，即漂移到公共切割点？

切点之间的距离最初（即合并位置0之前）与 h 成正比；平均而言是 $h/3$ 。从一个切点到下一个切点，每个切点增加的量（模 h ）大约是大小约为 c 的随机量。因此它们之间的距离随机增加或减少，增减量大致与 c 成正比。因此，距离变为零所需的步数可以近似为步长大致为 $c = h/k$ 的随机游走覆盖与 h 成正比的距离所需的步数。但已知随机游走覆盖的距离与步长的大小乘以步数的平方根成正比。因此所需的步数将大致与 k^2 成正比。由于随机游走中的每一步对应于在文件中前进大致等于块长度的距离，因此步数大约等于归一化松弛。因此，我们预期对于大的 k ，归一化松弛大致与 k^2 成正比。

4.4. The reverse slack of LBFS chunking

The definition of the cutpoints in LBFS chunking is not symmetrical with respect to left and right, so there is no reason to expect the $\leftarrow$ slack of this chunking method to equal the $\rightarrow$ slack. We therefore calculate the reverse slack here. This turns out to be easier than the preceding calculation of the $\rightarrow$ slack, in that we can obtain an exact answer rather than only a lower bound. The reason the reverse slack is easier to compute is that, when we have a candidate in the common part F^- of the two files F_1 and F_2 , the question whether it is a cutpoint depends only on F^- , not on the diverged, independent F_1^+ and F_2^+ . We can therefore obtain simple formulas for the probability that the reverse slack of a file is s .

This probability is the product of two factors, namely the probability that $-s$ is a cutpoint (in F^-) and the conditional probability, given that $-s$ is a cutpoint, that there is no other cutpoint in the interval $[-s + 1, 0]$. The first factor here is just the cutpoint probability, already computed as $1/(c + h)$ in Proposition 42. For the second factor, we must consider two cases, depending on the relative size of s and h . If $s \leq h$, then the second factor is 1, because the interval $[-s + 1, 0]$ is within the range where $-s$ blocks all candidates from being cutpoints. If $s > h$, however, then $-s$ blocks candidates only up to and including $-s + h$ and so we must still consider possible cutpoints in the interval $[-s + h + 1, 0]$. Notice that there is a cutpoint in this interval if and only if there is a candidate there, because the first candidate there, if any, will be a cutpoint. Thus, the second factor for our computation can be expressed as the probability, conditional on a cutpoint at $-s$, of having no candidates among the $s - h$ positions in $[-s + h + 1, 0]$. But the events “ i is a candidate” for such positions i are independent of each other and of the cutpoint at $-s$. Each of these events has probability $1/c$, so the probability that none of them occurs is $(1 - \frac{1}{c})^{s-h}$. Therefore, the probability that s is the reverse slack is, when $s > h$, given by $(1 - \frac{1}{c})^{s-h}/(c + h)$.

Therefore, the average $\leftarrow$ slack for the LBFS chunking method is given by

$$\sum_{s=0}^h s \frac{1}{c+h} + \sum_{s=h+1}^{\infty} s \frac{1}{c+h} \left(1 - \frac{1}{c}\right)^{s-h}.$$

A routine calculation, using the formulas in Section 2.3, yields the explicit form of the $\leftarrow$ slack:

$$\frac{h(h+1)}{2(c+h)} + c - 1 = \frac{h^2 - h + 2c^2 - 2c + 2ch}{2(c+h)}.$$

The normalized reverse slack, obtained by dividing this result by the expectation $c + h$ of the chunk length, is

$$\frac{h^2 - h + 2c^2 - 2c + 2ch}{2(c+h)^2} \approx \frac{k^2 + 2k + 2}{2(k+1)^2} = \frac{1}{2} + \frac{1}{2(k+1)^2},$$

where k is, as before, h/c , and where the approximation is for large h and c with fixed k .

In particular, when $k = 1/4$ as in [20], the normalized reverse slack is approximately 0.82.

If we allow k to vary, the expected normalized $\leftarrow$ slack, as a function of k (for large h and c) is monotonically decreasing, but there is no use choosing a large k in order to make the $\leftarrow$ slack small, for we have seen that this would make the $\rightarrow$ slack large. What we can reasonably do is to compute the value of k that minimizes the sum of the normalized reverse slack and our lower bound for the normalized slack. That will provide a lower bound for the normalized $\rightarrow$ slack plus $\leftarrow$ slack over all possible values of k . That minimum occurs at $(1 + \sqrt{17})/4 \approx 1.281$, and our lower bound for the sum of the two normalized slacks is approximately 1.41.

Corollary 51. *The sum of the normalized $\rightarrow$ slack and $\leftarrow$ slack of the interval filter method, with arbitrary k , is asymptotically (for large h and c) greater than 1.408.*

5. Interval filter methods

5.1. Definition of interval filter chunking

Any local chunking method might be called an interval filter method, because whether a position i is a cutpoint of a file F is determined by applying some criterion (or filter) C to the contents of F in some interval $[i - h, i + h]$ around i . (The notations C and h here are from Definition 21.) But we shall use the phrase “interval filter” in a more restrictive sense.

The first restriction is that whether i is a cutpoint will depend on the contents of the file only in an interval $[i - h, i]$. That is, when reading the file from left to right, one can recognize a cutpoint when one gets to it, without having to read any farther in the file. (This presupposes, for technical reasons, that we know where the file ends; see Remark 53 below.)

A second restriction concerns the particular form of the criterion for cutpoints. We assume that the set PFE of potential file entries has been partitioned into two pieces, U and V , and that the cutpoints of a file F are the positions matching the V in the pattern

$$\underbrace{U \dots U}_h V.$$

More formally:

4.4. LBFS 块划分的反向松弛

LBFS 分块中切点的定义相对于左右不对称，因此没有理由期望这种分块方法的 $\leftarrow$ 松弛等于 $\rightarrow$ 松弛。因此我们在此计算反向松弛。事实证明，计算反向松弛比之前的 $\rightarrow$ 松弛计算更容易，因为我们可以得到精确答案而不是仅得到下界。反向松弛之所以更容易计算的原因是，当我们有两个文件 F^- 和 F_1 的公共部分 F_2 ，中有一个候选时，它是否是切点的问题仅取决于 F^- ，而与分叉的独立 F_1^+ 和 F_2^+ 无关。因此我们可以得到文件反向松弛为 s 的概率的简单公式。

这个概率是两个因素的乘积，即 $-s$ 是切割点（在 F^- 中）的概率，以及在 $-s$ 是切割点的条件下，区间 $[-s + 1, 0]$ 中没有其他切割点的条件概率。这里的第一个因素就是切割点概率，已在命题 42 中计算为 $1/(c + h)$ 。对于第二个因素，我们必须考虑两种情况，取决于 s 和 h 的相对大小。如果 $s \leq h$ ，那么第二个因素是 1，因为区间 $[-s + 1, 0]$ 在 $-s$ 块所有候选成为切割点的范围内。如果 $s > h$ ，然而，那么 $-s$ 只阻挡候选直到并包括 $-s + h$ ，因此我们仍然必须考虑区间 $[-s + h + 1, 0]$ 中的可能切割点。请注意，如果该区间中有候选，那么它就有切割点，因为那里如果有候选，第一个候选将是切割点。因此，我们计算的第二个因素可以表示为在 $-s$ 处有切割点的条件下，区间 $[-s + h + 1, 0]$ 中 $s - h$ 位置没有候选的概率。但是，对于这些位置 i 的“ i 是候选”事件是相互独立且与 $-s$ 处的切割点无关的。每个这些事件有概率 $1/c$ ，因此它们都不发生的概率是 $(1 - \frac{1}{c})^{s-h}$ 。因此，当 $s > h$ 时， s 是反向松弛的概率由 $(1 - \frac{1}{c})^{s-h}/(c + h)$ 给出。

因此，LBFS 分块方法的平均 $\leftarrow$ 松弛由下式给出

$$\sum_{s=0}^h s \frac{1}{c+h} + \sum_{s=h+1}^{\infty} s \frac{1}{c+h} \left(1 - \frac{1}{c}\right)^{s-h}.$$

使用第 2.3 节的公式进行常规计算，即可得到 $\leftarrow$ 松弛的显式形式

$$\frac{h(h+1)}{2(c+h)} + c - 1 = \frac{h^2 - h + 2c^2 - 2c + 2ch}{2(c+h)}.$$

通过将此结果除以块长度+ h 的期望 c ，即可得到归一化反向松弛

$$\frac{h^2 - h + 2c^2 - 2c + 2ch}{2(c+h)^2} \approx \frac{k^2 + 2k + 2}{2(k+1)^2} = \frac{1}{2} + \frac{1}{2(k+1)^2},$$

其中 k 如前所述为 h/c ，且当 h 和 c 固定而 k 较大时，此近似成立

特别地，当 $k = 1/4$ 如 [20]，归一化反向松弛约为 0.82 时

如果我们允许 k 变化，则预期的归一化 $\leftarrow$ 松弛（作为 k 的函数，当 h 和 c 较大时）单调递减，但选择较大的 k 来使 $\leftarrow$ 松弛变小并无意义，因为我们已经看到这会使 $\rightarrow$ 松弛变大。我们合理可以做的，是计算使归一化反向松弛和归一化松弛下界之和最小的 k 值。这将给出所有可能的 k 值下归一化 $\rightarrow$ 松弛与 $\leftarrow$ 松弛之和的下界。该最小值出现在 $(1 + \sqrt{17})/4 \approx 1.281$ ，且这两个归一化松弛之和的下界约为 1.41

推论 51. 区间过滤器方法的归一化 $\rightarrow$ 松弛和 $\leftarrow$ 松弛之和，对于任意的 k ，当 h 和 c 足够大时，渐近地大于 1.408。

5. 区间过滤器方法

5.1. 区间过滤器分块的定义

任何局部分块方法都可以称为区间过滤器方法，因为位置 i 是否是文件 F 的切割点，是通过将某个标准（或过滤器） C 应用于文件 F 在位置 i 附近某个区间 $[i - h, i + h]$ 的内容来确定的。（这里的符号 C 和 h 来自定义 21。）但我们将在更严格的意义上使用“区间过滤器”这个短语。

第一个限制是， i 是否是切割点将仅取决于文件在区间 $[i - h, i]$ 内的内容。也就是说，当从左到右读取文件时，可以在到达切割点时识别它，而无需再读取文件中的任何其他内容。（这在技术上有前提，即我们知道文件在哪里结束；见下文注释 53。）

第二项限制涉及切点标准的特定形式。我们假设潜在文件条目集 PFE 已被划分为两部分， U 和 V ，并且文件 F 的切点是模式中匹配 V 的位置

$$\underbrace{U \dots U}_h V.$$

更正式地说：

Definition 52. The *interval filter chunking* with horizon h determined by a partition of PFE into U and V is the chunking method that declares a position i to be a cutpoint of a file F if and only if

- i is an h -internal position in F ,
- all of $F(i-h), \dots, F(i-1)$ are in U , and
- $F(i) \in V$.

This definition immediately shows that interval filter chunking is a local chunking method with horizon h .

Remark 53. For i to be a cutpoint of F , the first clause in the definition requires that the whole interval $[i-h, i+h]$ be included in the domain of F , but the values of F on the right part, $[i+1, i+h]$, of this interval are irrelevant. We could modify the first clause to require only that $[i-h, i]$ be included in the domain of F . Then there could be cutpoints within h of the end of a finite file. The modification would have the advantage that one could determine whether i is a cutpoint of F by reading F up to position i , without needing to know how much farther F goes. The modification would have the disadvantage of violating our definition of locality of chunking methods (Definition 21), which demands that all cutpoints be h internal. Both the advantage and the disadvantage are quite small; in particular, one ordinarily knows where a file ends. It is convenient for our purposes to use the definition as given, ensuring locality.

The particular form of the filter, h consecutive U 's followed by a V , ensures that no two distinct cutpoints will be within h positions of each other. Consequently, all chunks have length $> h$ except that the first chunk (in a finite or singly infinite file) might have length only h .

5.2. Statistics of interval filter chunking

In the following discussion of interval filter chunking, we assume that not only PFE but also U , V , and h are fixed. We use the notations

$$u = \frac{|U|}{|\text{PFE}|} \quad \text{and} \quad v = 1 - u = \frac{|V|}{|\text{PFE}|}$$

for the probabilities that a random element of PFE is in U and V , respectively. The independence of the entries at different positions in a random file immediately implies that an arbitrary position is a cutpoint with probability $p = u^h v$. And then, by Proposition 25, the expectation of the chunk length is $\mathbf{E}(L|\text{Cut}0) = 1/(u^h v)$. The following computation will give us the variance of the chunk length L ; it will also give the expectation without the need for Kac's theorem. Since we are concerned here with the chunk length, which was defined only for files with a cutpoint at 0, the probabilities, expectations, etc., in the following discussion are all conditional on the event Cut0.

Let S_k be the probability that the next cutpoint is at k and therefore the chunk length is k . Since the filter prevents cutpoints from being within h of each other, we have $S_k = 0$ for all $k \leq h$. For larger k , we have

$$S_k = u^h v \left(1 - \sum_{j=0}^{k-h-1} S_j \right).$$

The factor $u^h v$ here is the probability that k is a cutpoint. The remaining factor is the probability, conditional on k being a cutpoint, that there is no earlier cutpoint after 0, i.e., that no j in the range $[1, k-1]$ is a cutpoint. The upper limit on the sum is not $k-1$ but $k-h-1$ because the condition that k is a cutpoint already implies that no $j \in [k-h, k-1]$ is a cutpoint. A priori, the terms in the sum should be conditional probabilities of j being a cutpoint, conditional on k being a cutpoint. Fortunately, the conditioning here doesn't matter. Whether j is a cutpoint depends only on file entries at positions $\leq j \leq k-h-1$, while the condition that k is a cutpoint depends only on entries in positions $\geq k-h$. Notice also that the lower limit $j=0$ of the sum could be replaced by $j=h+1$ or anything in between, as $S_j = 0$ for $j \leq h$; the same observation applies to other sums, like the one defining $S(z)$ below.

To easily manipulate the formula for S_k , we introduce the generating function

$$S(z) := \sum_{k>h} S_k z^k.$$

Multiplying the formula for S_k by z^k and summing over k , we get

$$S(z) = u^h v \sum_{k>h} z^k - u^h v \sum_{k>h} \sum_{j=0}^{k-h-1} S_j z^k.$$

The first term here involves a geometric series and thus simplifies to $u^h v z^{h+1}/(1-z)$. To evaluate the sum in the second term, we interchange the order of summation; since the range of the variables j and k is given simply by $0 \leq j < k-h$, we get

定义52. 区间过滤器分块方法，其切割点由PFE划分为U和V确定的区间h，是指当且仅当位置i被声明为文件F的切割点时，该方法为分块方法。

- i 是 F 中的 h-内部位置，
- 所有 $F(i-h) \dots F(i-1)$ ，，都在 U 中，并且
- $F(i) \in V$ 。

这个定义立即表明区间过滤器分块是一种具有视野 h 的局部分块方法。

注释 53. 对于 i 成为 F 的切点，定义中的第一项要求整个区间 $[i-h, i+h]$ 包含在 F 的定义域中，但该区间右侧 $[i+1, i+h]$ 的 F 值无关紧要。我们可以修改第一项，要求只需 $[i-h, i]$ 包含在 F 的定义域中。那么，在有限文件末尾 h 范围内可能会有切点。这种修改的优点是，可以通过读取 F 到位置 i 来确定 i 是否是 F 的切点，而无需知道 F 还会延伸多远。这种修改的缺点是会违反我们对分块方法局部性的定义（定义 21），该定义要求所有切点都是 h-内部位置。这两个优缺点都相当小；特别是，通常知道文件在哪里结束。为了我们的目的，使用给定的定义更方便，确保了局部性。

过滤器特定的形式，即连续 h 个 U 后跟一个 V，确保没有两个不同的切点会在 h 个位置内。因此，所有块的长度都是 $>h$ ，但第一个块（在有限或单无限文件中）的长度可能只有 h。

5.2. 区间过滤器分块统计

在区间过滤器分块的以下讨论中，我们假设不仅 PFE，而且 U、V 和 h 都是固定的。我们使用符号

$$u = \frac{|U|}{|\text{PFE}|} \quad \text{and} \quad v = 1 - u = \frac{|V|}{|\text{PFE}|}$$

分别表示随机 PFE 元素在 U 和 V 中的概率。随机文件中不同位置条目的独立性立即意味着任意位置是切点的概率为 $p = u^h v$ 。然后，根据命题25，块长度的期望是 $\mathbf{E}(L|\text{Cut}0) = 1/(u^h v)$ 。接下来的计算将给出块长度 L 的方差；它也将给出无需卡茨定理即可得到的期望。由于我们这里关心的是块长度，该长度仅针对在0处有切点的文件定义，因此以下讨论中的概率、期望等都是基于事件 Cut0 的条件。

设 S_k 为下一个切割点位于 k 且因此块长度为 k 的概率。由于过滤器防止切割点彼此距离小于 h，所以我们有 $S_k = 0$ 对所有 $k \leq h$ 。对于更大的 k，我们有

$$S_k = u^h v \left(1 - \sum_{j=0}^{k-h-1} S_j \right).$$

这里的因子 $u^h v$ 是 k 是切割点的概率。剩余因子是条件概率，即给定 k 是切割点时，0 之后没有更早的切割点，即范围内没有 j 满足 $[1, k-1]$ 是切割点。求和的上限不是 $k-1$ 而是 $k-h-1$ ，因为 k 是切割点的条件已经意味着没有 $j \in [k-h, k-1]$ 是切割点。根据先验知识，求和中的项应该是给定 k 是切割点时 j 是切割点的条件概率。幸运的是，这里的条件无关紧要。j 是否是切割点仅取决于文件在位置 $\leq j \leq k-h-1$ 的条目，而 k 是切割点的条件仅取决于位置 $\geq k-h$ 的条目。请注意，求和的下限 $j=0$ 可以替换为 $j=h+1$ 或两者之间的任何值，因为 $S_j = 0$ 对于 $j \leq h$ ；同样的观察也适用于其他求和，例如下面定义 $S(z)$ 的求和。

为了方便操作 S_k 的公式，我们引入生成函数

$$S(z) := \sum_{k>h} S_k z^k.$$

将 S_k 的公式乘以 z^k 并对 k 求和，我们得到

$$S(z) = u^h v \sum_{k>h} z^k - u^h v \sum_{k>h} \sum_{j=0}^{k-h-1} S_j z^k.$$

第一项涉及一个几何级数，因此简化为 $u^h v z^{h+1}/(1-z)$ 。为了计算第二项的和，我们交换求和的顺序；由于变量 j 和 k 的范围简单地由 $0 \leq j < k-h$ 给出，我们得到

$$\begin{aligned}
\sum_{j=0}^{\infty} \sum_{k=h+j+1}^{\infty} S_j z^k &= \sum_{j=0}^{\infty} S_j \frac{z^{h+j+1}}{1-z} \\
&= \frac{z^{h+1}}{1-z} \sum_{j=0}^{\infty} S_j z^j \\
&= \frac{z^{h+1}}{1-z} S(z).
\end{aligned}$$

Combining the two terms and remembering that $u^h v$ equals the cutpoint probability p , we get

$$S(z) = \frac{pz^{h+1}}{1-z} (1 - S(z)),$$

and solving for $S(z)$ we get

$$S(z) = \frac{pz^{h+1}}{1-z+pz^{h+1}}.$$

The expectation of L is the derivative of $S(z)$ evaluated at $z = 1$, so we compute

$$S'(z) = \frac{pz^h(h+1-hz)}{(1-z+pz^{h+1})^2}.$$

When $z = 1$, this reduces to $1/p$, as predicted by Kac's theorem via Proposition 25. We get $\mathbf{E}(L^2)$ by multiplying $S'(z)$ by z , differentiating again, and then setting $z = 1$, because

$$\begin{aligned}
(zS'(z))' &= \left(z \sum_k S_k k z^{k-1} \right)' \\
&= \left(\sum_k S_k k z^k \right)' \\
&= \sum_k S_k k^2 z^{k-1},
\end{aligned}$$

and setting $z = 1$ yields $\sum_k S_k k^2 = \mathbf{E}(L^2)$. So we differentiate

$$pz^{h+1}(h+1-hz)(1-z+pz^{h+1})^{-2},$$

set $z = 1$ in the result, and simplify to get

$$\mathbf{E}(L^2) = \frac{2}{p^2} - \frac{2h+1}{p}.$$

Finally, the variance of the chunk length is

$$\mathbf{Var}(L) = \mathbf{E}(L^2) - \mathbf{E}(L)^2 = \frac{1}{p^2} - \frac{2h+1}{p} = \frac{1-(2h+1)p}{p^2}.$$

Comparing this to Corollary 43, we see that, for the same cutpoint probability p (hence the same average chunk length $1/p$) and the same minimum chunk length h , our interval filter method gives a smaller variance $\mathbf{Var}(L)$ than the LBFS method. As indicated in the introduction, smaller variance of the chunk length is generally desirable.

As explained earlier, there are other ways to assess desirability of chunking methods. The slack and reverse slack, introduced for just this purpose, and also the probability of long chunks will be considered below. But first, we look briefly at the criterion that simply compares the expected and minimum chunk lengths.

Having enforced a minimum chunk length of h by our choice of filter, it is reasonable to aim next for chunks that are not too big, and one might do this by choosing the parameters u and v (subject to $u + v = 1$ of course) so as to minimize $\mathbf{E}(L)$. To do this, it is convenient to work with the reciprocal of $\mathbf{E}(L)$, namely $u^h v = u^h(1-u)$, which we want to maximize. Differentiating it with respect to u , we get $hu^{h-1} - (h+1)u^h$, which vanishes at $u = 0$ and at $u = h/(h+1)$. Unless $h = 0$ (in which case we would be dealing with a pure point filter method), $u = 0$ minimizes $u^h v$; the maximum we want is at $u = h/(h+1)$, so $v = 1/(h+1)$. The cutpoint probability for this choice of u and v is

$$u^h v = \frac{h^h}{(h+1)^{h+1}} = \left(1 - \frac{1}{h+1}\right)^h \frac{1}{h+1} \approx \frac{1}{eh},$$

and so the expected chunk length is approximately eh . The average chunk is approximately e times the minimum possible chunk size.

$$\begin{aligned}
\sum_{j=0}^{\infty} \sum_{k=h+j+1}^{\infty} S_j z^k &= \sum_{j=0}^{\infty} S_j \frac{z^{h+j+1}}{1-z} \\
&= \frac{z^{h+1}}{1-z} \sum_{j=0}^{\infty} S_j z^j \\
&= \frac{z^{h+1}}{1-z} S(z).
\end{aligned}$$

将两项结合起来，并记住 $u^h v$ 等于断点概率 p ，我们得到

$$S(z) = \frac{pz^{h+1}}{1-z} (1 - S(z)),$$

并求解 $S(z)$ ，得到

$$S(z) = \frac{pz^{h+1}}{1-z+pz^{h+1}}.$$

L 的期望是 $S(z)$ 在 $z = 1$ 处的导数，因此我们计算

$$S'(z) = \frac{pz^h(h+1-hz)}{(1-z+pz^{h+1})^2}.$$

当 $z = 1$ 时，这简化为 $1/p$ ，正如卡茨定理通过命题25所预测的那样。我们通过将 $S'(z)$ 乘以 z ，再次求导，然后设置 $z = 1$ 来得到 $\mathbf{E}(L^2)$ ，因为

$$\begin{aligned}
(zS'(z))' &= \left(z \sum_k S_k k z^{k-1} \right)' \\
&= \left(\sum_k S_k k z^k \right)' \\
&= \sum_k S_k k^2 z^{k-1},
\end{aligned}$$

并设置 $z = 1$ 得到 $\sum_k S_k k^2 = \mathbf{E}(L^2)$ 。因此我们求导

$$pz^{h+1}(h+1-hz)(1-z+pz^{h+1})^{-2},$$

在结果中设置 $z = 1$ 并简化，得到

$$\mathbf{E}(L^2) = \frac{2}{p^2} - \frac{2h+1}{p}.$$

最后，块长度的方差是

$$\mathbf{Var}(L) = \mathbf{E}(L^2) - \mathbf{E}(L)^2 = \frac{1}{p^2} - \frac{2h+1}{p} = \frac{1-(2h+1)p}{p^2}.$$

将其与引理43比较，我们发现，对于相同的断点概率 p （因此具有相同的平均块长度 $1/p$ ）和相同的最小块长度 h ，我们的区间过滤器方法给出的方差 $\mathbf{Var}(L)$ 小于 LBFS 方法。正如引言中所指出的，块长度的方差越小通常越理想。

如前所述，还有其他方法可以评估分块方法的理想性。为了这个目的而引入的松弛和反向松弛，以及长块的概率，将在下文中考。但首先，我们简要看一下仅比较预期和最小块长度的标准。

通过我们选择的过滤器强制实施最小块长度 h ，接下来合理的目标是块不要太大，可以通过选择参数 u 和 v （当然， $u + v = 1$ ）来最小化 $\mathbf{E}(L)$ ，从而实现这一点。为此，使用 $\mathbf{E}(L)$ 的倒数，即 $u^h v = u^h(1-u)$ （我们希望最大化它），会方便得多。将其对 u 求导，我们得到 $hu^{h-1} - (h+1)u^h$ ，它在 $u = 0$ 和 $u = h/(h+1)$ 处为零。除非 $h = 0$ （在这种情况下，我们将处理纯点过滤器方法），否则 $u = 0$ 最小化 $u^h v$ ；我们想要的最大值在 $u = h/(h+1)$ ，所以 $v = 1/(h+1)$ 。这种 u 和 v 选择对应的断点概率是

$$u^h v = \frac{h^h}{(h+1)^{h+1}} = \left(1 - \frac{1}{h+1}\right)^h \frac{1}{h+1} \approx \frac{1}{eh}.$$

因此，预期的块长度大约是 eh 。平均块的大小大约是 e 倍的最小可能块大小。

5.3. Slack of the interval filter method

Consider a random $F \in \text{PFE}^{\mathbb{M}}$, as in the definition of slack, and use the notation $F_1, F_2, F_1^-, F_2^-, F^+$ as there. Write P_k for the probability that the first common cutpoint ≥ 0 of the files F_1 and F_2 is at k . So $\mathbf{E}(\text{Slack}) = \sum_k P_k k$.

For $0 \leq k < h$, the probability that k is a common cutpoint is $u^{2h-k}v$. The reason is that for k to be a common cutpoint requires $F^+(k)$ to be in V (probability v), $F^+(j)$ to be in U for $j=0, 1, \dots, k-1$ (k events of probability u each), and both $F_1^-(j)$ and $F_2^-(j)$ to be in U for $j=k-h, \dots, -1$ ($2(h-k)$ events of probability u each). All these events are independent, so we just multiply their probabilities. Furthermore, still assuming $0 \leq k < h$, we know that, if k is a common cutpoint, then it is the first one, because distinct cutpoints can never be within h of each other. Thus,

$$P_k = u^{2h-k}v \quad \text{for } 0 \leq k < h.$$

For $k \geq h$, on the other hand, the probability that k is a common cutpoint is given by the simpler formula $u^h v$, since we just require $F(k)$ to be in V and $F(j)$ to be in U for the h values $k-h, \dots, k-1$ of j . But the probability that k is the first common cutpoint is more complicated, since we must exclude the possibility of earlier common cutpoints. More precisely, when k is a common cutpoint, then none of $k-h, \dots, k-1$ can be a cutpoint (of either file), but we must exclude the possibility of a common cutpoint $j \in [0, k-h-1]$. Note that, since we only consider values of j smaller than $k-h$, the event that such a j is a cutpoint is independent of the event that k is a cutpoint; indeed, the former depends on file contents at positions $\leq j < k-h$ while the latter depends on file contents at positions $\geq k-h$, and these are independent. Thus, we have

$$P_k = u^h v \left(1 - \sum_{j=0}^{k-h-1} P_j \right) \quad \text{for } k \geq h.$$

Note that, although the events “ j is a common cutpoint” for distinct values of j need not be mutually exclusive (if the j values differ by more than h), the events “ j is the first common cutpoint” are mutually exclusive, so the sum in our formula correctly represents the probability of their union – and this is precisely the probability that some $j < k-h$ is a common cutpoint.

As before, manipulation of these formulas for P_k becomes easier if we introduce the generating function

$$P(z) = \sum_{k=0}^{\infty} P_k z^k.$$

If we take the formulas above for P_k , multiply by z^k and sum over k , we get

$$P(z) = \sum_{k=0}^{h-1} u^{2h-k} v z^k + \sum_{k=h}^{\infty} u^h v z^k - \sum_{k=h}^{\infty} u^h v \sum_{j=0}^{k-h-1} P_j z^k.$$

We simplify each of the three terms on the right, and for notational convenience we remember that the product $u^h v$ is the cutpoint probability, for which we have the shorter notation p . The first term on the right side is

$$u^{2h} v \sum_{k=0}^{h-1} \left(\frac{z}{u} \right)^k = p u^h \frac{\left(\frac{z}{u} \right)^h - 1}{\frac{z}{u} - 1}.$$

We introduce the abbreviation

$$\alpha(z) = \frac{\left(\frac{z}{u} \right)^h - 1}{\frac{z}{u} - 1},$$

so that the first term on the right side of the formula for $P(z)$ becomes $p u^h \alpha(z)$. The second term is

$$\sum_{k=h}^{\infty} p z^k = \frac{p z^h}{1-z}.$$

In the third term, we interchange the order of summation, obtaining

$$u^h v \sum_{j=0}^{\infty} \sum_{k=j+h+1}^{\infty} P_j z^k = p \sum_{j=0}^{\infty} P_j \frac{z^{h+j+1}}{1-z} = \frac{p z^{h+1}}{1-z} P(z).$$

Inserting these simplifications into our formula for $P(z)$ and multiplying by $1-z$ to clear denominators, we get

$$(1-z)P(z) = (1-z)p u^h \alpha(z) + p z^h - p z^{h+1} P(z).$$

5.3. 区间过滤器方法的松弛

考虑一个随机 $F \in \text{PFE}^{\mathbb{M}}$ ，如松弛的定义中所述，并使用符号 $F_1, F_2, F_1^-, F_2^-, F^+$ 表示。将 P_k 写作文件 F_1 和 F_2 的第一个公共切割点 ≥ 0 位于 k 的概率。所以 $\mathbf{E}(\text{Slack}) = \sum_k P_k k$ 。

对于 $0 \leq k < h$ ， k 是一个公共切点的概率是 $u^{2h-k}v$ 。原因是 k 要成为公共切点需要 $F^+(k)$ 在 V 中（概率 v ）， $F^+(j)$ 在 U 中对于 $j=0, 1, \dots, k-1$ ，，（ k 是概率为 u 的每个事件），并且 $F_1^-(j)$ 和 $F_2^-(j)$ 都在 U 中对于 $j=k-h, \dots, -1$ ($2(h-k)$)，，（概率为 u 的每个事件）。所有这些事件都是独立的，所以我们只需要将它们的概率相乘。此外，仍然假设 $0 \leq k < h$ ，我们知道，如果 k 是一个公共切点，那么它是第一个，因为不同的切点永远不会彼此之间相距 h 。因此，

$$P_k = u^{2h-k}v \quad \text{for } 0 \leq k < h.$$

对于 $k \geq h$ ，而言， k 是公共切割点的概率由更简单的公式 $u^h v$ 给出，因为我们只需要 $F(k)$ 在 V 中，且 $F(j)$ 在 U 中，对于 j 的 h 值 $k-h, \dots, k-1$ 。但 k 是第一个公共切割点的概率更为复杂，因为我们必须排除更早出现的公共切割点的可能性。更精确地说，当 k 是公共切割点时， $k-h, \dots, k-1$ ，中的任何一个都不能是切割点（属于任何文件），但我们必须排除公共切割点 $j \in [0, k-h-1]$ 的可能性。请注意，由于我们只考虑小于 k 的 j 值 $-h$ ，因此这样的 j 是切割点的事件与 k 是切割点的事件独立；实际上，前者取决于文件在位置 $\leq j < k-h$ 的内容，而后者取决于文件在位置 $\geq k-h$ 的内容，这两者是独立的。因此，我们有

$$P_k = u^h v \left(1 - \sum_{j=0}^{k-h-1} P_j \right) \quad \text{for } k \geq h.$$

请注意，尽管对于不同的 j 值，“ j 是一个公共切割点”这两个事件不一定互斥（如果 j 值之差大于 h ），但“ j 是第一个公共切割点”这两个事件是互斥的，因此我们公式中的和正确地表示了它们并集的概率——而这正是某些 $j < k-h$ 是公共切割点的概率。

如前所述，如果我们引入生成函数，对 P_k 的这些公式进行操作会更容易。

$$P(z) = \sum_{k=0}^{\infty} P_k z^k.$$

如果我们取上述关于 P_k 的公式，乘以 z^k 并对 k 求和，我们得到

$$P(z) = \sum_{k=0}^{h-1} u^{2h-k} v z^k + \sum_{k=h}^{\infty} u^h v z^k - \sum_{k=h}^{\infty} u^h v \sum_{j=0}^{k-h-1} P_j z^k.$$

我们简化右边的三个项，为了方便记号，我们记住乘积 $u^h v$ 是切割点概率，我们对其有更短的记号 p 。右边的第一项是

$$u^{2h} v \sum_{k=0}^{h-1} \left(\frac{z}{u} \right)^k = p u^h \frac{\left(\frac{z}{u} \right)^h - 1}{\frac{z}{u} - 1}.$$

我们引入缩写

$$\alpha(z) = \frac{\left(\frac{z}{u} \right)^h - 1}{\frac{z}{u} - 1},$$

以便公式 $P(z)$ 右边的第一项变为 $p u^h \alpha(z)$ 。第二项是

$$\sum_{k=h}^{\infty} p z^k = \frac{p z^h}{1-z}.$$

在第三学期，我们交换求和的顺序，得到

$$u^h v \sum_{j=0}^{\infty} \sum_{k=j+h+1}^{\infty} P_j z^k = p \sum_{j=0}^{\infty} P_j \frac{z^{h+j+1}}{1-z} = \frac{p z^{h+1}}{1-z} P(z).$$

将这些简化条件代入 $P(z)$ 的公式中，并乘以 $1-z$ 以消去分母，我们得到

$$(1-z)P(z) = (1-z)p u^h \alpha(z) + p z^h - p z^{h+1} P(z).$$

The expectation of the slack is obtained by differentiating $P(z)$ to get $\sum_k P_k k z^{k-1}$ and then setting $z=1$ to get $\sum_k P_k k$. To evaluate $P'(1)$, we differentiate the last displayed formula with respect to z and substitute $z=1$ in the result. The differentiation is simplified by the observation that, whenever a factor $1-z$ in our formula survives in the derivative, it will be annihilated by the substitution; in particular, we can ignore the term involving $\alpha'(z)$. The computation produces

$$-P(1) = -pu^h\alpha(1) + hp - (h+1)pP(1) - pP'(1).$$

Remembering that $P(1) = \sum_k P_k = 1$ and solving for $P'(1)$, we get

$$P'(1) = \frac{1}{p} - u^h\alpha(1) - 1.$$

The definition of $\alpha(z)$ gives

$$u^h\alpha(1) = u^h \frac{(\frac{1}{u})^h - 1}{\frac{1}{u} - 1} = \frac{1 - u^h}{\frac{1}{u} - 1}.$$

Inserting this into the formula for $P'(1)$, we finally get, using also that $1-u=v$,

$$\mathbf{E}(\text{Slack}) = \frac{1}{p} - \frac{\frac{1}{u} - u^h}{\frac{1}{u} - 1} = \frac{1}{p} - \frac{1 - u^{h+1}}{v}.$$

The expectation of the normalized slack is obtained by dividing by the expected chunk length, i.e., multiplying by $p = u^h v$, which gives

$$1 - u^h + u^{2h+1}.$$

If, as suggested at the end of the preceding subsection, we choose $u = h/(h+1)$, so as to minimize the expected chunk length, then the normalized slack is, on average

$$1 - \left(1 - \frac{1}{h+1}\right)^h + \left(1 - \frac{1}{h+1}\right)^{2h+1} \approx 1 - \frac{1}{e} + \frac{1}{e^2},$$

where the approximation is good for large h .

On the other hand, one might want to choose u and v so as to minimize the expected normalized slack. By differentiating, one finds that the minimum occurs when $u^h = (h+1)/(2h+1) \approx 1/2$,

$$u \approx \sqrt[h]{\frac{1}{2}} \quad \text{and so} \quad v \approx \frac{\ln 2}{h}.$$

The minimum value of the expected normalized slack is thus approximately 3/4.

5.4. Probability of long chunks

We estimate next the probability of getting long chunks in the interval filter method. This estimate will be relevant later in two computations. One concerns the probability of a long interval containing no cutpoints; the other concerns the right slack of the method.

As before, we assume that u , v , and h are given, and we write p for the cutpoint probability $p = u^h v$.

Let q_k be the conditional probability that there is no cutpoint in the interval $[1, k]$ given that there is a cutpoint at 0. Let $Q(z)$ be the generating function

$$Q(z) = \sum_{k=0}^{\infty} q_k z^k.$$

Before proceeding to estimate q_k , we comment on the condition “there is a cutpoint at 0.” It obviously implies that the file entry at 0 is in V . It implies more, namely that the h previous file entries are in U , but this additional information has no effect on the probability of a cutpoint at any positive position. To get a cutpoint at some $k > 0$, we need h consecutive elements of U at positions $k-h$ to $k-1$ and, because of the element of V at position 0, no positions farther to the left can contribute to a cutpoint at k . Thus, q_k could also be described as the probability that a random file F has a cutpoint at k , given that $F(0) \in V$.

We have almost computed the probabilities q_k in Section 5.2. We obtained there the generating function $S(z)$ for the probabilities S_k that the first positive cutpoint is at k , given a cutpoint at 0. This S_k can be described as the probability, given a cutpoint at 0, that there is no cutpoint in $[1, k-1]$ but there is one in $[1, k]$. That is,

$$S_k = q_{k-1} - q_k$$

通过微分 $P(z)$ 得到 $\sum_k P_k k z^{k-1}$, 然后令 $z=1$ 得到 $\sum_k P_k k$ 。为了计算 $P'(1)$, 我们对最后显示的公式关于 z 进行微分, 并将 $z=1$ 代入结果中。由于观察到每当公式中的因子 $1-z$ 在导数中仍然存在时, 它都会被代入操作抵消; 特别是, 我们可以忽略涉及 $\alpha'(z)$ 的项。计算结果为

$$-P(1) = -pu^h\alpha(1) + hp - (h+1)pP(1) - pP'(1).$$

记住 $P(1) = \sum_k P_k = 1$ 并求解 $P'(1)$, 我们得到

$$P'(1) = \frac{1}{p} - u^h\alpha(1) - 1.$$

$\alpha(z)$ 的定义给出

$$u^h\alpha(1) = u^h \frac{(\frac{1}{u})^h - 1}{\frac{1}{u} - 1} = \frac{1 - u^h}{\frac{1}{u} - 1}.$$

将其代入 $P'(1)$ 的公式中, 我们最终得到, 同时使用 $1-u=v$,

$$\mathbf{E}(\text{Slack}) = \frac{1}{p} - \frac{\frac{1}{u} - u^h}{\frac{1}{u} - 1} = \frac{1}{p} - \frac{1 - u^{h+1}}{v}.$$

归一化松弛的期望通过除以预期的块长度获得, 即乘以 $p = u^h v$, 得到

$$1 - u^h + u^{2h+1}.$$

如果, 如前一节末尾所建议的, 我们选择 $u = h/(h+1)$ 以最小化预期的块长度, 那么归一化松弛平均为

$$1 - \left(1 - \frac{1}{h+1}\right)^h + \left(1 - \frac{1}{h+1}\right)^{2h+1} \approx 1 - \frac{1}{e} + \frac{1}{e^2},$$

在 h 较大时, 近似效果较好。

另一方面, 人们可能希望选择 u 和 v 以最小化预期的标准化松弛。通过求导, 可以发现最小值在 $u^h = (h+1)/(2h+1) \approx 1/2$ 处出现,

$$u \approx \sqrt[h]{\frac{1}{2}} \quad \text{and so} \quad v \approx \frac{\ln 2}{h}.$$

因此, 预期的标准化松弛的最小值大约为 3/4。

5.4. 长块的概率

接下来, 我们估计区间过滤方法中长块出现的概率。这个估计将在后续两个计算中相关。一个涉及长区间不包含切点的概率; 另一个涉及方法的右松弛。

如前所述, 我们假设 u 、 v 和 h 已知, 并将切点概率 $p = u^h v$ 写作 p 。

设 q_k 是在区间 $[1, k]$ 中没有切割点的条件概率, 已知在 0 处存在切割点。设 $Q(z)$ 是生成函数

$$Q(z) = \sum_{k=0}^{\infty} q_k z^k.$$

在估计 q_k 之前, 我们评论一下条件 “在 0 处存在切割点”。这显然意味着 0 处的文件条目在 V 中。它还意味着前 h 个文件条目在 U 中, 但这额外的信息对任何正位置切割点的概率没有影响。要在某个 $k > 0$ 处获得切割点, 我们需要在位置 $k-h$ 到 $k-1$ 之间有 h 个连续的 U 元素, 并且由于位置 0 处的 V 元素, 没有更靠左的位置可以贡献 k 处的切割点。因此, q_k 也可以描述为随机文件 F 在 $F(0) \in V$ 条件下在 k 处存在切割点的概率

我们几乎计算了第5.2节的概率 q_k 。在那里, 我们得到了生成函数 $S(z)$, 用于计算在0处有一个切割点的情况下, 第一个正切割点位于 k 的概率 S_k 。这个 S_k 可以描述为在0处有一个切割点的情况下, $k[1, k-1]$ 处没有切割点, 但在 k 处有一个切割点的概率。也就是说,

$$S_k = q_{k-1} - q_k$$

for all $k \geq 1$. Multiplying this equation by z^k , summing over all $k \geq 1$, and taking into account that $S_0 = 0$ and $q_0 = 1$, we find that

$$S(z) = zQ(z) - Q(z) + 1.$$

Solving for $Q(z)$ and using the formula for $S(z)$ computed in Section 5.2, we find that

$$Q(z) = \frac{1 - S(z)}{1 - z} = \frac{1}{1 - z + pz^{h+1}}.$$

Because this generating function is rational, one can, in principle, expand it in partial fractions, expand those fractions as geometric series, and thus obtain explicit formulas for the probabilities q_k . This approach, unfortunately, presupposes that one knows the roots of the polynomial $1 - z + pz^{h+1}$; these roots enter into the partial fraction expansion, and (unless there are multiple roots) the formula for q_k is a linear combination (with constant coefficients) of the $h + 1$ terms w^k , where w ranges over the reciprocals of the $h + 1$ roots of the polynomial. In other words, w ranges over the solutions of the equation $w^{h+1} - w^h + p = 0$, which is most conveniently (for our purposes) written as $w^h(1 - w) = p$. For large k , the formula for q_k will be dominated by the w^k term for the largest of the roots. Most of the rest of this section will be devoted to estimating this largest root and thus estimating the rate at which q_k approaches 0 as $k \rightarrow \infty$. But first we indicate, in the following remark, an alternative approach to the computation of q_k ; the reader can safely skip it, as it will not be used directly in the subsequent work.

Remark 54. The inclusion–exclusion principle provides a formula for q_k as follows. For each subset A of $\{1, 2, \dots, k\}$, let $r(A)$ be the probability that a random file with a cutpoint at 0 also has cutpoints at all the members of A (and possibly additional points as well). Notice that $r(A) = 0$ if any element of A is $\leq h$ or if any two distinct elements differ by $\leq h$, because no file has two cutpoints separated by a distance h or less. For all other choices of A , the events of having cutpoints at the various elements of A are independent of each other and of the condition that there is a cutpoint at 0, so $r(A) = p^{|A|}$. The inclusion–exclusion principle, applied to this situation, says that

$$q_k = \sum_{A \subseteq \{1, \dots, k\}} (-1)^{|A|} r(A) = \sum_l (-1)^l p^l N_l,$$

where N_l is the number of l -element subsets of $\{1, 2, \dots, k\}$ that are good in the sense that they have no elements $\leq h$ and no two distinct elements a distance $\leq h$ apart. Fortunately, these good subsets are quite easy to count. Notice first that, if $A = \{a_1 < a_2 < \dots < a_l\}$ is good, then $0 < a_1 - h$ (because A has no elements $\leq h$) and $a_1 - h < a_2 - 2h < \dots < a_l - lh$ (because a_i and a_{i+1} differ by more than h). Thus, we can compress A to a set of l positive integers $\tilde{A} = \{a_i - ih: 1 \leq i \leq l\}$, which is obviously a subset of $\{1, 2, \dots, k - lh\}$. Conversely, any l -element subset of $\{1, 2, \dots, k - lh\}$ arises as $\tilde{A}$ from a unique good A . Therefore,

$$N_l = \binom{k - lh}{l}$$

and

$$q_k = \sum_l (-1)^l p^l \binom{k - lh}{l}.$$

(The variable l can be allowed to range over all integers, but the binomial coefficient will vanish unless l is in the range of reasonable values, $0 \leq l \leq k/(h + 1)$.) This formula for q_k , though quite explicit, does not seem to be directly amenable to estimating the asymptotic behavior of the sequence of q_k 's, mainly because of cancellations between the positive and negative terms in the sum. It leads, however, to a recursive formula that can be more useful for asymptotic estimates. Indeed, the familiar Pascal-triangle identity for the binomial coefficients allows us to write

$$q_k = \sum_l (-1)^l p^l \binom{k - lh - 1}{l} + \sum_l (-1)^l p^l \binom{k - lh - 1}{l - 1}.$$

The first sum here is simply q_{k-1} . The second can be rewritten, by factoring out $-p$ and changing the summation variable to $m = l - 1$, as

$$-p \sum_l (-1)^m p^m \binom{k - h - 1 - mh}{m} = -p q_{k-h-1}.$$

Therefore, we get the linear recursion equation

$$q_k = q_{k-1} - p q_{k-h-1}.$$

对于所有 $k \geq 1$ 。将这个方程乘以 z^k ，对所有 $k \geq 1$ ，求和，并考虑到 $S_0 = 0$ 和 $q_0 = 1$ ，我们发现

$$S(z) = zQ(z) - Q(z) + 1.$$

求解 $Q(z)$ 并使用第 5.2 节中计算的 $S(z)$ 公式，我们发现

$$Q(z) = \frac{1 - S(z)}{1 - z} = \frac{1}{1 - z + pz^{h+1}}.$$

由于这个生成函数是有理的，原则上可以将其展开为部分分式，将那些分式展开为几何级数，从而得到概率 q_k 的显式公式。这种方法的缺点是，它预先假定了人们知道多项式 $1 - z + pz^{h+1}$ 的根；这些根会进入部分分式展开中，并且（除非有重根）， q_k 的公式是 $h + 1$ 项 w^k （其中 w 取遍多项式 $h + 1$ 根的倒数）的线性组合（系数为常数）；换句话说， w 取遍方程 $w^{h+1} - w^h + p = 0$ 的解，为了我们（的目的），这个方程最方便地写为 $w^h(1 - w) = p$ 。对于大的 k ， q_k 的公式将主要由最大的根对应的 w^k 项主导。本节的大部分内容将致力于估计这个最大的根，从而估计 q_k 在 $k \rightarrow \infty$ 趋近于 0 的速率。但在接下来的注释中，我们指出计算 q_k 的另一种方法；读者可以安全地跳过它，因为它不会在后续工作中直接使用。

备注54。包含-排除原理为 q_k 提供了一个公式。对于 $\{1, 2, \dots, k\}$ 的每个子集 A ，令 $r(A)$ 为随机文件在 0 处有切割点，并且在所有 A 的成员处也有切割点（以及可能还有其他点）的概率。请注意， $r(A) = 0$ 如果 A 的任何元素是 $\leq h$ ，或者如果 A 的任何两个不同元素相差 $\leq h$ ，因为没有任何文件有两个相距 h 或更小的切割点。对于 A 的所有其他选择，在 A 的不同元素处有切割点的事件是相互独立的，并且与存在 0 处切割点的条件无关，所以 $r(A) = p^{|A|}$ 。将包含-排除原理应用于这种情况，它表明

$$q_k = \sum_{A \subseteq \{1, \dots, k\}} (-1)^{|A|} r(A) = \sum_l (-1)^l p^l N_l,$$

其中 N_l 是 $\{1, 2, \dots, k\}$ 中所有 l 元素子集的数量，这些子集在意义上是好的，即它们没有元素 $\leq h$ ，并且任何两个不同元素之间的距离都不小于 $\leq h$ 。幸运的是，这些好的子集很容易计数。首先请注意，如果 $A = \{a_1 < a_2 < \dots < a_l\}$ 是好的，那么 $0 < a_1 - h$ （因为 A 没有元素 $\leq h$ ）和 $a_1 - h < a_2 - 2h < \dots < a_l - lh$ （因为 a_i 和 a_{i+1} 相差超过 h ）。因此，我们可以将 A 压缩为一个 l 个正整数的集合 $\tilde{A} = \{a_i - ih: 1 \leq i \leq l\}$ ，这显然是 $\{1, 2, \dots, k - lh\}$ 的子集。反之， $\{1, 2, \dots, k - lh\}$ 的任何 l 元素子集都唯一地作为 A 来自一个好的 A 。因此，

$$N_l = \binom{k - lh}{l}$$

and

$$q_k = \sum_l (-1)^l p^l \binom{k - lh}{l}.$$

(变量 l 可以允许取遍所有整数，但二项式系数将消失，除非 l 在合理的值范围内， $0 \leq l \leq k/(h + 1)$.) 对于 q_k 的公式，虽然相当明确，但似乎不能直接用于估计序列 q_k 's 的渐近行为，主要是因为和中的正项和负项之间存在抵消。然而，它却导出了一个递归公式，该公式对于渐近估计可能更有用。实际上，二项式系数的熟悉帕斯卡三角形恒等式允许我们写出

$$q_k = \sum_l (-1)^l p^l \binom{k - lh - 1}{l} + \sum_l (-1)^l p^l \binom{k - lh - 1}{l - 1}.$$

这里的第一个和简单地是 q_{k-1} 。第二个可以通过提取 $-p$ 并更改求和变量为 $m = l - 1$ 来重写为

$$-p \sum_l (-1)^m p^m \binom{k - h - 1 - mh}{m} = -p q_{k-h-1}.$$

因此，我们得到线性递归方程

$$q_k = q_{k-1} - p q_{k-h-1}.$$

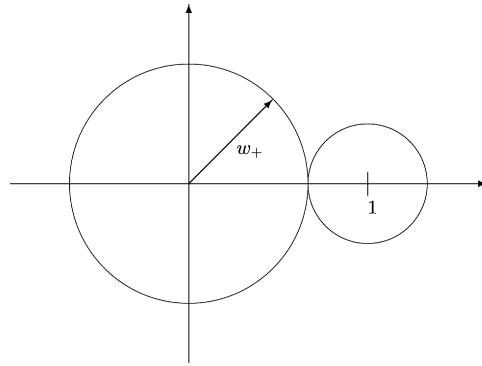


Fig. 1. A cover for the roots of $w^h(1-w)=p$.

The usual technique for solving such recursions, namely looking for constants λ such that λ^k solves the recursion, forming a linear combination of such solutions for the various possible λ 's, and choosing the constant coefficients in the linear combination to match initial conditions, leads to the following equation for λ :

$$\lambda^{h+1} = \lambda^h - p.$$

This is exactly the equation whose roots w figured in the discussion preceding this remark. So we have reached, via a different route, the same conclusion: The asymptotic behavior of q_k for large k is exponential, $q_k \approx \text{constant} \cdot \lambda^k$, where λ is the largest (in absolute value) root of $w^h(1-w)=p$.

We now turn to the task of estimating the largest root of $w^h(1-w)=p$. Recall that the cutpoint probability p was obtained as $p = u^h v = u^h(1-u)$, so u is a root of our equation. Recall also that, for a fixed h , the maximum possible value of p is $p_{\max} = h^h/(h+1)^{h+1}$, attained at $u = h/(h+1)$.

To study the solutions of $w^h(1-w)=p$, let us concentrate first on non-negative real solutions, and so let us consider the graph of the function $f(w) = w^h(1-w)$. For non-negative w , it starts at the origin, where it has a root of multiplicity h ; it increases up to the point where $w = h/(h+1)$ and $f(w) = h^h/(h+1)^{h+1}$ (the maximum we computed earlier), and then it decreases to the point where $w = 1$ and $f(w) = 0$. As w continues to increase, $f(w)$ continues to decrease, i.e., to become more and more negative.

Thus, for p in the range $0 \leq p < p_{\max}$, there are two non-negative real solutions w for $w^h(1-w)=p$, both of which are simple roots except that when $p=0$ the root $w=0$ has multiplicity h . Let us write w_+ for the larger and w_- for the smaller of these roots. For $p = p_{\max}$, there is a single non-negative real root, with multiplicity 2; we let both w_+ and w_- denote this root. Thus, both w_+ and w_- are continuous functions of p in $[0, p_{\max}]$. We can ignore any values of p outside this interval, because they cannot arise as the cutpoint probability of the interval filter method.

We propose now to show that w_+ is the root of largest absolute value for the equation $w^h(1-w)=p$, not just among the non-negative real roots already considered but also among all roots in the complex plane. Notice that there is no problem (and no interest) in the case $p=0$; here we know all the roots: an h -fold root at $0 = w_-$ and a simple root at $1 = w_+$. So we may assume $p \in (0, p_{\max}]$. Temporarily fix p (and thus w_+) and consider the following two disks in the complex plane. The *left disk* is centered at the origin and has radius w_+ ; the *right disk* is centered at 1 and has radius $1 - w_+$. The situation is illustrated in Fig. 1. Notice that the two disks are tangent at their common boundary point w_+ . Our ultimate goal is to show that all the roots of $w^h(1-w)=p$ lie in the left disk, but first we establish the easier result that all these roots lie in the union of the two disks.

To see this, suppose w is a root that lies outside the left disk. So $|w| > w_+$ and thus $|w|^h > w_+^h$. But

$$|w|^h \cdot |1-w| = p = w_+^h \cdot (1-w_+),$$

and so we must have $|1-w| < 1-w_+$. That is, w lies in the right disk.

To show that the roots actually all lie in the left disk, we allow p to vary and consider what happens if p starts at 0 and gradually increases toward $p_{\max}$. Of course, at each stage of the process, there is a root w_+ at the point of tangency of our two disks. Our concern is with the behavior of the other h roots. Initially, when $p=0$, these roots are all at the center 0 of the left disk. (The left disk is initially the unit disk and the right disk a single point.) Now as p increases, the left disk shrinks, the right disk grows, and the roots we are interested in move around continuously. Can they escape from the left disk? The preceding paragraph shows that the only way to escape from the left disk is to enter the right disk. Since the disks touch only at w_+ , an escaping root would have to coincide, at the moment of its escape, with w_+ . That is, w_+ would have to be (at least) a double root of our equation $w^h(1-w)=p$. But we know, from our analysis of the non-negative real roots, that w_+ is a simple root for $p < p_{\max}$; it becomes a double root only when p reaches $p_{\max}$. Thus, the only possible moment when a root can escape from the left disk is at the very end of the range of relevant p values. That is, for all $p \in [0, p_{\max}]$, all the roots are still in the left disk.

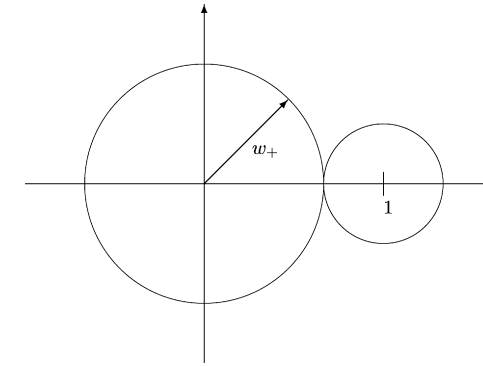


图1. $w^h(1-w)=p$ 的根的封面。

求解此类递归的常用技术，即寻找满足 λ^k 的常数 λ ，将各种可能的 λ 形成的线性组合，并选择线性组合中的常数系数以匹配初始条件，最终得到 λ 的下方程：

$$\lambda^{h+1} = \lambda^h - p.$$

这正好是讨论本注释之前所涉及的根的方程。因此，我们通过不同的途径得出了相同的结论： q_k 在 k 很大时的渐近行为是指数的， $q_k \approx \text{常数} \cdot \lambda^k$ ，其中 λ 是 $w^h(1-w)=p$ 中绝对值最大的根。

我们现在转向估计 $w^h(1-w)=p$ 的最大根的任务。回想一下，断点概率 p 是通过 $p = u^h v = u^h(1-u)$ 得到的，所以 u 是我们方程的一个根。回想一下，对于固定的 h ， p 的最大可能值是 $p_{\max} = h^h/(h+1)^{h+1}$ ，在 $u = h/(h+1)$ 处达到。

为研究 $w^h(1-w)=p$ 的解，让我们首先集中考虑非负实数解，因此让我们考虑函数 $f(w) = w^h(1-w)$ 的图像。对于非负的 w ，它从原点开始，在那里有一个重数为 h 的根；它增加到 $w = h/(h+1)$ 和 $f(w) = h^h/(h+1)^{h+1}$ （我们之前计算的最大值），然后减少到 $w = 1$ 和 $f(w) = 0$ 的点。随着 w 继续增加， $f(w)$ 继续减少，即变得越来越负。

因此，对于 p 在 $0 \leq p < p_{\max}$ 范围内，对于 $w^h(1-w)=p$ 有两个非负实数解 w ，它们都是单根，但当 $p=0$ 时，根 $w=0$ 的重数为 h 。让我们将较大的根记为 w_+ ，较小的根记为 w_- 。对于 $p = p_{\max}$ ，有一个重数为 2 的非负实数根；我们让 w_+ 和 w_- 都表示这个根。因此， w_+ 和 w_- 都是 p 在 $[0, p_{\max}]$ 上的连续函数。我们可以忽略此区间外的任何 p 值，因为它们不可能作为区间过滤器方法的断点概率出现。

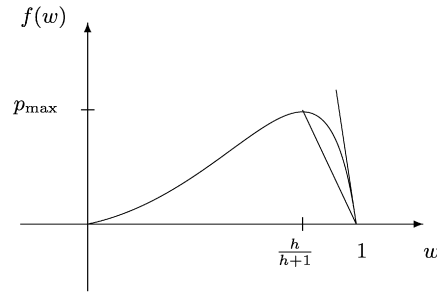
我们现提出要证明 w_+ 是方程 $w^h(1-w)=p$ 绝对值最大的根，不仅是在已经考虑的非负实根中，而是在复平面上所有根中。注意在这种情况下没有问题（也没有兴趣）：因为我们知道所有根：一个 h 重根在 $0 = w_-$ 处，以及一个单根在 $1 = w_+$ 处。所以我们假设 $p \in (0, p_{\max}]$ 。暂时固定 p （从而固定 w_+ ），并考虑复平面上的以下两个圆盘。左圆盘以原点为中心，半径为 w_+ ；右圆盘以 1 为中心，半径为 $1 - w_+$ 。情况如图 1 所示。注意这两个圆盘在它们的公共边界点 w_+ 处相切。我们的最终目标是证明 $w^h(1-w)=p$ 的所有根都在左圆盘中，但首先我们建立更容易的结果：这些根都在两个圆盘的并集中。

要看到这一点，假设 w 是一个位于左圆盘之外的根。因此 $|w| > w_+$ ，所以 $|w|^h > w_+^h$ 。但是

$$|w|^h \cdot |1-w| = p = w_+^h \cdot (1-w_+),$$

因此我们必须有 $|1-w| < 1-w_+$ 。也就是说， w 位于右圆盘内。

为了证明所有根实际上都在左圆盘内，我们允许 p 变化，并考虑当 p 从 0 开始逐渐增加到 $p_{\max}$ 时会发生什么。当然，在过程的每个阶段，我们的两个圆盘在切点处都有一个根 w_+ 。我们的关注点是其他 h 个根的行为。最初，当 $p=0$ 时，这些根都位于左圆盘的中心 0。（左圆盘最初是单位圆盘，右圆盘是一个点。）现在随着 p 增加，左圆盘缩小，右圆盘变大，我们感兴趣的根会连续移动。它们能从左圆盘逃逸出去吗？前一段说明，唯一能从左圆盘逃逸出去的方法是进入右圆盘。由于圆盘只在 w_+ 处相切，一个逃逸的根在逃逸的瞬间必须与 w_+ 重合。也就是说， w_+ 必须是（至少）我们方程 $w^h(1-w)=p$ 的一个重根。但我们知道，从我们对非负实根的分析来看， w_+ 对于 $p < p_{\max}$ 是一个单根，它只有在 p 达到 $p_{\max}$ 时才会变成重根。因此，根唯一可能从左圆盘逃逸出去的时刻就是相关 p 值范围的最末端。也就是说，对于所有 $p \in [0, p_{\max}]$ ，所有根仍然都在左圆盘内。

Fig. 2. A graph of $f(w)$ for estimating w_+ as a function of p .

This completes the verification of our claim that w_+ is always the largest root in absolute value.

There remains the question of evaluating or at least estimating w_+ as a function of p (where we continue to regard h as fixed). The two following rough estimates will suffice for our purposes; both are based on the fact that the function $f(w) = w^h(1-w)$ has a negative second derivative throughout the interval $[h/(h+1), 1]$ (in fact for all $w > (h-1)/(h+1)$). Recall that this is the interval over which w_+ ranges as $p = f(w)$ varies from $p_{\max}$ down to 0. Knowing that the graph of f is concave, we have that this graph lies below its tangents and above its chords on this interval.

The tangent to the graph of f at the point $w = 1, p = 0$ is the line $p + w = 1$, because $f'(1) = -1$. Since the graph is below this tangent, we conclude that $w_+ \leq 1 - p$ (with equality only at $p = 0$).

The chord of the graph of f joining the point $w = 1, p = 0$ and the point $w = h/(h+1), p = p_{\max}$ has the equation $w = 1 - p/(p_{\max}(h+1))$. Since the graph lies above the chord, we have

$$w_+ \geq 1 - \frac{1}{p_{\max}(h+1)}p \approx 1 - ep,$$

where the approximation is good for large h . Fig. 2 illustrates the above estimations. (For the sake of visibility, Fig. 2 is drawn with different scales along the two axes.)

Inserting these estimates of w_+ into our previous results concerning the probabilities q_k of chunks longer than k , we find that, for large k , q_k lies between a constant multiple of $(1 - ep)^k$ and a constant multiple of $(1 - p)^k$. The constant here arises from the partial fraction expansion of the generating function $Q(z)$. If we express k in terms of the average chunk length $1/p$, i.e., if we set $k = M/p$, so that q_k is the probability that the chunk containing 0 is at least M times the average length, then we can approximate

$$(1 - ep)^k = ((1 - ep)^{1/ep})^{eM} \approx e^{-eM}$$

and

$$(1 - p)^k = ((1 - p)^{1/p})^M \approx e^{-M}.$$

Thus, as a function of M , the probability of a chunk M times as long as the average decreases exponentially and lies between a constant multiple of e^{-eM} and a constant multiple of e^{-M} . The actual factor in the exponent depends on p ; the upper estimate e^{-M} is more accurate for very small p (where the graph of f is close to the tangent used in obtaining this estimate), while the lower estimate e^{-eM} is more accurate for relatively large p , i.e., close to $p_{\max}$.

One can, of course, obtain more accurate estimates of the decay rate of q_k by estimating w_+ more accurately. For example, one can express w_+ as a Taylor series in $1 - p$ or as a Puiseux series in $p - p_{\max}$.

5.5. Reverse slack of the interval filter method

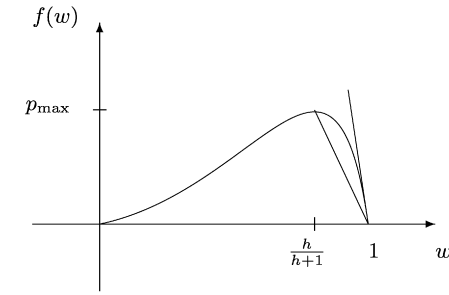
The estimates of q_k in Section 5.4 enable us to estimate the reverse slack of the interval filter method as follows. Consider a random $F \in \text{PFE}^{\mathbb{D}}$. The probability that its $\leftarrow$ slack is a particular natural number k is the product of the probability of a cutpoint at $-k$ and the conditional probability, given a cutpoint at $-k$, of having no further cutpoints in $[-k+1, 0]$. The former factor is $p = u^h v$ and the latter is, thanks to shift-invariance, q_k . (We are in the pleasant situation that neither factor is influenced by F_1^+ and F_2^+ ; only F^- is relevant, and so we are essentially dealing with a file in $\text{PFE}^{\mathbb{Z}}$.)

The expectation of the $\leftarrow$ slack is therefore

$$\mathbb{E}(\leftarrow \text{Slack}) = \sum_{k=0}^{\infty} k p q_k = p Q'(1),$$

where Q is the generating function obtained in Section 5.4,

$$Q(z) = \sum_{k=0}^{\infty} q_k z^k = \frac{1}{1 - z + p z^{h+1}}.$$

图2. $f(w)$ 的图，用于估计 w_+ 作为 p 的函数。

这完成了我们关于 w_+ 总是绝对值最大的根的断言的验证。

仍然存在一个问题：如何评估或至少估计 w_+ 作为 p 的函数（我们继续将 h 视为固定）。以下两个粗略估计足以满足我们的目的；两者都基于这样一个事实：函数 $f(w) = w^h(1-w)$ 在整个区间 $[h/(h+1), 1]$ （实际上对于所有 $w > (h-1)/(h+1)$ ）具有负的二阶导数。回想一下，这是 w_+ 在 $p = f(w)$ 从 $p_{\max}$ 变化到 0 时取值的区间。知道 f 的图形是凹的，我们有这个图形位于该区间内的切线之下和弦线之上。

f 在点 $w = 1, p = 0$ 处的切线是 $p + w = 1$ ，因为 $f'(1) = -1$ 。由于图形位于该切线之下，我们得出结论： $w_+ \leq 1 - p$ （仅在 $p = 0$ 处取等号）。

函数 f 的图上连接点 $w = 1, p = 0$ 和点 $w = h/(h+1), p = p_{\max}$ 的弦的方程为 $w = 1 - p/(p_{\max}(h+1))$ 。由于图位于弦的上方，我们有

$$w_+ \geq 1 - \frac{1}{p_{\max}(h+1)}p \approx 1 - ep,$$

其中当 h 较大时近似良好。图 2 说明了上述估计。（为了可见性，图 2 沿两个轴绘制了不同的比例。）

将 w_+ 的这些估计值代入我们之前关于块长度大于 k 的概率 q_k 的结果中，我们发现，对于较大的 k ， q_k 介于 $(1 - ep)^k$ 的一个常数倍和一个 $(1 - p)^k$ 的常数倍之间。这里的常数来自生成函数 $Q(z)$ 的部分分式展开。如果我们用平均块长度 $1/p$ 表示 k ，即如我们将 $k = M/p$ 设置为 q_k 是包含 0 的块长度至少是平均长度的 M 倍的概率，那么我们可以近似

$$(1 - ep)^k = ((1 - ep)^{1/ep})^{eM} \approx e^{-eM}$$

and

$$(1 - p)^k = ((1 - p)^{1/p})^M \approx e^{-M}.$$

因此，作为 M 的函数，长度是平均长度 M 倍的块的概率呈指数下降，并介于 e^{-eM} 的一个常数倍和一个 e^{-M} 的常数倍之间。指数中的实际因子取决于 p ；上估计 e^{-M} 在 p 非常小（即 f 的图像接近用于获得此估计的切线）时更准确，而下估计 e^{-eM} 在 p 相对较大（即接近 $p_{\max}$ ）时更准确。

当然，可以通过更准确地估计 w_+ 来获得 q_k 衰减率的更准确估计。例如，可以将 w_+ 表示为 $1 - p$ 的泰勒级数或 $p - p_{\max}$ 的普伊苏级数。

5.5. 区间过滤器方法的反向松弛

第 5.4 节中 q_k 的估计使我们能够按如下方式估计区间过滤器方法的反向松弛。考虑一个随机 $F \in \text{PFE}^{\mathbb{D}}$ 。其 $\leftarrow$ 松弛为特定自然数 k 的概率是切割点 at $-k$ 的概率与给定在 $-k$ 处有切割点时的条件概率（即在该处没有进一步切割点的概率）的乘积。前者因子是 $p = u^h v$ ，后者则由于平移不变性而为 q_k 。（我们处于一个令人愉快的情况，即这两个因子均不受 F_1^+ 和 F_2^+ 的影响；只有 F^- 是相关的，因此我们基本上是在处理一个 $\text{PFE}^{\mathbb{Z}}$ 中的文件。）

因此， $\leftarrow$ 松弛的期望是

$$\mathbb{E}(\leftarrow \text{Slack}) = \sum_{k=0}^{\infty} k p q_k = p Q'(1),$$

其中 Q 是在第 5.4 节中得到的生成函数，

$$Q(z) = \sum_{k=0}^{\infty} q_k z^k = \frac{1}{1 - z + p z^{h+1}}.$$

Differentiating, we find

$$Q'(z) = -(1 - z + pz^{h+1})^{-2}(-1 + p(h+1)z^h)$$

and so

$$Q'(1) = p^{-2}(1 - p(h+1))$$

and

$$\mathbf{E}(\leftarrow \text{Slack}) = \frac{1}{p} - (h+1).$$

The normalized $\leftarrow \text{slack}$ therefore has expectation $1 - p(h+1)$.

If h is fixed, then the value of p that minimizes the normalized $\leftarrow \text{slack}$ is $p_{\max} = h^h/(h+1)^{h+1}$, and the minimum of the normalized $\leftarrow \text{slack}$ is

$$1 - p_{\max}(h+1) = 1 - \left(\frac{h}{h+1}\right)^h \approx 1 - \frac{1}{e}$$

for large h .

6. Local maximum chunking

As the name suggests, local maximum chunking selects, as the cutpoints of a file F , those positions i where the entry $F(i)$ attains a local maximum. Recall that we fixed a linear ordering of PFE, so the notion of (strict) maximum makes sense. “Local” means that $F(i)$ is a maximum within a radius of h positions to either side. Here is the formal definition.

Definition 55. *Local maximum chunking* with horizon h is the chunking method that declares a position i to be a cutpoint of a file F if and only if

- i is h -internal to F and
- $F(i) > F(j)$ for all $j \neq i$ in the interval $[i-h, i+h]$.

We refer to the interval $[i-h, i+h]$ as the *window* around i . Clearly (and fortunately for the terminology), local maximum chunking is a local chunking method.

We shall often assume that the number $|\text{PFE}|$ of potential file entries is much larger than the horizon h . In applications, we might typically have $|\text{PFE}| = 2^{32}$ and $h = 2^7$, so the assumption is justified in practice. The main purpose of the assumption is to excuse us from paying attention to the strictness of the inequality $F(i) > F(j)$ in the definition. Of course, if we had written $F(i) \geq F(j)$ instead, then there could be more cutpoints in a file, points that are tied for maximum with some other point in their window. When $|\text{PFE}| \gg h$, ties for maximum become very unlikely, so we can safely ignore them when estimating the statistical properties of local maximum chunking. We shall use phrases like “ignoring ties” to refer to the assumption that $|\text{PFE}| \gg h$ and to indicate how it is being used.

Remark 56. According to the so-called “birthday paradox,” one would need the stronger inequality $|\text{PFE}| \gg h^2$ to ensure that all ties are unlikely. This is no problem, for two reasons. First, the typical values quoted above satisfy the stronger inequality. Second, we often don’t care about avoiding all ties but only ties for maximum, and for this $|\text{PFE}| \gg h$ suffices.

6.1. Statistics of local maximum chunking

It is clear from the definition of local maximum chunking that, if i is a cutpoint, then no other point in its window can be a cutpoint. Therefore, all chunks have length at least $h+1$, except that the first chunk (in a finite or singly infinite file) can have length h .

Ignoring ties, we easily see that the cutpoint probability for local maximum chunking is $p = 1/(2h+1)$. Indeed, if i is any h -internal position of a file, then exactly one of the $2h+1$ positions in the window around i must have the largest file entry in this window (because there are no ties for maximum), and each of the positions has an equal chance, $1/(2h+1)$.

It follows, by Kac’s theorem via Proposition 25, that the expectation of the chunk length is $2h+1$.

Remark 57. Abstaining from the assumption that there is no tie for the maximum, we still have a formula for the cutpoint probability, namely

$$p = \sum_{k=0}^{m-1} \frac{1}{m} \left(\frac{k}{m}\right)^{2h},$$

求导后, 我们发现

$$Q'(z) = -(1 - z + pz^{h+1})^{-2}(-1 + p(h+1)z^h)$$

因此

$$Q'(1) = p^{-2}(1 - p(h+1))$$

and

$$\mathbf{E}(\leftarrow \text{Slack}) = \frac{1}{p} - (h+1).$$

归一化的 $\leftarrow \text{松弛}$ 因此有期望 $1 - p(h+1)$ 。

如果 h 是固定的, 那么使归一化的 $\leftarrow \text{松弛}$ 最小的 p 的值是 $p_{\max} = h^h/(h+1)^{h+1}$, 归一化的 $\leftarrow \text{松弛}$ 的最小值是

$$1 - p_{\max}(h+1) = 1 - \left(\frac{h}{h+1}\right)^h \approx 1 - \frac{1}{e}$$

对于大的 h 。

6. 局部最大值分块

顾名思义, 局部最大值分块选择文件 F 的切点为那些位置 i , 其中 $F(i)$ 达到局部最大值。回想我们固定了 PFE 的线性排序, 所以 (严格) 最大值的概念是有意义的。“局部”意味着 $F(i)$ 在其左右两侧 h 个位置范围内是一个最大值。以下是正式定义。

定义 55. 局部最大值分块方法与视野 h 是一种分块方法, 它将位置 i 声明为文件 F 的切割点, 当且仅当

- i 是 F 的 h -内部条件期望
- $F(i) > F(j)$ 对于所有 $j \neq i$ 在区间 $[i-h, i+h]$ 。

我们称区间 $[i-h, i+h]$ 为围绕 i 的窗口。显然 (而且对术语来说幸运的是), 局部最大值分块方法是局部分块方法。

我们将经常假设潜在文件条目数 $|\text{PFE}|$ 远大于视野 h 。在实际应用中, 我们可能通常有 $|\text{PFE}| = 2^{32}$ 和 $h = 2^7$, 因此该假设在实践中是合理的。该假设的主要目的是让我们不必关注定义中不等式 $F(i) > F(j)$ 的严格性。当然, 如果我们写了 $F(i) \geq F(j)$, 那么文件中可能会有更多的切点, 这些点是与其窗口中其他点并列的最大值。当 $|\text{PFE}| \gg h$ 时, 并列最大值的可能性非常小, 因此我们在估计局部最大值分块统计特性时可以安全地忽略它们。我们将使用“忽略并列”等短语来指代该假设 $|\text{PFE}| \gg h$, 并表明它是如何被使用的。

备注 56. 根据所谓的“生日悖论”, 需要更强的不等式 $|\text{PFE}| \gg h^2$ 来确保所有平局都不太可能。这没有问题, 原因有两个。首先, 上述典型值满足更强的不等式。其次, 我们通常不在乎避免所有平局, 而只在乎最大值的平局, 对此 $|\text{PFE}| \gg h$ 就足够了。

6.1. 局部最大值分块统计

从局部最大值块的定义中可以清楚地看出, 如果 i 是一个切割点, 那么在其窗口中的其他任何点都不能是切割点。因此, 所有块的长度至少为 $h+1$, 但第一个块 (在有限文件或单无限文件中) 可以有长度 h 。

忽略平局, 我们很容易看出局部最大值分块法的断点概率是 $p = 1/(2h+1)$ 。确实, 如果 i 是文件的一个 h -内部位置, 那么 i 周围窗口中的 $2h+1$ 位置中必须恰好有一个拥有这个窗口中最大的文件条目 (因为最大值没有平局), 并且每个位置有相等的概率, $1/(2h+1)$ 。根据卡茨定理和命题25, 可知块长度的期望是 $2h+1$ 。

备注 57. 我们暂不假设最大值不存在并列的情况, 仍然有一个关于断点概率的公式, 即

$$p = \sum_{k=0}^{m-1} \frac{1}{m} \left(\frac{k}{m}\right)^{2h},$$

where we have abbreviated $|PFE|$ as m , so $PFE = \{0, 1, \dots, m-1\}$. The term indexed by k in this sum represents the probability that the file value at i is k (probability $1/m$) and the other $2h$ file values in the window are in the range $[0, k-1]$ (probability k/m for each position and thus $(k/m)^{2h}$ for all $2h$ positions). As we saw in Section 2.3, this sum is approximately $1/(2h+1)$ for large m , in agreement with the “no ties” estimate above. Also, we exhibited in Section 2.3 the more precise formula

$$p = \frac{1}{m^{2h+1}} \frac{1}{2h+1} \sum_{k=0}^{2h} \binom{2h+1}{k} B_k m^{2h+1-k} = \frac{1}{2h+1} - \frac{1}{2m} + \frac{h}{6m^2} + \dots$$

As $m \rightarrow \infty$, all terms except $1/(2h+1)$ approach zero.

Unfortunately, we do not have good estimates for the slack of the local maximum chunking method or for the variance of the chunk length. We briefly indicate in this section what we know and where the difficulty arises in trying to go farther.

Consider first the slack. Because of the left-right symmetry of the local maximum method, it is clear that the expectations of the slack and reverse slack are equal. We therefore confine attention to the former.

Consider a random $F \in PFE^{\mathbb{M}}$. What is the probability that its slack is a particular number $k \geq 0$? For $k \leq h$, all that is required is that k be a common cutpoint of F_1 and F_2 ; it will be the first non-negative cutpoint because cutpoints are never within a distance h of each other. The windows centered at k in the two files together contain $3h-k+1$ positions, namely the $k+h+1$ non-negative positions from 0 through $k+h$ and $2(h-k)$ negative positions, $k-h$ through -1 in each of the two files. Thus, the probability that k is the slack is $1/(3h-k+1)$.

Consequently, the probability that the slack is at most h is

$$\sum_{k=0}^h \frac{1}{3h-k+1} = \sum_{j=2h+1}^{3h+1} \frac{1}{j} \approx \ln(3h+1) - \ln(2h) \approx \ln \frac{3}{2}.$$

For $k > h$, however, the situation is more complicated. The probability that k is a common cutpoint is actually simpler; it is $1/(2h+1)$ because the window $[k-h, k+h]$ lies entirely in the common part F^+ of the two files. What is difficult is finding the probability, given that k is a cutpoint, that it is the first one ≥ 0 . The fact that k is a cutpoint gives some information about the values of F^+ in $[k-h, k-1]$; they are more likely to be smaller than they would be if we knew nothing about k , for all these values are $\leq F(k)$. And that increases the probability that positions just to the left of this window (positions $k-2h$ to $k-h+1$) are cutpoints; their windows overlap the region where F^+ has smaller than usual values, so they have a better chance of being a local maximum. It is this indirect effect of the condition “ k is a cutpoint” on the cutpoint probabilities between h and $2h$ positions earlier that is difficult to compute and has prevented us from estimating the slack of the local maximum method.

The problem of computing the slack is indirectly related to the problem of computing the variance of the chunk length, as follows. Obeying Pólya’s dictum (as quoted by Halmos in [14]), “if you can’t solve a problem, then there is an easier problem that you can’t solve — find it!” notice that the difficulty described above, which prevents us from computing the expected slack of a random $F \in PFE^{\mathbb{M}}$, also affects the simpler case of a random file $F \in PFE^{\mathbb{Z}}$, if we consider the obvious analog of the slack, namely the smallest non-negative cutpoint. We know that each point $k \in \mathbb{Z}$ has probability $1/(2h+1)$ of being a cutpoint, but its probability of being the first non-negative cutpoint is subject, when $k > h$, to the same complications encountered in trying to compute the slack.

Instead of the first non-negative cutpoint of a doubly infinite file F , it is convenient to consider the first positive cutpoint, which we call $\rho(F)$. The expectation is merely increased by 1, because the chunking method is shift-invariant. The notation ρ matches that of Section 2.2, the number of iterations of the Bernoulli shift needed to bring F into the set Cut0 of files with a cutpoint at 0. So the expectation of the first positive cutpoint is given by Proposition 11 in terms of the probability of Cut0 and the conditional variance of ρ , conditional on the event Cut0. The probability of Cut0 is just the cutpoint probability, $1/(2h+1)$. And $\text{Var}(\rho|A)$ is just the variance of the chunk length.

Thus, the problem of computing the variance of the chunk length turns out to be equivalent to a simplified version of the problem of computing the average slack. Note that this equivalence is not specific to local maximum chunking but applies to any shift-invariant chunking method for which the average chunk length is known.

7. Probability of long chunks

This section is about another measure of quality of local chunking methods, namely the probability of getting exceptionally large chunks. More precisely, we deal with the probability that a long interval $[1, l]$ contains no cutpoints of a random, doubly infinite file. Since very large chunks are undesirable, one wants this probability to be small. But one does not want to achieve this by making all the chunks too small, since very small chunks are also undesirable. In order to fairly compare different chunking methods, it is therefore reasonable to choose their parameters so that the average chunk sizes agree and then to ask about the probabilities of significantly larger chunks. In this section, we carry out a comparison of the chunking methods we have discussed, computing the probability of finding no cutpoint in an interval $[1, l]$ where l is a specified multiple of the average chunk size.

其中我们将 $|PFE|$ 简记为 m , 因此 $PFE = \{0, 1, \dots, m-1\}$. 这个求和项中索引为 k 的部分表示文件值在 i 处为 k 的概率 (概率 $1/m$) , 以及窗口中其他 $2h$ 个文件值在范围 $[0, k-1]$ 内的概率 (每个位置的概率为 k/m , 因此 $(k/m)^{2h}$ 对于所有 $2h$ 个位置)。正如我们在第 2.3 节中看到的, 这个求和项对于较大的 m 大约等于 $1/(2h+1)$, 与上述“无并列”估计一致。此外, 我们在第 2.3 节中还展示了更精确的公式

$$p = \frac{1}{m^{2h+1}} \frac{1}{2h+1} \sum_{k=0}^{2h} \binom{2h+1}{k} B_k m^{2h+1-k} = \frac{1}{2h+1} - \frac{1}{2m} + \frac{h}{6m^2} + \dots$$

随着 $m \rightarrow \infty$ 的增大, 除了 $1/(2h+1)$ 以外的所有项都趋近于零。

不幸的是, 我们还没有局部最大值分块方法的松弛或块长度方差的良好估计。在本节中, 我们简要说明我们所知的内容以及尝试进一步前进时遇到的困难。首先考虑松弛。由于局部最大值方法的左右对称性, 很明显松弛和反向松弛的期望是相等的。因此, 我们将注意力集中在前者。

考虑一个随机 $F \in PFE^{\mathbb{M}}$ 。它的松弛是特定数字 $k \geq 0$ 的概率是多少? 对于 $k \leq h$, 只需要 k 是 F_1 和 F_2 的一个公共切点; 它将是第一个非负切点, 因为切点彼此之间的距离永远不会小于 h 。在两个文件中, 以 k 为中心的窗口总共包含 $3h-k+1$ 个位置, 即从 0 到 $k+h$ 的 $k+h+1$ 个非负位置和 $2(h-k)$ 个负位置, 每个文件中从 $k-h$ 到 -1 。因此, k 是松弛的概率是 $1/(3h-k+1)$ 。

因此, 松弛量不超过 h 的概率是

$$\sum_{k=0}^h \frac{1}{3h-k+1} = \sum_{j=2h+1}^{3h+1} \frac{1}{j} \approx \ln(3h+1) - \ln(2h) \approx \ln \frac{3}{2}.$$

然而, 对于 $k > h$, 情况更为复杂。 k 是公共切点的概率实际上更简单; 这是因为窗口 $[k-h, k+h]$ 完全位于两个文件的公共部分 F^+ 中。困难之处在于, 在已知 k 是切点的情况下, 找到它是第一个 ≥ 0 的概率。 k 是切点这一事实为 F^+ 在 $[k-h, k-1]$ 中的值提供了一些信息; 它们比我们不知道 k 的情况下要小, 因为所有这些值都是 $\leq F(k)$ 。这增加了该窗口左侧位置 (位置 $k-2h$ 到 $k-h+1$) 是切点的概率; 它们的窗口与 F^+ 值通常较小的区域重叠, 因此它们更有可能是局部最大值。正是条件 “ k 是切点” 对 h 和 $2h$ 位置之间切点概率的间接影响难以计算, 这也阻碍了我们估计局部最大值方法的松弛量。

计算松弛的问题与计算块长度方差的问题间接相关, 具体如下。遵循Polya的箴言 (哈洛斯引用 [14]), ‘如果你不能解决一个问题, 那么有一个更容易的问题你不能解决——找到它!’ 注意到上述困难, 即阻止我们计算随机 $F \in PFE^{\mathbb{M}}$ 的预期松弛, 也影响随机文件 $F \in PFE^{\mathbb{Z}}$ 的更简单情况, 如果我们考虑松弛的明显类比, 即最小的非负切割点。我们知道每个点 $k \in \mathbb{Z}$ 有概率 $1/(2h+1)$ 成为切割点, 但当 $k > h$ 时, 它成为第一个非负切割点的概率也受到计算松弛时遇到的同样复杂性的影响。

与其考虑双无穷文件F的第一个非负切割点, 不如考虑第一个正切割点, 我们称之为 $\rho(F)$ 。期望值仅增加了1, 因为分块方法是移位不变的。符号 ρ 与第2.2节的符号匹配, 即需要伯努利移位多少次才能将F带入具有0处切割点的文件集合Cut0。因此, 第一个正切割点的期望值由命题11根据Cut0的概率和给定事件Cut0的 ρ 的条件方差给出。Cut0的概率就是切割点概率, $1/(2h+1)$ 。而 $\text{Var}(\rho|A)$ 就是块长度的方差。

因此, 计算块长度的方差的问题, 结果被证明等同于计算平均松弛度问题的简化版本。请注意, 这种等价性并非特定于局部最大值块划分方法, 而是适用于任何已知平均块长度的平移不变块划分方法。

7. 长块的概率

本节讨论局部分块方法的另一种质量度量, 即获得异常大块的概率。更精确地说, 我们处理的是长区间 $[1, l]$ 中不包含随机双无穷文件的切点的概率。由于非常大的块是不受欢迎的, 人们希望这个概率很小。但人们不希望通过使所有块都太小来实现这一点, 因为非常小的块也是不受欢迎的。为了公平地比较不同的分块方法, 因此合理地选择它们的参数, 使得平均块大小一致, 然后询问显著更大的块的概率。在本节中, 我们对讨论过的分块方法进行比较, 计算在区间 $[1, l]$ 中找到不包含切点的概率, 其中 l 是平均块大小的指定倍数。

As we want to show that the probability of such a long cut-less interval is smaller (and thus better) for the local maximum method than for its competitors, we shall estimate the former from above and the latter from below.

The most complicated computation here will be for the local maximum method, so we arrange our notation and conventions to maximize convenience there. As before, we let h be the horizon for the local maximum method, so the expected chunk length is $2h + 1$. We shall estimate the probability of finding no cutpoints in the interval $[1, 2hM]$, which is essentially (glossing over the distinction between $2h$ and $2h + 1$) M average chunk lengths long. Here the multiplier M should be larger than 1, but it need not be huge. $M = 5$ might be a reasonable choice; i.e., we might want to have low probability that a particular chunk is more than 5 times the average length.

Our calculation will assume that there are no ties among the file entries in the interval under consideration. Considerably weaker assumptions would suffice, but this one is easy to use and is satisfied with high probability for typical values of the parameters.

7.1. Pure point filter method

We first obtain a lower estimate for the probability of long chunks in the pure point filter method. As indicated above, for a fair comparison, we adjust the parameter c of the pure point filter method to produce the same average chunk length $2h + 1$ as the local maximum method. Then each position in a random file has, independently, probability $1/(2h + 1)$ of being a cutpoint. Therefore, the probability of no cutpoint among $2hM$ points is

$$\left(1 - \frac{1}{2h + 1}\right)^{2hM}.$$

We resist the temptation to approximate $1 - (1/(2h + 1))$ by $e^{-1/(2h+1)}$, because this is an approximation from above and we want one from below. Instead, we use that

$$\frac{2h + 1}{2h} = 1 + \frac{1}{2h} < e^{1/(2h)},$$

so

$$1 - \frac{1}{2h + 1} = \frac{2h}{2h + 1} > e^{-1/(2h)},$$

and therefore the probability of no cut point in $[1, 2hM]$ in the pure point filter model is

$$\left(1 - \frac{1}{2h + 1}\right)^{2hM} > e^{-2hM/2h} = e^{-M}.$$

7.2. LBFS method

We next perform the analogous computation for the LBFS chunking method. That method had two parameters, previously called c and h . The latter notation is no longer usable, since we are now using h as the horizon of the local maximum method. Fortunately, in our earlier discussion of the LBFS method, we introduced the notation k for the ratio h/c , and this letter is still available. So we shall carry out the computation for the LBFS method with $1/c$ as the probability of any position being a candidate and with kc as the horizon. We think of k as fixed; for example k was $1/4$ in the version of the LBFS method proposed in [20]. We adjust c to make the expected chunk length $c + kc$ match our $2h + 1$. Thus, $c = (2h + 1)/(k + 1)$.

Now the probability that the interval $[1, 2hM]$ contains no cutpoint is obviously bounded below by the probability that this interval contains no candidate, namely

$$\left(1 - \frac{1}{c}\right)^{2hM} = \left(1 - \frac{k + 1}{2h + 1}\right)^{2hM}.$$

We estimate this from below by the same technique already used for the pure point filter method. We have

$$\frac{2h + 1}{2h - k} = 1 + \frac{k + 1}{2h - k} \leq e^{\frac{k+1}{2h-k}}$$

and therefore

$$1 - \frac{k + 1}{2h + 1} = \frac{2h - k}{2h + 1} \geq e^{-\frac{k+1}{2h-k}}.$$

Thus, the probability that there is no cutpoint in $[1, 2hM]$ is at least

$$e^{-(k+1)2hM/(2h-k)}.$$

This estimate is useless for very large k , but for reasonable k , small compared to h , this lower bound is essentially $e^{-(k+1)M}$. For example, when $k = 1/4$, we have the lower bound $e^{-5M/4}$. More generally, as long as $k \ll h$, we get a lower bound that decreases only exponentially with M .

由于我们希望表明这种无切点的长区间对于局部最大值方法 (local maximum method) 来说比其竞争对手 (competitors) 的概率更小 (因此更好), 我们将前者从上方估计, 后者从下方估计。

这里最复杂的计算将是局部最大值方法, 因此我们调整符号和约定以最大化该方法的便利性。如前所述, 我们让 h 成为局部最大值方法的视野, 因此预期的块长度是 $2h + 1$ 。我们将估计在区间 $[1, 2hM]$, 中找到没有切点的概率, 这基本上是 (忽略 $2h$ 和 $2h + 1$ 之间的区别) M 个平均块长度长。在这里, 乘数 M 应该大于 1, 但不必很大。 $M = 5$ 可能是一个合理的选择; 也就是说, 我们可能希望某个特定块的长度不会超过平均长度的 5 倍的概率较低。

我们的计算将假设在所考虑的区间内, 文件条目之间没有平局。更弱的假设就足够了, 但这个假设易于使用, 并且对于典型参数值具有很高的概率成立。

7.1. 纯点过滤器方法

我们首先对纯点过滤器方法中长块的概率获得一个下界估计。如上所述, 为了公平比较, 我们将纯点过滤器方法的参数 c 调整为产生与局部最大值方法相同的平均块长度 $2h + 1$ 。然后, 随机文件中的每个位置独立地有 $1/(2h + 1)$ 的概率成为切割点。因此, 在 $2hM$ 个点中没有切割点的概率是

$$\left(1 - \frac{1}{2h + 1}\right)^{2hM}.$$

我们抵制了用 $1 - (1/(2h + 1))$ 近似 $e^{-1/(2h+1)}$ 的诱惑, 因为这是一个从上近似, 而我们想要一个从下近似。相反, 我们使用

$$\frac{2h + 1}{2h} = 1 + \frac{1}{2h} < e^{1/(2h)},$$

so

$$1 - \frac{1}{2h + 1} = \frac{2h}{2h + 1} > e^{-1/(2h)},$$

因此, 在纯点过滤器模型中, $[1, 2hM]$ 没有切割点的概率是

$$\left(1 - \frac{1}{2h + 1}\right)^{2hM} > e^{-2hM/2h} = e^{-M}.$$

7.2. LBFS 方法

我们接下来对 LBFS 分块方法执行类似的计算。该方法有两个参数, 以前称为 c 和 h 。后者不再适用, 因为我们现在将 h 作为局部最大值方法的视野。幸运的是, 在我们的早期关于 LBFS 方法的讨论中, 我们引入了 h/c 的比率 k 的符号, 这个字母仍然可用。因此, 我们将使用 $1/c$ 作为任何位置成为候选的概率, 并使用 kc 作为视野来对 LBFS 方法进行计算。我们认为 k 是固定的; 例如, k 是在 [20] 中提出的 LBFS 方法的版本中的 $1/4$ 。我们调整 c 以使预期块长度 $c + kc$ 与我们的 $2h + 1$ 匹配。因此, $c = (2h + 1)/(k + 1)$ 。

现在, 区间 $[1, 2hM]$ 不包含切割点的概率显然下界为该区间不包含候选的概率, 即

$$\left(1 - \frac{1}{c}\right)^{2hM} = \left(1 - \frac{k + 1}{2h + 1}\right)^{2hM}.$$

我们用纯点过滤器方法已使用的相同技术从下方估计这个概率。我们有

$$\frac{2h + 1}{2h - k} = 1 + \frac{k + 1}{2h - k} \leq e^{\frac{k+1}{2h-k}}$$

因此

$$1 - \frac{k + 1}{2h + 1} = \frac{2h - k}{2h + 1} \geq e^{-\frac{k+1}{2h-k}}.$$

因此, $[1, 2hM]$ 中没有切割点的概率至少为

$$e^{-(k+1)2hM/(2h-k)}.$$

对于非常大的 k , 这个估计没有用, 但对于合理的 k (相对于 h 较小), 这个下界基本上是 $e^{-(k+1)M}$ 。例如, 当 $k = 1/4$ 时, 我们有下界 $e^{-5M/4}$ 。更一般地, 只要 $k \ll h$, 我们就得到一个仅随 M 指数下降的下界。

7.3. Interval filter method

Although our main goal is to show that the local maximum method makes long intervals without cutpoints far less likely than the previously known LBFS chunking method, we include also a rough estimate for the corresponding probability in the interval filter method. As with the point filter and LBFS chunking methods, we shall find that the probability of finding no cutpoint in a long interval decreases (only) exponentially with respect to the interval's length.

We already computed in Section 5.4 the conditional probability q_l of finding no cutpoint in $[1, l]$ given that there is a cutpoint at 0. Now we shall compute the unconditional probability of finding no cutpoint in $[1, l]$. We shall obtain it by combining conditional probabilities, the conditions being the various possibilities for what happens at and shortly before position 0. Since we are interested in long intervals, we shall assume that $l > h$.

For each j in the range $0 \leq j < h$, let C_j be the event that the entry $F(-j)$ at position $-j$ is in V and all the entries in positions $-j+1$ to 0 are in U . So $\text{Prob}(C_j) = vu^j$. The conditional probability, given C_j , of having no cutpoint in $[1, l]$ is

$$u^l + \sum_{i=1}^{h-j} u^{i-1} v q_{l-i}.$$

The first term here is the probability that there is no element of V , and therefore certainly no cutpoint, at any position in $[1, l]$. The term indexed by i in the sum is the probability that V occurs somewhere in the interval $[1, l]$, that the first such occurrence is at i , and that no cutpoint occurs thereafter, in $[i+1, l]$. We restrict i to range only up to $h-j$ because, if the first positive i where V occurs were at position $h-j+1$ or later, then it would be preceded by at least h consecutive U 's, from position $-j+1$ to $i-1$ inclusive. Then this i would be a cutpoint, so this situation does not contribute to the probability we are computing.

Let D be the event that none of the C_j occur, i.e., the event that all file entries from position $-h+1$ to 0 are U 's. This event has probability u^h and the conditional probability, given D , of finding no cutpoint in $[1, l]$ is simply the probability that all l of the file entries from position 1 to l are in U ; the reason is that, if there were any V in this range, then the first one would, because of D , be a cutpoint.

Thus, the probability of finding no cutpoint in $[1, l]$ is given by

$$\sum_{j=0}^{h-1} u^j v \left(u^l + \sum_{i=1}^{h-j} u^{i-1} v q_{l-i} \right) + u^{h+l}.$$

The sum over j of $u^j v u^l$ is just a finite geometric series; evaluating it and remembering that $1-u=v$, we find simply $u^l - u^{h+l}$. The second term here cancels the u^{h+l} that arose from D , so what remains is u^l . The double sum over j and i can be simplified somewhat by reversing the order of summation. The final result is that the probability that the interval $[1, l]$ contains no cutpoint is

$$u^l + v \sum_{i=1}^h q_{l-i} u^{i-1} - p \sum_{i=1}^h q_{l-i}.$$

Note that i ranges only up to h , so for large (compared to h) values of l , all the subscripts of q 's in this formula are large, so we can use the asymptotic estimates from Section 5.4. Thus, we find that all terms in our formula decrease exponentially as l grows; specifically the probability of finding no cutpoint in $[1, l]$ is asymptotically a constant multiple of w_+^l . From our rough estimates of w_+ in Section 5.4, we can infer that, if $l = M/p$, so that the interval $[1, l]$ is M times as long as an average chunk, then the probability that $[1, l]$ contains no cutpoint is asymptotically Ae^{-BM} for some constants A and B , with B lying between 1 and e .

7.4. Local maximum method

In this subsection, we estimate from above the probability that the local maximum method with horizon h produces no cutpoint in the interval $[1, 2hM]$. Recall that we assume that $|PFE|$ is so large that we can ignore the possibility of two relevant positions having equal entries in a random file. Here the relevant positions are not just the interval $[1, 2hM]$ but an additional h positions at either end, since these are within the windows of positions 1 and $2hM$ and may thus affect cutpoints within $[1, 2hM]$.

Observe that whether a position i is a local maximum depends only on the relative ordering of the values of F on the interval $[i-h, i+h]$, not on the actual values of F . Thus, we could replace our probability space of random files with the finite space of all linear orderings of the relevant interval of positions $[1-h, 2hM+h]$; all of the $(L+2h)!$ orderings are equally probable.

Formally, this means that we use the function Φ that assigns to each $F \in \text{PFE}^{\mathbb{Z}}$ (without ties in the relevant segment) the ordering induced on $[1-h, 2hM+h]$ by

$$m < n \Leftrightarrow F(m) < F(n),$$

7.3. 区间过滤方法

尽管我们的主要目标是证明局部最大值方法使得没有切割点的长区间比之前已知的 LBFS 分块方法远不可能，但我们也包括了区间过滤器方法中相应概率的粗略估计。与点过滤器和 LBFS 分块方法一样，我们会发现长区间中找不到切割点的概率随区间长度的增加而（仅）指数下降。

我们在第 5.4 节已经计算了在 $[1, l]$ 中给定在 0 处存在切割点的情况下，未找到切割点的条件概率 q_l 。现在我们将计算在 $[1, l]$ 中未找到切割点的无条件概率。我们将通过结合条件概率来获得它，条件是 0 处及稍前位置可能发生的情况的各种可能性。由于我们关心的是长区间，我们将假设 $l > h$ 。

对于范围内的每个 j ，令 C_j 是位置 $-j$ 处的条目 $F(-j)$ 在 V 中且位置 $-j+1$ 到 0 处的所有条目都在 U 的事件。因此 $\text{Prob}(C_j) = vu^j$ 。给定 C_j ，在 $[1, l]$ 中未找到切割点的条件概率是

$$u^l + \sum_{i=1}^{h-j} u^{i-1} v q_{l-i}.$$

这里的第一个项是 V 中没有任何元素的概率，因此在 $[1, l]$ 的任何位置上都没有切割点，这是肯定的。求和中的第 i 个项是 V 在区间 $[1, l]$ 中出现的概率，且首次出现的位置是 i ，并且之后在 $[i+1, l]$ 中没有切割点。我们将 i 限制在 $h-j$ 之内，因为如果 V 首次出现的第一个正 i 在位置 $h-j+1$ 或更晚，那么它之前会有至少 h 个连续的 U ，从位置 $-j+1$ 到 $i-1$ 包含在内。那么这个 i 将是一个切割点，因此这种情况不会对我们计算的概率有贡献。

设 D 为事件 C_j 全部不发生的事件，即位置 $-h+1$ 到 0 的所有文件条目都是 U 的事件。这个事件的概率是 u^h ，给定 D 的情况下，在 $[1, l]$ 中没有找到切割点的条件概率，简单地就是位置 1 到 l 的所有文件条目都在 U 中的概率；原因是，如果这个范围内有任何 V ，那么由于 D ，第一个 V 将是一个切割点。

因此，在 $[1, l]$ 中找到不存在切割点的概率为

$$\sum_{j=0}^{h-1} u^j v \left(u^l + \sum_{i=1}^{h-j} u^{i-1} v q_{l-i} \right) + u^{h+l}.$$

对 j 求和的 $u^j v u^l$ 仅仅是一个有限几何级数；计算它并记住 $1-u=v$ ，我们得到简单的 $u^l - u^{h+l}$ 。这里的第二项抵消了由 D 产生的 u^{h+l} ，因此剩下的是 u^l 。通过反转求和顺序，双重求和 j 和 i 可以在一定程度上简化。最终结果是，区间 $[1, l]$ 中不存在切割点的概率为

$$u^l + v \sum_{i=1}^h q_{l-i} u^{i-1} - p \sum_{i=1}^h q_{l-i}.$$

注意 i 只范围到 h ，所以对于大于 h 的 l 值，此公式中 q 的所有下标都很大，因此我们可以使用第 5.4 节的渐近估计。因此，我们发现我们公式中的所有项都随着 l 的增长呈指数下降；具体来说，在 $[1, l]$ 中找到不存在切割点的概率渐近地是 w_+^l 的一个常数倍。根据第 5.4 节对 w_+ 的粗略估计，我们可以推断出，if $l = M/p$ ，因此如果区间 $[1, l]$ 是平均块长的 M 倍长，那么 $[1, l]$ 中不存在切割点的概率渐近地为 Ae^{-BM} ，其中 A 和 B 是一些常数， B 介于 1 和 e 之间。

7.4. 局部最大值方法

在本小节中，我们估计局部最大值方法在区间 $[1, 2hM]$ 中不产生切点的概率。回想一下，我们假设 $|PFE|$ 非常大，因此可以忽略两个相关位置在随机文件中具有相同条目的可能性。这里的“相关位置”不仅包括区间 $[1, 2hM]$ ，还包括两端各 h 个位置，因为这些位置位于位置 1 和 $2hM$ 的窗口内，因此可能影响区间 $[1, 2hM]$ 内的切点。

注意到位置 i 是否为局部最大值仅取决于 F 在区间 $[i-h, i+h]$ 上的相对排序，而与 F 的实际值无关。因此，我们可以用所有相关位置区间的线性排序的有限空间来替换随机文件的概率空间。所有 $[1-h, 2hM+h]$; L ($h+2$ 排序都是等可能的。

形式上，这意味着我们使用函数 Φ ，它将每个 $F \in \text{PFE}^{\mathbb{Z}}$ （在相关片段中没有并列）映射到 $[1-h, 2hM+h]$ 上由所诱导的排序上。

$$m < n \Leftrightarrow F(m) < F(n),$$

we observe that our probability measure on $\text{PFE}^{\mathbb{Z}}$ projects via Φ to the uniform measure on the set of orderings, and we observe that the notion of local maximum (as well as the other notions that will play a role in our computations) depend on F only via $\Phi(F)$.

7.4.1. Splitting intervals without local maxima

To estimate from above the probability of the event “no local maximum in $[1, 2hM]$,” we first show that this event is included in some other events whose probability is easier to estimate. That is, we analyze sequences F that have no local maximum in $[1, 2hM]$, and we establish some other properties that all such sequences must have.

Accordingly, we consider a temporarily fixed F with no local maximum in $[1, 2hM]$, and we deduce some properties of F .

For each $n \in [1, 2hM]$, define $\mu(n)$ to be the element of $[n - h, n + h]$ where F has the largest value. Since n is not a local maximum, $\mu(n) \neq n$. Partition $[1, 2hM]$ into two pieces according to the relative order of $\mu(n)$ and n ; that is, define

$$A = \{n \in [1, 2hM]: \mu(n) < n\} \quad \text{and} \quad B = \{n \in [1, 2hM]: \mu(n) > n\}.$$

Lemma 58. A is an initial segment and B a final segment of $[1, 2hM]$.

Proof. By symmetry, it suffices to prove one of the two assertions; we choose the second. It suffices to show that, if $n \in B$ and $n < 2hM$, then $n + 1 \in B$. In the window $W = [n + 1 - h, n + 1 + h]$ centered at $n + 1$, all the points to the left of $n + 1$ are also in the window $[n - h, n + h]$ centered at n and are distinct from $\mu(n)$. They therefore have F -values smaller than $F(\mu(n))$. Furthermore $\mu(n) \in W$. So the points of W to the left of $n + 1$ cannot serve as $\mu(n + 1)$. $\square$

The argument in this proof extends easily to show that, when $n \in B$ and $n < 2hM$, then $\mu(n + 1)$ is either $\mu(n)$ or $n + 1 + h$, whichever has the larger F -value.

The lemma shows that an interval $[1, 2hM]$ without local maxima can be split into two subintervals that are without local maxima in a stronger, one-sided sense. In A , every element n is prevented from being a local maximum by something to its left in its window (i.e., something in $[n - h, n - 1]$, namely $\mu(n)$), while in B , everything is prevented from being a local maximum by something to its right (i.e., in $[n + 1, n + h]$).

7.4.2. Greedy increasing sequence

We temporarily confine our attention to the subinterval B of $[1, 2hM]$ where (for our still fixed F) we have, for each n , a $\mu(n) \in [n + 1, n + h]$ with a larger F -value than n has. (To avoid possible confusion, we note that $\mu(n)$ need not be in B ; it could be larger than $2hM$.) Of course, what we do with B can also be done symmetrically with A .

Let (g_k) be the $\rightarrow$ greedy increasing sequence in B for (the restriction to B of) the file F , as defined in Section 2.4. Since g_{k+1} is defined as the smallest $n > g_k$ with $F(n) > F(g_k)$ and since, by definition of B , $\mu(g_k)$ is such an n , we have

$$g_{k+1} \leq \mu(g_k) \leq g_k + h.$$

Thus, the greedy increasing sequence increases in steps of at most h and therefore has at least $\lceil |B|/h \rceil$ terms. (We can round $|B|/h$ up to an integer, rather than down, because g_0 is the first element of B and therefore g_k is no larger than the $(kh + 1)$ st element of B .) Notice that this fact makes $F \upharpoonright B$ quite atypical. Indeed, as we saw in Section 2.4 the expectation of the length of the greedy sequence in an interval of size $|B|$ is approximately $\ln |B|$. So the greedy sequence for F is far longer than expected when $|B|$ is sufficiently large compared to h .

Recall from Proposition 12 that the elements g_k of the greedy sequence are exactly the $\rightarrow$ maxima of F in B .

7.4.3. Good cuts

By a *cut* in $[1, 2hM]$, we mean a partition of $[1, 2hM]$ into two subintervals A' and B' , with A' lying to the left of B' . That is, $A' = [1, c]$ and $B' = [c + 1, 2hM]$, for some $c \in [0, 2hM]$; this allows the possibility that A' or B' could be empty. We think of this cut as being located between c and $c + 1$, and so we say that it lies just to the right of c and just to the left of $c + 1$. We also use terminology like “consecutive cuts” in the same sense.

A cut (A', B') will be called

- *right-good* if B' has at least one $\rightarrow$ maximum in every h consecutive elements,
- *left-good* if A' has at least one $\leftarrow$ maximum in every h consecutive elements, and
- *good* if it is both left- and right-good.

Our results above show that, when F has no local maximum in the interval $[1, 2hM]$, this interval admits at least one good cut, namely the partition into the specific pieces A and B defined above. It will be useful to know that, in fact, there are usually several good cuts.

Lemma 59. *At least one of the following three statements is true.*

我们观察到，我们的概率测度在 $\text{PFE}^{\mathbb{Z}}$ 上通过 Φ 投影到排序集合上的统一测度上，并且我们观察到局部最大值（以及将在我们的计算中起作用的其它概念）仅通过 $\Phi(F)$ 依赖于 F 。

7.4.1. 不含局部最大值的区间拆分

为估计事件 “ $[1, 2hM]$ 中无局部最大值” 的概率，我们首先证明该事件包含在其他概率更易估计的事件中。也就是说，我们分析在 $[1, 2hM]$ 中无局部最大值的序列 F ，并建立所有此类序列必须具有的其他性质。

因此，我们考虑一个暂时固定的 F （在 $[1, 2hM]$ 中无局部最大值），并推导出 F 的一些性质。

对于每个 $n \in [1, 2hM]$ ，定义 $\mu(n)$ 为 $[n - h, n + h]$ 中 F 取最大值的元素。由于 n 不是局部最大值， $\mu(n) \neq n$ 。根据 $\mu(n)$ 和 n 的相对顺序，将 $[1, 2hM]$ 拆分为两部分；也就是说，定义

$$A = \{n \in [1, 2hM]: \mu(n) < n\} \quad \text{and} \quad B = \{n \in [1, 2hM]: \mu(n) > n\}.$$

词条58. A 是 $[1, 2hM]$ 的初始段， B 是其最终段。

证明。由对称性可知，只需证明两个断言中的任意一个；我们选择第二个。只需证明，如果 $n \in B$ 和 $n < 2hM$ ，则 $n + 1 \in B$ 。在以 $n + 1$ 为中心的窗口 $W = [n + 1 - h, n + 1 + h]$ 中， $n + 1$ 左侧的所有点也都在以 n 为中心的窗口 $[n - h, n + h]$ 中，并且与 $\{n\}$ 不同。因此它们的 F 值小于 $F(\mu(n))$ 。此外 $\mu(n) \in W$ 。所以 W 在 $n + 1$ 左侧的点不能作为 $\mu(n + 1)$ 。 $\square$

此证明中的论证可以很容易地推广，以表明，当 $n \in B$ 和 $n < 2hM$ 时， $\mu(n + 1)$ 要么是 $\{n\}$ ，要么是 $n + 1 + h$ ，以具有较大 F 值者为准。

该词元表明，一个没有局部最大值的区间 $[1, 2hM]$ 可以被拆分成两个子区间，这两个子区间在更强的单边意义上也没有局部最大值。在 A 中，每个元素 n 都被其窗口（即 $[n - h, n - 1]$ 中的 $\mu(n)$ ）左侧的某个元素阻止成为局部最大值，而在 B 中，所有元素都被其窗口右侧的某个元素（即 $[n + 1, n + h]$ ）阻止成为局部最大值。

7.4.2. 贪心递增序列

我们暂时将注意力集中在区间 $[1, 2hM]$ 的子区间 B 上，其中（对于我们仍然固定的 F ）对于每个 n ，a $\mu(n) \in [n + 1, n + h]$ 都有一个比 n 具有更大 F 值的元素。（为了避免可能的混淆，我们指出 $\mu(n)$ 不必在 B 中；它可能大于 $2hM$ 。）当然，我们对 B 所做的一切也可以对称地应用于 A 。

设 (g_k) 是 B 中文件 F （如第 2.4 节定义的 B 的限制）的 $\rightarrow$ 贪婪递增序列。由于 g_{k+1} 被定义为最小的 $n > g_k$ ，且 $F(n) > F(g_k)$ ，并且根据 B 的定义， $\mu(g_k)$ 是这样的一个 n ，所以我们有

$$g_{k+1} \leq \mu(g_k) \leq g_k + h.$$

因此，贪婪递增序列以不超过 h 的步长递增，因此至少有 $\lceil |B|/h \rceil$ 个项。（我们可以将 $|B|/h$ 向上舍入为整数，而不是向下舍入，因为 g_0 是 B 的第一个元素，因此 g_k 不会大于 B 的 $(kh + 1)$ 个元素。）请注意，这一事实使 $F \upharpoonright B$ 非常不典型。实际上，正如我们在第 2.4 节看到的，大小为 $|B|$ 的区间中贪婪序列的期望长度大约为 $\ln |B|$ 。因此，当 $|B|$ 与 h 相比足够大时， F 的贪婪序列远比预期长得多。

从命题12中回忆起，贪婪序列的元素 g_k 恰好是 F 在 B 中的 $\rightarrow$ 极大值。

7.4.3. 好的切割

在 $[1, 2hM]$ 中，我们所说的切割是指将 $[1, 2hM]$ 划分为两个子区间 A' 和 B' ，其中 A' 位于 B' 的左侧。也就是说， $A' = [1, c]$ 和 $B' = [c + 1, 2hM]$ ，对于某个 $c \in [0, 2hM]$ ；这允许 A' 或 B' 可能为空。我们将此切割视为位于 c 和 $c + 1$ 之间，因此我们称它位于 c 的右侧且位于 $c + 1$ 的左侧。我们还使用“连续切割”等术语，含义相同。

一个切割 (A', B') 将被调用

- 如果 B' 在每 h 个连续元素中至少有一个 $\rightarrow$ 最大值，则称为右-好
- 如果 A' 在每 h 个连续元素中至少有一个 $\leftarrow$ 最大值，并且
- 如果它既是左-好的又是右-好的，则称为好

我们上述结果表明，当 F 在区间 $[1, 2hM]$ 中没有局部最大值时，该区间至少存在一个良好的切割，即上述定义的具体片段 A 和 B 的划分。实际上，知道通常存在多个良好切割将很有用。

引理 59。以下三个陈述中至少有一个是真的。

- (1) The cut $(\emptyset, [1, 2hM])$ is good.
- (2) The cut $([1, 2hM], \emptyset)$ is good.
- (3) There are at least $h + 1$ consecutive cuts, all of which are good.

Proof. The cut $(\emptyset, [1, 2hM])$ is vacuously left-good, so if it is right-good then we have the first alternative of the lemma. So we assume for the rest of the proof that $(\emptyset, [1, 2hM])$ is not right-good. Symmetrically, we assume that $([1, 2hM], \emptyset)$ is not left-good.

If a cut $([1, c], [c + 1, 2hM]) \neq ([1, 2hM], \emptyset)$ is right-good, then so is the next cut to the right, $([1, c + 1], [c + 2, 2hM])$, and therefore, by induction, so are all cuts further right. The reason is that each $\rightarrow$ maximum for $[c + 1, 2hM]$ except $c + 1$ is also a $\rightarrow$ maximum for $[c + 2, 2hM]$. (Note that a $\rightarrow$ maximum for $[c + 2, 2hM]$ need not be a $\rightarrow$ maximum for $[c + 1, 2hM]$, because its F -value may be smaller than $F(c + 1)$.) Let p be the largest number in $[1, 2hM]$ such that the cut just to the left of p is not right-good; this exists because the cut just to the left of 1, i.e., the cut $(\emptyset, [1, 2hM])$, is not right-good. Since right-goodness is preserved when one moves a cut to the right, we see that the right-good cuts are exactly those that are to the right of p . Because our original cut (A, B) is good, we know that $p \in A$.

Similarly, let q be the smallest number in $[1, 2hM]$ such that the cut just to the right of q is not left-good, and observe that $q \in B$. In particular, $p < q$. Also note that the left-good cuts are exactly those that are to the left of q . Therefore, the good cuts are those that lie between p and q . To show that there are at least $h + 1$ of these, suppose not. That means $q \leq p + h$.

We assume, for the rest of the proof, that $F(p) < F(q)$. This entails no loss of generality, because the other case, $F(q) < F(p)$, can be treated symmetrically.

Consider the $\rightarrow$ greedy increasing sequence in the interval $[p, 2hM]$. It begins with $g_0 = p$, and its next term g_1 is the first $n \in [p + 1, 2hM]$ such that $F(p) < F(n)$. Now q is such an n , so we have $g_1 \leq q \leq p + h$. That is, the difference between the first two elements of this greedy sequence is at most h .

All $\rightarrow$ maxima for $[p + 1, 2hM]$ that are to the right of g_1 are also $\rightarrow$ maxima for $[p, 2hM]$. Indeed, the only way a $\rightarrow$ maximum n for $[p + 1, 2hM]$ could fail to be a $\rightarrow$ maximum for $[p, 2hM]$ is to have $F(n) < F(p)$. But if $n > g_1$, then its being a $\rightarrow$ maximum for $[p + 1, 2hM]$, which contains g_1 , implies that $F(n) > F(g_1) > F(p)$. So n cannot fail to be a $\rightarrow$ maximum for $[p, 2hM]$. Now the difference between any two consecutive $\rightarrow$ maxima for $[p + 1, 2hM]$ is at most h because the cut just to the left of $p + 1$ is right-good (by definition of p). So the difference between consecutive $\rightarrow$ maxima for $[p, 2hM]$ to the right of g_1 is at most h also. This fact, together with the result of the preceding paragraph, shows that the cut just to the left of p is right-good, contrary to the definition of p . This contradiction (together with the analogous contradiction, to the definition of q , when $F(p) > F(q)$) completes the proof of the lemma. $\square$

Corollary 60. There is a good cut (A', B') such that the cardinalities $|A'|$ and $|B'|$ are divisible by h .

Proof. The conclusion of the corollary is obvious if either of the first two alternatives in the lemma holds. Under the third alternative, we have $h + 1$ consecutive good cuts (we actually need only h), so one of them must be at a distance from the left end that is divisible by h . That is, some good cut (A', B') has $|A'|$ divisible by h . $|B'|$ has the same divisibility property because $2hM$ does. $\square$

We summarize the preceding work as follows.

Proposition 61. For any $F \in \text{PFE}^{\mathbb{Z}}$ (without ties) that has no local maximum in $[1, 2hM]$, there exists a c , with $0 \leq c \leq 2M$, such that the cut $([1, ch], [ch + 1, 2hM])$ is good.

7.4.4. Probabilities of $\rightarrow$ maxima

We now un-fix F ; that is, F will now be a random element of $\text{PFE}^{\mathbb{Z}}$. Our ultimate goal is to estimate, from above, **Prob**(F has no local maximum in $[1, 2hM]$). In view of the results obtained above, we begin by estimating, for a fixed c , the probability that the cut $([1, ch], [ch + 1, 2hM])$ is right-good, i.e., that every h consecutive elements of $[ch + 1, 2hM]$ include at least one $\rightarrow$ maximum. There will be a similar estimate for the probability that $([1, ch], [ch + 1, 2hM])$ is left-good, and afterward we shall combine these estimates and sum over all c to estimate the probability that there is no local maximum in $[1, 2hM]$.

Recall from Propositions 14 and 17 that, as n ranges over the interval $J = [ch + 1, 2hM]$, the events “ n is a $\rightarrow$ maximum” are probabilistically independent and their probabilities are given by $1/(n - ch)$. We can now easily compute the probability that a given subinterval of length h in $[ch + 1, 2hM]$, say $[ch + a + 1, ch + a + h]$, contains a $\rightarrow$ maximum of J . Indeed, the probability that this interval contains no $\rightarrow$ maximum for J is given by a telescoping product:

$$\prod_{i=1}^h \left(1 - \frac{1}{a+i}\right) = \prod_{i=1}^h \frac{a+i-1}{a+i} = \frac{a}{a+h}.$$

So the complementary probability, that there is at least one $\rightarrow$ maximum of J in $[ch + a + 1, ch + a + h]$, is $h/(h + a)$.

- (1) 割集 $(\emptyset, [1, 2hM])$ 是好的。
- (2) 割集 $([1, 2hM], \emptyset)$ 是好的。
- (3) 至少有 h 个 $+1$ 连续割集, 且它们都是好的。

证明。切割 $(\emptyset, [1, 2hM])$ 空洞地左好, 因此如果它右好, 则我们得到引理的第一个替代方案。所以我们假设在证明的其余部分中 $(\emptyset, [1, 2hM])$ 不是右好的。对称地, 我们假设 $([1, 2hM], \emptyset)$ 不是左好的。

如果一个割集 $([1, c], [c + 1, 2hM]) \neq ([1, 2hM], \emptyset)$ 是右好的, 那么它右侧的下一个割集 $([1, c + 1], [c + 2, 2hM])$ 也是右好的, 并且因此, 通过归纳法, 所有右侧的割集都是右好的。原因是每个 $\rightarrow$ 最大值对于 $[c + 1, 2hM]$ (除了 $c + 1$) 也是 $\rightarrow$ 最大值对于 $[c + 2, 2hM]$ 。(注意: 一个 $\rightarrow$ 最大值对于 $[c + 2, 2hM]$ 不一定是 $\rightarrow$ 最大值对于 $[c + 1, 2hM]$, 因为它的 F -值可能小于 $F(c + 1)$.)。设 p 是 $[1, 2hM]$ 中最大的数, 使得 p 左侧的割集不是右好的; 这个数存在, 因为 1 左侧的割集, 即割集 $(\emptyset, [1, 2hM])$, 不是右好的。由于当将割集向右移动时右好性得以保持, 因此我们可以看到右好的割集正好是 p 右侧的割集。因为我们的原始割集 (A, B) 是好的, 所以我们知道 $p \in A$ 。

类似地, 令 q 是 $[1, 2hM]$ 中最小的数, 使得 q 右侧的切割不是左好的, 并观察到 $q \in B$ 。特别是, $p < q$ 。还请注意, 左好的切割正好是位于 q 左侧的那些。因此, 好的切割是位于 p 和 q 之间的那些。为了证明至少有 $h + 1$ 个这样的切割, 假设不是这样。这意味着 $q \leq p + h$ 。

我们假设, 在证明的其余部分中, $F(p) < F(q)$ 。这不会损失普遍性, 因为其他情况, $F(q) < F(p)$, 可以对称地处理。

考虑区间 $[p, 2hM]$ 中的 $\rightarrow$ 贪婪递增序列。它以 $g_0 = p$ 开头, 其下一个项 g_1 是第一个 $n \in [p + 1, 2hM]$ 满足 $F(p) < F(n)$ 的数, 所以 q 是这样的一个 n , 因此我们有 $g_1 \leq q \leq p + h$ 。也就是说, 这个贪婪序列前两个元素之间的差至多为 h 。

所有 $\rightarrow$ 最大值对于 $[p + 1, 2hM]$ 在 g_1 右侧的, 也都是 $\rightarrow$ 最大值对于 $[p, 2hM]$ 。实际上, $\rightarrow$ 最大值 n 对于 $[p + 1, 2hM]$ 只有在不成为 $\rightarrow$ 最大值对于 $[p, 2hM]$ 的情况下才会失败, 即需要 $F(n) < F(p)$ 。但如果 $n > g_1$, 那么它作为 $\rightarrow$ 最大值对于 $[p + 1, 2hM]$, (该值包含 g_1 ,) 意味着 $F(n) > F(g_1) > F(p)$ 。因此 n 不可能不成为 $\rightarrow$ 最大值对于 $[p, 2hM]$ 。现在任意两个连续的 $\rightarrow$ 最大值对于 $[p + 1, 2hM]$ 之间的差值至多为 h , 因为 $p + 1$ 左侧的切割是右良的 (根据 p 的定义)。因此, g_1 右侧的连续 $\rightarrow$ 最大值对于 $[p, 2hM]$ 之间的差值也至多为 h 。这一事实连同前一段的结果表明, p 左侧的切割是右良的, 这与 p 的定义相矛盾。这一矛盾 (连同当 $F(p) > F(q)$ 完成时, 根据 q 的定义产生的类似矛盾) 共同证明了引理。 $\square$

推论60。存在一个良好的切割 (A', B') , 使得基数 $|A'|$ 和 $|B'|$ 可以被 h 整除。

证明。如果引理中的前两个备选方案中的任何一个成立, 则推论的结论是明显的。在第三个备选方案下, 我们有 $h + 1$ 连续的良好切割 (我们实际上只需要 h), 所以其中之一必须距离左端的位置可以被 h 整除。也就是说, 某个良好的切割 (A', B') 有 $|A'|$ 可以被 h 整除。 $|B'|$ 具有相同的可整除性, 因为 $2hM$ 也具有。 $\square$

我们将前面的工作总结如下。

命题61. 对于任何 $F \in \text{PFE}^{\mathbb{Z}}$ (无并列) 在 $[1, 2hM]$, 中没有局部最大值, 存在一个 c , 有 $0 \leq c \leq 2M$, 使得切割 $([1, ch], [ch + 1, 2hM])$ 是良好的。

7.4.4. $\rightarrow$ 极大值的概率

我们现在解除 F 的固定状态; 也就是说, F 现在将是 $\text{PFE}^{\mathbb{Z}}$ 的一个随机元素。我们的最终目标是估计, 从上界来看, $\text{Prob}(F$ 在 $[1, 2hM])$ 中没有局部最大值。鉴于上述结果, 我们首先估计固定 c 时, 切割 $([1, ch], [ch + 1, 2hM])$ 是右好的概率, 即, $[ch + 1, 2hM]$ 的每个 h 个连续元素中至少包含一个 $\rightarrow$ 最大值。之后, 我们将对 $([1, ch], [ch + 1, 2hM])$ 是左好的概率进行类似的估计, 然后我们将结合这些估计并对所有 c 求和, 以估计 $[1, 2hM]$ 中没有局部最大值的概率。

回顾命题 14 和 17, 当 n 在区间 $J = [ch + 1, 2hM]$ 范围内变化时, “ n 是一个 $\rightarrow$ 最大值” 这些事件在概率上是独立的, 它们的概率由 $1/(n - ch)$ 给出。我们现在可以轻松计算 $[ch + 1, 2hM]$, 中长度为 h 的给定子区间的概率, 比如 $[ch + a + 1, ch + a + h]$, 包含 J 的一个 $\rightarrow$ 最大值。实际上, 这个区间不包含 J 的 $\rightarrow$ 最大值的概率由一个望远镜积给出:

$$\prod_{i=1}^h \left(1 - \frac{1}{a+i}\right) = \prod_{i=1}^h \frac{a+i-1}{a+i} = \frac{a}{a+h}.$$

所以, 补充概率, 即 $[$ 中至少有一个 $\rightarrow$ 最大值 $J]$, 在 $+$ 中, $+1$ 中, $+$ 中, $+$ 中, $]$, 中, $/($ 中, $+$ 中, $)$ 中。

Break the interval $[ch + 1, 2hM]$ into $2M - c$ subintervals, which we call *blocks*, of length h . The event that the cut $([1, ch], [ch + 1, 2hM])$ is right-good is included in the event that each of these blocks contains a $\rightarrow$ maximum of J , and the probability of the latter event can be computed by combining the computation in the preceding paragraph with the independence result in Lemma 17. Numbering the blocks from 1 to $2M - c$, we can apply the computation from the preceding paragraph, with $a = (j - 1)h$, to see that the j th block contains a $\rightarrow$ maximum of J with probability $h/(h + (j - 1)h) = 1/j$. Thus, by independence, the probability that every block contains a $\rightarrow$ maximum of J is

$$\prod_{j=1}^{2M-c} \frac{1}{j} = \frac{1}{(2M - c)!}.$$

This probability therefore provides an upper bound for the probability that the cut $([1, ch], [ch + 1, 2hM])$ is right-good.

Similarly, the probability that the cut $([1, ch], [ch + 1, 2hM])$ is left-good is bounded above by $1/c!$. Furthermore, all the events “ n is a $\rightarrow$ maximum of $[ch + 1, 2hM]$ ” are independent of the events “ m is a $\leftarrow$ maximum of $[1, ch]$.” This is because the former events refer only to the relative ordering of values of F at places $> ch$ while the latter refer only to the relative ordering values of F at places $\leq ch$. Therefore, the probability that $([1, ch], [ch + 1, 2hM])$ is good is bounded above by

$$\frac{1}{(2M - c)!} \cdot \frac{1}{c!} = \binom{2M}{c} \frac{1}{(2M)!}.$$

The event that $[1, 2hM]$ contains no local maximum is included in the union of the events that $([1, ch], [ch + 1, 2hM])$ is good, where c ranges from 0 to $2M$. Thus, the probability of no local maximum in $[1, 2hM]$ is bounded above by

$$\sum_{c=0}^{2M} \binom{2M}{c} \frac{1}{(2M)!} = \frac{2^{2M}}{(2M)!}.$$

7.4.5. Comparison with other methods

We compare the probability of unpleasantly long chunks under the point filter, LBFS, interval filter, and local maximum chunking methods. Recall from the preceding calculations that, when the parameters of all three methods are adjusted to produce the same average chunk length $2h + 1$, the probabilities of finding no cutpoints in the interval $[1, 2hM]$ (approximately M average chunks) are

$> e^{-M}$	for the pure point filter method
$> e^{-(k+1)M/(1-k/2h)} \approx e^{-(k+1)M}$	for the LBFS method
$> Ae^{-BM}$	for the interval filter method
$< 2^{2M}/(2M)!$	for the local maximum method,

where in the case of LBFS $k = h/c$ is the expected number of candidates in a segment of length equal to the horizon, and where in the case of the interval filter method $1 < B < e$.

Notice that the pure point filter, LBFS, and interval filter methods give probabilities that decrease “only” exponentially as $M \rightarrow \infty$, while the local maximum method gives a probability that decreases more rapidly. This can be seen by comparing the logarithms of the probabilities, using Stirling’s approximation for the factorial. The four logarithms are $-M$, $-(k + 1)M$, $-BM + \ln A$, and asymptotically $-2M(\ln M - 1)$, respectively. The last is, for large enough M , much smaller (i.e., more negative) than the others because of the $\ln M$ factor.

Unfortunately, this comparison of behaviors as $M \rightarrow \infty$ can be misleading. The reason is that our computation for the local maximum method assumed that there are no ties, i.e., that no file entry is repeated in the interval $[1 - h, 2hM + h]$. That assumption is reasonable as long as $(2hM)^2 \ll |\text{PFE}|$, but not as $M \rightarrow \infty$; in fact the assumption is obviously false once M is large enough.

So a true comparison should use relatively small values of M . These are also the values of M that are relevant for practical purposes. We would like to have small probabilities for cut-less intervals of length, say, 5 average chunk lengths. The corresponding probabilities for very large M will be too small to worry about.

It turns out that, once $M \geq 4$, our upper bound $2^{2M}/(2M)!$ for the local maximum method is smaller than the lower bound e^{-M} for the pure point filter method and also the lower bound $e^{-5M/4}$ for the LBFS method when $k = 1/4$. If we increase M to 7, the local maximum method gives a probability smaller by almost a factor 1000 than the LBFS method with $k = 1/4$.

Remark 62. Our upper bound for the probability of no local maximum in $[1, 2hM]$ is rather rough; we sacrificed a good deal of information by using only the fact that each of the $2M - c$ blocks contains a $\rightarrow$ maximum, when in fact every interval of length h in J must contain a $\rightarrow$ maximum. The reason for this sacrifice is to obtain independence and thereby facilitate the computation. Because the blocks are pairwise disjoint, the events that they contain $\rightarrow$ maxima of J are independent. If we used all subintervals of J of length h , rather than only the blocks, we would lose the disjointness and thus the

将区间 $[中, 2hM]$ 分成 $2M - 中$ 子区间, 我们称之为块, 长度为 h 。事件 $([1, 中], [中, + 1中, 2hM])$ 是右好的, 包含在每个这些块中包含一个 $\rightarrow$ 最大值 J 的事件中, 后者的概率可以通过结合前一段的计算与引理 17 中的独立性结果来计算。将块编号从 1 到 $2M - 中$, 我们可以应用前一段的计算, $a = (j - 1)h$, 来看到第 j 个块包含一个 $\rightarrow$ 最大值 J 的概率是 $h/(h + (j - 1)h) = 1/j$ 。因此, 根据独立性, 每个块都包含一个 $\rightarrow$ 最大值 J 的概率是

$$\prod_{j=1}^{2M-c} \frac{1}{j} = \frac{1}{(2M - c)!}.$$

因此, 该概率为切分 $([1, ch], [ch + 1, 2hM])$ 为右好的概率提供了上界。

类似地, 切分 $([1, ch], [ch + 1, 2hM])$ 为左好的概率被 $1/c!$ 上界。此外, 所有事件 “ n 是 $\rightarrow ch$ $[ch + 1, 2hM]$ 的 $\leftarrow ch$ ” 都独立于事件 “ m 是 $[1, ch]$ 的 $> ch$ ”。这是因为前者事件仅涉及 F 在位置 $\leq ch$ 处的值相对顺序, 而后者事件仅涉及 F 在位置 $([1, ch$ 处的值相对顺序。因此, 切分 $], [ch + 1, 2hM])$ 为好的概率被上界。

$$\frac{1}{(2M - c)!} \cdot \frac{1}{c!} = \binom{2M}{c} \frac{1}{(2M)!}.$$

$[1, 2hM]$ 不包含局部最大值的事件包含在 $([1, ch], [ch + 1, 2hM])$ 为好的事件的并集中, 其中 c 的范围从 0 到 $2M$ 。因此, $[1, 2hM]$ 中无局部最大值的概率被上界。

$$\sum_{c=0}^{2M} \binom{2M}{c} \frac{1}{(2M)!} = \frac{2^{2M}}{(2M)!}.$$

7.4.5. 与其他方法比较

我们比较了在点过滤器、LBFS、区间过滤器和局部最大值分块方法下, 出现不愉快长块的概率。回想一下前面的计算, 当调整所有三种方法的参数以产生相同的平均块长度 $2h + 1$ 时, 在区间 $[1, 2hM]$ (大约 M 个平均块) 内找到无切点的概率是

$> e^{-M}$	for the pure point filter method
$> e^{-(k+1)M/(1-k/2h)} \approx e^{-(k+1)M}$	for the LBFS method
$> Ae^{-BM}$	for the interval filter method
$< 2^{2M}/(2M)!$	for the local maximum method,

其中, 在 LBFS 的情况下 $k = h/c$ 是等于视距长度的片段中的预期候选数量, 而在区间过滤器方法的情况下 $1 < B < e$ 。

请注意, 纯点过滤器、LBFS 和区间过滤器方法给出的概率随着 $M \rightarrow \infty$ 按指数 “仅” 减少, 而局部最大值方法给出的概率减少得更快。这可以通过比较概率的对数, 并使用斯特林近似公式来计算阶乘来看。这四个对数分别是 $-M$, $-(k + 1)M$, $-BM + \ln A$, 以及渐近地 $-2M(\ln M - 1)$ 。最后一个, 对于足够大的 M , 由于 $\ln M$ 因子, 它比其他值小得多 (即更负)。

不幸的是, 将行为作为 $M \rightarrow \infty$ 进行比较可能会产生误导。原因是我们的局部最大值方法计算假设没有平局, 即, 在区间 $[1 - h, 2hM + h]$ 中没有任何文件条目重复。只要 $(2hM)^2 \ll |\text{PFE}|$, 这一假设是合理的, 但 $M \rightarrow \infty$; 实际上, 一旦 M 足够大, 这一假设显然是错误的。

因此, 真正的比较应该使用相对较小的 M 值。这些也是实际应用中相关的 M 值。我们希望对于长度为 5 个平均块长度的无切分区, 有较小的概率。对于非常大的 M , 相应的概率将小到可以忽略不计。

结果表明, 一旦 $M \geq 4$, 局部最大值方法的上限 $2^{2M}/(2M)!$ 小于纯点过滤器方法的下限 e^{-M} , 并且也小于 LBFS 方法的下限 $e^{-5M/4}$, 当 $k = 1/4$ 。如果我们将 M 增加到 7, 局部最大值方法给出的概率比使用 $k = 1/4$ 的 LBFS 方法小了近 1000 倍。

备注62。我们对于 $[1, 2hM]$ 中不存在局部最大值的概率上界相当粗糙; 我们通过仅使用每个 $2M - c$ 块包含一个 $\rightarrow$ 最大值这一事实, 牺牲了大量信息, 而实际上 J 中每个长度为 h 的区间都必须包含一个 $\rightarrow$ 最大值。这种牺牲的原因是为了获得独立性, 从而简化计算。由于块是两两不交的, 它们包含 J 的 $\rightarrow$ 最大值的事件是独立的。如果我们使用 J 的所有长度为 h 的子区间, 而不仅仅是块, 我们会失去不交性, 从而失去

independence. If the computation could be completed despite this loss, it would surely yield a tighter upper bound than the one we obtained.

8. Computing local maxima

In this section, we discuss ways of finding the local maxima in a (finite) file. For the other chunking methods that we have discussed — point filter, LBFS, and interval filter — it is clear that the cutpoints of a file F can be found in a single pass through the file, performing some elementary test (divisibility by a given c or membership in U) on each file entry, and counting (to see whether we are at a blocked location for LBFS or whether we have matched an interval filter). The most obvious algorithm for determining local maxima, namely to compare each file entry with each of the $2h$ others in its window, is far less efficient, as it requires $2h$ operations per file position. Fortunately, there are better algorithms. We shall describe two of them. One is rather straightforward and needs just two comparisons per file entry. That is, the total number of comparisons needed is no more than twice the length of the file. The second algorithm is more sophisticated and needs, on average, only

$$1 + \frac{\ln h}{h} + O\left(\frac{1}{h}\right)$$

comparisons per file entry. Since h is large, this amounts to barely more than 1 comparison per file entry, so the local maximum chunking method does not require significantly more work than others to find the cutpoints.

Remark 63. For operation on a modern CPU, the important metric is the number of branch mispredictions encountered during chunk computation. Modern CPUs use advanced branch prediction hardware to opportunistically continue computation assuming a branch predicate evaluates to the same value as it did when it was previously encountered. This pays off as long as branch predicates are biased to evaluate to the same value, but is of no help if branch predicates evaluate to different truth values in random alternation. Counting the number of comparisons per file entry provides an upper bound on branch mispredictions, and is therefore a good abstract measure for the running time of the chunking methods.

We begin by describing the more straightforward of the two algorithms. The idea is to read the file F , from left to right, producing a list of the local maximum positions, and keeping track of the additional information that is relevant to the computation of later local maxima. More precisely, the algorithm will keep track of pairs $(i, F(i))$ that might affect future decisions about what is or is not a local maximum. Those pairs are of two sorts.

First, there are the pairs $(i, F(i))$ that might turn out to be local maxima, but are not yet known to be local maxima. This means that $F(i)$ is larger than the h immediately preceding values of F as well as the subsequent values that have been read so far, but the number of these subsequent values is $< h$. So i looks as though it could be a cutpoint but some values in its window remain to be read, so it may yet turn out not to be a cutpoint. Notice that pairs of this sort must always have i within the last h positions that have been read; once we read farther than that, we will know whether i is a cutpoint, so it will either be put on the output list of local maxima or dropped from consideration.

Second, there are the pairs $(i, F(i))$ that might prevent some position that we haven't yet visited from being a cutpoint. That is, we might in the future read $F(j)$ at position j and decide that j cannot be a cutpoint because $F(i) \geq F(j)$. In principle, any of the pairs $(i, F(i))$ among the last h that were read could play this role, but many of them can safely be ignored. Specifically, suppose that we have read a larger value at a later position, say $F(i') \geq F(i)$ with $i' > i$. Then any future j that is prevented from being a cutpoint by $(i, F(i))$ is also prevented by $(i', F(i'))$. Indeed, we have $F(i') \geq F(i) \geq F(j)$ and, if i is in the window of j , then so is i' because $i < i' < j$.

This means that the only i 's for which we have to remember $(i, F(i))$ because it might prevent a future j from being a cutpoint are those i , within the last h positions read, for which $F(i)$ is larger than all later F -values already read. Proposition 12, applied in the right-to-left direction, tells us that these values of i constitute the $\leftarrow$ greedy sequence in the interval of the last h positions read.

Notice that, if there is an $(i, F(i))$ of the first sort, a candidate for being a cutpoint, then it is also of the second sort, since it is within the last h positions read and its F -value is larger than the later ones. Summarizing, we see that our algorithm should maintain the following information as it reads through the file F :

- $(i, F(i))$ for i in the $\leftarrow$ greedy subsequence of the interval of the last h positions read, and
- one additional bit, telling whether the leftmost position in the greedy sequence (the one with the largest F -value) is a candidate for being a cutpoint, i.e., whether its F -value exceeds the h immediately preceding F -values.

Thus, our algorithm acts as follows while reading the file from left to right. It maintains a list A of pairs and a bit γ , and it (gradually) outputs a list of cutpoints. At position i , the algorithm performs the following steps, in the given order.

- (1) Read $F(i)$.
- (2) Go through A , in order, deleting any pairs $(j, F(j))$ with $F(j) \leq F(i)$. Stop when and if a pair is not deleted.

独立性。如果尽管这种损失计算仍能完成，它肯定会得到比我们获得的更紧的上界。

8. 计算局部最大值

在本节中，我们讨论在（有限）文件中查找局部最大值的方法。对于我们已经讨论过的其他分块方法——点过滤器、LBFS 和区间过滤器——很明显，文件 F 的切点可以在一次文件遍历中找到，对每个文件条目执行一些基本测试（除以给定 c 或属于 U 的成员资格），并计数（以查看我们是否处于 LBFS 的阻塞位置或是否匹配了区间过滤器）。确定局部最大值的最明显算法，即将每个文件条目与其窗口中的其他 $2h$ 个条目进行比较，效率远低，因为它需要每个文件位置 $2h$ 次操作。幸运的是，有更好的算法。我们将描述其中的两种。一种是相当直接的，每个文件条目只需要两次比较。也就是说，所需的比较总数不超过文件长度的两倍。第二种算法更复杂，平均只需要

$$1 + \frac{\ln h}{h} + O\left(\frac{1}{h}\right)$$

每个文件条目的比较次数。由于 h 很大，这相当于每个文件条目 barely more than 1 次比较，因此局部最大值分块方法 does not requiresignificantly more work than others to find the cutpoints。

备注63。对于在现代 CPU 上的操作，重要的指标是在 chunk 计算过程中遇到的分支误预测数量。现代 CPU 使用高级分支预测硬件，opportunistically 继续计算，假设分支谓词的值与之前遇到时相同。只要分支谓词倾向于评估为相同的值，这种方法就会得到回报，但如果分支谓词在随机交替中评估为不同的真值，则毫无帮助。计算每个文件条目的比较次数为分支误预测提供了上限，因此是 chunking 方法运行时间的良好抽象度量。

我们首先描述这两种算法中更直接的一种。思路是按从左到右的顺序读取文件 F ，生成局部最大值位置列表，并跟踪与后续局部最大值计算相关的附加信息。更精确地说，该算法将跟踪可能影响未来关于什么是或不是局部最大值的决策的 $(i, F(i))$ 对。这些对分为两种类型。

首先，有 $(i, F(i))$ 这对可能成为局部极大值，但目前尚不清楚是否为局部极大值。这意味着 $F(i)$ 比紧邻其前的 F 值以及已读取的后续值都要大，但这些后续值的数量是 $< h$ 。因此， i 看起来可能是一个切割点，但其窗口中仍有某些值尚未读取，所以它最终可能并非切割点。请注意，这类对中的 i 必须位于最近读取的 h 个位置之内；一旦我们读取到更远的位置，就会知道 i 是否为切割点，因此它将被添加到局部极大值的输出列表中，或者被排除在考虑范围之外。

其次，存在 $(i, F(i))$ 这样的对，它们可能会阻止我们尚未访问的某个位置成为切割点。也就是说，我们未来可能会在位置 j 读取 $F(j)$ 并决定 j 不能成为切割点，因为 $F(i) \geq F(j)$ 。原则上，最后读取的 h 个对中任意一个 $(i, F(i))$ 都可能扮演这个角色，但其中许多对可以安全地忽略。具体来说，假设我们在较后的位置读取了一个较大的值，比如 $F(i') \geq F(i)$ with $i' > i$ 。那么，任何被 $(i, F(i))$ 阻止成为切割点的未来 j ，也会被 $(i', F(i'))$ 阻止。确实，我们有 $F(i') \geq F(i) \geq F(j)$ ，并且，如果 i 在 j 的窗口中，那么 i' 也因为 $i < i' < j$ 而在窗口中。

这意味着，我们只需要记住那些在最后读取的 h 个位置内的 i ，使得 $(i, F(i))$ ，因为它们可能会阻止未来的 j 成为切割点，这些 i 满足 $F(i)$ 大于所有后来已经读取的 F 值。命题 12，从右到左应用，告诉我们这些 i 值构成了最后读取的 h 个位置区间的 $\leftarrow$ 贪婪序列。

请注意，如果有 $(i, F(i))$ 是第一种类型的候选切割点，那么它也是第二种类型的，因为它在最后读取的 h 个位置内，并且其 F 值大于后来的值。总结来说，我们看到我们的算法在读取文件 F 时应该维护以下信息：

- $(i, F(i))$ 对于 i 在最后读取的 h 个位置区间的 $\leftarrow$ 贪婪子序列中，以及
- 一个额外的位，用于判断贪婪序列中最左侧的位置（即 F 值最大的那个位置）是否是一个切割点的候选，也就是说，它的 F 值是否超过了它之前紧邻的 F 值。

因此，我们的算法在从左到右读取文件时按如下方式运行。它维护一个包含对 A 的列表和一个比特 γ ，并（逐步）输出一个包含切割点的列表。在位置 i ，算法按给定顺序执行以下步骤：

- (1) 读取 $F(i)$ 。
- (2) 依次遍历 A ，删除任何满足 $F(j) \leq F(i)$ 的配对 $(j, F(j))$ 。当且仅当存在一个未被删除的配对时停止。

- (3) If $(i - h, F(i - h))$ is in Δ , add it to the output list of cutpoints and set $\gamma := 0$.
- (4) Add $(i, F(i))$ to the beginning of Δ and, if Δ has no other elements, set $\gamma := 1$.
- (5) Delete $(i - h, F(i - h))$ from Δ (if it's present).

When the algorithm has finished processing position i in this manner, Δ contains $(j, F(j))$ for j in the $\leftarrow$ greedy sequence for $[i - h + 1, i]$, γ is 1 if and only if the last (leftmost) element of Δ is still a candidate to be a cutpoint, and the output produced so far consists of all the cutpoints at positions $\leq i - h$. Note that the algorithm maintains the property that the pairs $(j, F(j))$ in Δ always occur in order of decreasing j and increasing $F(j)$. This is why the sentence beginning “Stop” in instruction (2) is justified; the elements of Δ that are not inspected have $F(j)$ at least as large as the non-deleted one that triggered the stop, and so they should also not be deleted.

To estimate the number of comparisons performed (while executing instruction (2)) during a run of this algorithm, we associate to each comparison a position in the file as follows. If the comparison of the newly read $F(i)$ with an earlier $F(j)$ results in the deletion of $(j, F(j))$ from Δ (because $F(j) \leq F(i)$), then associate position j to this comparison; we refer to this as “association with deletion.” Otherwise, i.e., if $F(j) > F(i)$, then associate position i to the comparison; we refer to this as “association without deletion.”

Since any $(j, F(j))$ enters Δ just once, it is deleted at most once, and so j has at most one comparison associated to it with deletion. Furthermore, because of the “stop” part of instruction (2), each i has at most one comparison associated to it without deletion. So altogether, each position has at most two comparisons associated to it. Thus, the total number of comparisons performed by the algorithm is at most twice the length of the file.

Remark 64. We can be more precise about the number of comparisons. The only way a position i can avoid having a comparison associated to it with deletion is to have $F(i) > F(j)$ for all $j > i$ in its window. Call such a point a right semi-maximum. The only way i can avoid having a comparison associated to it without deletion is to have $F(i) \geq F(j)$ for all $j < i$ in its window. Call such a point a left semi-maximum. (Note the asymmetry: right semi-maxima satisfy a strict inequality and left semi-maxima only a non-strict one. Of course, this doesn't matter if PFE is big enough and we ignore ties.) Thus, the number of comparisons performed by this algorithm is twice the length of the file, minus the sum of the number of right and left semi-maxima. Since a local maximum is both a right and a left semi-maximum, it follows that the number of comparisons is at most twice the number of positions that are not local maxima.

We now turn to a more sophisticated algorithm, which, compared to the preceding one, cuts the number of comparisons almost in half, on average.

The algorithm splits the file into blocks of length $h + 1$, and it processes the blocks in order, from the leftmost to the rightmost. For brevity, we ignore the trivialities arising if the length of the file isn't exactly divisible by $h + 1$.

For a position i to be a local maximum, it is necessary (but not sufficient) that $F(i) > F(j)$ for all j in the block containing i , because all such j are in the window $[i - h, i + h]$ centered at i . We call i a *candidate* if it fulfills this necessary condition. For a candidate to be a local maximum it must, in addition, have an F -value greater than that of any position within h in the immediately previous and immediately following blocks. If the algorithm finds that a candidate fails to satisfy this additional requirement, we shall say that it *kills* the candidate; we use the phrase *live candidate* to mean a candidate that has not (yet) been killed.

When processing a block B , the algorithm will produce the following information:

- (1) If, when it starts processing B , there is a live candidate in the immediately previous block, it will decide whether that candidate is a local maximum.
- (2) It will produce the $\leftarrow$ greedy sequence of B , in decreasing order of positions (and thus increasing order of F -values).
- (3) It will decide whether the last term in the $\leftarrow$ greedy sequence is a candidate.
- (4) If the last term is a candidate, it will decide whether it is to be killed because of a larger or equal F -value in the previous block (and within the candidate's window).

Recall that, by Proposition 12 (applied with right and left reversed), the $\leftarrow$ greedy sequence in item (2) consists of those positions in B where the F -value is greater than all later F -values in B . It can thus be found by reading the F -values in the block B , from right to left, keeping track of the highest value seen so far, and adding to the $\leftarrow$ greedy sequence any position where the newly read value exceeds the largest previously seen value. Notice that the algorithm processes the blocks in left-to-right order but processes the positions within any block in right-to-left order.

In connection with item (3), notice that the last element of the $\leftarrow$ greedy sequence will always have an F -value $\geq$ all other F -values for the block B . But for a candidate, we need strict inequality here. So item (3) amounts to checking for ties for the highest F -value in the block.

Most of the algorithm's work goes into item (2), so we begin our description there. Notice that, if we were just producing the $\leftarrow$ greedy sequence (and not doing anything about items (1), (3), and (4)), this would involve h comparisons. Each position in the block, except the rightmost, must have its F -value compared, at the time the algorithm reads it, with the F -value of the currently last element of the sequence under construction. So this task requires approximately one (exactly $\frac{h}{h+1}$) comparison per position in the file.

- (3) 如果 $(i - h, F(i - h))$ 在 Δ 中, 将其添加到切割点输出列表并设置 $\gamma := 0$ 。
- (4) 将 $(i, F(i))$ 添加到 Δ 的开头, 并且如果 Δ 没有其他元素, 则设置 $\gamma := 1$ 。
- (5) 从 Δ (如果存在) 中删除 $(i - h, F(i - h))$ 。

当算法以这种方式处理完位置 i 后, Δ 包含 $(j, F(j))$ 对于 $\leftarrow$ 贪婪序列中的 $[i - h + 1]$ 中的 j 。如果 Δ 的最后一个 (最左边的) 元素仍然是候选切割点时, γ 为 1, 并且到目前为止生成的输出包含所有位于位置 $\leq i - h$ 的切割点。请注意, 算法维护的性质是, $(j, F(j))$ 在 Δ 中的对总是按 j 递减和 $F(j)$ 递增的顺序出现。这就是为什么指令 (2) 中以 “停止” 开头的句子是合理的; 未被检查的 Δ 元素的 $F(j)$ 至少与触发停止的非删除元素一样大, 因此它们也不应该被删除。

为了估计在此算法运行期间执行指令 (2) 时进行的比较次数, 我们将每个比较与文件中的一个位置相关联, 具体方法如下。如果新读取的 $F(i)$ 与之前的 $F(j)$ 的比较导致删除 $(j, F(j))$ 从 Δ (因为 $F(j) \leq F(i)$), 则将位置 j 与此比较相关联; 我们称此为 “关联删除”。否则, 即如果 $F(j) > F(i)$, 则将位置 i 与比较相关联; 我们称此为 “不关联删除”。

由于任何 $(j, F(j))$ 只进入 Δ 一次, 它最多被删除一次, 因此 j 最多关联一个带删除的比较。此外, 由于指令 (2) 的 “停止” 部分, 每个 i 最多关联一个不带删除的比较。因此, 总的来说, 每个位置最多关联两个比较。因此, 该算法执行的总比较次数最多是文件长度的两倍。

备注 64。我们可以更精确地说明比较次数。位置 i 避免关联带删除比较的唯一方式是, 在其窗口中对于所有 $j > i$ 都有 $F(i) > F(j)$ 。将这样的点称为右半最大值。位置 i 避免关联不带删除比较的唯一方式是, 在其窗口中对于所有 $j < i$ 都有 $F(i) \geq F(j)$ 。将这样的点称为左半最大值。(注意不对称性: 右半最大值满足严格不等式, 而左半最大值仅满足非严格不等式。当然, 如果 PFE 足够大并且忽略平局, 这并不重要。) 因此, 该算法执行的比较次数是文件长度的两倍, 减去右半最大值和左半最大值数量的和。由于局部最大值既是右半最大值也是左半最大值, 因此比较次数最多不超过不是局部最大值的位置数量的两倍。

我们现在转向一个更复杂的算法, 与前面的算法相比, 它在平均情况下将比较次数减少了一半多。

该算法将文件分割成长度为 $h + 1$ 的块, 并按顺序处理这些块, 从最左边的块到最右边的块。为了简洁起见, 我们忽略了如果文件长度不能被 $h + 1$ 整除时产生的琐碎问题。

要使位置 i 成为局部最大值, 必要条件 (但不是充分条件) 是 $F(i) > F(j)$ 对于包含 i 的块中的所有 j 都成立, 因为所有这样的 j 都在以 i 为中心、宽度为 h $[i - h, i + h]$ 的窗口内。如果 i 满足这个必要条件, 我们就称其为候选。对于一个候选来说, 要成为局部最大值, 它还必须在紧邻的前一个和后一个块中, 任何位置 h 内的 F 值都大于它。如果算法发现候选不满足这个附加要求, 我们将说它杀死了该候选; 我们使用短语 “活候选” 来指尚未被杀死 (的候选)。

在处理块 B 时, 算法将生成以下信息:

- (1) 如果在开始处理 B 时, 前一个块中存在一个活跃候选, 它将决定该候选是否为局部最大值, 局部最大值。
- (2) 它将生成 B 的 $\leftarrow$ 贪婪序列, 按位置降序排列 (因此按 F 值升序排列)。
- (3) 它将决定 $\leftarrow$ 贪婪序列的最后一个项是否为候选。
- (4) 如果最后一个项是候选, 它将决定是否因在更大的或相等的 F 值中而被杀死。前一个块 (以及候选窗口内)。

回想一下, 根据命题 12 (将左右反转应用), 第 (2) 项中的 $\leftarrow$ 贪婪序列由 B 中所有 F 值大于 B 中后续所有 F 值的那些位置组成。因此, 可以通过从右到左读取块 B 中的 F 值, 跟踪当前已看到的最高值, 并将任何新读取值超过先前所见最大值的位添加到 $\leftarrow$ 贪婪序列中来找到它。请注意, 该算法按从左到右的顺序处理块, 但在任何块内按从右到左的顺序处理位置。

与第 (3) 项相关, 请注意, $\leftarrow$ 贪婪序列的最后一个元素将始终具有一个 F 值, 该值 $\geq$ 大于块 B 中的所有其他 F 值。但对于候选, 这里需要严格不等式。因此, 第 (3) 项相当于检查块中最高 F 值的平局情况。

该算法的大部分工作都用于项目 (2), 所以我们从那里开始描述。请注意, 如果我们只是生成 $\leftarrow$ 贪婪序列 (并且不处理项目 (1)、(3) 和 (4)), 这将涉及 h 次比较。除了最右侧的位置外, 块中的每个位置都必须在算法读取它时将其 F 值与正在构建的序列的当前最后一个元素的 F 值进行比较。因此, 这项任务需要大约每文件位置一次 (确切地说是 $\frac{h}{h+1}$) 的比较。

Item (3) can be handled simultaneously with the construction of the $\leftarrow$ greedy sequence. Whenever a position is put into the sequence, call it a candidate (tentatively). If another position is put in later (because it has a larger F -value) kill the old candidate while making the newly added position a candidate. Also, if another position in the block is found to have the same F -value as the current candidate, then kill the candidate (even though it remains in the $\leftarrow$ greedy sequence and the later position with the same F -value is not added to the sequence). Thus, item (3) requires no additional comparisons.

Let us refer to the process just described, running through B in reverse order to handle items (2) and (3), the *ordinary* run through B . If, when we start processing block B , there is no live candidate in the immediately previous block, then the ordinary run through B handles item (1) vacuously and we need only consider item (4), which we shall do later. But if there is a live candidate in the preceding block, then the ordinary run must be modified in order to handle item (1), and we now describe this modification.

Suppose, therefore, that position m is a live candidate in the block just before B . Being a candidate, it has a larger F -value than all other elements of its block. Furthermore, being live, it has a larger F -value than all positions to its left in its window, i.e., in $[m-h, m-1]$, because otherwise it would have been killed during the processing of its own block – see item (4). So it will be a local maximum unless there is a larger or equal F -value at a position that is in B and $\leq m+h$. Our task is to detect such an F -value, if there is one, and then kill m . And we must do this without excessive comparisons of F -values.

Begin processing B by the ordinary run until you reach position $m+h$. (The point is that, until this moment, you're working with positions outside the window of m and thus irrelevant to item (1).) When you reach $m+h$, there is a branching according to whether the current last position in the $\leftarrow$ greedy sequence, say g , has $F(g) \geq F(m)$ or $F(g) < F(m)$.

Suppose first that $F(g) \geq F(m)$. (To avoid confusion, notice that this case hypothesis doesn't kill m because g is beyond the right end of the window of m .) In this case, continue going leftward through the block B , but, instead of comparing F -values with $F(g)$ (as the ordinary run would), compare them with $F(m)$. As long as they are $< F(m)$, they can be ignored as they don't kill m and they don't go into the $\leftarrow$ greedy sequence (because $F(g) \geq F(m)$). If you find an F -value equal to $F(m)$, then that kills m , but the position still doesn't go into the $\leftarrow$ greedy sequence; once m is killed, resume the ordinary run from the next position on the left. Finally, if you find an F -value strictly greater than $F(m)$, then kill m , and resume the ordinary run from the current position. Notice that, in this last situation, one position has its F -value compared with both $F(m)$ and $F(g)$. So the total number of comparisons will not be h as computed above for the ordinary run but $h+1$; that's still an average of only one comparison per position in B .

Now consider the other case, where $F(g) < F(m)$. In this case, continue with the ordinary run but, whenever a position g' is added to the $\leftarrow$ greedy list, check whether $F(g') \geq F(m)$. If so, then kill m and continue with the ordinary run and no further comparisons with $F(m)$. If, on the other hand, $F(g') < F(m)$, then m remains live and the next addition to the $\leftarrow$ greedy sequence will also need a comparison with $F(m)$. Notice that, if m should be killed, then this procedure will kill it. Specifically, if m should be killed, then there is an x in its window and in B with $F(x) \geq F(m)$, because, as noted above, any other reason for killing m would have done so while the previous block was processed. The rightmost such x will be added to the $\leftarrow$ greedy sequence, will therefore have $F(x)$ compared to $F(m)$ by our algorithm, and will thus kill m .

In the case just considered, it is possible for many positions to have their F -values compared with the F -values at both m and the current last position in the $\leftarrow$ greedy list. Indeed, this happens whenever a position $\leq m+h$ is added to the $\leftarrow$ greedy list as long as m remains live. A priori, the number of such occurrences is bounded by the length of the $\leftarrow$ greedy list, which is, as we saw in Section 2.4, on average approximately $\ln h$. In fact, we shall get a much better bound, on average, later, but first we finish the description of the algorithm by showing how to handle item (4).

Item (4) is handled by a separate process after the completion of the run through B (either the ordinary run or the modification to handle item (1)). Suppose, therefore that this run has been completed and that it resulted in a candidate, namely the last term g of the $\leftarrow$ greedy sequence, and that this candidate hasn't been killed yet (i.e., no point to its left in B had the same F -value). For item (4), we must check whether some point x in the window of g and in the previous block had a larger F -value, $F(x) > F(g)$. The key observation is that, if this happens, then the rightmost such x is in the $\leftarrow$ greedy sequence of the previous block. Indeed, for all $y > x$ in that block, we have $F(x) > F(g) \geq F(y)$. So to look for such an x , it suffices to look through the terms of the previously computed $\leftarrow$ greedy sequence of the preceding block. Since the length of the $\leftarrow$ greedy sequence in any block is, on average, only approximately $\ln h$, we can handle item (4) with only $\ln h$ additional comparisons, i.e., only $\frac{\ln h}{h}$ comparisons per position, on average.

This completes the description of the algorithm and most, but not all, of the estimation of how many comparisons of F -values are needed. On average, we have, during the processing of a block, at most

- h comparisons to handle items (2) and (3),
- $\ln h$ comparisons to handle item (1) in the case where $F(g) < F(m)$ and $F(g') < F(m)$ for many subsequent elements g' of the $\leftarrow$ greedy sequence, and
- $\ln h$ comparisons to handle item (4).

We now show that the $\ln h$ associated to item (1) can be reduced greatly, namely to a constant. Thus, the final estimate for the average number of comparisons will be $h + \ln h + O(1)$ per block or $1 + \frac{\ln h}{h} + O(\frac{1}{h})$ per position. Notice that, even without this improvement, we already know that the average number of comparisons per position is at most $1 + \frac{2\ln h}{h}$.

项目(3)可以与 $\leftarrow$ 贪婪序列的构建同时处理。每当一个位置被放入序列中时, 称其为候选 (暂定)。如果稍后另一个位置被放入 (因为它具有更大的F值), 则杀死旧候选, 同时将新添加的位置设为候选。此外, 如果在块中找到另一个具有与当前候选相同F值的位置, 则杀死候选 (即使它仍然在 $\leftarrow$ 贪婪序列中, 而具有相同F值的稍后位置没有被添加到序列中)。因此, 项目(3)不需要额外的比较。

让我们参考刚才描述的过程, 通过 B 以逆序运行以处理项目 (2) 和 (3), 即普通的 B 运行。如果在开始处理块 B 时, 前一个块中没有活动的候选, 那么普通运行通过 B 空值处理项目 (1), 我们只需考虑项目 (4), 我们稍后将处理它。但如果前一个块中有活动的候选, 那么普通运行必须修改以处理项目 (1), 我们现在描述这种修改。

因此, 假设位置 m 是 B 前一个块中的一个活动候选。作为候选, 它比其块中的所有其他元素具有更大的 F 值。此外, 作为活动状态, 它比其窗口中的所有左侧位置具有更大的 F 值, 即 $[m-h, m-1]$ 中, 因为否则在处理其自己的块时它会被杀死——见项目 (4)。所以它将是局部最大值, 除非在 B 和 $\leq m+h$ 中的某个位置存在更大的或相等的 F 值。我们的任务是检测是否存在这样的 F 值 (如果存在), 然后杀死 m 。我们必须在不进行过多 F 值比较的情况下完成此操作。

使用常规方法开始处理 B , 直达到位置 $m+h$ 。(关键在于, 在此刻之前, 你处理的是 m 窗口外的位置, 因此与条目 (1) 无关。) 当你达到 $m+h$ 时, 会根据当前贪婪序列 $\leftarrow$ 的最后一个位置, 比如 g , 是否具有 $F(g) \geq F(m)$ 或 $F(g) < F(m)$ 进行分支。

假设首先 $F(g) \geq F(m)$ 。(为了避免混淆, 请注意此情况假设不会影响 m , 因为 g 位于 m 窗口的右侧末端。) 在这种情况下, 继续向左通过块 B , 但, 与常规方法不同, 不是将 F 值与 $F(g)$ 比较, 而是与 $F(m)$ 比较。只要它们是 $< F(m)$, 就可以忽略, 因为它们不会影响 m , 也不会进入贪婪序列 (因为 $F(g) \geq F(m)$)。如果你找到一个等于 $F(m)$ 的 F 值, 那么它会影响 m , 但位置仍然不会进入贪婪序列; 一旦 m 受影响, 就从左侧下一个位置恢复常规方法。最后, 如果你找到一个严格大于 $F(m)$ 的 F 值, 那么会影响 m , 并从当前位置恢复常规方法。请注意, 在这种情况下, 一个位置的 F 值会与 $F(m)$ 和 $F(g)$ 都进行比较。因此, 总比较次数不会像上面计算的那样是 h , 而是 $h+1$; 这仍然平均每个位置只有一个比较。

现在考虑另一种情况, 其中 $F(g) < F(m)$ 。在这种情况下, 继续执行常规运行, 但每当一个位置 g' 被添加到 $\leftarrow$ 贪婪序列中时, 检查是否 $F(g') \geq F(m)$ 。如果是, 则杀死 m 并继续执行常规运行, 不再与 $F(m)$ 进行比较。另一方面, 如果 $F(g') < F(m)$, 则 m 保持活跃, 并且下一个添加到 $\leftarrow$ 贪婪序列中的元素也需要与 $F(m)$ 进行比较。请注意, 如果 m 应该被杀死, 那么这个程序会杀死它。具体来说, 如果 m 应该被杀死, 那么它的窗口和 B 中有一个 x 满足 $F(x) \geq F(m)$, 因为如上所述, 任何其他杀死 m 的原因在处理前一个块时已经将其杀死。这样的最右边的 x 将被添加到 $\leftarrow$ 贪婪序列中, 因此将与 $F(x)$ 通过我们的算法与 $F(m)$ 进行比较, 从而杀死 m 。

在刚才讨论的情况下, 许多位置都有可能将其 F 值与 $\leftarrow$ 贪婪列表中 m 和当前最后一个位置处的 F 值进行比较。实际上, 只要 m 仍然活跃, 每当一个位置 $\leq m+h$ 被添加到 $\leftarrow$ 贪婪列表中时, 这种情况就会发生。从先验角度看, 此类情况出现的次数受 $\leftarrow$ 贪婪列表长度的限制, 正如我们在第 2.4 节所见, 其平均长度大约为 $\ln h$ 。事实上, 稍后我们将得到一个更好的平均界限, 但首先我们通过展示如何处理项目 (4) 来完成算法的描述。

项目 (4) 在 B 运行完成后由一个单独的过程处理 (无论是普通运行还是修改以处理项目 (1))。因此, 假设该运行已完成, 并且它产生了一个候选, 即 $\leftarrow$ 贪婪序列的最后一个项 g , 并且这个候选尚未被杀死 (即 B 中其左侧没有具有相同 F 值的点)。对于项目 (4), 我们必须检查 g 的窗口和前一个块中是否存在某个点 x 具有更大的 F 值 $F(x) > F(g)$ 。关键观察是, 如果这种情况发生, 那么最右侧的 x 位于前一个块的 $\leftarrow$ 贪婪序列中。实际上, 对于该块中的所有 $y > x$, 我们有 $F(x) > F(g) \geq F(y)$ 。因此, 为了寻找这样的 x , 只需遍历先前计算的前一个块的 $\leftarrow$ 贪婪序列的项即可。由于任何块中 $\leftarrow$ 贪婪序列的长度平均仅为大约 $\ln h$, 我们可以仅使用 $\ln h$ 次额外的比较来处理项目 (4), 即每个位置平均仅进行 $\ln h$ 次比较。

这完成了算法的描述, 以及 F 值比较次数估计的大部分内容, 但并非全部。平均而言, 在处理一个块的过程中, 我们最多有

- h 次比较来处理项目 (2) 和 (3),
- $\ln h$ 次比较来处理在 $F(g) < F(m)$ 和 $F(g') < F(m)$ 的情况下, 对于许多后续元素中的项目 (1) g' of 的 $\leftarrow$ 贪婪序列, 以及
- $\ln h$ 次比较来处理项目 (4)。

我们现在证明, 与项目 (1) 相关的 $\ln h$ 可以大大减少, 即减少为一个常数。因此, 平均比较次数的最终估计将是 $h + \ln h + O(1)$ 每块或 $1 + \frac{\ln h}{h} + O(\frac{1}{h})$ 每个位置。请注意, 即使没有这个改进, 我们也已经知道每个位置的平均比较次数最多是 $1 + \frac{2\ln h}{h}$ 。

Remark 65. Before proceeding with the proof, we give a rough, intuitive argument for why we might expect the number of comparisons needed for (1) to be less than the estimate $\ln h$. If there is a live candidate in the preceding block, its expected position is at the middle of that block, and so our algorithm begins paying attention to item (1) around the middle of the current block. Once it begins paying attention, it performs an extra comparison when adding an element to the $\leftarrow$ greedy sequence. But, if attention begins at the middle of the block, then there is a 50% chance that nothing will be added to the $\leftarrow$ greedy sequence from that point on, because there is a 50% chance that the largest F -value in the block is in the right half of the block, in which case the $\leftarrow$ greedy sequence is already complete when the algorithm reaches the middle of the block. Of course, a real proof must take into account that the live candidate in the preceding block isn't known to be at the midpoint, so its variability must be taken into account. That is what the following argument does.

We wish to bound, from above, the expectation of the number X of additional comparisons introduced by the part of the algorithm that handles item (1). To begin, we imagine some changes in the algorithm, which make X worse, i.e., bigger, but which simplify the computation of its expectation. Of course by bounding the expectation of the imagined, larger X , we obtain a fortiori the same bound for the expectation of the actual X .

The first imagined change is that we pretend that the position m of a maximum of F in the preceding block is a live candidate, whether or not it really is one. This clearly increases the number X of extra comparisons, because we will be doing comparisons for the sake of item (1) in some cases where the actual algorithm can ignore item (1) because there is no live candidate. In the case where the maximum is attained at several positions, we choose one of them at random to serve as m . (This is one of the cases where there is really no live candidate.) Observe for future reference that m is equally likely to be any of the h positions in the preceding block, and therefore the place where our algorithm will begin doing extra work is equally likely to be immediately to the left of any of the $h + 1$ positions in the current block B .

The second imagined change is that, once we get into the range where the extra work is done, every element added to the $\leftarrow$ greedy sequence contributes an extra comparison. This makes X worse because in the actual algorithm the extra comparisons would end when and if the $\leftarrow$ greedy sequence acquires a member g with $F(g) \geq F(m)$.

With both imagined changes, X is simply the number of positions that are put into the $\leftarrow$ greedy sequence and are $\leq m + h$. As noted above, m is a random variable uniformly distributed in the preceding block, and so $m + h$ is uniformly distributed in the range from the rightmost element of the preceding block to the next-to-rightmost element of the current block B . (In other words, the range of values of $m + h$ is B shifted one step to the left.) Let us write r for the location of $m + h$ in this range, but counted from right to left (the direction that the algorithm goes while processing B). Thus, $r = 1$ means the next-to-rightmost element of B ; $r = h + 1$ means the rightmost element of the preceding block. All $h + 1$ values of r are equally likely. For any particular r , the elements of the $\leftarrow$ greedy sequence that contribute to X are those whose distance from the right end of B is $> r$. (For example, if $r = 1$ then all but the rightmost element of B can contribute to X , while if $r = h + 1$ then nothing contributes to X .) The position at distance j from the right end of B has probability $1/j$ of being in the $\leftarrow$ greedy sequence; see Section 2.4. Therefore, the expectation of X (as increased by our imagined changes) is

$$\frac{1}{h+1} \sum_{r=1}^{h+1} \sum_{j=r+1}^h \frac{1}{j}.$$

This double sum is easy to estimate by interchanging the order of summation. Specifically, for any fixed j in the relevant range $2 \leq j \leq h$, the fraction $\frac{1}{j}$ occurs $j - 1$ times, namely once for each r in the range $1 \leq r \leq j - 1$. So these terms contribute $\frac{j-1}{j} < 1$. This happens for each of the $h - 1$ values of j , so the double sum is $< h - 1 < h + 1$, and the expectation of X is therefore < 1 .

9. Evaluation and experiments

The chunking methods were at first developed and tested without a theoretical analysis. As part of the initial development of RDC we conducted several measurements on different chunking methods. Many of the experiments are reported in [28]. One immediately recognized advantage with the local-maximum chunking method was that it only required to be configured with a minimum chunk length, while the point-filter method required both a minimum length and a cut-point probability (in the form of a bit-mask). However, the best configuration could potentially be identified once and for all, making this advantage irrelevant. A more important advantage was soon recognized experimentally: the local-maximum chunking method produced higher-quality compression. Eventually we set out to develop a theoretical analysis of chunking methods whose results are reported above. In this section, we report some experimental results.

In the experiments we measured the effect of chunking methods as well as other aspects of RDC on a large corpus of internal Microsoft design documents as well as hard-disk file images (known as virtual hard-disks). The design documents were mostly in the Microsoft Office Word, Power-Point and Excel formats. This corpus of around 10,000 Microsoft Office files was chosen to represent user scenarios for the RDC protocol. We used a smaller set of around 20 virtual hard-disks for similar experiments. Each virtual hard-disk contains from a few hundred MB up to 10 GB of data. The experiments exercised RDC when the receiver could use chunks from several other local files in addition to the file being transmitted; the use of multiple local files allows RDC to reduce bandwidth usage. The initial evaluation therefore covered much more than just the

注释 65。在继续证明之前，我们给出一个粗略的直观论证，解释为什么我们可能期望 (1) 所需的比较次数小于估计的 $\ln h$ 。如果前一个块中有一个活动的候选，其期望位置在该块的中部，因此我们的算法从当前块的中部开始关注项目 (1)。一旦开始关注，当向 $\leftarrow$ 贪婪序列添加元素时，它会执行一次额外的比较。但是，如果关注从块的中部开始，那么有 50% 的可能性从那时起不会有任何内容被添加到 $\leftarrow$ 贪婪序列中，因为该块中最大的 F 值有 50% 的概率位于块的右半部分，在这种情况下，当算法到达块的中部时， $\leftarrow$ 贪婪序列已经完成。当然，一个真正的证明必须考虑到前一个块中的活动候选并不一定位于中点，因此必须考虑其可变性。这就是以下论证所做的事情。

我们希望从上方界定算法中处理项目 (1) 所引入的额外比较次数 X 的期望。首先，我们想象算法的一些变化，这些变化会使 X 变差，即变大，但会简化其期望的计算。当然，通过界定想象的、更大的 X 的期望，我们自然地得到了实际 X 期望的相同界限。

第一个想象中的变化是，我们假装前一个块中 F 的最大值的位置 m 是一个活跃候选，无论它是否真的是一个。这显然增加了额外比较的次数 X ，因为我们在某些情况下会为项目 (1) 进行比较，而实际算法可以忽略项目 (1)，因为没有活跃候选。在最大值在多个位置达到的情况下，我们随机选择其中一个作为 m 。（这是真正没有活跃候选的情况之一。）请注意， m 等可能地是前一个块中的 h 个位置中的任何一个，因此我们的算法开始进行额外工作的位置等可能地是当前块 B 中 $h + 1$ 位置中的任何一个的立即左侧。

第二个假想的变化是，一旦我们进入额外工作完成的范围，添加到 $\leftarrow$ 贪婪序列中的每个元素都会贡献一次额外的比较。这使 X 变得更差，因为在实际算法中，额外的比较会在 $\leftarrow$ 贪婪序列获得一个具有 $F(g) \geq F(m)$ 的成员 g 时结束。

对于这两个假想的变化， X 简单地是放入 $\leftarrow$ 贪婪序列且是 $\leq m + h$ 的位置数量。如上所述， m 是在先前块中均匀分布的随机变量，因此 $m + h$ 在从先前块的最后一个元素到当前块 B 的倒数第二个元素的范围中均匀分布。（换句话说， $m + h$ 的值范围是 B 向左移动一步。）让我们将 $m + h$ 在此范围内的位置写为 r ，但按从右到左的方向计数（即算法处理 B 时行的方向）。因此， $r = 1$ 表示 B ； $r = h + 1$ 表示先前块的最后一个元素。所有 r 的 h 值都是等可能的。对于任何特定的 r ，对 X 有贡献的 $\leftarrow$ 贪婪序列中的元素是那些距离 B 右端为 $> r$ 的元素。（例如，如果 $r = 1$ ，则 B 的所有元素除了最右端的元素都可以对 X 有贡献，而如果 $r = h + 1$ ，则没有任何元素对 X 有贡献。）距离 B 右端 j 的位置在 $\leftarrow$ 贪婪序列中的概率为 $1/j$ ；见第 2.4 节。因此， X （由于我们的假想变化而增加）的期望是

$$\frac{1}{h+1} \sum_{r=1}^{h+1} \sum_{j=r+1}^h \frac{1}{j}.$$

这个双重求和很容易通过交换求和顺序来估计。具体来说，对于相关范围内固定的 j $2 \leq j \leq h$ ，分数 $\frac{1}{j}$ 出现 $j - 1$ 次，即对于范围 $1 \leq r \leq j - 1$ 中的每个 r 都出现一次。因此这些项贡献了 $\frac{j-1}{j} < 1$ 。对于 $h - 1$ 个 j 值中的每一个都会发生这种情况，所以双重求和是 $< h - 1 < h + 1$ ，因此 X 的期望是 < 1 。

9. 评估与实验

分块方法最初在没有理论分析的情况下开发和测试的。作为 RDC 初始开发的一部分，我们对不同的分块方法进行了一些测量。许多实验结果报告在 [28] 中。人们立即认识到局部最大分块方法的一个优点是它只需要配置最小块长度，而点过滤方法则需要最小长度和切割点概率（以位掩码的形式）。然而，最佳配置可以一次性确定，这使得这个优势变得无关紧要。一个更重要的优势很快通过实验被认识到：局部最大分块方法产生了更高质量的压缩。最终我们着手开发分块方法的理论分析，其结果已在前面报告。在本节中，我们报告了一些实验结果。

在实验中，我们测量了分块方法以及其他 RDC 方面的效果，对象是一个包含大量内部 Microsoft 设计文档以及硬盘文件镜像（称为虚拟硬盘）的大语料库。设计文档大多是 Microsoft Office Word、PowerPoint 和 Excel 格式。这个包含约 10,000 个 Microsoft Office 文件的语料库被选来代表 RDC 协议的用户场景。我们使用了一个较小的约 20 个虚拟硬盘的集合进行类似的实验。每个虚拟硬盘包含几百 MB 到 10 GB 的数据。实验在接收器可以使用除正在传输的文件之外来自几个其他本地文件的块时进行 RDC；使用多个本地文件允许 RDC 减少带宽使用。因此，初始评估涵盖的内容远不止

isolated effect of chunking methods. Nevertheless, some of the experiments have specifically covered the efficiency of the chunking methods, and we recall and present some of the results in the next subsection. We then describe a few selected measurements that are targeted more narrowly on evaluating the chunking methods.

9.1. Computation overhead

While developing the RDC protocol we were mainly interested in measuring the combined overhead of chunking and other factors dominating file transfer. And we measured the combined client and server overhead for file transmission. We studied in particular the following two extreme scenarios. In one scenario, the client happens to have the exact file being transmitted, and in the other scenario the client has none of the chunks being transmitted. We compared RDC against RSYNC and two other widely available utilities for differential compression, namely xdelta [19], and BSDiff [21]. (The LBFS system is less appropriate for such an experiment because normally it relies on a database that contains the chunks of multiple files.) On a Pentium 3 machine we measured an average number of cycles per byte of the transmitted file. In the first scenario we had 31 cycles for RDC, 45 for RSYNC, 39 for xdelta, and 2580 for BSDiff. In the second scenario, we had 36 cycles per byte for RDC, 32 for RSYNC, 410 for xdelta and 2780 cycles for BSDiff. Thus, RDC and RSYNC appear comparable as far as the CPU overhead is concerned, while the two other utilities require much more CPU. (In fact they require more memory as well.)

Early performance testing of the RDC protocol indicated that calculating chunk boundaries and signatures (that is the hash values of the chunks) is a significant CPU bottleneck. Low-level machine-architecture specific optimizations were used, with significant benefit, to boost performance of the inner loops in the chunking routines.

We also used the optimization described below in Section 10.2. That optimization results in a chunking algorithm that is hash-less in the sense that it bypasses the rolling hash stage for the local-maximum chunking method. To compare the hash-less algorithm with the one based on a rolling hash we measured the average number of cycles required to compute the chunks. For the expected chunk length of 256 bytes, a 64 bit Pentium 4 machine requires 8.4 cycles per byte for the hash-less algorithm and 18.7 cycles per byte for the original local-maximum algorithm based on a fast (and low quality) rolling hash. Our fast rolling hash uses just two bitwise xor operations and one bitwise rotation per processed byte. The point-filter method cannot avoid the rolling hash stage but it takes advantage of branch predicting hardware while determining cut-points. As a result, it requires only 7.6 cycles per byte using the low quality hash. By definition low quality hashing has too many collisions. From the point of view of collisions, the hash-less method works as a perfect hash: there are no collisions. The point-filter method using the higher quality Karp–Rabin hash took 15.8 cycles per byte.

Overall, the overhead of computing chunk boundaries based on the local-maximum method seems reasonable for chunks of size 256 bytes and up. However we later noticed that the point filter method was preferable in scenarios (different from RDC scenarios) where chunks of size 30–40 bytes should be generated with as little CPU overhead as possible. In the context of such small chunks branch-predicting hardware is critical.

9.2. Intra-file compression

In the following experiments we measure how the local maximum, the interval filter and the non-pure point-filter chunking methods (used in LBFS) compare. We observe the distribution of chunk sizes on real-world (non-random) files, and we measure the compressibility of long chunks as well as the effectiveness of the chunking methods for identifying repeated chunks in the same files.

Our experiments use three files from the RDC target domain. The first is the Outlook folder file (.ost file) of the first author. It contains 2 years of email messages and its size is 1.17 GB. The second file is a virtual hard disk (.vhd file) for Windows Server 2008, Enterprise edition. The file contains a full operating system installation and its size is 1.71 GB. Our third file is an 11 MB power-point presentation (.ppt file).

For the local-maximum chunking method we set the horizon size to 90 so that the expected chunk size is 181. For the interval-filter method we use a bit-mask with 6 bits so that the expected chunk size is $e \cdot 2^6 = 173$. And for the point-filter method we use a bit-mask with 7 bits and a minimal chunk length of 20 so that the expected chunk length is 148 bytes. However, as the experimental data show, the average chunk sizes vary depending on the files and tend to be slightly higher than the expected chunk sizes based on random files.

The effectiveness of finding repeated chunks in the same file is reflected in the overall compression ratio obtained by the methods. We observe that the local-maximum method finds up to 10% more repeated bytes than the other methods. Our graphs in Figs. 3, 4 and 5 also give an indication of the distribution of chunk sizes. The horizontal line indicates chunk sizes. The vertical scale shows the total number of bytes used by chunks of the size indicated by the horizontal line.

9.3. Distribution of long chunks

In Section 5.4 we analyzed the probability of long chunks for the three chunking methods. As an experimental counterpart, Fig. 6 summarizes how much of each file resides in chunks that are of size at most L , $2L$, $3L$, and up to $21L$ where L is the expected chunk length. In all three figures, the local-maximum method contains more chunk data close to the average chunk length; it is represented by the left-most and top-most line. For example, for the Outlook folder file, 43% of the

分块方法的孤立效应。然而，一些实验特别涵盖了分块方法的效率，我们回顾并将在下一小节中呈现其中一些结果。然后，我们描述了一些更专门针对评估分块方法的选择性测量。

9.1. 计算开销

在开发 RDC 协议时，我们主要关注于测量分块和其他主导文件传输的因素所带来的总开销。我们还测量了文件传输的客户端和服务端端的总开销。我们特别研究了以下两种极端场景。在一个场景中，客户端恰好拥有正在传输的完整文件，而在另一个场景中，客户端没有任何正在传输的块。我们将 RDC 与 RSYNC 以及另外两种广泛使用的差分压缩工具——xdelta [19], 和 BSDiff [21] 进行了比较。（LBFS 系统不太适合进行此类实验，因为它通常依赖于包含多个文件块的数据库。）在一台奔腾 3 机器上，我们测量了传输文件每个字节的平均周期数。在第一种场景中，RDC 为 31 周期，RSYNC 为 45 周期，xdelta 为 39 周期，而 BSDiff 为 2580 周期。在第二种场景中，RDC 为每个字节 36 周期，RSYNC 为 32 周期，xdelta 为 410 周期，BSDiff 为 2780 周期。因此，就 CPU 开销而言，RDC 和 RSYNC 看起来相当，而另外两种工具则需要更多的 CPU。（实际上它们还需要更多的内存。）

早期的 RDC 协议性能测试表明，计算块边界和签名（即块的哈希值）是一个显著的 CPU 瓶颈。通过使用针对底层机器架构的特定优化，在块处理例程的内循环性能提升方面取得了显著效益。

我们还在第 10.2 节中使用了下面描述的优化方法。该优化方法产生了一种无哈希的分块算法，在局部最大分块方法的意义上绕过了滚动哈希阶段。为了将无哈希算法与基于滚动哈希的算法进行比较，我们测量了计算块所需的平均周期数。对于 256 字节预期块长度，64 位奔腾 4 机器对于无哈希算法需要每个字节 8.4 个周期，而对于基于快速（且质量低）滚动哈希的原始局部最大算法需要每个字节 18.7 个周期。我们的快速滚动哈希每个处理字节只使用两次按位异或操作和一次按位旋转。点过滤方法无法避免滚动哈希阶段，但在确定切割点时利用了分支预测硬件。因此，它在使用低质量哈希时只需要每个字节 7.6 个周期。根据定义，低质量哈希有太多的冲突。从冲突的角度来看，无哈希方法就像一个完美的哈希：没有冲突。使用更高质量 Karp–Rabin 哈希的点过滤方法需要每个字节 15.8 个周期。

总体而言，基于局部最大值方法计算块边界的高开销对于 256 字节及以上的块来说似乎是合理的。然而，我们后来注意到在需要尽可能减少 CPU 开销以生成 30–40 字节块的场景（不同于 RDC 场景）中，点过滤器方法更可取。在这样的小块中，分支预测硬件至关重要。

9.2. 文件内压缩

在以下实验中，我们测量局部最大值、区间过滤器和非纯点过滤器块方法（用于 LBFS）的比较情况。我们观察真实世界（非随机）文件上的块大小分布，并测量长块的压缩性，以及块方法在相同文件中识别重复块的有效性。

我们的实验使用了来自 RDC 目标域的三个文件。第一个是第一作者 Outlook 文件夹文件（.ost 文件），其中包含 2 年的电子邮件消息，大小为 1.17 GB。第二个文件是 Windows Server 2008 企业版的虚拟硬盘（.vhd 文件），其中包含完整的操作系统安装，大小为 1.71 GB。我们的第三个文件是一个 11 MB 的演示文稿（.ppt 文件）。

对于局部最大值分块方法，我们将视野大小设置为 90，以便预期块大小为 181。对于区间过滤方法，我们使用一个 6 位的掩码，以便预期块大小为 $e \cdot 2^6 = 173$ 。而对于点过滤方法，我们使用一个 7 位的掩码和最小块长度为 20，以便预期块长度为 148 字节。然而，正如实验数据显示，平均块大小会根据文件而变化，并且往往会略高于基于随机文件计算的预期块大小。

在同一个文件中查找重复块的有效性体现在这些方法获得的总体压缩比上。我们观察到，局部最大值方法比其他方法最多多找到 10% 的重复字节。我们图 3、图 4 和图 5 中的图表也给出了块大小的分布情况。水平线表示块大小。垂直刻度显示了水平线所指示大小的块所使用的总字节数。

9.3. 长块分布

在 5.4 节中，我们分析了三种分块方法下长块的概率。作为实验的对照，图 6 总结了每个文件中有多少数据存在于块大小最多为 L 、 $2L$ 、 $3L$ 以及最多为 $21L$ 的块中，其中 L 是预期的块长度。在所有三个图中，局部最大值方法包含更多接近平均块长度的块数据；它由最左侧和最上侧的线表示。例如，对于 Outlook 文件夹文件，43% 的块长度最多为 L ，90% 的块长度最多为 $2L$ 。在所有三个图中，第二好的线表示区间过滤方法。最低和最右侧的线表示点过滤方法，该方法在较长的块中有更多文件数据。例如，对于 Outlook，要覆盖 90% 的文件数据需要包括大小最多为 $5L$ 的块（与局部最大值方法的 $2L$ 形成对比）。

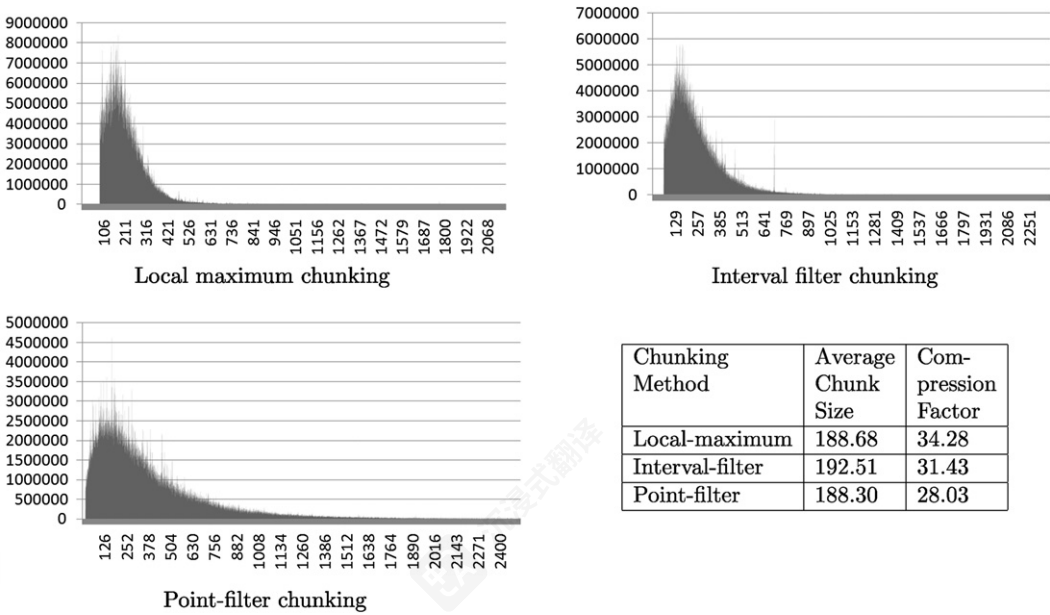


Fig. 3. Chunking-based compression of the Outlook folder file.

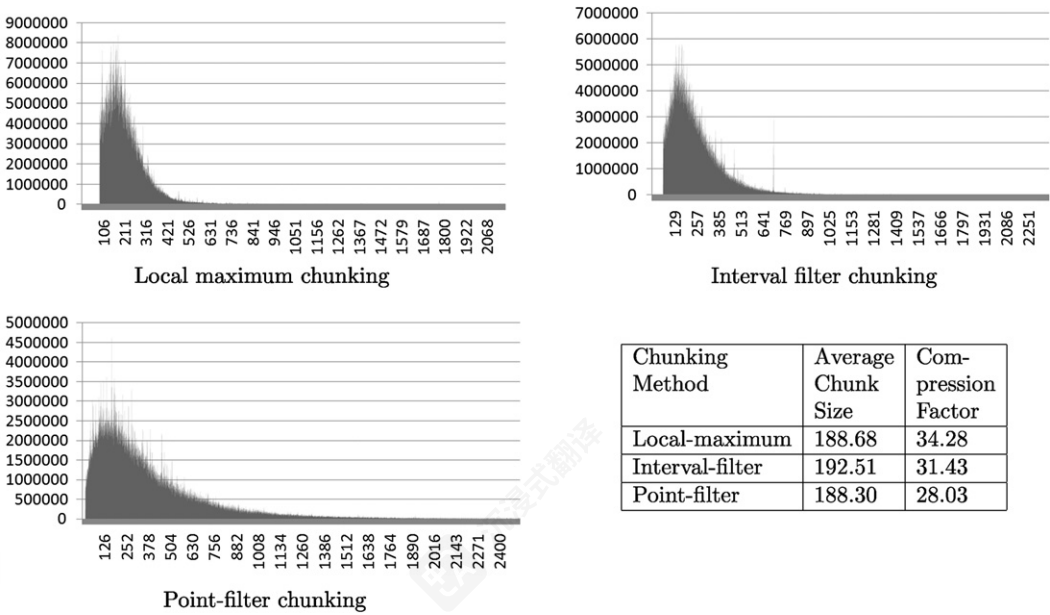


图 3. 基于分块压缩的 Outlook 文件夹文件

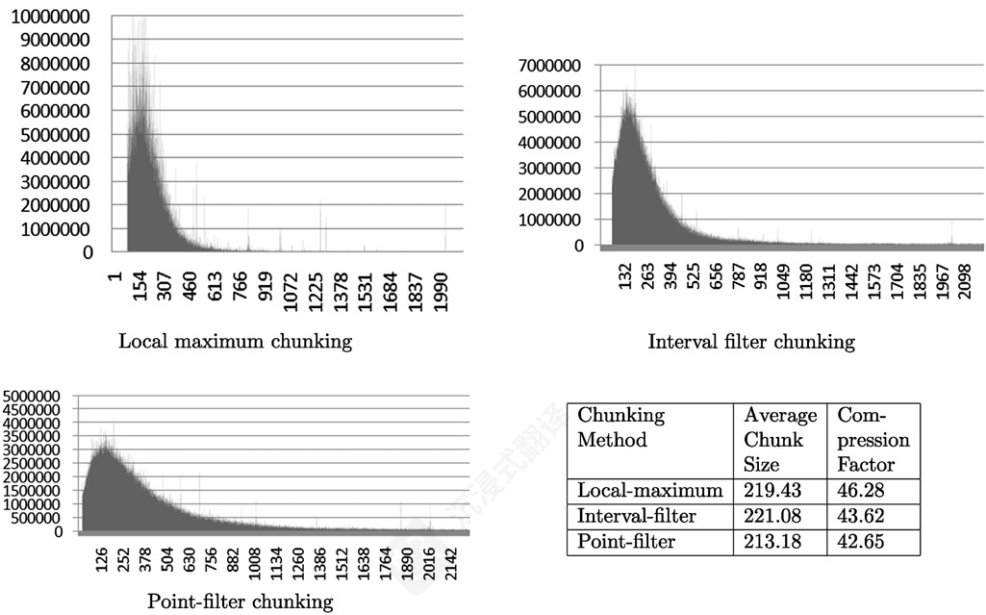


Fig. 4. Chunking-based compression of the Windows Server virtual hard disk.

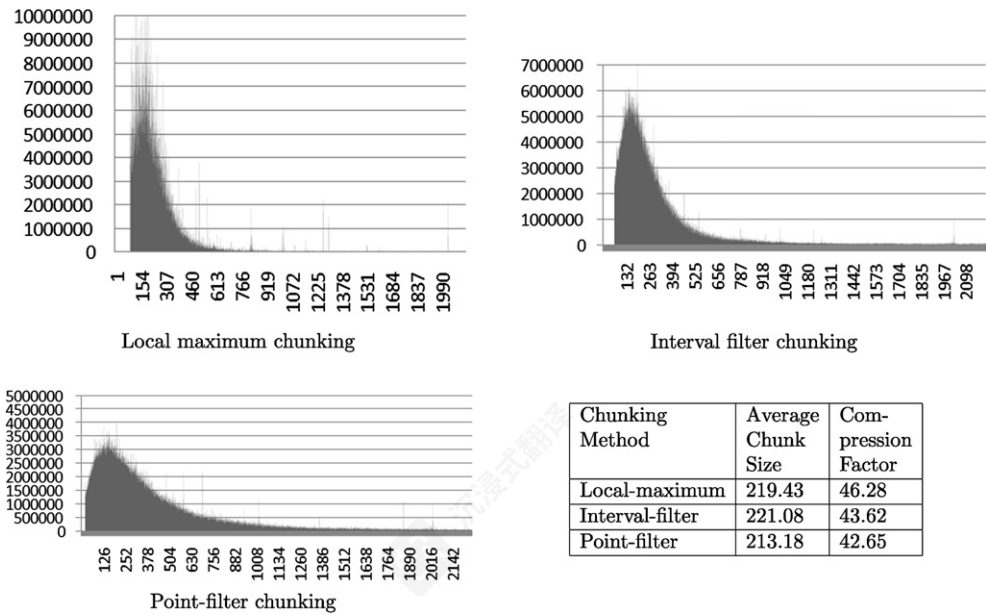


图 4. 基于分块压缩的 Windows 服务器虚拟硬盘

chunks are of length at most L , and 90% of the chunks are of length at most $2L$. The second best lines in all three figures represent the interval-filter method. The lowest and right-most lines represent the point-filter method which has more file data in longer chunks. For example, for Outlook, to cover 90% of the file data requires including chunks of size up to $5L$ (as opposed to $2L$ in the local-maximum method).

9.4. Run-length compression

For the local-maximum method and the Outlook file, we furthermore measured the compressibility of the chunks as a function of their size. We performed two experiments. The first was inspired by Remark 8. We used a simple run-length encoder that replaced repeated bytes and pairs of repeated bytes by a code. For example, it would replace the string *abababcccdffg* by *x3aby4cz3dfg*, where *x* is used as a control character for double-byte repetition, *y* for single byte

对于Outlook，要覆盖90%的文件数据需要包括大小最多为5L的块（与局部最大值方法的2L形成对比）。

9.4. 游程压缩

对于局部最大值方法和Outlook文件，我们进一步测量了块的可压缩性，将其作为块大小的函数。我们进行了两个实验。第一个实验受注释8的启发。我们使用了一个简单的行程长度编码器，该编码器用代码替换重复的字节和重复的字节对。例如，它会将字符串*abababcccdffg*替换为*x3aby4cz3dfg*，其中*x*用作双字节重复的控制字符，*y*用于单字节重复，而*z*用于其他情况。在第二个实验中，我们应用了标准的Lempel–Ziv压缩算法对块进行压缩，以更准确地测量可压缩性。

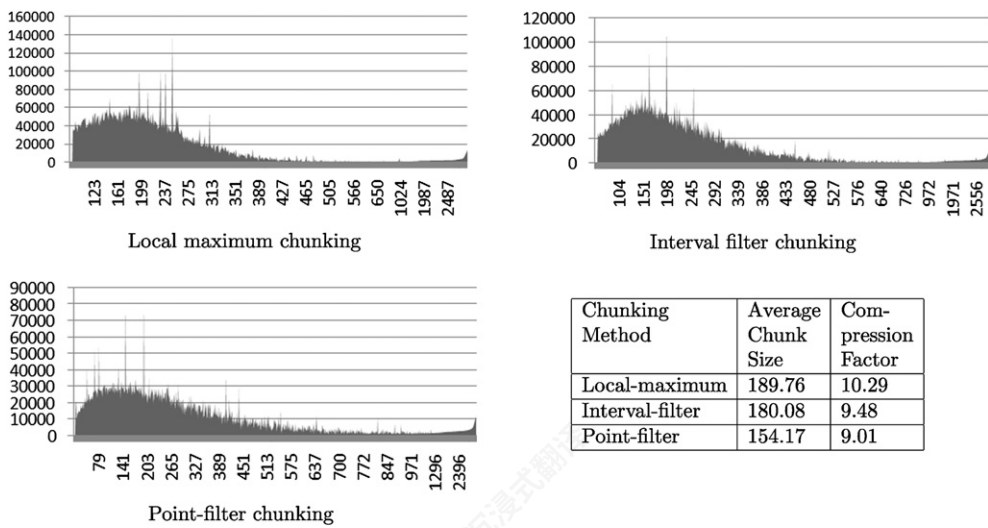


Fig. 5. Chunking-based compression of the power-point file.

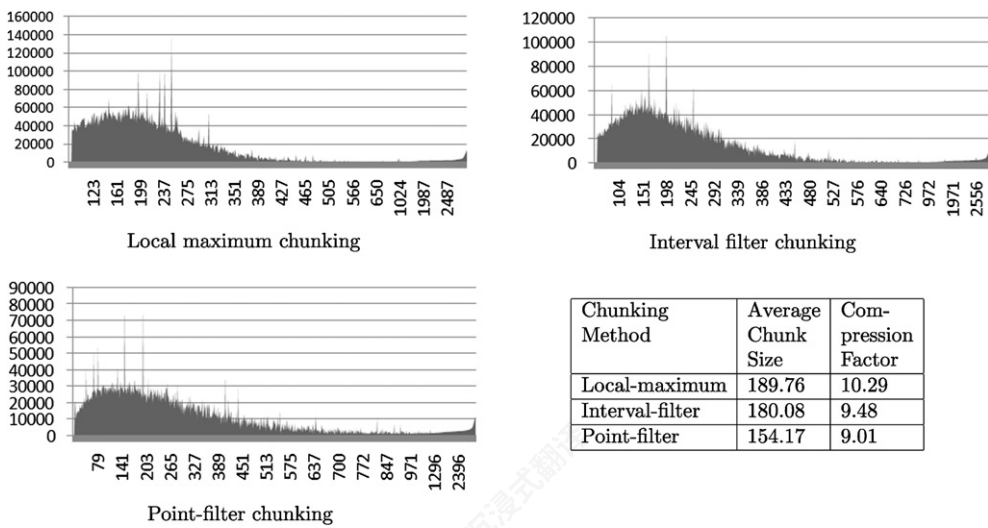


图 5. 基于块划分的演示文稿文件压缩。

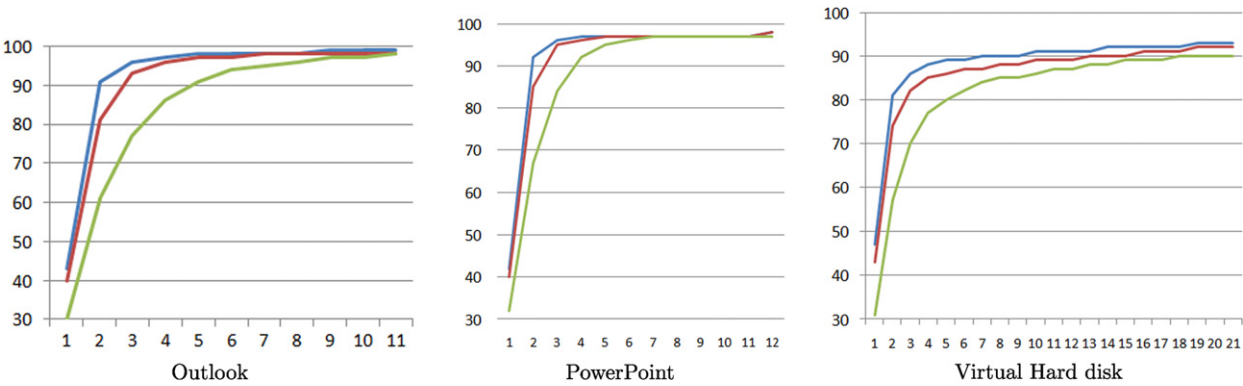


Fig. 6. Distribution of long chunks.

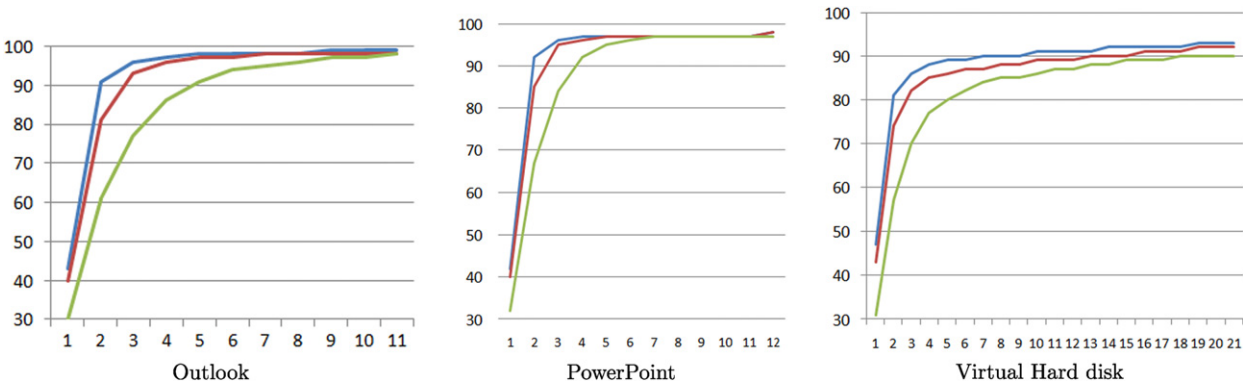


图 6. 长块分布。

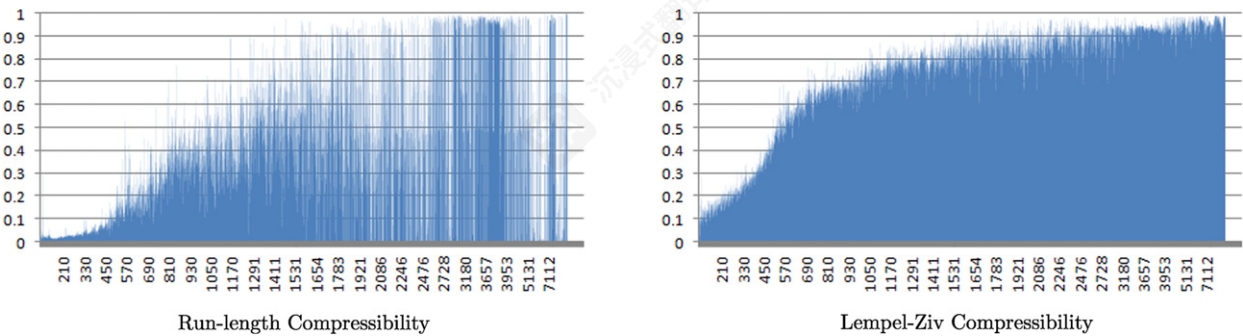


Fig. 7. Compressibility of file data by chunk lengths.

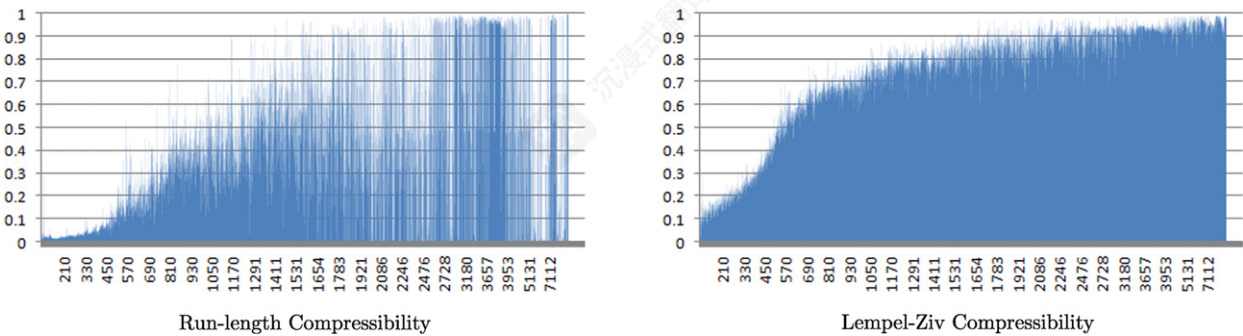


图 7. 文件数据通过块长度压缩的可压缩性。

repetition, and z is used otherwise. In the second experiment, we applied the standard Lempel–Ziv compression algorithm on the chunks to measure the compressibility more accurately.

Fig. 7 summarizes the results. The x-axes of both histograms show the chunk lengths and the y-axis show the compression ratio. A ratio of 1 indicates the perfect compression. Note that practically all long chunks are highly compressed and that the run-length encoder is often sufficient for the long chunks.

在第二个实验中，我们应用了标准的Lempel–Ziv压缩算法对块进行压缩，以更准确地测量可压缩性。

图7总结了结果。两个直方图的x轴显示块长度，y轴显示压缩比。比率为1表示完美压缩。请注意，实际上所有长块都被高度压缩，并且游程编码器通常足以处理长块。

10. Related work, variants, and open problems

10.1. Previous work

After we developed the main results on local maximum chunking, we learned of a related technique proposed in [26]. That paper introduces the concept of local algorithms for document fingerprinting.

In the situation treated in [26], at least one distinguished position, whose file entry (a hash value) is to be part of the fingerprint, must be chosen within each interval of w consecutive positions. The task of choosing these distinguished values is analogous to the task of choosing cutpoints in remote differential compression. In both cases, the chosen positions should be neither too close together nor too far apart, and in both cases agreements between substantial segments of two files should result in corresponding positions being chosen in the two files. Because of the analogy and for the sake of brevity, we shall write “cutpoint” to refer to the distinguished positions, even though [26] does not envision actually cutting files there.

The winnowing method proposed in [26] chooses a cutpoint by considering each interval of hashes with length w and selecting the index with the minimal hash value; if there is a tie, the right-most position with minimal hash value is chosen. As a result, successive cutpoints may be arbitrarily close to each other, but their distance never exceeds w . Winnowing is a local algorithm because whether a position is a cutpoint depends only on hash values within w positions to both sides of it. Analogously to the probability distribution of local maxima, the *density* of local minima is computed as $2/(w + 1)$. It is also shown in [26] that any local scheme for choosing distinguished points never farther than w apart must have density at least $\frac{1.5}{1+w}$. It is also shown in [26] that, for the pure point filter method or any impure variant thereof, if the method ensures at least one cutpoint in every interval of length w , then the density of cutpoints is at least $\frac{1+\ln w}{w}$. Finally, [26] gives an algorithm for determining the cut-points.

Our results on local maximum chunking can be viewed as complementary: The winnowing scheme of [26] imposes an absolute upper bound w on the distance between consecutive cutpoints (which we’ll call the chunk length) and seeks to prevent the chunks from being too much shorter. Our local maximum method imposes an absolute lower bound $h + 1$ on the chunk length and seeks to prevent the chunks from being too much longer. This is why we strive for small slack and for low probability of long chunks.

Finally, we provide an analysis of the average number of file comparisons per file entry, which translates into branch mispredictions.

10.2. Hash-less local maxima

All of the chunking methods we have considered relied on a pre-processing step that uses a hash function to distill characters in an interval of length w into machine representable numbers. This is useful when a cut-point can be determined by using arithmetic or logical operations on the numbers that are directly supported by the CPU. In the point filter, LBFS, and interval-filter chunking methods, these operations consisted of masking selected bits to determine cut-points. In the local maximum approach, the relevant operation is comparison.

It is however, possible to skip the pre-processing step in the local maximum method and treat a block of w consecutive characters as an $8 \cdot w$ bit number. For numbers of this size, the comparison operation is not directly provided by the CPU, and so we have to supply our own comparison operation to find local maxima among these numbers.

On average, and in practice, it seems that even a naïve procedure for finding locally maximal substrings by means of lexicographic byte-wise comparisons is superior, in terms of running time, to pre-processing $|F|$ with a rolling hash. While a lexicographic byte-wise comparison over words with w bytes requires in the worst case w comparisons per position, it is likely that far fewer comparisons (often just one) are required because the most significant byte of the largest number so far is likely to be larger than a random byte.

One straightforward refinement of the naïve procedure is to record the number of repeated characters at the currently scanned position. This allows processing files consisting of large blocks of the same characters independently of w , but it does not help in the case of files consisting of large blocks of periodic patterns (longer than a single character). Obviously, the first refinement may be generalized to also take periodic patterns into account. In general, it may be of theoretical interest to consider variants of Boyer–Moore [13,8,3] string matching algorithms that avoid repetitive scanning of the same characters. In contrast to Knuth–Morris–Pratt and Boyer–Moore string matching algorithms, this problem is not that of finding a fixed pattern, and we cannot rely on a one-pass pre-processing step.

10.3. Open problems

Our calculations leave several problems open:

- What is the slack of the local maximum method? Is it smaller than (our lower bounds for) the slack of the point filter and LBFS chunking methods?
- What is the variance of the chunk length for the local maximum method?
- Replace our estimates for the probabilities of long cutpoint-free intervals by exact values (or tighter estimates).

10. 相关工作、变体和开放问题

10.1. 以往工作

在我们就局部最大值块处理开发主要结果后，我们了解到 [26]中提出的一种相关技术。该论文介绍了用于文档指纹的局部算法的概念。

在 [26], 处理的情况中，在每个 w 个连续位置组成的区间内，必须选择至少一个特殊位置，其文件条目（一个哈希值）要成为指纹的一部分。选择这些特殊值的工作类似于在远程差分压缩中选择切点的工作。在这两种情况下，选择的位置既不能靠得太近，也不能离得太远，而且两种情况下，两个文件的大段内容之间的协议应该导致在两个文件中选择相应的位置。由于这种类比，并且为了简洁起见，我们将用“切点”来指代这些特殊位置，即使 [26] 并不打算在那里实际切割文件。

[26] 中提出的 winnowing 方法通过考虑长度为 w 的哈希值区间，并选择具有最小哈希值的索引来选择切点；如果有平局，则选择具有最小哈希值的右端位置。因此，连续的切点可以任意接近，但它们的距离永远不会超过 w 。Winnowing 是一个局部算法，因为一个位置是否是切点只取决于它两侧各 w 个位置内的哈希值。类似于局部最大值的概率分布，局部最小值的密度计算为 $2/(w + 1)$ 。[26] 还表明，任何选择特殊点且距离不超过 w 的局部方案都必须具有至少 $\frac{1.5}{1+w}$ 的密度。[26] 还表明，对于纯点过滤器方法或任何其不纯变体，如果该方法确保每个长度为 w 的区间内至少有一个切点，则切点的密度至少为 $\frac{1+\ln w}{w}$ 。最后，[26] 给出了一个确定切点的算法。

我们在局部最大值分块上的结果可以视为互补的：[26] 的筛选方案对连续切点（我们称之为块长度）之间的距离施加绝对上限 w ，并试图防止块变得太短。我们的局部最大值方法对块长度施加绝对下限 $h + 1$ ，并试图防止块变得太长。这就是为什么我们追求小的松弛和低的较长块的概率。

最后，我们提供了每条文件条目平均文件比较次数的分析，这转化为分支误预测。

10.2. 无哈希局部最大值

我们所考虑的所有分块方法都依赖于一个预处理步骤，该步骤使用哈希函数将长度为 w 的区间内的字符提炼成机器可表示的数字。当切点可以通过对CPU直接支持的数字进行算术或逻辑运算来确定时，这很有用。在点过滤器、LBFS和区间过滤器分块方法中，这些运算包括掩码选定位以确定切点。在局部最大值方法中，相关运算是比较。

然而，在局部最大值方法中可以跳过预处理步骤，将 w 个连续字符的块视为一个 $8 \cdot w$ 位数字。对于这种规模的数字，比较操作不是由CPU直接提供的，因此我们必须提供自己的比较操作来在这些数字中找到局部最大值。

平均而言，在实践中，即使通过字典序字节比较来查找局部最大子串的简单方法，在运行时间方面也比使用滚动哈希进行预处理 | F | 要优越。虽然对于 w 字节的单词进行字典序字节比较在最坏情况下每个位置需要 w 次比较，但很可能需要的比较次数要少得多（通常只需一次），因为迄今为止最大的数字的最高字节很可能比随机字节要大。

对简单方法的直接改进之一是记录当前扫描位置处重复字符的数量。这允许独立于 w 处理由大量相同字符组成的文件块，但在由大量周期性模式（长度超过单个字符）组成的文件块的情况下则无济于事。显然，第一个改进可以推广到也考虑周期性模式。一般来说，考虑避免对相同字符进行重复扫描的Boyer–Moore [13,8,3] 字符串匹配算法的变体可能具有理论意义。与Knuth–Morris–Pratt和Boyer–Moore字符串匹配算法不同，这个问题不是寻找固定模式的问题，我们无法依赖单次预处理步骤。

10.3. 开放性问题

我们的计算留下了几个悬而未决的问题：

- 局部最大值方法的松弛是多少？它是否小于（我们针对点过滤器和LBFS分块方法的下界）的松弛？
- 局部最大值方法的块长度方差是多少？
- 用精确值（或更紧的估计）替换我们对于长无切割点区间的概率估计。

Another natural question concerns the ratio of expected chunk size to minimum chunk size. We would like this ratio to be as small as possible. The absolute minimum value, 1, is attainable but only by chopping the file into chunks of constant size, and we have seen that this is a bad method because adding one character to a file can destroy all agreement between chunks. The LBFS method can attain ratios arbitrarily close to 1 by taking $h \gg c$ (i.e., $k \gg 1$), but we have seen that this also makes agreement between chunks difficult to attain; recall in particular Remark 50. A reasonable question is how small the ratio can be for local chunking methods. (Locality excludes both constant-length chunks and the LBFS chunking method with $h > 0$.) The interval filter attains a ratio approaching e (for large h), and the local maximum method does somewhat better with a ratio approximately 2. Can a local method do better yet?

Acknowledgments

We are indebted to Dan Teodosiu for initiating and driving the development of the RDC protocols. Joe Porkka developed RDC and its associated protocols that are included in the DFS replication system. Laci Lovász and Yuval Perez helped drawing our attention to ergodic theory. We are also thankful for numerous interactions with Mark Manasse, Akhil Wable and Le Wang on RDC.

References

- [1] M. Aigner, G.M. Ziegler, Proofs from THE BOOK, Springer-Verlag, 2001.
- [2] M. Ajtai, R. Burns, R. Fagin, D.D.E. Long, L. Stockmeyer, Compactly encoding unstructured inputs with differential compression, J. ACM 49 (3) (May 2002) 318–367.
- [3] C. Charras, T. Lecroq, Exact String Matching Algorithms, King's College London Publications, 2004.
- [4] Microsoft Corporation, Distributed file system replication, More information on DFSR is available from http://msdn.microsoft.co/library/en-us/stgmngmt/fs/distributed_file_system_replication__dfs_r_.asp, 2006.
- [5] Microsoft Corporation, MSN IM 8, <http://www.msn.com>, 2006.
- [6] Microsoft Corporation, Remote differential compression reference, <http://windowssdk.msdn.microsoft.com/en-us/library/ms715305.aspx>, 2006.
- [7] Microsoft Corporation, Windows meeting space, Windows Vista, More information is available from <http://www.microsoft.com/windows/products/windowsvista/features/details/meetingspace.mspx>, 2007.
- [8] M. Crochemore, C. Hancart, T. Lecroq, Algorithmique du texte, Vuibert, 2001.
- [9] M. Drinic, D. Kirovski, PPMexe: PPM for compressing software, in: IEEE Data Compression Conference, 2002, pp. 192–201.
- [10] P. Erdős, G. Szekeres, A combinatorial problem in geometry, Compos. Math. 2 (1935) 464–470.
- [11] W. Feller, An Introduction to Probability Theory and Its Applications, third ed., John Wiley & Sons, 1968.
- [12] R.L. Graham, D.E. Knuth, O. Patashnik, Concrete Mathematics, Addison–Wesley, 1994.
- [13] D. Gusfield, Algorithms on Strings, Trees and Sequences, Cambridge University Press, Cambridge, UK, 1997.
- [14] P. Halmos, E. Moise, G. Piranian, The problem of learning to teach, Amer. Math. Monthly 82 (1975) 466–476.
- [15] J.W. Hunt, T.G. Szymansky, A fast algorithm for computing longest common subsequences, Comm. ACM 20 (5) (May 1977) 350–353.
- [16] J.W. Hunt, M.D. McIlroy, An algorithm for differential file comparison, Computer Science Technical Report 41, Bell Labs, June 1976.
- [17] R.M. Karp, M.O. Rabin, Efficient randomized pattern-matching algorithms, IBM J. Res. Dev. 31 (2) (1987) 249–260.
- [18] J. Langford, Multiround RSYNC, Technical Report, Dept. of Computer Science, Carnegie-Mellon University. The report can be obtained from <http://www.cs.cmu.edu/~jcl/research/mrsync/mrsync.ps>.
- [19] J. MacDonald, File system support for delta compression, Master's thesis, Department of Electrical Engineering and Computer Science, University of California at Berkeley, 2000.
- [20] A. Muthitacharoen, B. Chen, D. Mazieres, A low-bandwidth network file system, in: Proceedings of the 18th ACM Symposium on Operating Systems Principles, 2001, pp. 174–187.
- [21] C. Percival, Naive differences of executable code, Technical Report, Oxford Computing Laboratory, University of Oxford, 2002.
- [22] K. Petersen, Ergodic Theory, Cambridge University Press, 1983.
- [23] M.O. Rabin, Fingerprinting by random polynomials, Technical Report TR-15-81, Harvard University, Center for Research in Computing Technology, 1981.
- [24] D. Rasch, R.C. Burns, In-place RSYNC: File synchronization for mobile and wireless devices, in: USENIX Annual Technical Conference, FREENIX Track, USENIX, 2003, pp. 91–100.
- [25] M.J. Rochkind, The source code control system, IEEE Trans. Software Eng. SE-1 (4) (December 1975) 364–370.
- [26] S. Schleimer, D.S. Wilkerson, A. Aiken, Winnowing: Local algorithms for document fingerprinting, in: A.Y. Halevy, Z.G. Ives, A. Doan (Eds.), SIGMOD Conference, ACM, 2003, pp. 76–85.
- [27] T. Suel, N. Memon, Algorithms for delta compression and remote file synchronization, in: Khalid Sayood (Ed.), Lossless Compression Handbook, Academic Press, December 2002.
- [28] D. Teodosiu, N. Bjørner, Y. Gurevich, M. Manasse, J. Porkka, A. Wable, Optimizing file replication over limited-bandwidth networks using remote differential compression, Technical Report, Microsoft Corporation, August 2006.
- [29] W.F. Tichy, RCS – A system for version control, Software Pract. Ex. 15 (7) (1985) 637–654.
- [30] D. Trendafilov, N. Memon, T. Suel, zdelta: An efficient delta compression tool, Technical Report TR-CIS-2002-02, Polytechnic University, Brooklyn, NY, June 2002.
- [31] A. Tridgell, Efficient algorithms for sorting and synchronization, PhD thesis, The Australian National University, February 1999.

另一个自然的问题是预期块大小与最小块大小的比率。我们希望这个比率尽可能小。绝对最小值1是可以达到的，但只有通过将文件分割成固定大小的块才能实现，而我们已经看到这是一种糟糕的方法，因为向文件中添加一个字符可能会破坏块之间的所有一致性。LBFS方法可以通过取 $h \gg c$ (i.e., $k \gg 1$)来达到任意接近1的比率，但我们已经看到这也使得块之间的一致性难以实现；特别是要回忆第50条注释。一个合理的问题是局部分块方法的比率可以小到多少。（局部性排除了固定长度的块和带 $h > 0$ 的LBFS分块方法。）区间过滤器达到接近 e （对于较大的 h ）的比率，局部最大值方法略好一些，比率约为2。局部方法还能做得更好吗？

致谢

我们感谢 Dan Teodosiu 促成了 RDC 协议的开发和推动。Joe Porkka 开发了 RDC 及其相关协议，这些协议包含在 DFS复制系统中。Laci Lovász 和 Yuval Perez 帮助我们关注遍历理论。我们也感谢与 Mark Manasse、Akhil Wable 和 Le Wang 在 RDC 方面的多次交流。

参考文献

- [1] M. Aigner, G.M. Ziegler, Proofs from THE BOOK, Springer-Verlag, 2001.
- [2] M. Ajtai, R. Burns, R. Fagin, D.D.E. Long, L. Stockmeyer, Compactly encoding unstructured inputs with differential compression, J. ACM 49 (3) (May 2002) 318–367.
- [3] C. Charras, T. Lecroq, Exact String Matching Algorithms, King's College London Publications, 2004.
- [4] Microsoft Corporation, Distributed file system replication, More information on DFSR is available from http://msdn.microsoft.co/library/en-us/stgmngmt/fs/distributed_file_system_replication__dfs_r_.asp, 2006.
- [5] Microsoft Corporation, MSN IM 8, <http://www.msn.com>, 2006.
- [6] Microsoft Corporation, Remote differential compression reference, <http://windowssdk.msdn.microsoft.com/en-us/library/ms715305.aspx>, 2006.
- [7] Microsoft Corporation, Windows meeting space, Windows Vista, More information is available from <http://www.microsoft.com/windows/products/windowsvista/features/details/meetingspace.mspx>, 2007.
- [8] M. Crochemore, C. Hancart, T. Lecroq, Algorithmique du texte, Vuibert, 2001.
- [9] M. Drinic, D. Kirovski, PPMexe: PPM for compressing software, in: IEEE Data Compression Conference, 2002, pp. 192–201.
- [10] P. Erdős, G. Szekeres, A combinatorial problem in geometry, Compos. Math. 2 (1935) 464–470.
- [11] W. Feller, An Introduction to Probability Theory and Its Applications, third ed., John Wiley & Sons, 1968.
- [12] R.L. Graham, D.E. Knuth, O. Patashnik, Concrete Mathematics, Addison–Wesley, 1994.
- [13] D. Gusfield, Algorithms on Strings, Trees and Sequences, Cambridge University Press, Cambridge, UK, 1997.
- [14] P. Halmos, E. Moise, G. Piranian, The problem of learning to teach, Amer. Math. Monthly 82 (1975) 466–476.
- [15] J.W. Hunt, T.G. Szymansky, A fast algorithm for computing longest common subsequences, Comm. ACM 20 (5) (May 1977) 350–353.
- [16] J.W. Hunt, M.D. McIlroy, An algorithm for differential file comparison, Computer Science Technical Report 41, Bell Labs, June 1976.
- [17] R.M. Karp, M.O. Rabin, Efficient randomized pattern-matching algorithms, IBM J. Res. Dev. 31 (2) (1987) 249–260.
- [18] J. Langford, Multiround RSYNC, Technical Report, Dept. of Computer Science, Carnegie-Mellon University. The report can be obtained from <http://www.cs.cmu.edu/~jcl/research/mrsync/mrsync.ps>.
- [19] J. MacDonald, File system support for delta compression, Master's thesis, Department of Electrical Engineering and Computer Science, University of California at Berkeley, 2000.
- [20] A. Muthitacharoen, B. Chen, D. Mazieres, A low-bandwidth network file system, in: Proceedings of the 18th ACM Symposium on Operating Systems Principles, 2001, pp. 174–187.
- [21] C. Percival, Naive differences of executable code, Technical Report, Oxford Computing Laboratory, University of Oxford, 2002.
- [22] K. Petersen, Ergodic Theory, Cambridge University Press, 1983.
- [23] M.O. Rabin, Fingerprinting by random polynomials, Technical Report TR-15-81, Harvard University, Center for Research in Computing Technology, 1981.
- [24] D. Rasch, R.C. Burns, In-place RSYNC: File synchronization for mobile and wireless devices, in: USENIX Annual Technical Conference, FREENIX Track, USENIX, 2003, pp. 91–100.
- [25] M.J. Rochkind, The source code control system, IEEE Trans. Software Eng. SE-1 (4) (December 1975) 364–370.
- [26] S. Schleimer, D.S. Wilkerson, A. Aiken, Winnowing: Local algorithms for document fingerprinting, in: A.Y. Halevy, Z.G. Ives, A. Doan (Eds.), SIGMOD Conference, ACM, 2003, pp. 76–85.
- [27] T. Suel, N. Memon, Algorithms for delta compression and remote file synchronization, in: Khalid Sayood (Ed.), Lossless Compression Handbook, Academic Press, December 2002.
- [28] D. Teodosiu, N. Bjørner, Y. Gurevich, M. Manasse, J. Porkka, A. Wable, Optimizing file replication over limited-bandwidth networks using remote differential compression, Technical Report, Microsoft Corporation, August 2006.
- [29] W.F. Tichy, RCS – A system for version control, Software Pract. Ex. 15 (7) (1985) 637–654.
- [30] D. Trendafilov, N. Memon, T. Suel, zdelta: An efficient delta compression tool, Technical Report TR-CIS-2002-02, Polytechnic University, Brooklyn, NY, June 2002.
- [31] A. Tridgell, Efficient algorithms for sorting and synchronization, PhD thesis, The Australian National University, February 1999.